



Mooncake AI Infra

实习指南

从零开始掌握大模型推理系统核心技术

AI基础设施 · LLM训练推理 · PD分离 · KV Cache优化

清华大学 Mooncake 团队

2025年2月

目录

第一章：AI基础设施（AI Infra）基础入门

- 1.1 AI Infra的定义和范畴
- 1.2 AI训练和推理系统的整体架构
- 1.3 GPU/TPU/NPU等AI加速硬件详解
- 1.4 分布式训练基础
- 1.5 集群网络和通信
- 1.6 存储系统在AI中的作用
- 1.7 AI Serving系统的演进

第二章：LLM训练与推理基础

- 2.1 Transformer架构详解
- 2.2 LLM训练流程
- 2.3 训练优化技术
- 2.4 LLM推理的两个阶段
- 2.5 KV Cache的原理和作用
- 2.6 推理优化技术
- 2.7 批处理和调度策略

第三章：AI Infra最新前沿论文分析（2024-2025）

- 3.1 引言：LLM推理优化的核心挑战
- 3.2 OSDI 2024论文深度分析
- 3.3 FAST 2025最佳论文：Mooncake深度解析
- 3.4 ASPLOS 2024论文深度分析

3.5 其他重要工作

3.6 技术演进脉络分析

3.7 技术对比总结

第四章：Prefill-Decode (PD) 分离技术详解

4.1 为什么需要PD分离

4.2 PD分离的基本架构

4.3 Chunked Prefill vs PD Disaggregation

4.4 Mooncake的PD分离实现

4.5 vLLM的Disaggregated Prefilling

4.6 DistServe的设计

4.7 KV Cache传输优化

第五章：KV Cache存储与网络传输优化

5.1 KV Cache的基本原理

5.2 KV Cache的内存占用分析

5.3 PagedAttention机制

5.4 Prefix Caching技术

5.5 KV Cache的压缩和量化

5.6 分层存储架构

5.7 网络传输优化

第六章：Mooncake和AI Serving Stack

6.1 Mooncake项目背景

6.2 Mooncake的整体架构

6.3 核心组件详解

6.4 与vLLM的集成

6.5 与SGLang的集成

6.6 章明星团队介绍

6.7 开源生态与部署案例

第七章：AI Infra核心算法详解

7.1 Attention算法家族

7.2 推理加速算法

7.3 调度算法

7.4 压缩算法

7.5 MoE相关算法

第八章：技术图表集

AI Infra整体架构图

LLM训练和推理流程图

Transformer架构图

Mooncake架构图

PD分离架构对比图

第一章：AI基础设施（AI Infra）基础入门

本章专为即将加入Mooncake团队的实习生准备，旨在帮助你快速建立对AI基础设施的全面认知。无论你是计算机科学背景，还是来自其他专业，都能从本章中找到适合自己的学习路径。

1.1 AI Infra的定义和范畴

1.1.1 什么是AI基础设施？

AI基础设施（AI Infrastructure，简称AI Infra） 是支撑人工智能应用从研发到部署全流程的底层技术体系。如果说AI模型是"大脑"，那么AI Infra就是支撑这个大脑运转的"神经系统"、"血液循环系统"和"骨骼肌肉系统"。

让我们用一个形象的比喻来理解：

想象你要建造一座摩天大楼（AI应用）：- **算法工程师** 是建筑师，设计大楼的结构和外观 - **AI Infra工程师** 是土木工程师和施工队，负责地基、钢结构、水电系统、电梯等基础设施 - 没有稳固的基础设施，再漂亮的设计图也无法变成现实

AI Infra的核心使命是：**让AI模型的训练和部署更快、更便宜、更稳定。**

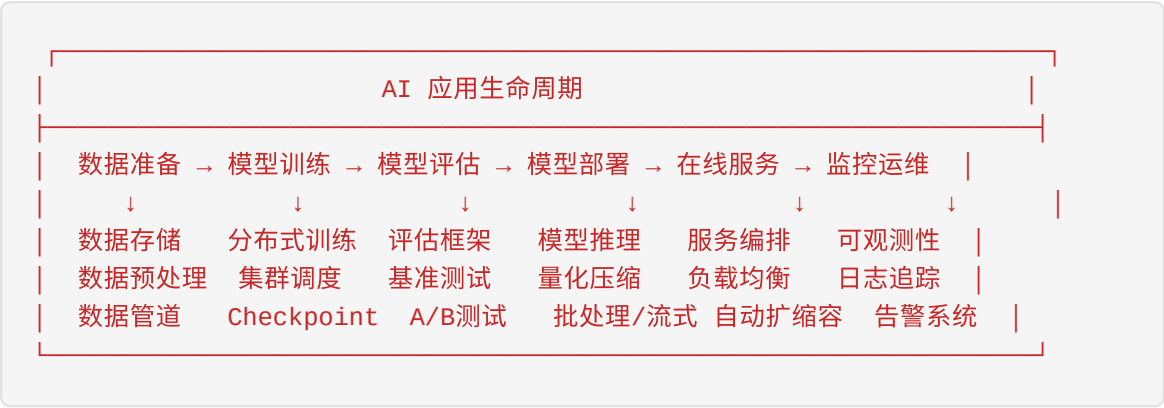
1.1.2 AI Infra的范畴与层次

AI Infra可以从多个维度进行划分：

按技术层次划分

层次	名称	核心组件	功能描述
L1	硬件层	GPU/TPU/NPU、CPU、内存、存储、网络设备	提供计算、存储、通信的物理基础
L2	系统软件层	驱动程序、CUDA、NCCL、容器运行时	硬件资源的管理和抽象
L3	平台层	Kubernetes、Slurm、vLLM、TGI	资源调度、任务编排、推理服务
L4	框架层	PyTorch、TensorFlow、JAX、DeepSpeed	模型开发和训练的工具链
L5	应用层	模型服务API、MLOps平台、监控系统	面向用户的最终服务

按生命周期划分



1.1.3 为什么AI Infra如此重要？

成本角度

训练一个GPT-4级别的大模型，成本可能高达**数千万到上亿美元**。AI Infra的优化可以直接影响：

- **训练时间**：从几个月缩短到几周，节省大量计算资源
- **硬件利用率**：从30%提升到80%，同等算力产出更多模型
- **能源消耗**：数据中心电费可能占运营成本的40%以上

性能角度

指标	优化前	优化后	提升倍数
训练吞吐量	100 samples/s	800 samples/s	8x
推理延迟	200ms	50ms	4x
模型加载时间	30分钟	2分钟	15x
GPU利用率	35%	85%	2.4x

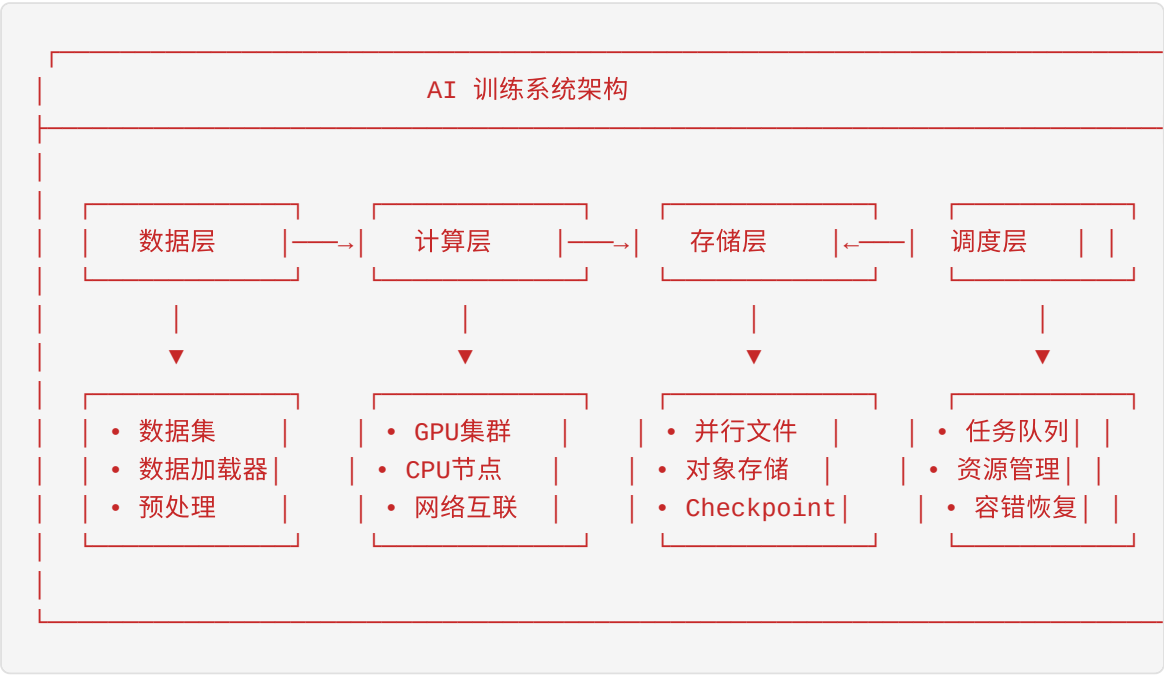
人才角度

AI Infra领域的人才缺口巨大。据统计： - 全球AI Infra工程师缺口超过**100万人** - 顶尖AI Infra工程师的年薪可达**50-100万美元** - Mooncake团队正处于这一领域的最前沿

1.2 AI训练和推理系统的整体架构

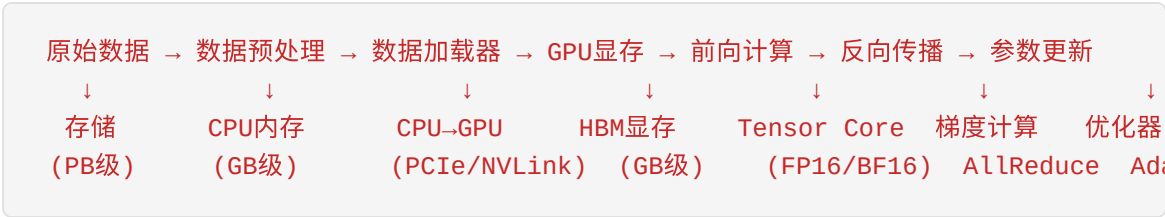
1.2.1 训练系统架构

一个典型的AI训练系统由以下核心组件构成：



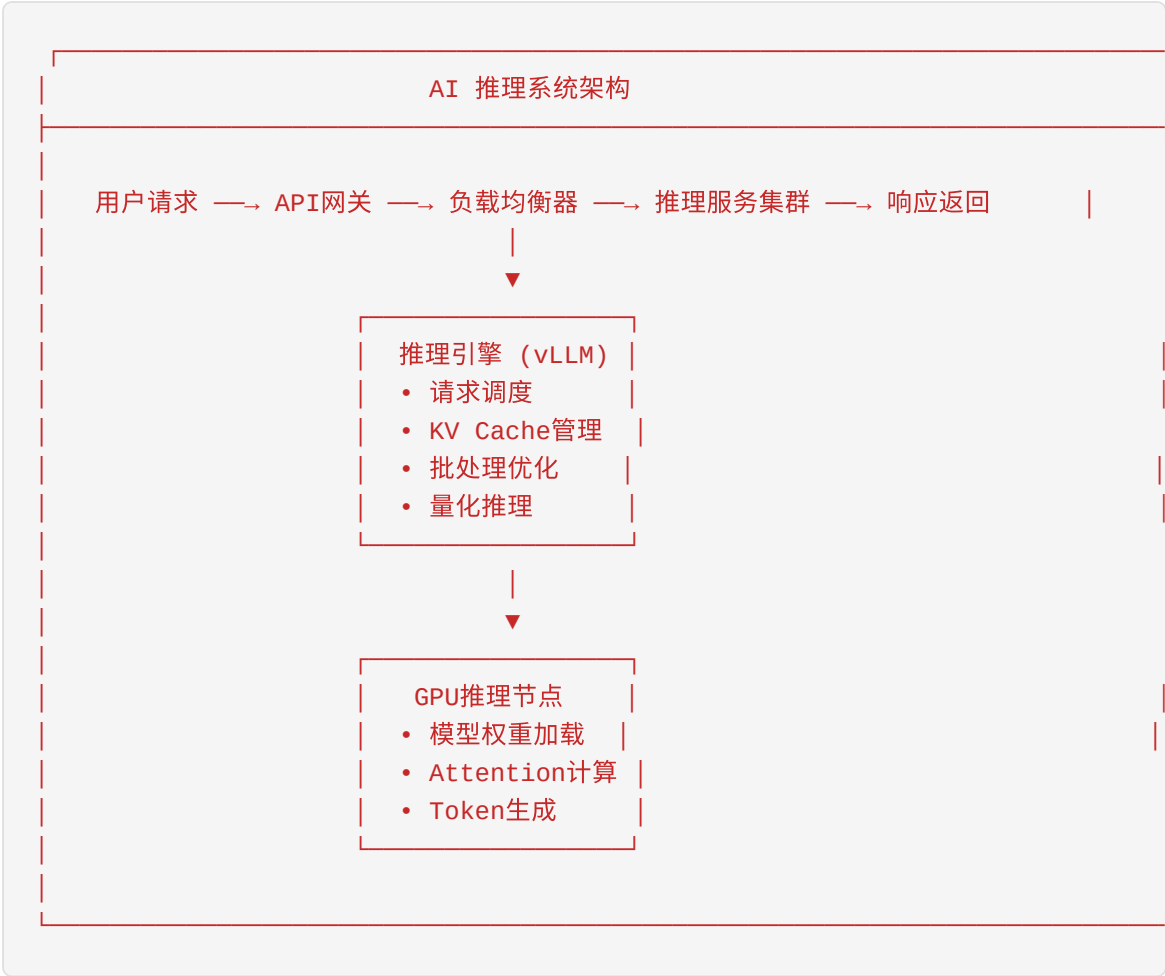
数据流详解

训练过程中的数据流动可以用下图表示：



1.2.2 推理系统架构

推理系统与训练系统有显著不同，更注重**低延迟**和**高吞吐**：

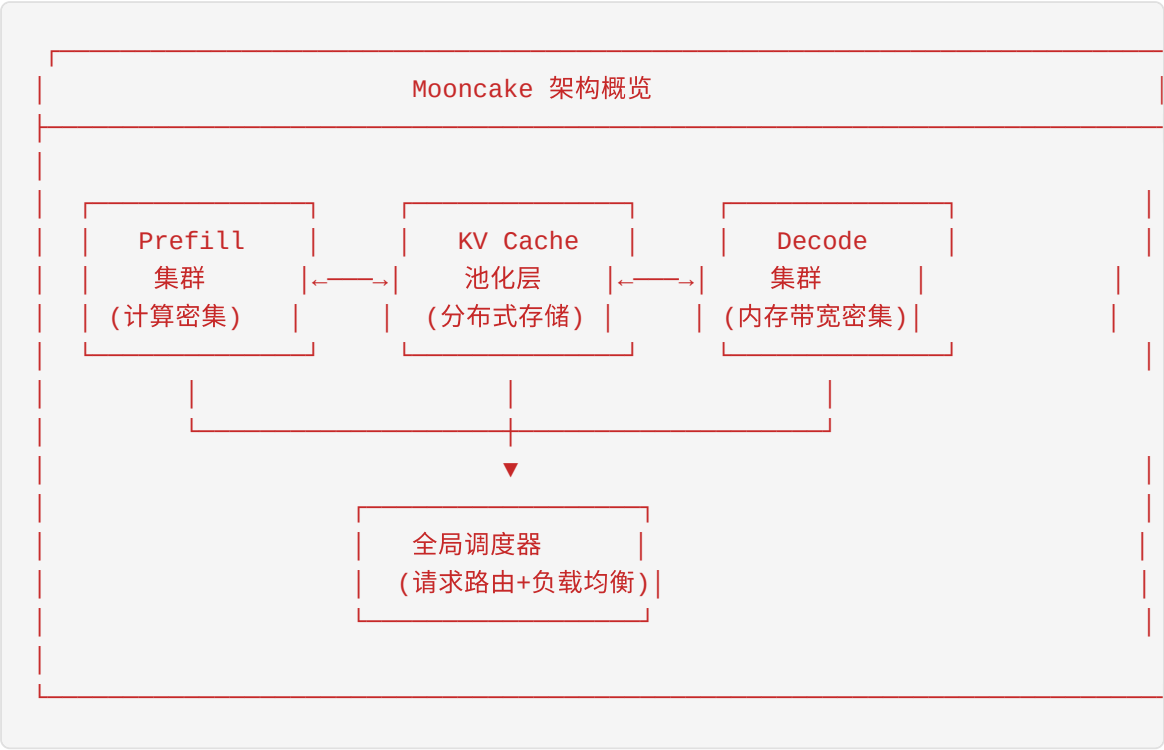


1.2.3 训练与推理的关键差异

维度	训练 (Training)	推理 (Inference)
计算目标	更新模型参数	生成预测结果
精度要求	FP32/FP16/BF16	INT8/INT4/FP16
批处理	固定batch size	动态batching
内存使用	参数+梯度+优化器状态	参数+KV Cache
通信模式	AllReduce频繁	主要是数据传输
容错要求	Checkpoint机制	快速失败恢复
优化重点	吞吐量	延迟+吞吐量

1.2.4 Mooncake架构简介

Mooncake是月之暗面（Moonshot AI）开源的**大模型推理服务平台**，其核心架构特点：



Mooncake的核心创新： 1. **Prefill-Decode分离**：将计算密集和内存带宽密集的任务分离到不同硬件 2. **KV Cache池化**：跨请求的KV Cache共享和复用 3. **全局调度**：基于负载的智能请求路由

1.3 GPU/TPU/NPU等AI加速硬件详解

1.3.1 为什么需要AI专用硬件？

在深入GPU架构之前，让我们先理解一个问题：**为什么不能用普通CPU来训练AI模型？**

CPU vs GPU 计算模式对比

CPU 架构（擅长串行任务）：

控制单元	ALU	缓存	控制单元	ALU	缓存	
(复杂)	(少)	(大)	(复杂)	(少)	(大)	
≈ 8-64 核心，每个核心强大但数量少						

GPU 架构（擅长并行任务）：

控制单元	ALU ALU ALU ... ALU	控制单元	ALU ALU ... ALU
(简单)	(大量简单计算单元)	(简单)	(大量简单计算单元)
≈ 数千个核心，每个核心简单但数量庞大			

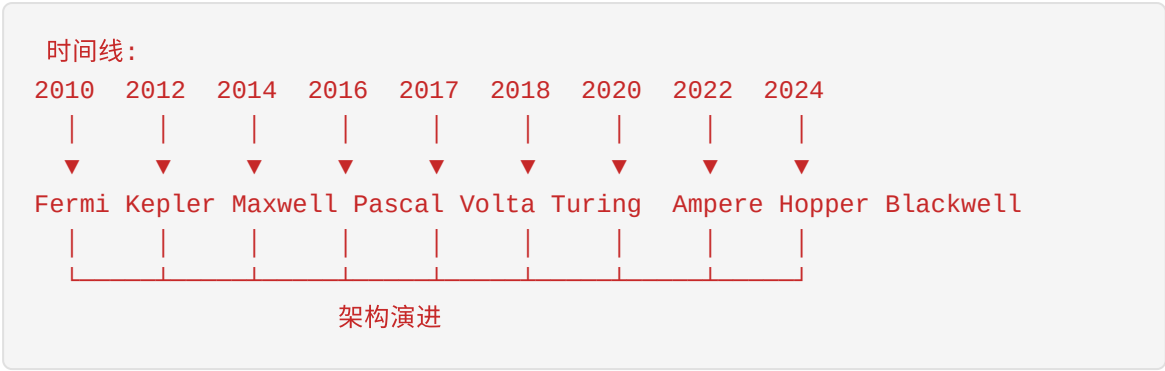
具体性能对比

操作类型	CPU (Intel Xeon)	GPU (NVIDIA A100)	加速比
矩阵乘法 (FP32)	3 TFLOPS	19.5 TFLOPS	6.5x
矩阵乘法 (FP16)	6 TFLOPS	312 TFLOPS	52x
内存带宽	200 GB/s	2,039 GB/s	10x
并行线程数	64	6,912	108x

核心洞察：AI计算的核心是大量的矩阵乘法和向量运算，这些操作天然适合并行化。GPU的设计理念就是“用数量换效率”。

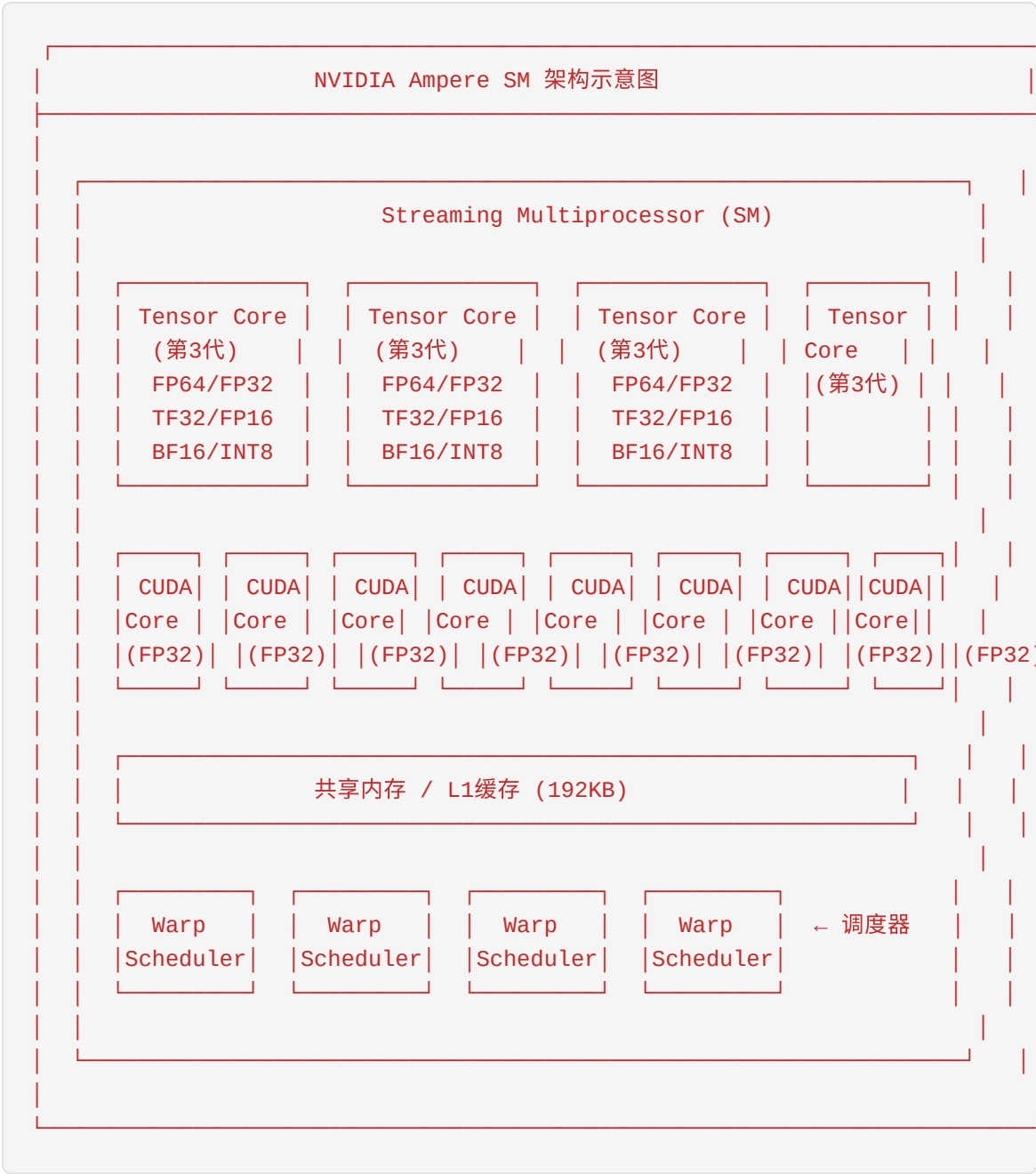
1.3.2 GPU架构深度解析

NVIDIA GPU架构演进史



SM（Streaming Multiprocessor）架构

SM是GPU的基本计算单元，类似于CPU的核心，但设计完全不同。



Tensor Core详解

Tensor Core是NVIDIA GPU的"秘密武器", 专门用于加速矩阵运算。

工作原理:

传统CUDA Core计算矩阵乘法：

$C = A \times B$

需要： $64 \times 64 \times 64 = 262,144$ 次乘加运算

每个CUDA Core每次做1个乘加 → 需要262,144个时钟周期

Tensor Core计算矩阵乘法：

使用4×4×4的矩阵乘加单元

一次操作完成： $D = A \times B + C$

只需要： $(64/4)^3 = 4,096$ 次操作

加速比： 64x

Tensor Core演进：

代数	架构	支持精度	核心特性
1st	Volta (V100)	FP16	首次引入，4×4×4矩阵运算
2nd	Turing (RTX)	FP16, INT8, INT4	增加整数精度支持
3rd	Ampere (A100)	FP64, FP32, TF32, FP16, BF16, INT8	TF32自动混合精度
4th	Hopper (H100)	同上 + FP8	Transformer Engine
5th	Blackwell	同上 + 更高性能	AI性能翻倍

HBM（高带宽内存）

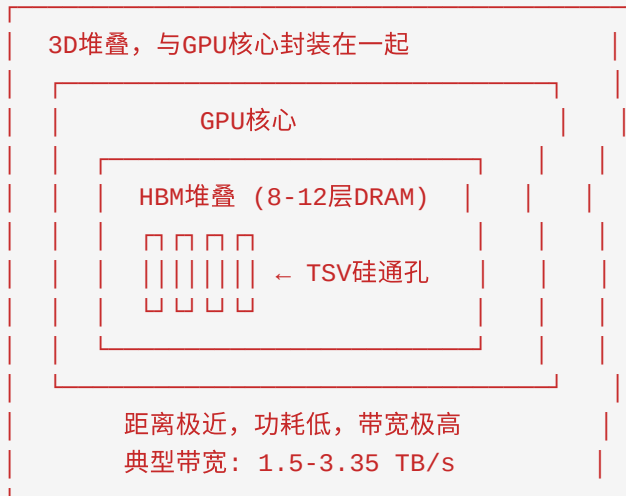
HBM是GPU的"生命线"，决定了数据能否及时喂给计算单元。

传统GDDR vs HBM对比：

GDDR6（游戏显卡常用）：



HBM2e/HBM3（数据中心GPU）：



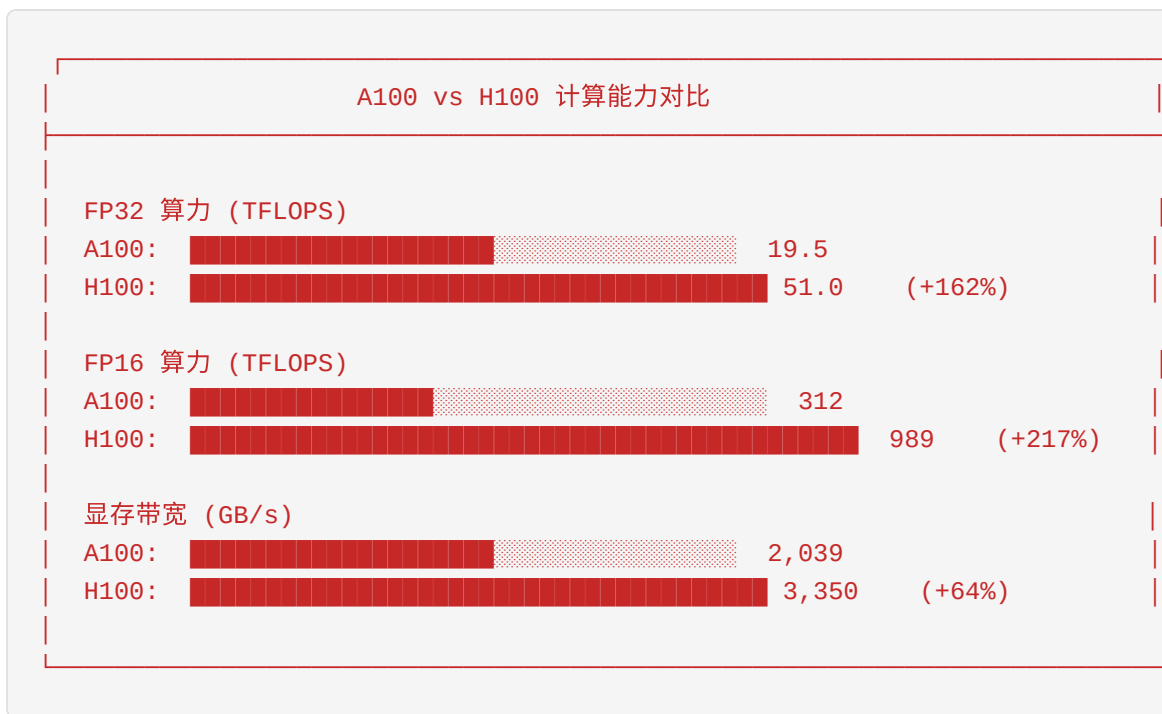
1.3.3 主流GPU型号详细对比

NVIDIA数据中心GPU家族

型号	架构	显存	显存带宽	FP32算力	FP16算力	TDP	发布时间
V100	Volta	32GB HBM2	900 GB/s	15.7 TFLOPS	125 TFLOPS	300W	2017
A100	Ampere	80GB HBM2e	2,039 GB/s	19.5 TFLOPS	312 TFLOPS	400W	2020
A800	Ampere	80GB HBM2e	2,039 GB/s	19.5 TFLOPS	312 TFLOPS	400W	2022
H100	Hopper	80GB HBM3	3.35 TB/s	51 TFLOPS	989 TFLOPS	700W	2022
H800	Hopper	80GB HBM3	3.35 TB/s	51 TFLOPS	989 TFLOPS	700W	2023
H200	Hopper	141GB HBM3e	4.8 TB/s	51 TFLOPS	989 TFLOPS	700W	2024
B200	Blackwell	192GB HBM3e	8 TB/s	~100 TFLOPS	~2000 TFLOPS	1000W	2024

A100 vs H100 深度对比

计算能力对比：



H100的关键创新：

1. **Transformer Engine：**
2. 自动在FP16和FP8之间切换
3. 大模型训练速度提升 **4-6倍**
4. 动态缩放保持精度
5. **第四代NVLink：**
6. 单链路带宽: 25 GB/s → 50 GB/s
7. 支持最多18条链路
8. GPU间总带宽: 900 GB/s
9. **DPX指令：**
10. 加速动态规划算法
11. 基因组学、路径规划等应用

A800/H800 特别说明

为什么会有A800/H800？

2022年，美国出口管制政策限制了高性能GPU对中国的出口。NVIDIA推出了“阉割版”的A800和H800：

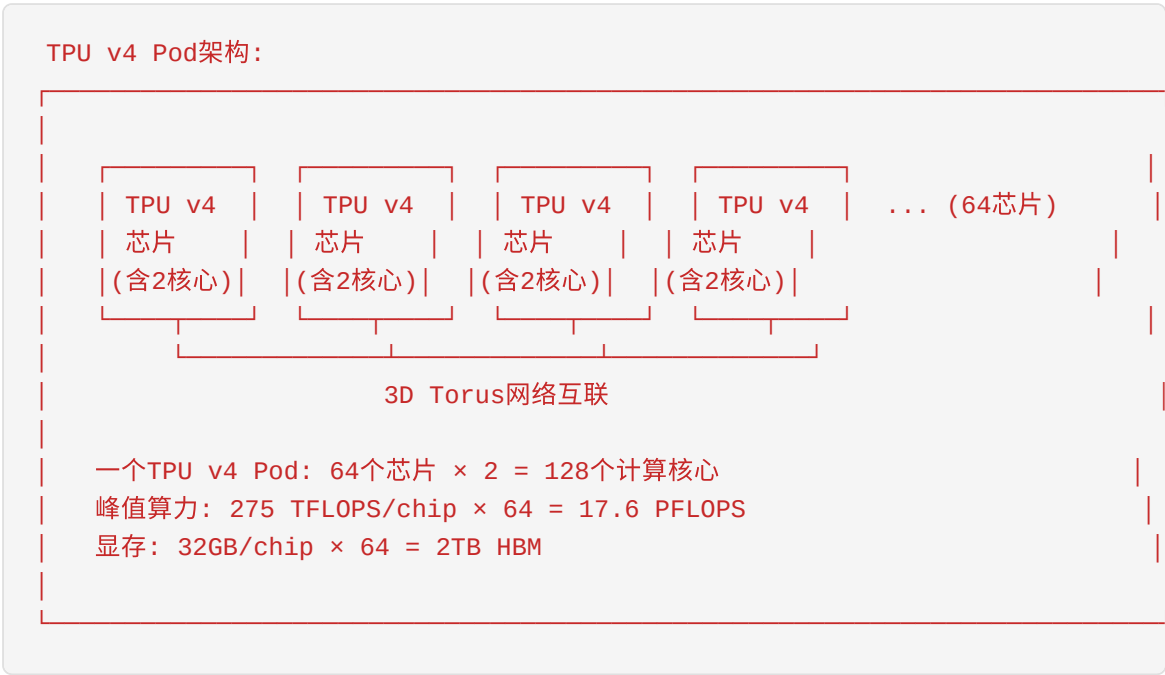
限制项	A100/H100	A800/H800
计算性能	完整	相同
显存带宽	完整	相同
NVLink带宽	600 GB/s	400 GB/s (-33%)
互联带宽	完整	降低

对训练的影响： - 小规模训练（单节点8卡）：影响较小 - 大规模训练（多节点）：通信成为瓶颈，训练效率下降 **10-30%**

1.3.4 TPU和NPU简介

Google TPU

TPU（Tensor Processing Unit） 是Google自研的AI专用芯片。



TPU vs GPU对比：

特性	TPU	GPU
制造商	Google	NVIDIA
软件生态	JAX/TensorFlow	PyTorch/TensorFlow
可用性	仅Google Cloud	广泛
灵活性	专用架构	通用架构
性价比	训练大模型较好	通用场景更好

国产NPU

华为昇腾（Ascend）： - Ascend 910B: 320 TFLOPS FP16, 32GB HBM - 支持CANN框架，兼容PyTorch

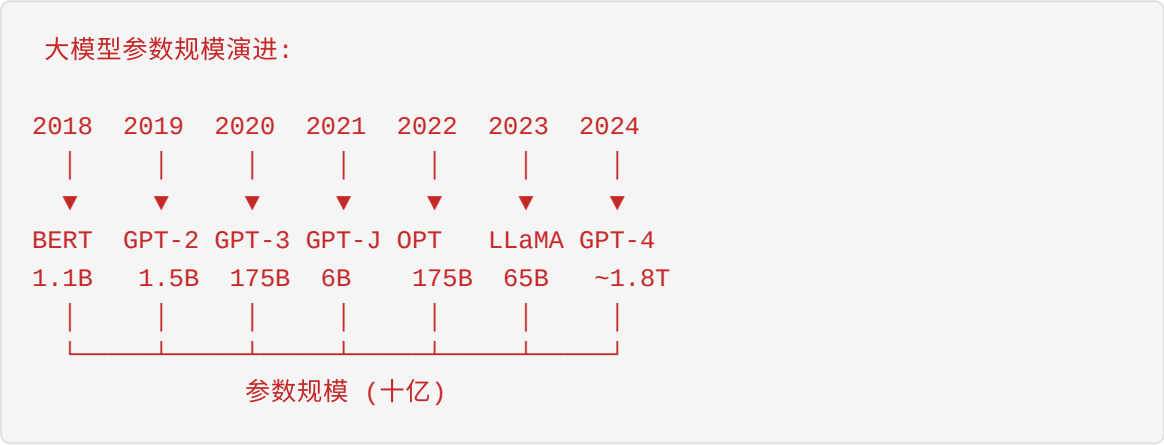
寒武纪（Cambricon）： - MLU370: 256 TFLOPS FP16 - 支持MagicMind推理框架

海光DCU： - 兼容CUDA生态 - 适用于国产化替代场景

1.4 分布式训练基础

1.4.1 为什么需要分布式训练？

模型规模爆炸



显存需求计算：

训练一个175B参数的GPT-3模型，需要多少显存？

参数存储:

- 模型参数: $175\text{B} \times 2\text{字节 (FP16)} = 350\text{ GB}$
- 梯度: $175\text{B} \times 2\text{字节} = 350\text{ GB}$
- 优化器状态 (Adam): $175\text{B} \times 2 \times 4\text{字节} = 1,400\text{ GB}$
- 激活值: 约500 GB (取决于batch size和序列长度)

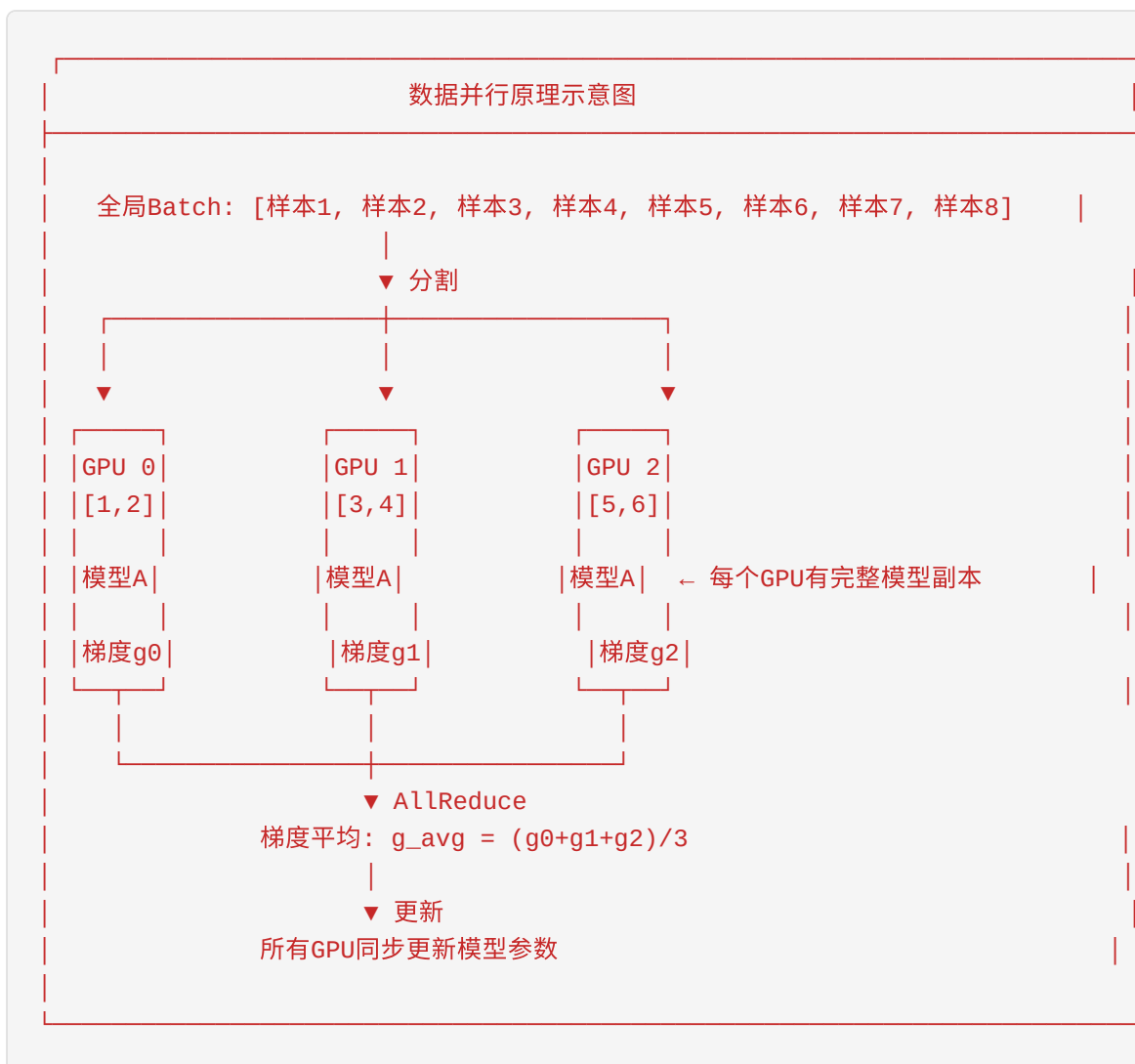
总计: 约 2.6 TB 显存

单张A100 (80GB) 可以存储: $80\text{GB} / 2.6\text{TB} \approx 3\%$ 的模型

需要: $2.6\text{TB} / 80\text{GB} \approx 33\text{张 A100}$ (至少需要4个节点)

1.4.2 数据并行 (Data Parallelism, DP)

核心思想: 每个GPU保存完整的模型副本, 处理不同的数据批次。



详细流程:

```
# 伪代码示意
def data_parallel_training():
    # 1. 每个GPU加载相同的模型副本
    model = load_model().to(local_gpu)

    # 2. 数据分发 (Scatter)
    local_batch = global_batch[rank::world_size]

    # 3. 前向传播
    loss = model(local_batch)

    # 4. 反向传播
    loss.backward() # 计算本地梯度

    # 5. 梯度同步 (AllReduce)
    all_reduce(gradients, op=AVERAGE)

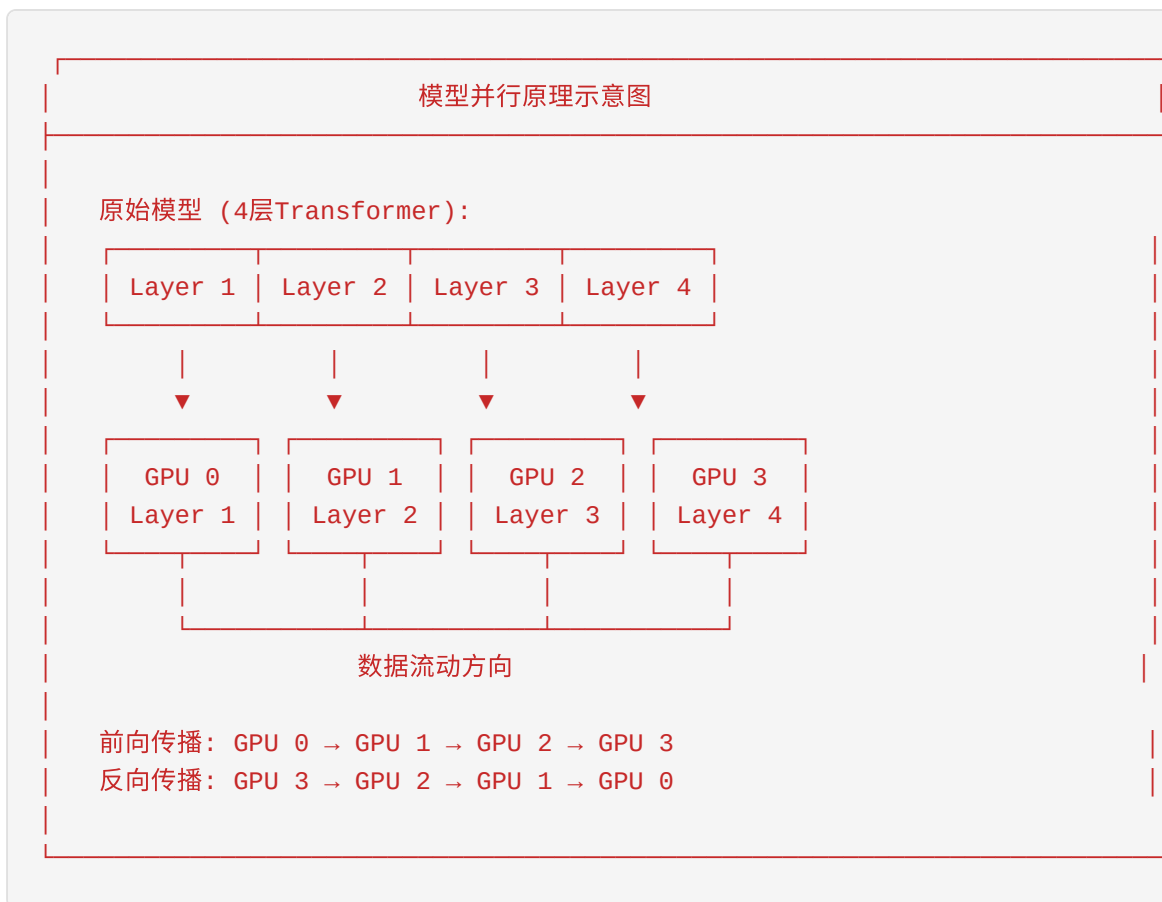
    # 6. 参数更新
    optimizer.step() # 所有GPU更新相同的参数
```

优点： - 实现简单，易于理解 - 加速比接近线性（理想情况下，4卡 = 4倍速度）

缺点： - 每个GPU需要存储完整的模型 - 模型大小受限于单卡显存 - 通信开销随GPU数量增加

1.4.3 模型并行 (Model Parallelism, MP)

核心思想： 将模型切分到不同GPU，每个GPU只存储部分参数。



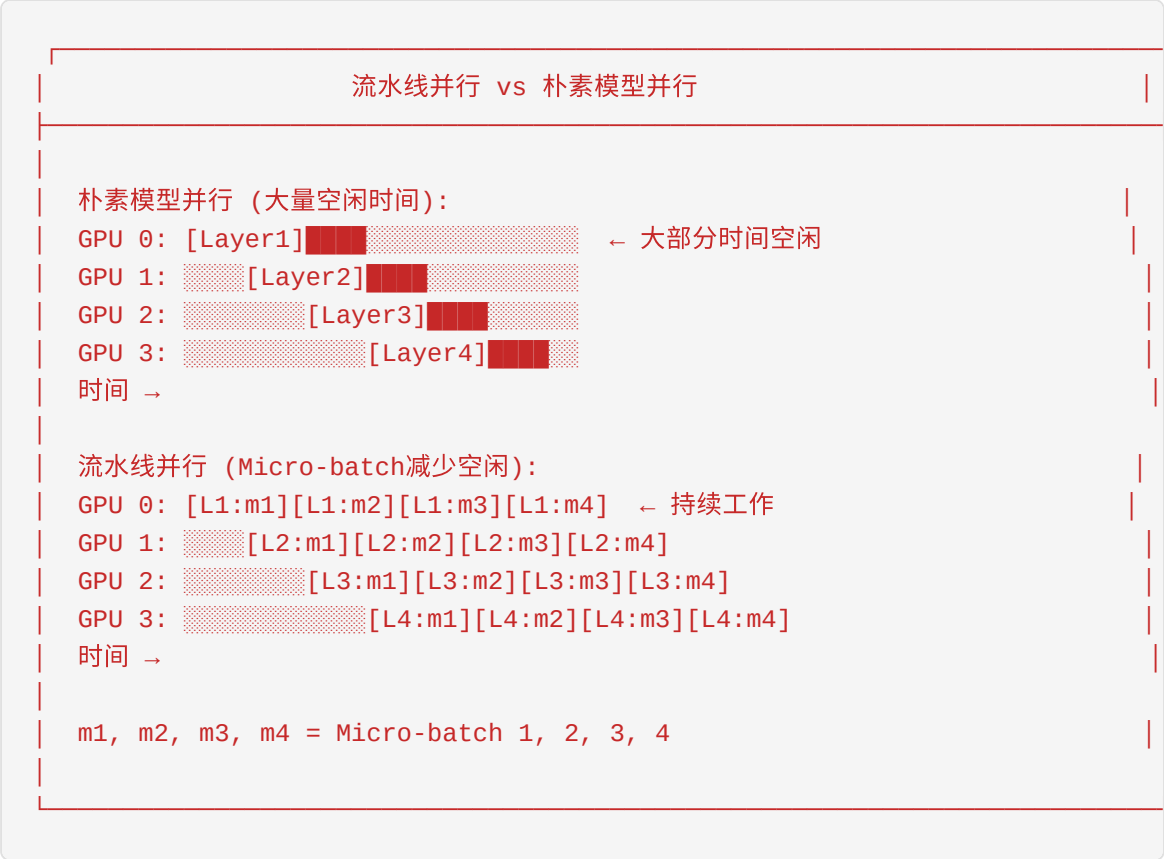
详细流程:

```
# 伪代码示意
def model_parallel_forward(x, rank):
    if rank == 0:
        # GPU 0: 第一层
        x = layer1(x)
        send(x, dest=1) # 发送给GPU 1
        return None
    elif rank == 1:
        x = recv(src=0) # 从GPU 0接收
        x = layer2(x)
        send(x, dest=2)
        return None
    elif rank == 2:
        x = recv(src=1)
        x = layer3(x)
        send(x, dest=3)
        return None
    elif rank == 3:
        x = recv(src=2)
        x = layer4(x)
        return x # 最终结果
```

- 优点：** - 可以训练超大模型（不受单卡显存限制） - 适合模型层数多、参数量大的场景
- 缺点：** - GPU利用率低（流水线气泡） - 通信频繁，延迟敏感 - 实现复杂

1.4.4 流水线并行（Pipeline Parallelism, PP）

核心思想： 结合数据并行和模型并行，将数据分批处理以减少空闲时间。



GPipe vs PipeDream：

特性	GPipe	PipeDream
更新方式	同步（等待所有micro-batch）	异步（立即更新）
内存开销	高（存储多个激活值）	低
实现复杂度	简单	复杂
收敛稳定性	好	需要额外处理

1.4.5 张量并行（Tensor Parallelism, TP）

核心思想： 将单层内的计算拆分到多个GPU。

张量并行原理

矩阵乘法拆分 (以Linear层为例):

$Y = X \times W$ (X: [batch, in_dim], W: [in_dim, out_dim])

按列拆分 (Column Parallel):

$W = [W1 \mid W2]$ 拆分到2个GPU

GPU 0: $Y1 = X \times W1$

GPU 1: $Y2 = X \times W2$

结果: $Y = [Y1 \mid Y2]$ (Concatenate)

按行拆分 (Row Parallel):

$W = \begin{bmatrix} W1 \\ W2 \end{bmatrix}$ 拆分到2个GPU

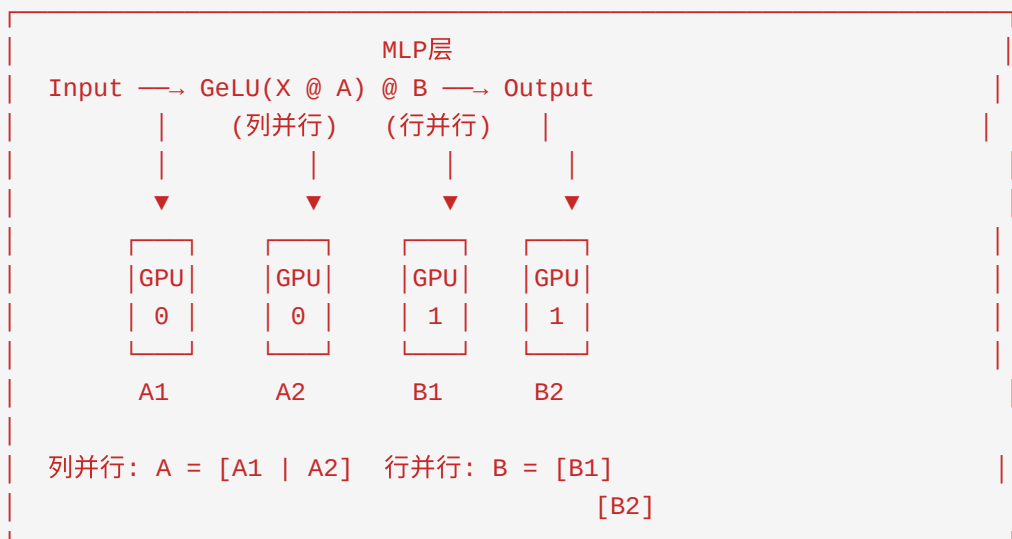
GPU 0: $Y1 = X1 \times W1$

GPU 1: $Y2 = X2 \times W2$

结果: $Y = Y1 + Y2$ (AllReduce)

Megatron-LM的张量并行策略:

Transformer层的张量并行:



1.4.6 3D并行组合

核心思想：将数据并行(DP)、流水线并行(PP)、张量并行(TP)组合使用。



并行策略选择指南：

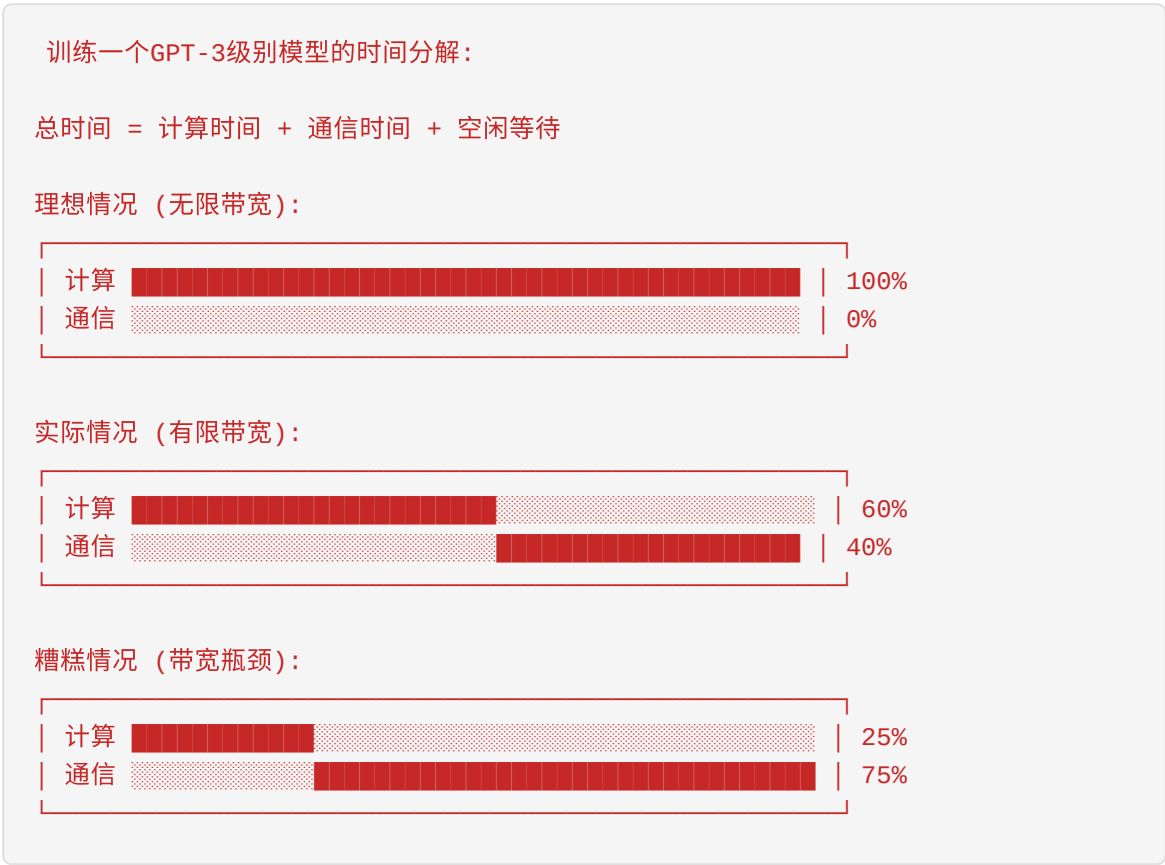
场景	推荐配置	说明
小模型 (<10B)	DP only	简单高效
中等模型 (10-100B)	DP + TP	单节点内TP，跨节点DP
大模型 (100B-1T)	DP + PP + TP	3D并行
超大模型 (>1T)	ZeRO + 3D	结合显存优化

1.5 集群网络和通信

1.5.1 为什么网络如此重要？

在分布式训练中，网络是连接各个GPU的“高速公路”。如果高速公路太窄，即使GPU再快，也会被数据传输拖慢。

网络带宽对训练的影响

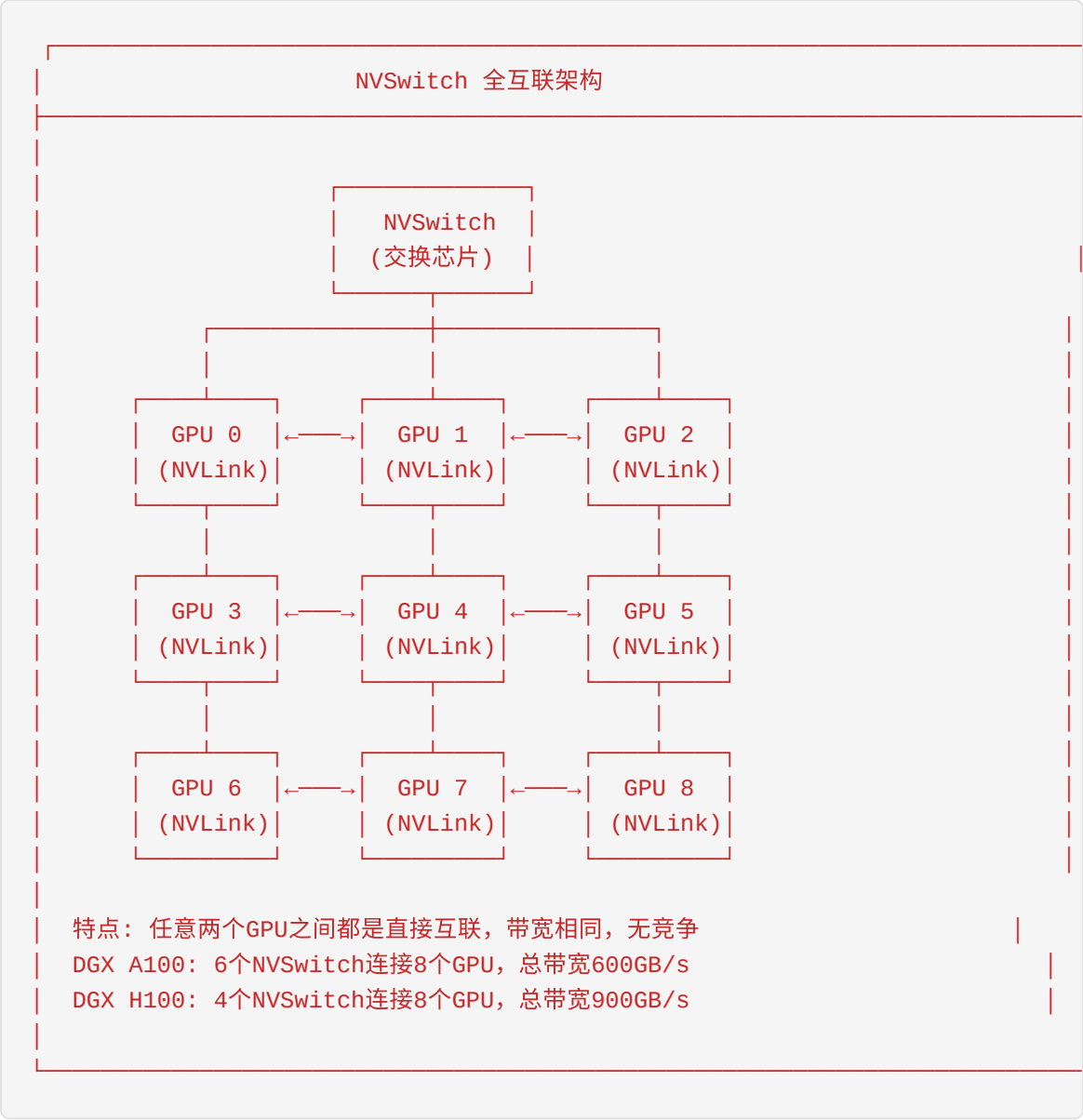


1.5.2 GPU互联技术对比

单机内部互联

技术	带宽	延迟	用途	特点
PCIe 4.0 x16	32 GB/s	~1μs	CPU-GPU, GPU-存储	通用，成本较低
PCIe 5.0 x16	64 GB/s	~1μs	新一代服务器	带宽翻倍
NVLink 3.0	50 GB/s/链路	~0.5μs	GPU-GPU	专用高速互联
NVLink 4.0	50 GB/s/链路	~0.5μs	H100 GPU互联	支持最多18链路
NVSwitch	900 GB/s (全互联)	~0.5μs	多GPU全互联	无阻塞交换

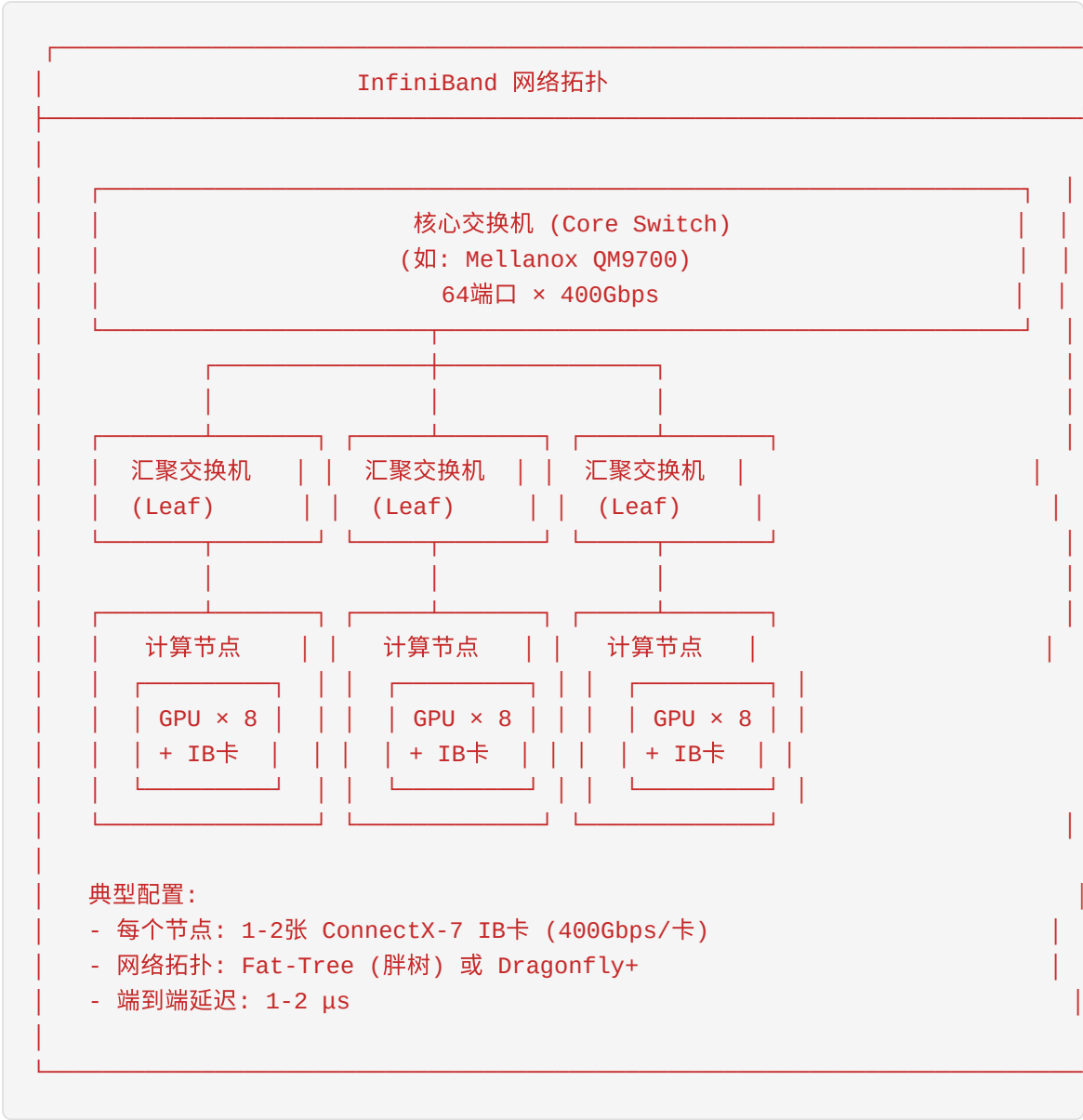
NVSwitch架构详解



1.5.3 跨节点网络技术

InfiniBand (IB)

InfiniBand是高性能计算(HPC)领域的王者，也是AI训练集群的首选网络。

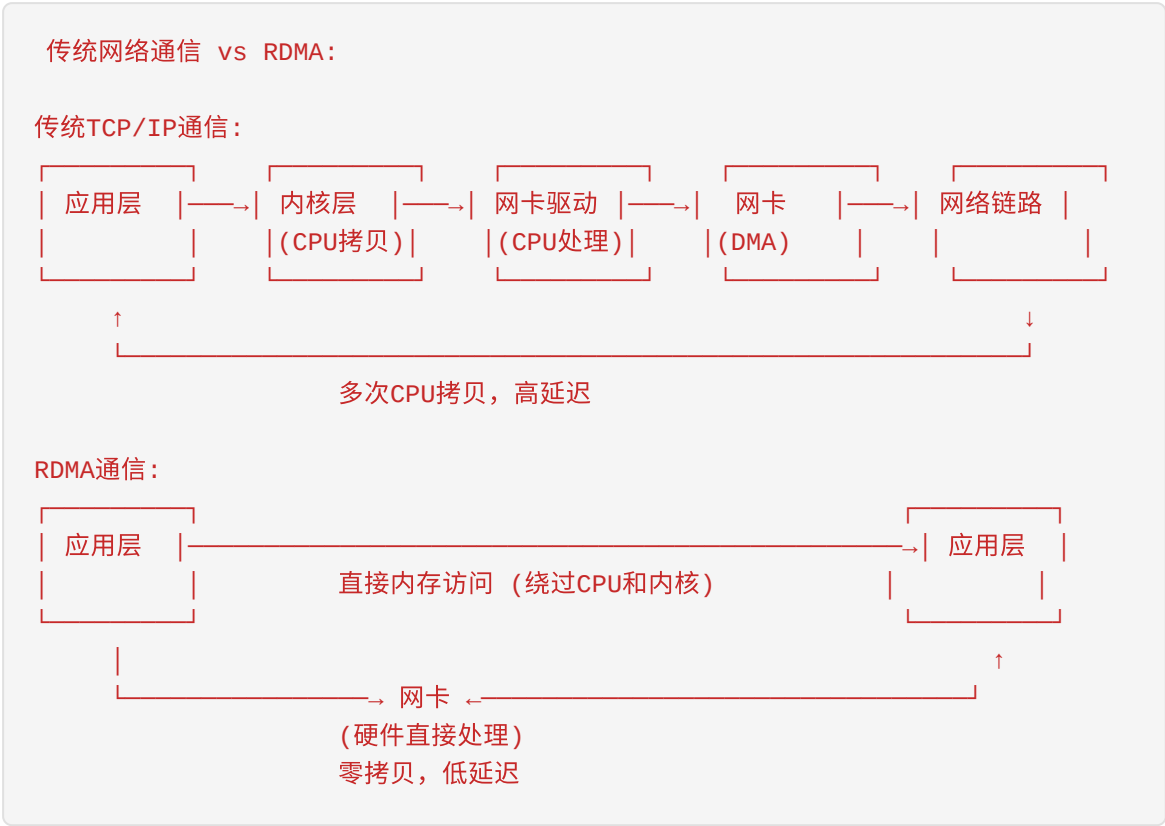


InfiniBand演进：

版本	单链路带宽	常见配置	总带宽	发布时间
IB FDR	14 Gbps	4x	56 Gbps	2011
IB EDR	25 Gbps	4x	100 Gbps	2014
IB HDR	50 Gbps	4x	200 Gbps	2018
IB NDR	100 Gbps	4x	400 Gbps	2022
IB XDR	200 Gbps	4x	800 Gbps	2024

RDMA (Remote Direct Memory Access)

RDMA是InfiniBand的核心技术，允许一台机器直接访问另一台机器的内存，无需CPU介入。



RDMA优势： - 零拷贝：数据直接从应用内存到网卡，无需内核缓冲 - 内核旁路：应用直接操作网卡，无需系统调用 - CPU卸载：通信由网卡处理，CPU专注于计算

RoCE (RDMA over Converged Ethernet)

RoCE是在以太网上实现RDMA的技术，成本比InfiniBand更低。

特性	InfiniBand	RoCEv2
网络类型	专用网络	标准以太网
需要特殊交换机	是	否 (需支持PFC/ECN)
性能	最优	接近IB
成本	高	中等
兼容性	专用生态	与以太网兼容

1.5.4 NCCL集合通信库

NCCL (NVIDIA Collective Communications Library) 是NVIDIA提供的多GPU通信库，是分布式训练的基石。

NCCL核心操作

NCCL 集合通信操作

1. AllReduce (最常用的操作)

GPU 0: [1] ┌
GPU 1: [2] ┤ ─→ AllReduce(SUM) ─→ GPU 0-3: [10] (1+2+3+4)
GPU 2: [3] ┤
GPU 3: [4] └

2. AllGather

GPU 0: [A] ┌
GPU 1: [B] ┤ ─→ AllGather ─→ GPU 0-3: [A|B|C|D]
GPU 2: [C] ┤
GPU 3: [D] └

3. ReduceScatter

GPU 0-3: [A|B|C|D] ─→ ReduceScatter(SUM)
─→ GPU 0: [A0+B0+C0+D0]
─→ GPU 1: [A1+B1+C1+D1]
─→ GPU 2: [A2+B2+C2+D2]
─→ GPU 3: [A3+B3+C3+D3]

4. Broadcast

GPU 0: [DATA] ─→ Broadcast ─→ GPU 0-3: [DATA] (全部复制)
GPU 1-3: []

5. Reduce

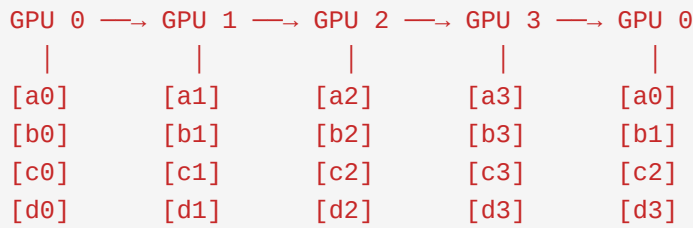
GPU 0: [1] ┌
GPU 1: [2] ┤ ─→ Reduce(SUM) ─→ GPU 0: [10] (只保留在root)
GPU 2: [3] ┤
GPU 3: [4] └

NCCL AllReduce算法

Ring AllReduce (环形算法):

Ring AllReduce 过程示意 (4个GPU):

阶段1: Scatter-Reduce (每个GPU获得部分结果)



Step 1: GPU 0发送a0给GPU 1, GPU 1计算a0+a1

Step 2: GPU 1发送a0+a1给GPU 2, GPU 2计算a0+a1+a2

Step 3: GPU 2发送a0+a1+a2给GPU 3, GPU 3计算a0+a1+a2+a3 (完整和)

...

阶段2: AllGather (将完整结果广播给所有GPU)

每个GPU将自己的完整和部分结果传递给下一个GPU

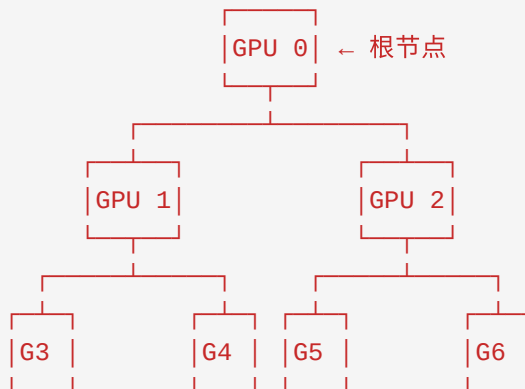
最终所有GPU都有完整的求和结果

时间复杂度: $2 \times (N-1)/N \times \text{数据量}/\text{带宽}$ (N为GPU数量)

Tree AllReduce (树形算法):

Tree AllReduce 过程示意 (8个GPU):

Reduce阶段 (从叶子到根):



数据逐级汇总, 根节点获得完整结果

Broadcast阶段 (从根到叶子):

结果从根节点逐级广播给所有GPU

优点: 延迟更低, 适合小规模数据

缺点: 带宽利用率不如Ring算法

NCCL性能调优

```
# NCCL环境变量调优示例
import os

# 1. 选择AllReduce算法
# TREE: 树形算法, 延迟低
# RING: 环形算法, 带宽利用率高
# COLLNET: 使用SHARP交换机卸载
os.environ['NCCL_ALGO'] = 'RING'

# 2. 设置缓冲区大小
os.environ['NCCL_BUFFSIZE'] = '2097152' # 2MB

# 3. 启用网络热插拔检测
os.environ['NCCL_NET_PLUGIN'] = 'none'

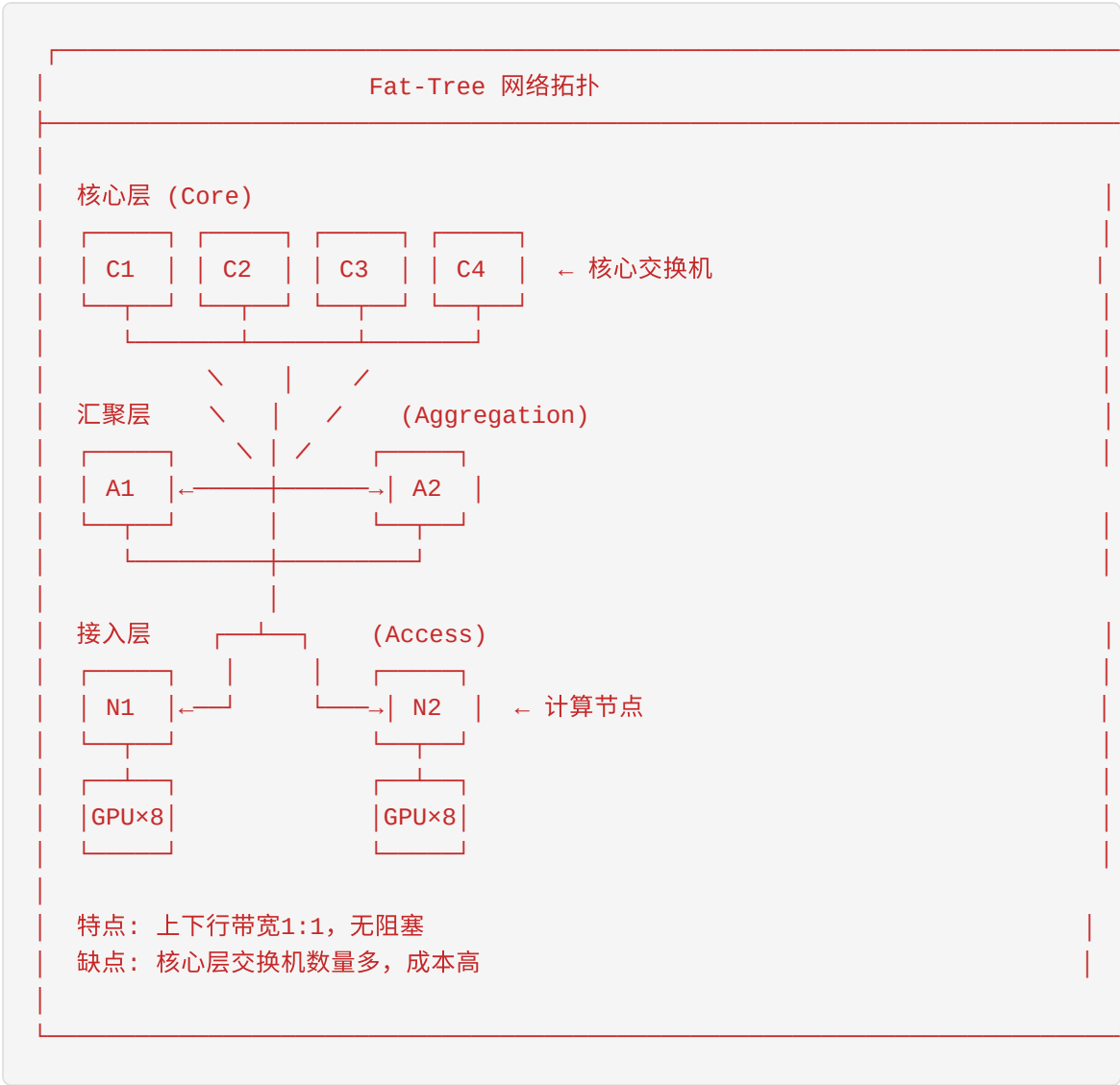
# 4. 调试模式
os.environ['NCCL_DEBUG'] = 'INFO' # 或 'WARN', 'ERROR'
os.environ['NCCL_DEBUG_SUBSYS'] = 'ALL'

# 5. 设置IB设备
os.environ['NCCL_IB_HCA'] = 'mlx5_0,mlx5_1' # 指定IB网卡

# 6. 禁用某些特性
os.environ['NCCL_P2P_DISABLE'] = '0' # 启用P2P
os.environ['NCCL_IB_DISABLE'] = '0' # 启用IB
```

1.5.5 网络拓扑设计

Fat-Tree（胖树）拓扑



3D Torus (3D环面) 拓扑

3D Torus 网络拓扑

3D Torus：每个节点在x, y, z三个方向都与邻居相连，形成环形

z方向视图：

0, 0	0, 1	0, 2	← 第0层
1, 0	1, 1	1, 2	← 第1层
2, 0	2, 1	2, 2	← 第2层

↑
(循环连接)

↑
(循环连接)

↑
(循环连接)

每个节点连接6个邻居 (+x, -x, +y, -y, +z, -z)

优点：

- 交换机数量少，成本低
- 可扩展性好
- 适合AllReduce等集合通信

缺点：

- 非最短路径可能拥塞
- 需要精心设计的通信模式

代表：Google TPU Pod, NVIDIA DGX SuperPOD

1.6 存储系统在AI中的作用

1.6.1 AI工作负载的存储需求

AI工作负载对存储系统有独特的需求：

AI存储需求金字塔：

热数据 (NVMe)	← Checkpoint（频繁读写） 容量：TB级 带宽：10+ GB/s
温数据 (SSD)	← 训练数据集 容量：10-100TB 带宽：1-5 GB/s
冷数据 (HDD)	← 原始数据、归档 容量：PB级 带宽：100-500 MB/s

1.6.2 训练数据存储

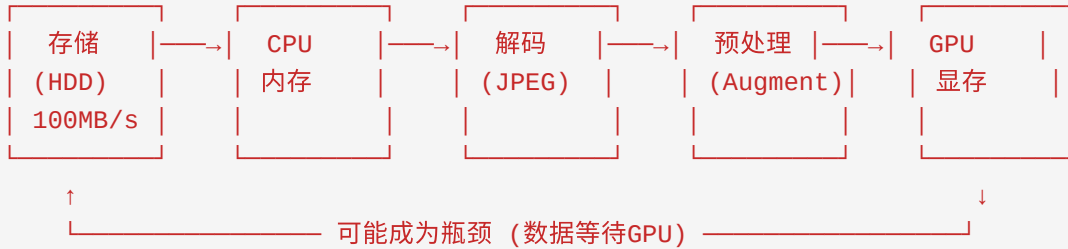
数据集规模对比

数据集	类型	大小	用途
ImageNet	图像	150 GB	图像分类基准
LAION-5B	图文对	240 TB	多模态预训练
The Pile	文本	800 GB	语言模型预训练
C4	文本	750 GB	T5训练
Common Crawl	网页	数百PB	大规模预训练

数据加载瓶颈

数据加载性能问题：

慢速路径：



优化后路径：



数据加载优化策略

```
# PyTorch DataLoader优化示例
from torch.utils.data import DataLoader
from torch.utils.data.distributed import DistributedSampler

# 1. 使用多进程数据加载
dataloader = DataLoader(
    dataset,
    batch_size=32,
    num_workers=8,          # 多进程加载
    pin_memory=True,        # 页锁定内存，加速CPU→GPU传输
    prefetch_factor=2,      # 每个worker预加载2个batch
    persistent_workers=True, # 保持worker进程，避免重复创建
)

# 2. 分布式采样器
sampler = DistributedSampler(
    dataset,
    num_replicas=world_size,
    rank=rank,
    shuffle=True,
)

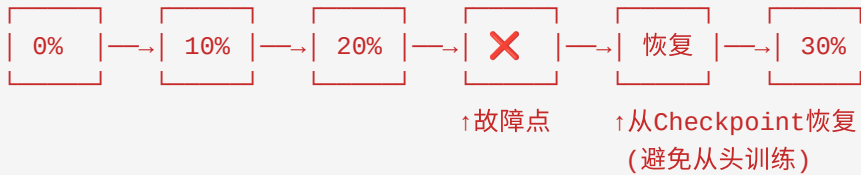
# 3. 使用WebDataset (适合大规模数据集)
# WebDataset将数据打包为tar格式，顺序读取效率高
from webdataset import WebDataset
dataset = WebDataset("path/to/data-{000..999}.tar")
```

1.6.3 Checkpoint存储

Checkpoint的重要性

Checkpoint = 模型的"存档点"

训练过程中：



Checkpoint内容

```
# 一个完整的Checkpoint包含：
checkpoint = {
    # 1. 模型参数
    'model_state_dict': model.state_dict(),

    # 2. 优化器状态（特别是Adam的动量）
    'optimizer_state_dict': optimizer.state_dict(),

    # 3. 学习率调度器
    'scheduler_state_dict': scheduler.state_dict(),

    # 4. 训练状态
    'epoch': current_epoch,
    'global_step': global_step,
    'best_loss': best_loss,

    # 5. 随机数种子（保证可复现）
    'torch_rng_state': torch.get_rng_state(),
    'cuda_rng_state': torch.cuda.get_rng_state_all(),

    # 6. 其他元数据
    'config': model_config,
    'training_args': args,
}

# 大模型Checkpoint大小估算：
# GPT-3（175B参数）：
# - FP16模型：175B × 2字节 = 350 GB
# - 优化器状态：175B × 2 × 4字节 = 1.4 TB（Adam需要2份动量）
# - 总计：~1.75 TB per checkpoint
```

Checkpoint策略

策略	频率	保留数量	适用场景
定期保存	每N步	最近K个	常规训练
最佳模型	验证提升时	1-3个	需要最优模型
故障恢复	每N分钟	1个	长时训练
分层保存	不同频率	多级	重要实验

```
# Checkpoint保存优化
import torch.distributed as dist

# 1. 异步保存（不阻塞训练）
def async_save_checkpoint(state, path):
    # 使用后台线程保存
    import threading
    thread = threading.Thread(target=torch.save, args=(state, path))
    thread.start()
    return thread

# 2. 仅rank 0保存（减少IO竞争）
if dist.get_rank() == 0:
    torch.save(checkpoint, save_path)
dist.barrier() # 等待rank 0完成

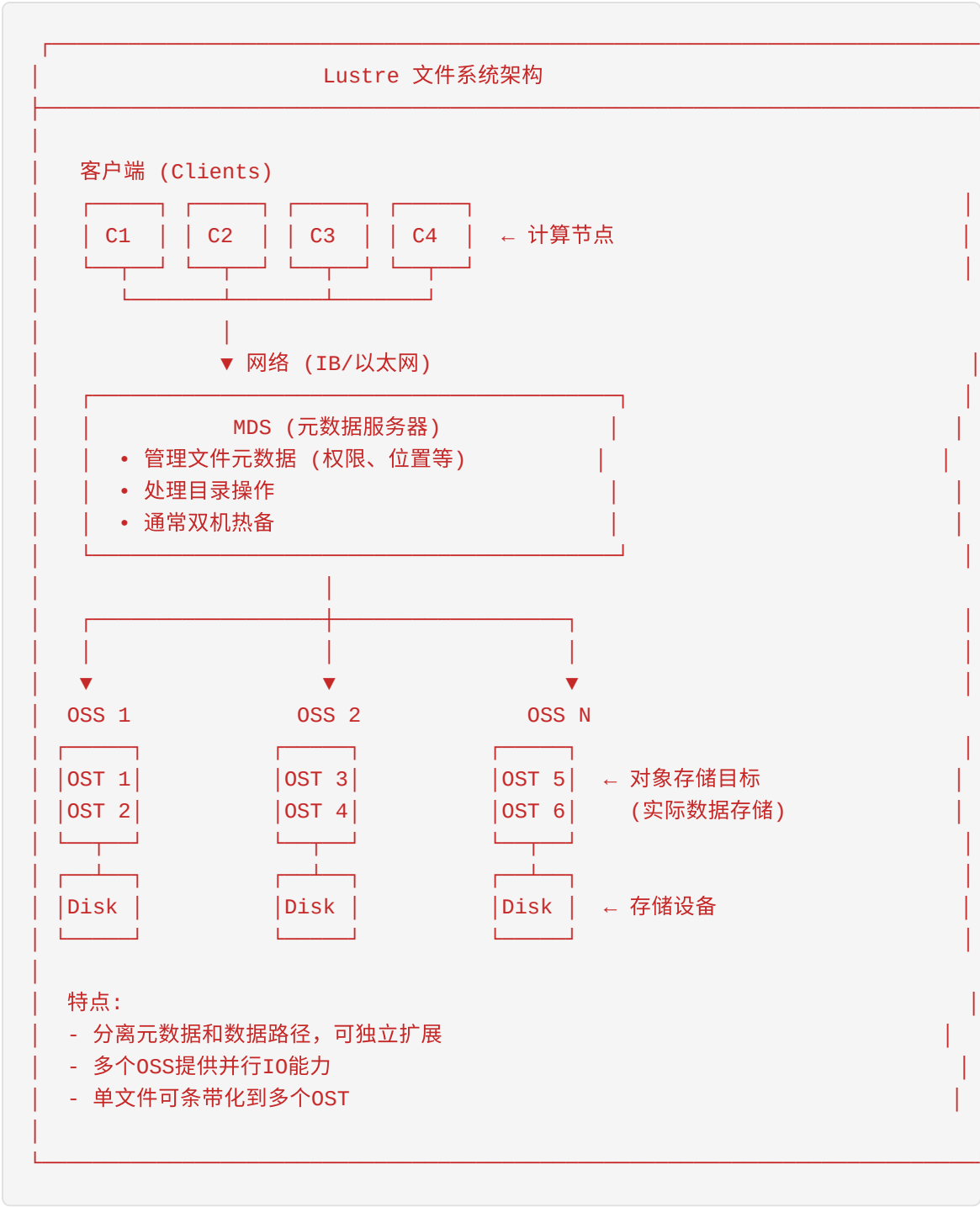
# 3. 分片保存（大模型）
# 每个rank保存自己的部分
local_state = {
    'model_shard': model.local_state_dict(),
    'optimizer_shard': optimizer.local_state_dict(),
}
torch.save(local_state, f"{save_path}/rank_{rank}.pt")
```

1.6.4 高性能文件系统

并行文件系统对比

文件系统	开发者	特点	适用场景
Lustre	Intel/开源	成熟稳定，大规模部署	HPC集群
GPFS/Spectrum Scale	IBM	企业级特性丰富	企业HPC
BeeGFS	ThinkParQ	易部署，性能好	中小型集群
Ceph	开源	统一存储，高可用	云原生
WekaFS	Weka	极致性能，NVMe优化	AI/ML
JuiceFS	开源	云原生，元数据分离	混合云

Lustre架构



存储性能基准

典型AI存储性能需求：

指标	最低要求	推荐配置	理想配置
顺序读带宽	1 GB/s	5 GB/s	10+ GB/s
顺序写带宽	500 MB/s	2 GB/s	5+ GB/s
随机读IOPS	10K	50K	100K+
元数据操作	1K ops/s	5K ops/s	10K+ ops/s
延迟	<10ms	<5ms	<1ms

1.6.5 存储优化最佳实践

```
# 1. 本地缓存策略
import os

# 将数据预读到本地NVMe SSD
local_cache = "/local_ssd/cache"
os.makedirs(local_cache, exist_ok=True)

# 首次访问时从共享存储复制
if not os.path.exists(f"{local_cache}/data"):
    os.system(f"cp -r /shared_storage/data {local_cache}/")

# 2. 内存缓存（适合小数据集）
# 将整个数据集加载到内存
import numpy as np
data = np.load("/shared_storage/dataset.npy") # 加载到内存

# 3. 使用内存文件系统（tmpfs）
# mount -t tmpfs -o size=100G tmpfs /dev/shm/cache

# 4. 数据预处理流水线优化
from torchvision import transforms

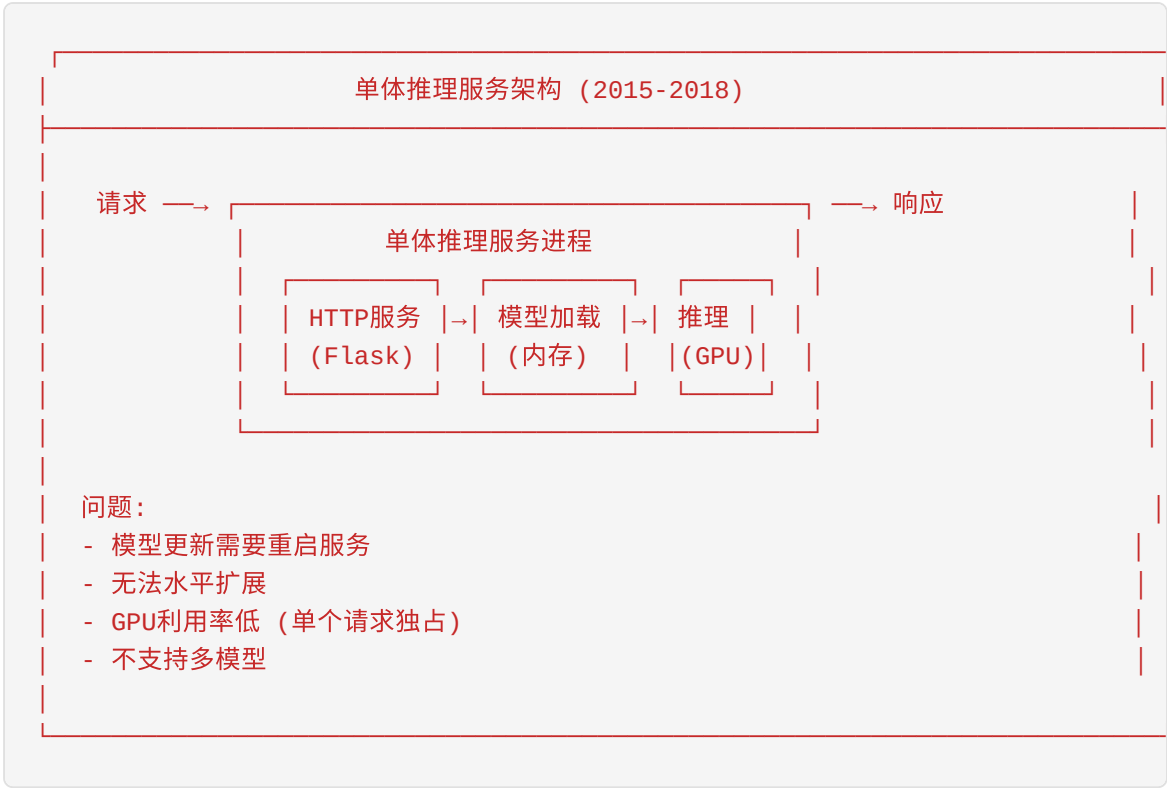
transform = transforms.Compose([
    transforms.RandomResizedCrop(224),
    transforms.RandomHorizontalFlip(),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                          std=[0.229, 0.224, 0.225]),
])

# 注意：预处理在CPU上完成，可能成为瓶颈
# 考虑使用NVIDIA DALI进行GPU加速预处理
```

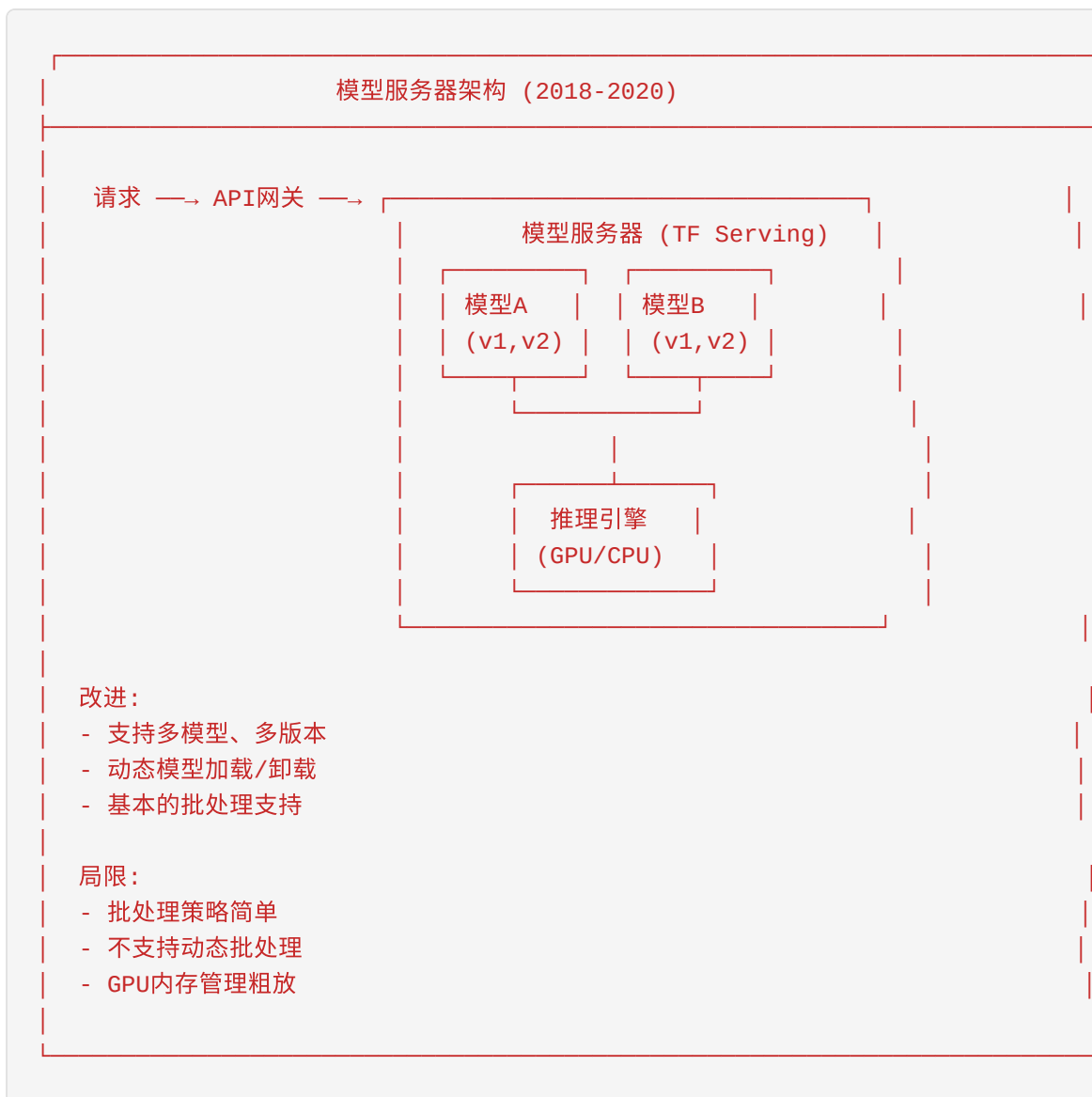
1.7 AI Serving系统的演进

1.7.1 从单体到分离式架构

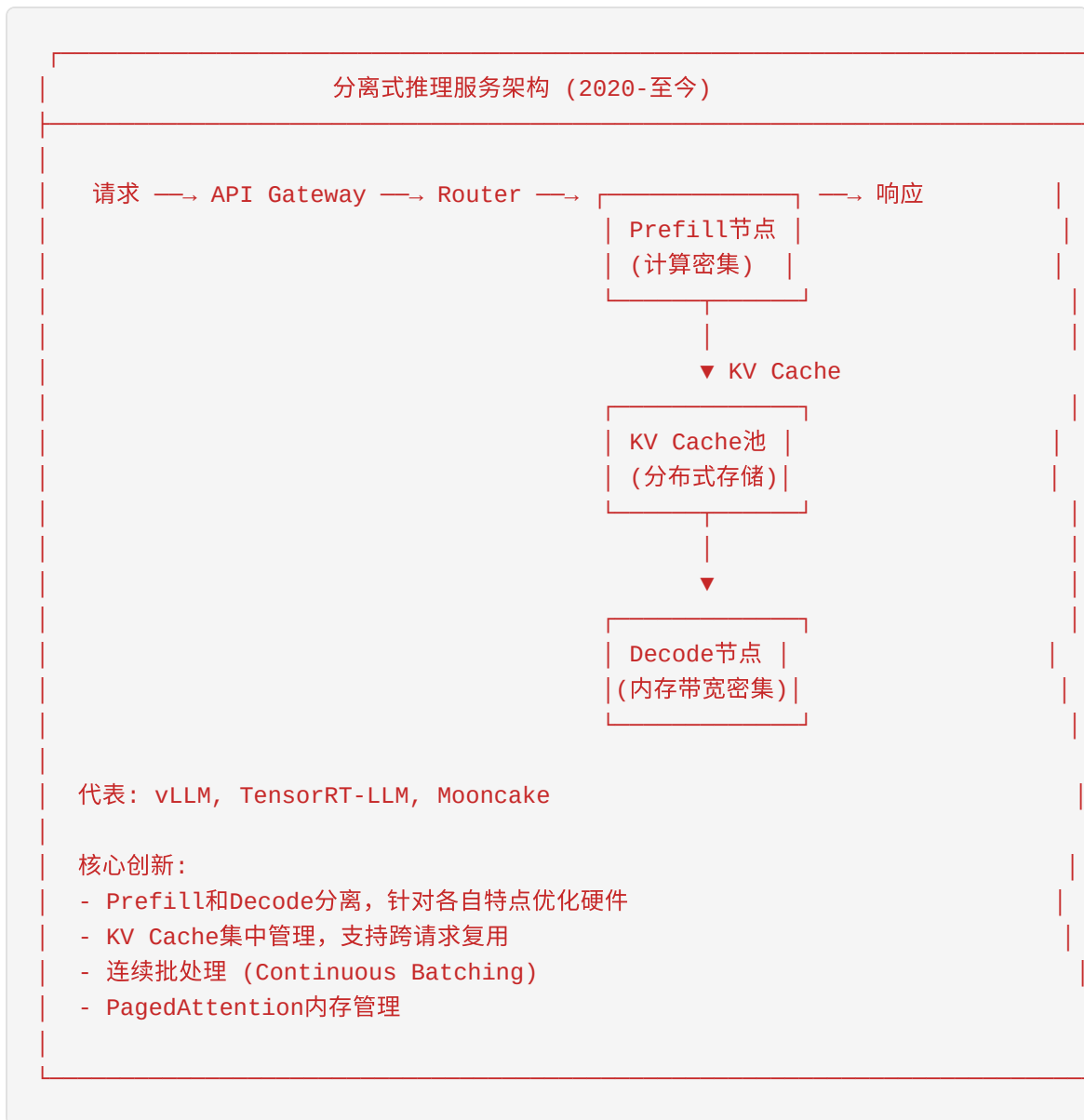
第一代：单体推理服务



第二代：模型服务器



第三代：分离式推理服务



1.7.2 大模型推理的核心挑战

内存瓶颈

GPT-3 175B模型推理内存需求:

模型权重: $175\text{B} \times 2\text{字节 (FP16)} = 350\text{ GB}$

└─ 至少需要 5张 A100 (80GB)

KV Cache: $\text{batch_size} \times \text{seq_len} \times \text{hidden_dim} \times \text{layers} \times 2 \text{ (K+V)}$

例如: $64 \times 2048 \times 12288 \times 96 \times 2 \times 2\text{字节} = 590\text{ GB}$

└─ 比模型权重还大!

总内存需求: $350\text{ GB} + 590\text{ GB} = 940\text{ GB}$

└─ 至少需要 12张 A100

计算 vs 内存带宽

大模型推理的两个阶段：

1. Prefill阶段 (Prompt处理)：

输入：	"请解释量子计算的原理..." (1000 tokens)	
计算：	并行的Self-Attention	
瓶颈：	计算能力 (FLOPS)	
时间：	~100ms	

2. Decode阶段 (Token生成)：

输入：	已生成的token	
计算：	逐个生成新token	
瓶颈：	内存带宽 (无法并行)	
时间：	~20ms/token × N tokens	

关键洞察：

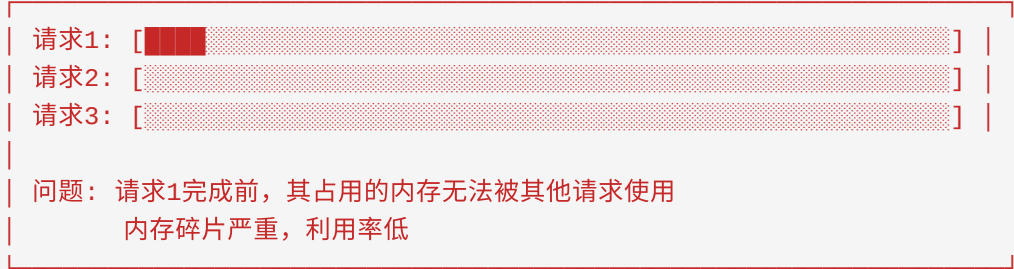
- Prefill是计算密集，需要高算力GPU
- Decode是内存带宽密集，需要高带宽显存
- 分离部署可以优化成本

1.7.3 vLLM核心创新

PagedAttention

PagedAttention 内存管理

传统方法（连续内存分配）：



Continuous Batching (连续批处理)

静态批处理 (传统):

[illegible]

问题：请求2完成后，需要等待其他请求，GPU空闲
无法动态添加新请求

连续批处理 (vLLM):

Step 1: [请求1 ] [请求2 ] [请求3 ] [请求4 ]

Step 2: [请求1 ] [请求2 ] [请求3 ] [请求4 ]

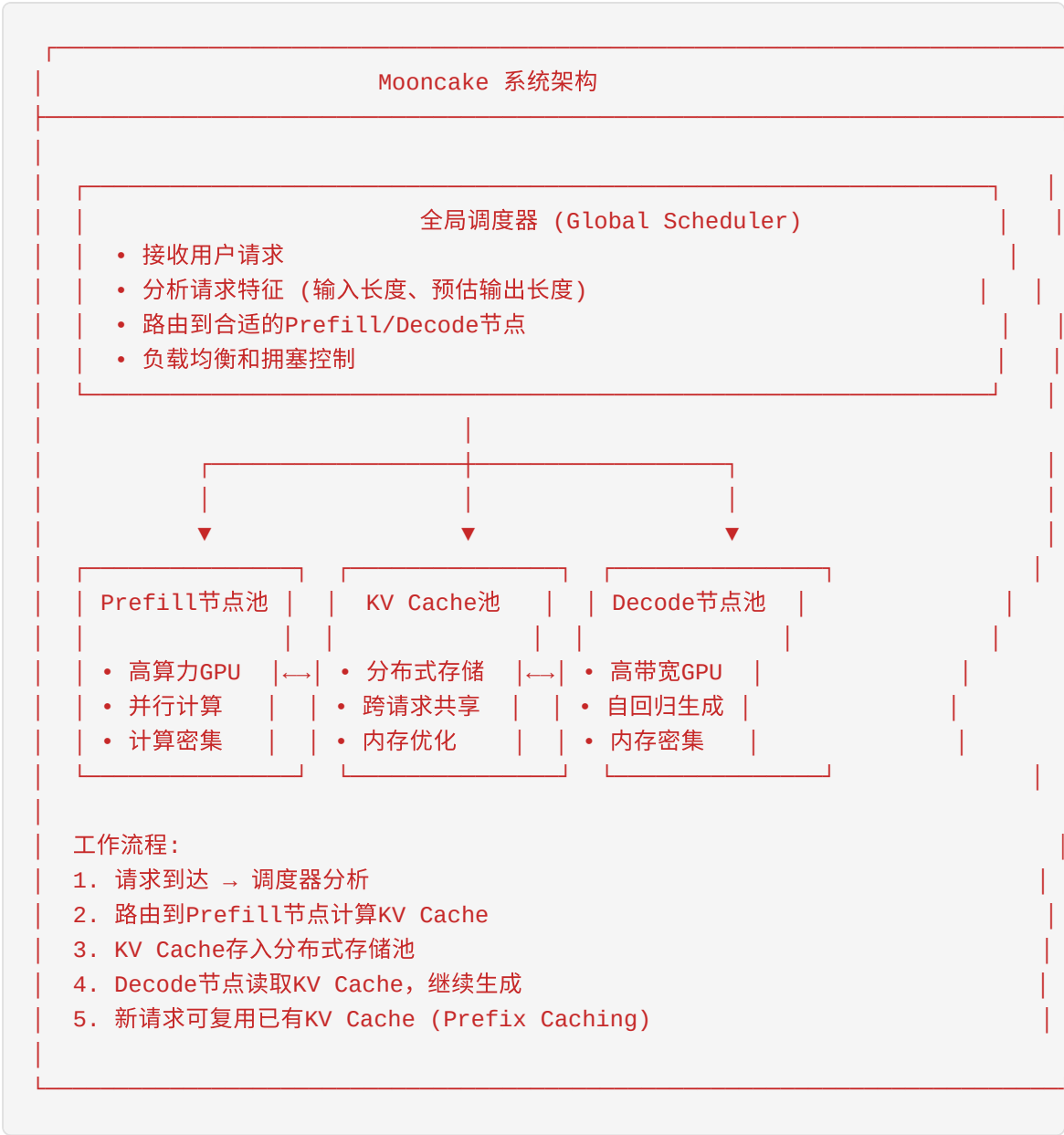
Step 3: [请求1 ] [请求2 ] [请求5 ] [请求3 ] ← 新请求加入

Step 4: [请求1 ] [请求3 ] [请求5 ] [请求6 ] ← 请求2完成

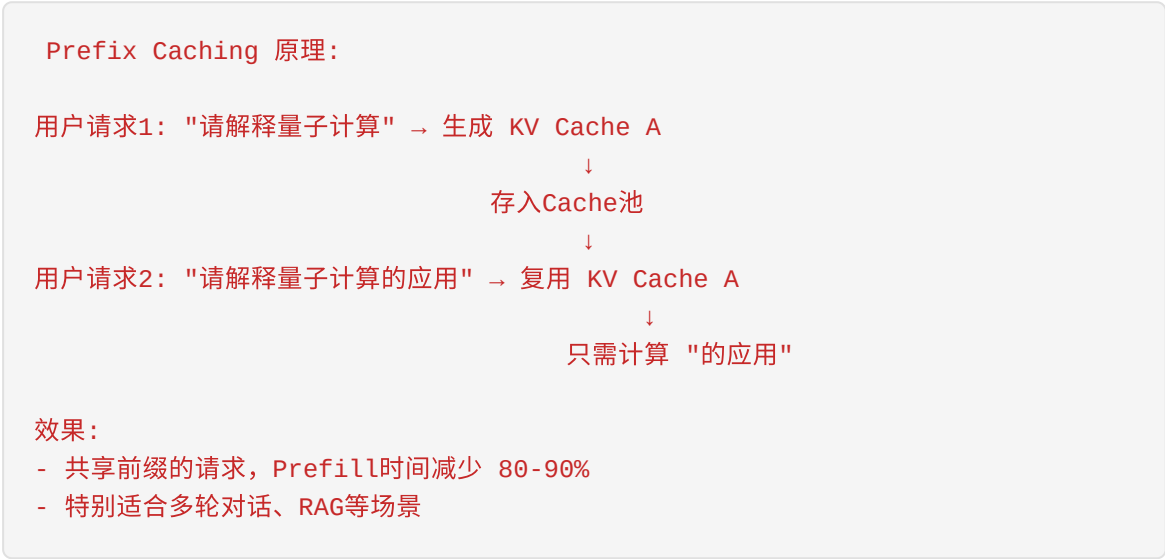
优势：请求完成立即退出，新请求随时加入
GPU利用率最大化
吞吐量提升 5-20倍

1.7.4 Mooncake架构详解

核心组件

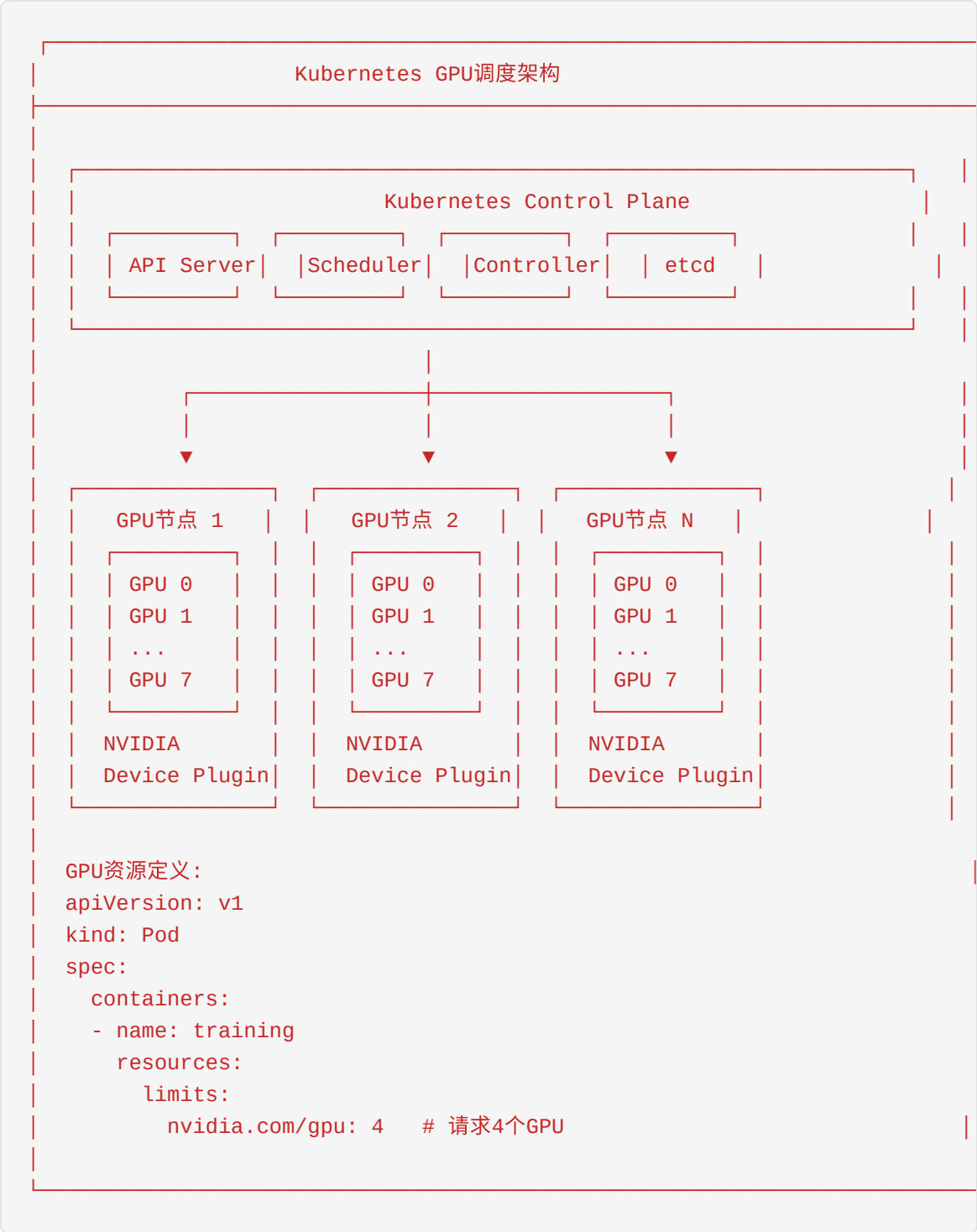


Prefix Caching



1.7.5 云原生AI基础设施

Kubernetes + GPU



GPU共享技术

GPU共享方案对比：

方案	技术	隔离性	适用场景
MPS	NVIDIA多进程服务	进程级	推理服务
MIG	多实例GPU	硬件级	多租户
vGPU	虚拟GPU	VM级	云桌面
Time-slicing	时间片轮转	无	开发测试

MIG (Multi-Instance GPU):

A100 (40GB)			
MIG 1	MIG 2	MIG 3	MIG4
10GB	10GB	10GB	10GB
独立	独立	独立	独立
计算单元	计算单元	计算单元	计算
每个MIG实例完全隔离，故障不影响其他实例			

1.7.6 AI Infra未来趋势

AI基础设施演进方向：

1. 异构计算

CPU + GPU + TPU + NPU + FPGA 协同工作	
不同任务使用最适合的硬件	

2. 存算一体

计算靠近存储，减少数据搬运	
CXL内存扩展，打破显存限制	

3. 边缘AI

模型压缩 + 边缘部署	
低延迟推理，保护隐私	

4. 绿色AI

能效优化，碳中和目标	
液冷技术，可再生能源	

5. 自动化运维

AIOps：智能监控、故障预测、自动调优	
降低运维成本，提高可靠性	

本章小结

本章全面介绍了AI基础设施的核心概念和技术：

核心知识点回顾

- 1. **AI Infra定义**：支撑AI应用全生命周期的底层技术体系
- 2. **硬件基础**：GPU架构、Tensor Core、HBM、主流型号对比
- 3. **分布式训练**：数据并行、模型并行、流水线并行、张量并行
- 4. **网络通信**：NCCL、RDMA、InfiniBand、网络拓扑

- 5. **存储系统**：数据加载、Checkpoint、并行文件系统
- 6. **推理服务**：从单体到分离式架构、vLLM创新、Mooncake架构

关键数字记忆

指标	数值	意义
A100显存带宽	2,039 GB/s	数据吞吐能力
H100 FP16算力	989 TFLOPS	计算能力
NVLink 4.0	50 GB/s/链路	GPU互联带宽
IB NDR	400 Gbps	跨节点网络
GPT-3训练成本	数百万美元	基础设施重要性

推荐学习路径

初学者：

GPU基础

→

数据并行

→

PyTorch

进阶：

NCCL

→

3D并行

→

Megatron

专家：

vLLM

→

Mooncake

→

论文阅读

关于Mooncake： Mooncake是月之暗面开源的分离式大模型推理架构，通过Prefill-Decode分离、KV Cache池化、Prefix Caching等创新技术，实现了业界领先的推理性能。作为Mooncake团队的实习生，你将有机会深入这些前沿技术的实现细节。

本章内容由Mooncake团队技术写作组编写，仅供内部学习使用。

第二章：LLM训练与推理基础

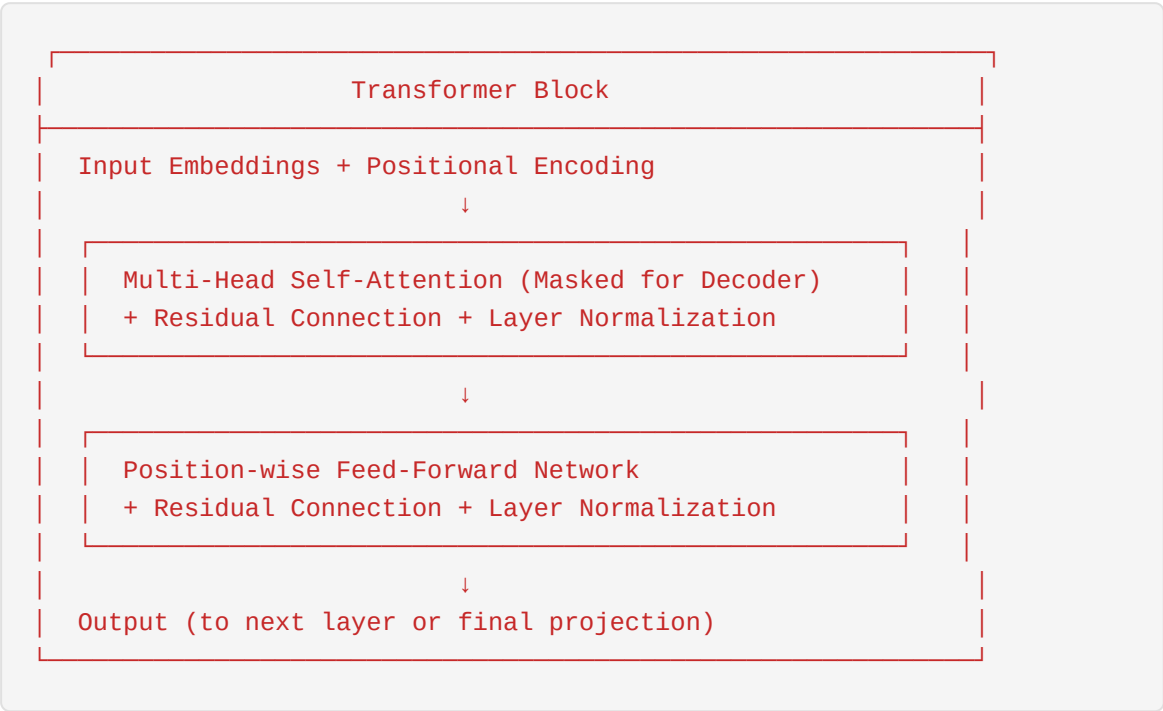
本章为即将加入Mooncake团队的实习生提供大语言模型（LLM）训练和推理的核心基础知识。内容涵盖从Transformer架构到实际部署优化的完整技术栈。

2.1 Transformer架构详解

Transformer架构由Vaswani等人在2017年的论文《Attention Is All You Need》中首次提出，它彻底改变了自然语言处理领域，成为现代大语言模型的基石。

2.1.1 整体架构概览

Transformer采用**编码器-解码器（Encoder-Decoder）**结构，但现代LLM（如GPT系列）主要使用**仅解码器（Decoder-only）**架构。



2.1.2 Self-Attention机制数学原理

Self-Attention（自注意力）是Transformer的核心机制，它允许模型在处理序列时关注输入的不同部分。

核心思想

对于输入序列 $\mathbf{X} \in \mathbb{R}^{n \times d_{\text{model}}}$ ，Self-Attention通过三个可学习的投影矩阵将其映射为：

- **Query矩阵**： $\mathbf{Q} = \mathbf{X}\mathbf{W}^Q$
- **Key矩阵**： $\mathbf{K} = \mathbf{X}\mathbf{W}^K$
- **Value矩阵**： $\mathbf{V} = \mathbf{X}\mathbf{W}^V$

其中 $\mathbf{W}^Q, \mathbf{W}^K, \mathbf{W}^V \in \mathbb{R}^{d_{\text{model}} \times d_k}$

Scaled Dot-Product Attention

注意力计算的核心公式：

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}}\right)\mathbf{V}$$

为什么要除以 $\sqrt{d_k}$ ？

当 d_k 较大时，点积的数值会变得很大，导致softmax函数的梯度变得非常小（饱和）。缩放因子 $\sqrt{d_k}$ 可以稳定梯度流：

$$\text{Var}\left(\frac{\mathbf{q} \cdot \mathbf{k}}{\sqrt{d_k}}\right) = \frac{d_k}{d_k} = 1$$

完整的Self-Attention计算流程

```
def self_attention(X, W_Q, W_K, W_V, d_k):
    """
    X: 输入矩阵 [batch_size, seq_len, d_model]
    W_Q, W_K, W_V: 投影矩阵 [d_model, d_k]
    """
    # 1. 线性投影
    Q = X @ W_Q    # [batch_size, seq_len, d_k]
    K = X @ W_K    # [batch_size, seq_len, d_k]
    V = X @ W_V    # [batch_size, seq_len, d_k]

    # 2. 计算注意力分数
    scores = Q @ K.transpose(-2, -1) # [batch_size, seq_len, seq_len]

    # 3. 缩放
    scores = scores / math.sqrt(d_k)

    # 4. Softmax归一化
    attn_weights = softmax(scores, dim=-1) # [batch_size, seq_len, seq_len]

    # 5. 加权求和
    output = attn_weights @ V # [batch_size, seq_len, d_k]

    return output, attn_weights
```

Masked Self-Attention (因果注意力)

在解码器中，为了防止模型在预测当前token时看到未来的信息，需要使用**因果掩码 (Causal Mask)**：

$$\text{MaskedAttention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \mathbf{Q} \cdot \text{softmax}\left(\frac{\mathbf{Q} \mathbf{K}^T}{\sqrt{d_k}} + \mathbf{M}\right) \cdot \mathbf{V}$$

其中掩码矩阵 $\mathbf{M}$ 定义为：

$$M_{ij} = \begin{cases} 0 & \text{if } i \geq j \\ \infty & \text{if } i < j \end{cases}$$


```
def create_causal_mask(seq_len):
    """创建下三角掩码矩阵"""
    mask = torch.triu(torch.ones(seq_len, seq_len), diagonal=1)
    mask = mask.masked_fill(mask == 1, float('-inf'))
    return mask

# 掩码矩阵示例 (seq_len=4)
# [[ 0, -inf, -inf, -inf],
#  [ 0,  0, -inf, -inf],
#  [ 0,  0,  0, -inf],
#  [ 0,  0,  0,  0]]
```

2.1.3 Multi-Head Attention

单一的注意力机制可能只关注一种类型的关系。Multi-Head Attention通过并行使用多组投影矩阵，让模型同时关注不同子空间的信息。

数学定义

$$\text{MultiHead}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) \mathbf{W}^O$$

其中每个head的计算：

$$\text{head}_i = \text{Attention}(\mathbf{X} \mathbf{W}_i^Q, \mathbf{X} \mathbf{W}_i^K, \mathbf{X} \mathbf{W}_i^V)$$

维度设置

参数	典型值	说明
d_{model}	768/1024/2048/4096	模型隐藏层维度
h	12/16/32/64	注意力头数
$d_k = d_v$	64/128	每个头的维度， $d_k = d_{\text{model}} / h$
$\mathbf{W}^O$	$d_{\text{model}} \times d_{\text{model}}$	输出投影矩阵

计算复杂度分析

Multi-Head Attention的计算复杂度为：

$$\mathcal{O}(n^2 \cdot d_{\text{model}})$$

其中 n 是序列长度。这是Self-Attention成为长序列处理瓶颈的主要原因。

```
class MultiHeadAttention(nn.Module):
    def __init__(self, d_model, num_heads):
        super().__init__()
        assert d_model % num_heads == 0

        self.d_model = d_model
        self.num_heads = num_heads
        self.d_k = d_model // num_heads

        # 投影矩阵
        self.W_Q = nn.Linear(d_model, d_model)
        self.W_K = nn.Linear(d_model, d_model)
        self.W_V = nn.Linear(d_model, d_model)
        self.W_O = nn.Linear(d_model, d_model)

    def forward(self, X, mask=None):
        batch_size, seq_len, _ = X.shape

        # 线性投影并分头
        Q = self.W_Q(X).view(batch_size, seq_len, self.num_heads, self.d_k)
        K = self.W_K(X).view(batch_size, seq_len, self.num_heads, self.d_k)
        V = self.W_V(X).view(batch_size, seq_len, self.num_heads, self.d_k)
        # Q, K, V: [batch_size, num_heads, seq_len, d_k]

        # 计算注意力
        scores = Q @ K.transpose(-2, -1) / math.sqrt(self.d_k)
        if mask is not None:
            scores = scores + mask
        attn_weights = F.softmax(scores, dim=-1)

        # 应用注意力到Value
        attn_output = attn_weights @ V # [batch_size, num_heads, seq_len,
        # 合并多头并投影
        attn_output = attn_output.transpose(1, 2).contiguous().view(
            batch_size, seq_len, self.d_model
        )
        output = self.W_O(attn_output)

        return output, attn_weights
```

2.1.4 Position-wise Feed-Forward Network (FFN)

FFN对每个位置独立地应用相同的全连接网络：

$$\text{FFN}(\mathbf{x}) = \max(0, \mathbf{x} \cdot \mathbf{W}_1 + \mathbf{b}_1) \cdot \mathbf{W}_2 + \mathbf{b}_2$$

或使用GELU激活函数（现代LLM更常用）：

$$\text{FFN}(\mathbf{x}) = \text{GELU}(\mathbf{x} \cdot \mathbf{W}_1 + \mathbf{b}_1) \cdot \mathbf{W}_2 + \mathbf{b}_2$$

维度设置

- 中间层维度：\$d_{\text{ff}} = 4 \times d_{\text{model}}\$（典型配置）
- \$\mathbf{W}_1\$: \$d_{\text{model}} \times d_{\text{ff}}\$
- \$\mathbf{W}_2\$: \$d_{\text{ff}} \times d_{\text{model}}\$

FFN的计算量占比

在标准Transformer中，FFN贡献了约 **2/3 的总参数量** 和 **约50% 的计算量**。

```
class FeedForward(nn.Module):
    def __init__(self, d_model, d_ff=None, dropout=0.1):
        super().__init__()
        if d_ff is None:
            d_ff = 4 * d_model

        self.fc1 = nn.Linear(d_model, d_ff)
        self.fc2 = nn.Linear(d_ff, d_model)
        self.dropout = nn.Dropout(dropout)
        self.activation = nn.GELU() # 现代LLM常用GELU

    def forward(self, x):
        x = self.fc1(x)
        x = self.activation(x)
        x = self.dropout(x)
        x = self.fc2(x)
        return x
```

2.1.5 Layer Normalization

Layer Normalization（层归一化）对每个样本的所有特征进行归一化，稳定训练过程：

$$\text{LayerNorm}(\mathbf{x}) = \gamma \odot \frac{\mathbf{x} - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta$$

其中： - \$\mu = \frac{1}{d} \sum_{i=1}^d x_i\$（均值） - \$\sigma^2 = \frac{1}{d} \sum_{i=1}^d (x_i - \mu)^2\$（方差） - \$\gamma, \beta\$ 是可学习的缩放和平移参数 -

ϵ 是数值稳定性的小常数（通常 10^{-6} ）

Pre-Norm vs Post-Norm

架构	公式	特点
Post-Norm （原始Transformer）	$\mathbf{x}_{out} = \text{LayerNorm}(\mathbf{x} + \text{Sublayer}(\mathbf{x}))$	训练不稳定但性能上限高
Pre-Norm （现代LLM常用）	$\mathbf{x}_{out} = \mathbf{x} + \text{Sublayer}(\text{LayerNorm}(\mathbf{x}))$	训练更稳定，收敛更快

现代LLM（GPT、LLaMA等）普遍采用Pre-Norm架构。

2.1.6 Positional Encoding

由于Self-Attention本身不具备位置信息，需要通过Positional Encoding注入位置信息。

正弦/余弦位置编码（原始Transformer）

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

$$PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

可学习位置编码

现代LLM（如GPT系列）通常使用可学习的位置嵌入：

```
self.position_embedding = nn.Embedding(max_seq_len, d_model)
```

RoPE（旋转位置编码）

LLaMA等模型使用RoPE（Rotary Position Embedding），通过旋转矩阵编码相对位置：

$$\text{RoPE}(\mathbf{x}, m) = \mathbf{x} \cdot e^{i m \theta}$$

RoPE的优势： - 天然支持相对位置 - 外推性能好（可处理比训练时更长的序列） - 与Self-Attention兼容性好

```
class RotaryEmbedding(nn.Module):
    def __init__(self, dim, max_seq_len=2048, base=10000):
        super().__init__()
        inv_freq = 1.0 / (base ** (torch.arange(0, dim, 2).float() / dim))
        self.register_buffer("inv_freq", inv_freq)

    def forward(self, seq_len):
        t = torch.arange(seq_len, device=self.inv_freq.device)
        freqs = torch.einsum("i,j->ij", t, self.inv_freq)
        emb = torch.cat((freqs, freqs), dim=-1)
        return emb.cos(), emb.sin()
```

2.2 LLM训练流程

大语言模型的训练通常分为三个阶段：预训练（Pre-training）、监督微调（SFT）和基于人类反馈的强化学习（RLHF）。

2.2.1 预训练（Pre-training）

预训练是LLM训练的第一阶段，目标是让模型学习通用的语言表示和世界知识。

训练目标：下一个Token预测

预训练采用自监督学习方式，目标是预测序列中的下一个token：

$$\mathcal{L}_{\text{pretrain}} = -\sum_{t=1}^T \log P(x_t | x_{<t}; \theta)$$

```
# 预训练伪代码
for batch in dataloader:
    input_ids = batch['input_ids']      # [batch_size, seq_len]
    labels = batch['labels']            # 向右偏移一位的input_ids

    logits = model(input_ids)           # [batch_size, seq_len, vocab_size]
    loss = cross_entropy(logits, labels)

    loss.backward()
    optimizer.step()
```

预训练数据

数据源	占比	说明
Common Crawl	60-80%	网页数据，需要清洗
书籍/文学	5-15%	高质量长文本
Wikipedia	3-5%	百科知识
代码	10-20%	GitHub等代码库
学术论文	2-5%	ArXiv等

典型预训练配置

模型	参数量	训练Token数	批次大小	学习率	训练时长
GPT-3	175B	300B	3.2M	0.6×10^{-4}	~数月
LLaMA-2	70B	2T	4M	1.5×10^{-4}	~数月
GPT-4	~1.8T	~13T	未知	未知	未知

Chinchilla Scaling Laws

DeepMind的Chinchilla研究表明，模型参数量 N 和训练token数 D 应满足：

$$D \approx 20N$$

即对于给定计算预算，模型大小和数据量应该同时扩展。例如： - 70B参数的模型应该训练约1.4T tokens - 这与早期只关注参数量的做法不同

2.2.2 监督微调（Supervised Fine-Tuning, SFT）

预训练后的模型需要通过SFT学习遵循指令和完成任务的能力。

SFT数据格式

SFT数据通常采用对话格式：

```
{
  "messages": [
    {"role": "system", "content": "你是一个有帮助的助手。"},
    {"role": "user", "content": "解释什么是机器学习。"},
    {"role": "assistant", "content": "机器学习是人工智能的一个分支..."}
  ]
}
```

训练目标

SFT只在assistant的回复上计算损失：

$$\mathcal{L}_{\text{SFT}} = -\sum_{(x, y) \in \mathcal{D}} \sum_{t=1}^{|y|} \log P(y_t | x, y_{<t}; \theta)$$

```
def compute_sft_loss(model, batch):
    """
    只在assistant回复上计算损失
    """
    input_ids = batch['input_ids']
    labels = batch['labels']
    loss_mask = batch['loss_mask'] # 1表示assistant回复的位置

    logits = model(input_ids)

    # 应用loss mask
    losses = F.cross_entropy(logits.view(-1, vocab_size), labels.view(-1),
                             loss_mask.view(-1))
    losses = losses * loss_mask.view(-1)

    return losses.sum() / loss_mask.sum()
```

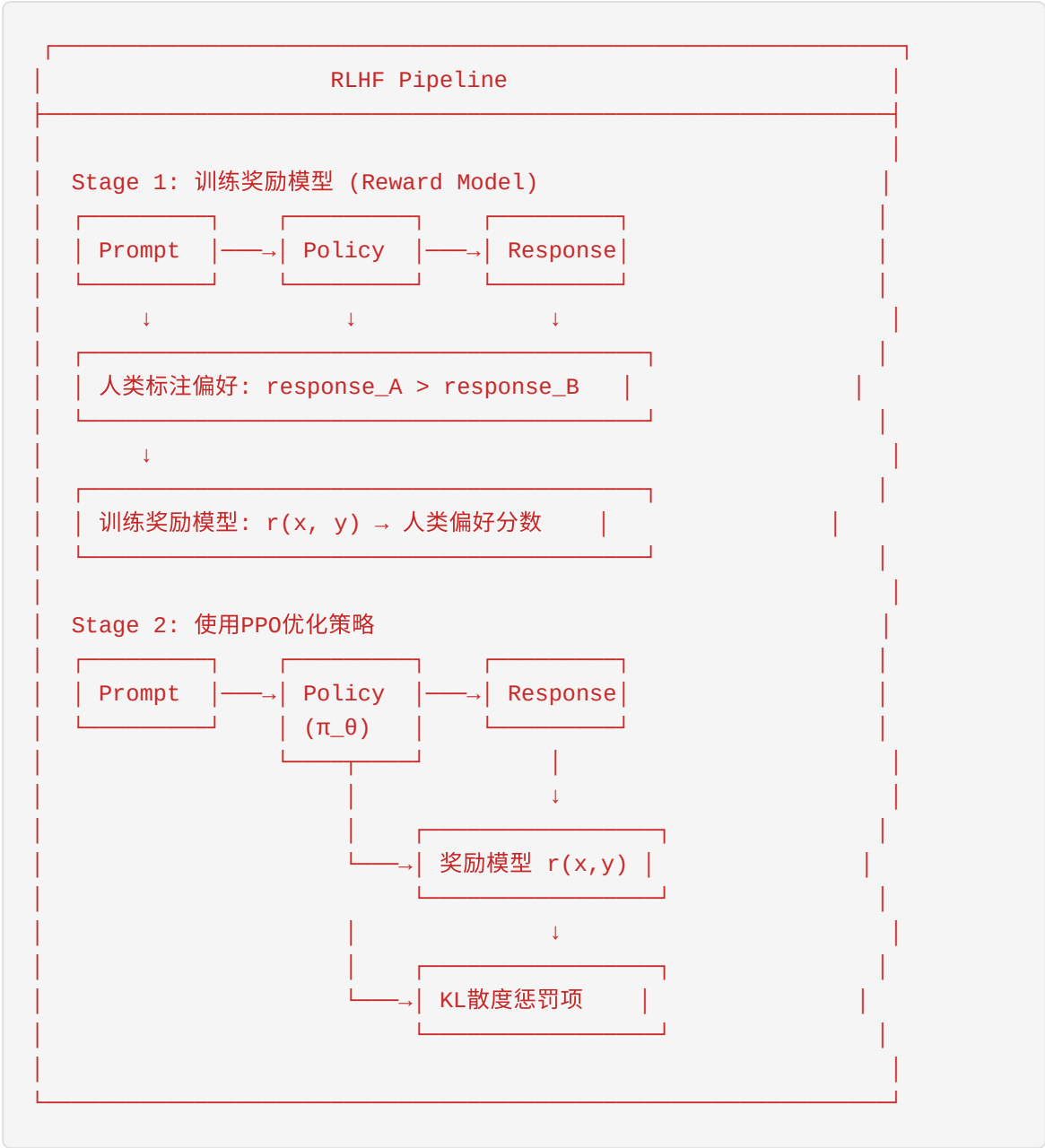
SFT vs 预训练的关键区别

方面	预训练	SFT
数据量	万亿级tokens	十万到百万级样本
数据质量	原始网页数据	人工标注/高质量数据
训练目标	无监督	有监督
学习率	较大 (~1e-4)	较小 (~1e-5)
训练轮数	1 epoch	3-5 epochs

2.2.3 RLHF（人类反馈强化学习）

RLHF通过人类偏好数据进一步对齐模型行为，使其输出更符合人类期望。

RLHF三阶段流程



奖励模型训练

对于偏好对 (x, y_w, y_l) ，其中 y_w 是人类偏好的回复， y_l 是不偏好的回复：

$$\mathcal{L}_{\text{RM}} = -\mathbb{E}_{(x, y_w, y_l)} \left[\log \left(\frac{\sigma(r(x, y_w) - r(x, y_l))}{1 + \sigma(r(x, y_w) - r(x, y_l))} \right) \right]$$

PPO (Proximal Policy Optimization)

PPO是RLHF中常用的优化算法，目标函数：

$$\mathcal{L}_{\text{PPO}} = \mathbb{E}_{(x, y)} \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip} \left(r_t(\theta), 1-\epsilon, 1+\epsilon \right) \hat{A}_t \right) \right]$$

其中： - $r_t(\theta) = \frac{\pi_{\theta}(y_t|x, y_{<t})}{\pi_{\theta_{\text{old}}}(y_t|x, y_{<t})}$ 是重要性采样比率 - $\hat{A}_t$ 是优势函数估计 - ϵ 是裁剪参数（通常0.1或0.2）

KL散度约束

为了防止策略偏离SFT模型太远，加入KL散度惩罚：

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{PPO}} - \beta \cdot \text{KL}(\pi_{\theta} \parallel \pi_{\text{SFT}})$$

DPO (Direct Preference Optimization)

DPO是RLHF的替代方案，直接用偏好数据优化策略，无需显式训练奖励模型：

$$\mathcal{L}_{\text{DPO}} = -\mathbb{E}_{(x, y_w, y_l)} \left[\log \sum \beta \log \frac{\pi_{\theta}(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta \log \frac{\pi_{\theta}(y_l|x)}{\pi_{\text{ref}}(y_l|x)} \right]$$

DPO的优势： - 更简单，无需训练奖励模型 - 训练更稳定 - 计算开销更小

2.3 训练优化技术

训练大语言模型需要巨大的计算资源，各种优化技术可以显著降低训练成本。

2.3.1 混合精度训练 (Mixed Precision Training)

混合精度训练使用FP16/BF16进行前向/反向传播，FP32进行参数更新，在保持精度的同时加速训练。

FP16 vs BF16

特性	FP16	BF16
指数位	5 bit	8 bit
尾数位	10 bit	7 bit
数值范围	$\pm 65,504$	$\pm 3.4 \times 10^{38}$
精度	较高	较低
适用场景	推理	训练

FP16: [sign:1][exponent:5][mantissa:10] → 范围小但精度高
BF16: [sign:1][exponent:8][mantissa:7] → 范围大但精度低
FP32: [sign:1][exponent:8][mantissa:23] → 完整精度

混合精度训练流程

```
from torch.cuda.amp import autocast, GradScaler

scaler = GradScaler()

for batch in dataloader:
    optimizer.zero_grad()

    # 前向传播使用FP16
    with autocast():
        outputs = model(batch)
        loss = criterion(outputs, targets)

    # 缩放损失并反向传播
    scaler.scale(loss).backward()

    # 梯度裁剪（在缩放空间进行）
    scaler.unscale_(optimizer)
    torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm)

    # 更新参数（使用FP32）
    scaler.step(optimizer)
    scaler.update()
```

性能提升

- **内存节省**：约50%（FP16 vs FP32）

- **速度提升**：1.5-3×（取决于GPU架构，Tensor Core加速）
- **A100/H100**：BF16是推荐格式，硬件原生支持

2.3.2 梯度累积（Gradient Accumulation）

当GPU内存不足以支持目标batch size时，可以使用梯度累积。

原理

将大batch拆分为多个小batch，累积梯度后再更新：

```
accumulation_steps = 4 # 累积4个小batch
effective_batch_size = batch_size * accumulation_steps

optimizer.zero_grad()
for i, batch in enumerate(dataloader):
    loss = model(batch) / accumulation_steps # 缩放损失
    loss.backward()

    if (i + 1) % accumulation_steps == 0:
        optimizer.step()
        optimizer.zero_grad()
```

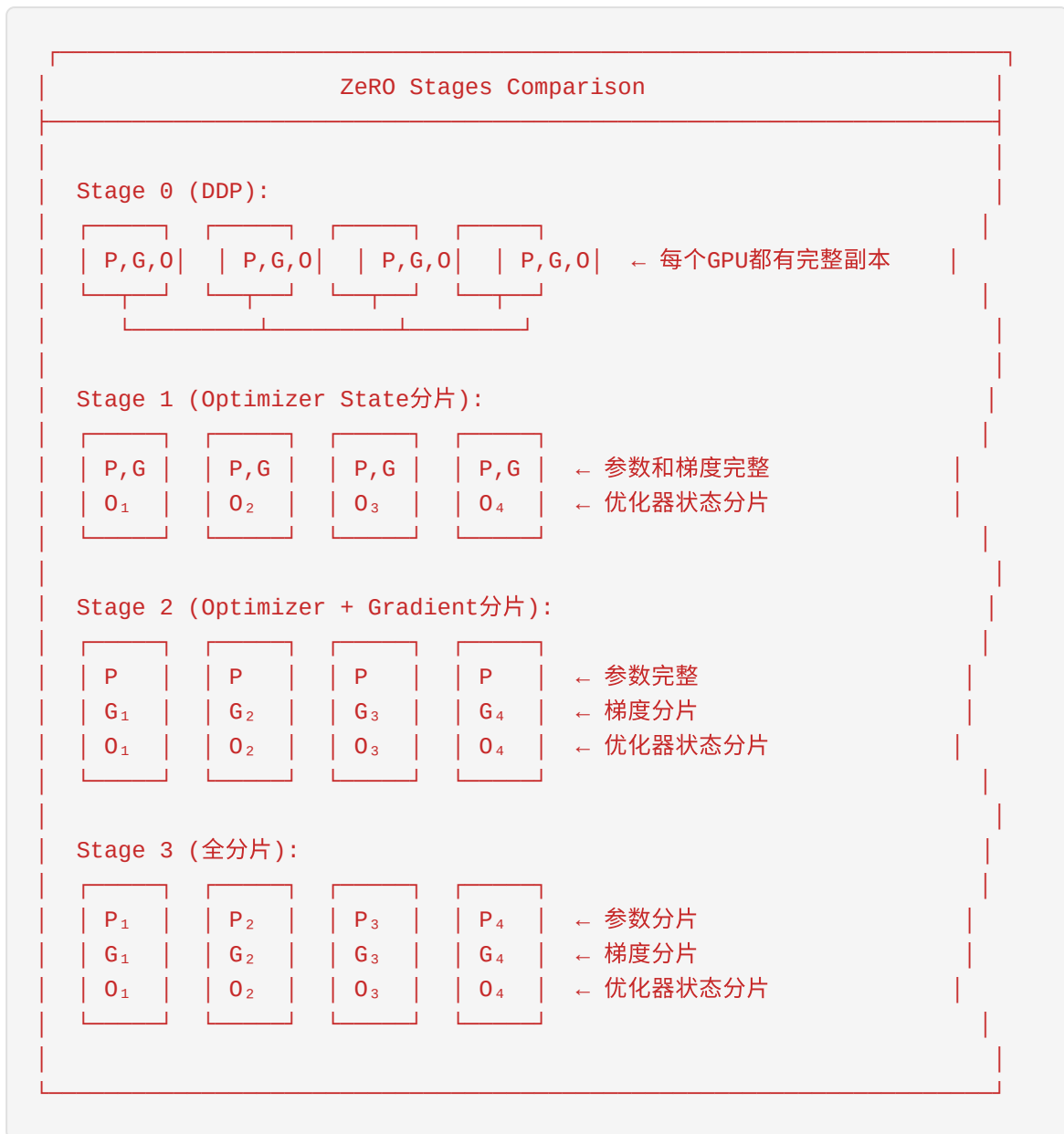
内存vs速度的权衡

配置	内存使用	训练速度	适用场景
无累积	基准	最快	内存充足
4步累积	25%	75%	内存受限
8步累积	12.5%	50%	严重受限

2.3.3 ZeRO优化器（Zero Redundancy Optimizer）

ZeRO是DeepSpeed提出的数据并行优化技术，将优化器状态、梯度和参数分片到多个GPU。

ZeRO的三个阶段



内存节省效果

对于参数数量为 Ψ 的模型：

ZeRO Stage	每GPU内存	相对DDP节省
DDP	16Ψ	1×
ZeRO-1	16Ψ	1×
ZeRO-2	$2\Psi + 14\Psi/N$	$\sim 8\times (N=8)$
ZeRO-3	$16\Psi/N$	$N\times$

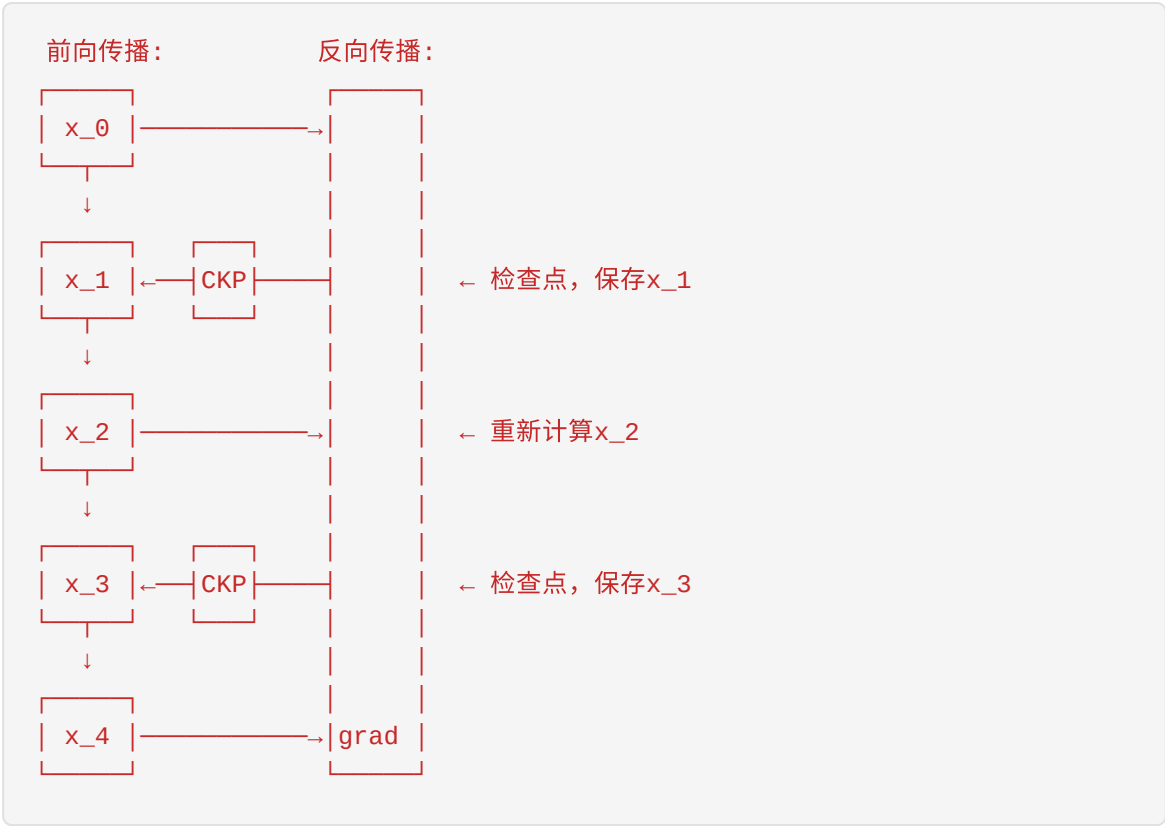
其中 N 是GPU数量。

```
# DeepSpeed ZeRO配置示例
deepspeed_config = {
    "zero_optimization": {
        "stage": 2, # 或 1, 3
        "offload_optimizer": {
            "device": "cpu",
            "pin_memory": True
        },
        "allgather_partitions": True,
        "allgather_bucket_size": 2e8,
        "overlap_comm": True,
        "reduce_scatter": True,
    },
    "train_batch_size": "auto",
    "train_micro_batch_size_per_gpu": "auto",
    "gradient_accumulation_steps": "auto",
    "fp16": {
        "enabled": True
    }
}
```

2.3.4 梯度检查点 (Gradient Checkpointing)

梯度检查点通过牺牲计算来换取内存：只保存部分中间激活值，其他在反向传播时重新计算。

原理



内存vs计算权衡

检查点策略	内存节省	额外计算	适用场景
无检查点	0%	0%	内存充足
每层检查点	~50%	~20%	标准选择
每2层检查点	~25%	~10%	平衡方案

```
from torch.utils.checkpoint import checkpoint

class CheckpointedTransformerLayer(nn.Module):
    def __init__(self, layer):
        super().__init__()
        self.layer = layer

    def forward(self, x):
        # 使用梯度检查点
        return checkpoint(self.layer, x)
```

2.3.5 训练优化技术总结

技术	内存节省	速度影响	实现复杂度
混合精度	~50%	+50-200%	低
梯度累积	可调	与累积步数成反比	低
ZeRO-1	~0%	-5%	低
ZeRO-2	~60-80%	-10%	中
ZeRO-3	~90%+	-15%	中
梯度检查点	~50%	+20%	低

实际组合示例： - 训练7B模型：混合精度 + ZeRO-2 + 梯度检查点 - 训练70B模型：混合精度 + ZeRO-3 + 梯度检查点 + CPU Offload

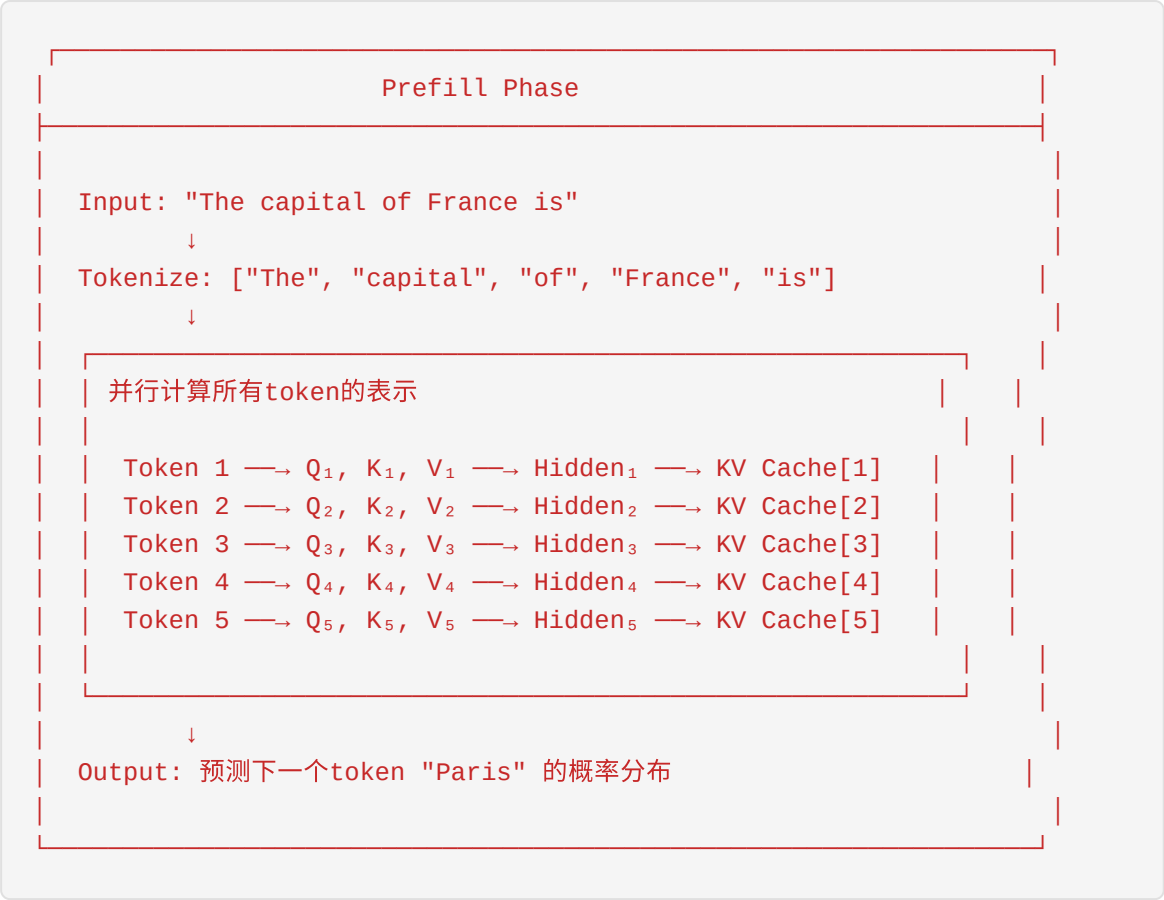
2.4 LLM推理的两个阶段

LLM推理过程可以分为两个截然不同的阶段：**Prefill阶段**和**Decode阶段**。理解这两个阶段的特性对于推理优化至关重要。

2.4.1 Prefill阶段（计算密集型）

Prefill阶段处理输入prompt，计算所有token的KV Cache，为后续生成做准备。

工作流程



计算特性

特性	描述
计算模式	矩阵-矩阵乘法（GEMM）
计算复杂度	$\mathcal{O}(n^2 \cdot d)$ ，其中 n 是序列长度
内存访问	可预测、连续
瓶颈	计算能力（Compute-bound）
GPU利用率	高（>80%）

性能指标

Prefill阶段的性能通常用 **Time-to-First-Token (TTFT)** 来衡量：

$$\text{TTFT} = \frac{\text{Prefill计算量}}{\text{GPU峰值算力} \times \text{利用率}}$$

对于7B模型在A100上的典型数据： - 1K tokens prompt: ~50-100ms TTFT - 4K tokens prompt: ~200-400ms TTFT - 8K tokens prompt: ~500-1000ms TTFT

2.4.2 Decode阶段（内存密集型）

Decode阶段是自回归生成过程，每次只生成一个新token，直到遇到结束符或达到最大长度。

工作流程



计算特性

特性	描述
计算模式	矩阵-向量乘法（GEMV）
计算复杂度	$\mathcal{O}(n \cdot d)$ 每步
内存访问	随机、分散（访问KV Cache）
瓶颈	内存带宽（Memory-bound）
GPU利用率	低（<30%）

为什么Decode是内存瓶颈？

计算强度分析：

Prefill (GEMM):
 计算量: $2 \times M \times N \times K$ FLOPs
 内存访问: $M \times K + N \times K + M \times N$ 元素
 计算强度: $\sim K$ (高)

Decode (GEMV):
 计算量: $2 \times M \times N$ FLOPs
 内存访问: $M + N + M \times N$ 元素
 计算强度: ~ 1 (低)

当计算强度 < 硬件计算/带宽比时，成为内存瓶颈
A100: $312 \text{ TFLOPS} / 2 \text{ TB/s} = 156 \text{ FLOPs/byte}$
Decode强度远小于156，因此是内存瓶颈

性能指标

Decode阶段的性能用 **Time-Per-Output-Token (TPOT)** 或 **Throughput (tokens/s)** 衡量：

$$\text{Throughput} = \frac{\text{Batch Size}}{\text{TPOT}}$$

对于7B模型在A100上的典型数据： - Batch Size = 1: ~50-100ms/token (10-20 tokens/s) - Batch Size = 8: ~60-120ms/token (60-130 tokens/s) - Batch Size = 32: ~80-160ms/token (200-400 tokens/s)

2.4.3 两个阶段性能特征对比

特性	Prefill	Decode
并行度	高（所有token并行）	低（逐个token）
计算类型	GEMM	GEMV
主要瓶颈	算力	内存带宽
GPU利用率	70-90%	10-30%
批处理收益	中等	高
优化重点	算力优化	内存优化、批处理
典型延迟	50-500ms	50-150ms/token

2.4.4 端到端推理时间估算

完整的推理请求时间：

$$\text{Total Time} = \text{TTFT} + (\text{Output Length} - 1) \times \text{TPOT}$$

示例：生成256 tokens，Prefill 1K tokens - TTFT = 100ms - TPOT = 80ms - Total Time = 100 + 255 × 80 = 20.5 seconds

2.5 KV Cache的原理和作用

KV Cache是LLM推理优化的核心技术，它避免了在Decode阶段重复计算历史token的Key和Value。

2.5.1 什么是KV Cache

在Transformer的Self-Attention中，每个token的计算需要所有前面token的Key和Value。KV Cache就是缓存这些中间结果。

直观理解

无KV Cache（每次重新计算）：

```
Step 1: 计算 token_1 的  $K_1, V_1$ 
Step 2: 计算 token_1,2 的  $K_{1,2}, V_{1,2} \rightarrow$  重复计算  $K_1, V_1$ 
Step 3: 计算 token_1,2,3 的  $K_{1,2,3}, V_{1,2,3} \rightarrow$  重复计算
...
时间复杂度:  $O(n^3)$ 
```

有KV Cache（只计算新token）：

```
Step 1: 计算 token_1 的  $K_1, V_1 \rightarrow$  Cache:  $[K_1, V_1]$ 
Step 2: 计算 token_2 的  $K_2, V_2 \rightarrow$  Cache:  $[K_{1,2}, V_{1,2}]$ 
Step 3: 计算 token_3 的  $K_3, V_3 \rightarrow$  Cache:  $[K_{1,2,3}, V_{1,2,3}]$ 
...
时间复杂度:  $O(n^2)$ 
```

数学原理

在Self-Attention中，第 t 个token的输出：

$$\text{Attention}(\mathbf{q}_t, \mathbf{K}_{\leq t}, \mathbf{V}_{\leq t}) = \frac{\exp(\frac{\mathbf{q}_t \cdot \mathbf{K}_{\leq t}}{\sqrt{d_k}})}{\sum_{i=1}^t \exp(\frac{\mathbf{q}_t \cdot \mathbf{K}_i}{\sqrt{d_k}})} \mathbf{V}_{\leq t}$$

其中： - $\mathbf{q}_t$ 是当前token的Query（需要实时计算） - $\mathbf{K}_{\leq t} = [\mathbf{K}_1, \mathbf{K}_2, \dots, \mathbf{K}_t]$ 是所有前面token的Key - $\mathbf{V}_{\leq t} = [\mathbf{V}_1, \mathbf{V}_2, \dots, \mathbf{V}_t]$ 是所有前面token的Value

KV Cache存储的就是 $\mathbf{K}_{\leq t}$ 和 $\mathbf{V}_{\leq t}$ 。

2.5.2 KV Cache内存占用计算

KV Cache的内存占用是推理系统设计的核心考量。

计算公式

对于单条请求：

$$\text{KV Cache Size} = 2 \times \text{num_layers} \times \text{num_heads} \times d_{\text{head}} \times \text{seq_len} \times \text{bytes_per_element}$$

简化公式（假设 $d_{\text{model}} = \text{num_heads} \times d_{\text{head}}$ ）：

$$\text{KV Cache Size} = 2 \times L \times h \times d_h \times s \times \text{prec} = 2 \times L \times d_{\text{model}} \times s \times \text{prec}$$

其中： - \$L\$: 层数 - \$h\$: 注意力头数 - \$d_h\$: 每个头的维度 - \$s\$: 序列长度 - prec: 精度字节数（FP16=2, FP32=4）

具体计算示例

LLaMA-2 7B模型（FP16）： - 层数 \$L = 32\$ - 隐藏维度 \$d_{\text{model}} = 4096\$ - 序列长度 \$s = 4096\$ - 精度 = FP16 (2 bytes)

$$\text{KV Cache} = 2 \times 32 \times 4096 \times 4096 \times 2 = 2,147,483,648 \text{ bytes} = 2 \text{ GB}$$

不同模型的KV Cache占用：

模型	参数量	层数	隐藏维度	4K序列	8K序列	32K序列
LLaMA-2	7B	32	4096	2.0 GB	4.0 GB	16.0 GB
LLaMA-2	13B	40	5120	3.1 GB	6.3 GB	25.0 GB
LLaMA-2	70B	80	8192	10.0 GB	20.0 GB	80.0 GB
GPT-4	~1.8T	120	18432	162 GB	324 GB	1.3 TB

批处理的KV Cache

对于batch size为 \$b\$ 的情况：

$$\text{Total KV Cache} = b \times 2 \times L \times d_{\text{model}} \times s \times \text{prec}$$

示例： batch_size=32， LLaMA-2 7B， 4K序列 $32 \times 2 \text{ GB} = 64 \text{ GB}$

这已经接近A100的80GB显存上限！

2.5.3 为什么需要KV Cache

性能对比

场景	无KV Cache	有KV Cache	加速比
7B模型, 512 tokens	5.2s	0.8s	6.5×
7B模型, 1024 tokens	21.0s	1.6s	13×
7B模型, 2048 tokens	84.0s	3.2s	26×

内存vs计算的权衡

KV Cache Trade-off:

内存 vs 计算

无KV Cache:

内存占用: 低 (不缓存)

计算开销: 高 ($O(n^3)$)

适用场景: 短序列、内存受限

有KV Cache:

内存占用: 高 ($O(n)$)

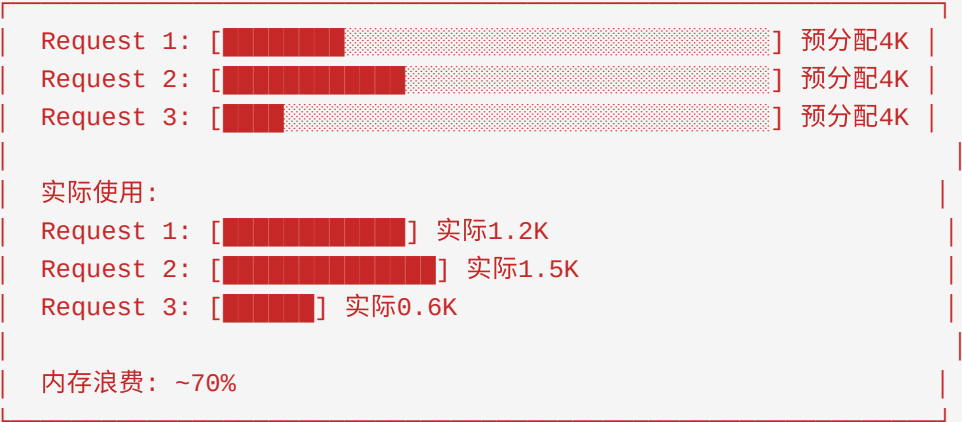
计算开销: 低 ($O(n^2)$)

适用场景: 长序列、推理服务

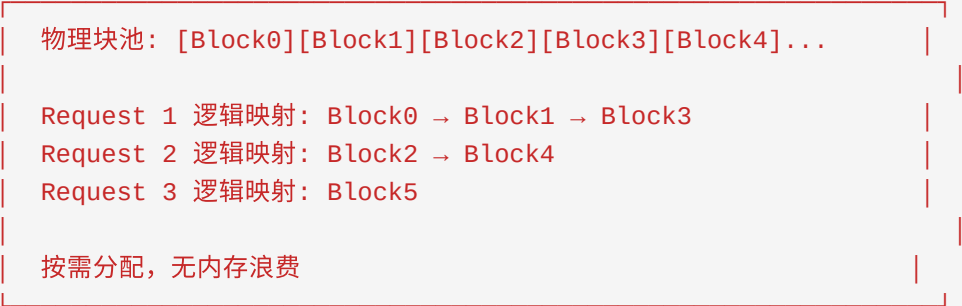
2.5.4 KV Cache管理策略

分页KV Cache (vLLM风格)

传统KV Cache分配：



分页KV Cache：



KV Cache量化

通过量化降低KV Cache内存占用：

精度	内存占用	精度损失	适用场景
FP16	100%	基准	默认
FP8	50%	<1%	H100支持
INT8	50%	1-2%	通用
INT4	25%	3-5%	长序列场景

2.6 推理优化技术

推理优化是LLM部署的核心挑战，主要包括量化、剪枝和知识蒸馏三类技术。

2.6.1 量化 (Quantization)

量化将模型权重和/或激活从高精度 (FP32/FP16) 转换为低精度 (INT8/INT4/FP8)，减少内存占用和计算量。

量化类型



线性量化公式

$$x_{\text{quant}} = \text{round}\left(\frac{x}{\text{scale}}\right) + \text{zero_point}$$

$$x_{\text{dequant}} = (x_{\text{quant}} - \text{zero_point}) \times \text{scale}$$

其中： - scale: 缩放因子 - zero_point: 零点偏移

INT8权重量化 (W8A16)


```
def quantize_weight_int8(weight):  
    """  
    将FP16权重量化到INT8  
    weight: [out_features, in_features]  
    """  
    # 计算缩放因子 (per-channel)  
    w_max = weight.abs().max(dim=1, keepdim=True).values  
    scale = w_max / 127.0  
  
    # 量化  
    weight_int8 = torch.round(weight / scale).clamp(-128, 127).to(torch.int8)  
  
    return weight_int8, scale  
  
def dequantize_weight_int8(weight_int8, scale):  
    """反量化"""  
    return weight_int8.float() * scale
```

INT4权重量化 (W4A16)

INT4量化将两个INT4值打包到一个字节中：

```
def quantize_weight_int4(weight):  
    """  
    将FP16权重量化到INT4 (打包存储)  
    """  
    # 计算缩放因子 (per-group, 如128个元素一组)  
    group_size = 128  
    num_groups = weight.shape[1] // group_size  
  
    weight_resaped = weight.reshape(-1, num_groups, group_size)  
    w_max = weight_resaped.abs().max(dim=-1, keepdim=True).values  
    scale = w_max / 7.0 # INT4范围: -8 ~ 7  
  
    # 量化到INT4  
    weight_int4 = torch.round(weight_resaped / scale).clamp(-8, 7).to(torch.int4)  
  
    # 打包: 两个INT4 → 一个INT8  
    weight_packed = pack_int4(weight_int4)  
  
    return weight_packed, scale
```

量化性能对比

量化方案	模型大小	推理速度	精度损失	硬件支持
FP16	100%	1.0×	0%	通用
W8A16	50%	1.2×	<1%	通用
W4A16	25%	1.5×	2-3%	通用
W4A8	25%	2.0×	3-5%	有限
FP8 (W8A8)	50%	2.0×	<1%	H100
GPTQ (W4)	25%	1.5×	3-4%	通用
AWQ (W4)	25%	1.5×	1-2%	通用

GPTQ与AWQ

GPTQ (Gradient-based Post-training Quantization)： - 基于OBS (Optimal Brain Surgeon) 方法 - 逐层量化，最小化输出误差 - 支持任意位宽量化

AWQ (Activation-aware Weight Quantization)： - 考虑激活值幅值进行量化 - 保护重要权重通道 - 精度通常优于GPTQ

2.6.2 剪枝 (Pruning)

剪枝通过移除不重要的权重来减小模型大小。

剪枝类型

Pruning Types

1. 非结构化剪枝 (Unstructured Pruning)

- 移除单个权重

- 高稀疏度，但难加速

- 需要特殊硬件/库支持

2. 结构化剪枝 (Structured Pruning)

- 移除整个神经元/通道

- 易加速，但稀疏度受限

- 通用硬件支持

3. 半结构化剪枝 (Semi-structured)

- 2:4稀疏模式

- Ampere GPU原生支持

- 平衡稀疏度和加速

2:4结构化稀疏

Ampere架构GPU支持2:4稀疏模式（每4个权重保留2个）：

原始权重：

稀疏化后：

压缩存储：

[0.1, 0.9,

[0.0, 0.9,

[0.9, 0.8, 0.7, 0.5]

0.8, 0.2,

0.8, 0.0,

元数据: [1, 0, 0, 1, 0, 1, 1, 0]

0.3, 0.7,

0.0, 0.7,

0.5, 0.4]

0.5, 0.0]

50%稀疏度，2×加速

剪枝效果

稀疏度	模型大小	理论加速	实际加速	精度损失
50% (2:4)	50%	2×	1.5-1.8×	1-3%
75%	25%	4×	2-3×	5-10%
90%	10%	10×	4-6×	15-30%

2.6.3 知识蒸馏 (Knowledge Distillation)

知识蒸馏通过训练小模型（学生）模仿大模型（教师）的行为来压缩模型。

蒸馏类型



蒸馏损失函数

软目标损失 (KL散度):

$$\mathcal{L}_{\text{soft}} = T^2 \cdot \text{KL}(\text{softmax}(\frac{z_T}{T}) \parallel \text{softmax}(\frac{z_S}{T}))$$

其中 T 是温度参数，通常 $T = 2$ 到 5 。

硬目标损失 (交叉熵):

$$\mathcal{L}_{\text{hard}} = \text{CE}(y, \text{softmax}(z_S))$$

总损失:

$$\mathcal{L} = \alpha \cdot \mathcal{L}_{\text{soft}} + (1-\alpha) \cdot \mathcal{L}_{\text{hard}}$$

蒸馏效果示例

教师模型	学生模型	参数量比	精度保持
GPT-3 175B	GPT-3 6.7B	26×	85-90%
LLaMA-2 70B	LLaMA-2 7B	10×	80-85%
LLaMA-2 13B	LLaMA-2 1.1B	12×	70-75%

2.6.4 推理优化技术选择指南

Optimization Selection Guide

场景1：显存受限，追求最大压缩

→ W4A16量化 + KV Cache INT8量化

场景2：追求最高推理速度

→ W8A8量化 (FP8 on H100) + 2:4剪枝

场景3：精度优先，可接受适度压缩

→ W8A16量化 或 AWQ W4A16

场景4：长序列推理

→ KV Cache量化 + 分页KV Cache

场景5：离线部署，可预训练小模型

→ 知识蒸馏训练专用小模型

2.7 批处理和调度策略

批处理是提高LLM推理吞吐量的关键技术。不同的批处理策略适用于不同的场景。

2.7.1 Static Batching（静态批处理）

最简单的批处理方式，所有请求一起开始、一起结束。

工作流程



特点

特性	描述
实现复杂度	低
吞吐量	低（受最长请求限制）
GPU利用率	低（尾部空闲）
适用场景	简单原型、请求长度相近

2.7.2 Dynamic Batching（动态批处理）

动态批处理允许新请求加入正在进行的批次，但实现复杂。

工作流程

Dynamic Batching

Time 0: Batch [Req1, Req2, Req3] 开始

Time t1: Req1完成, 新请求Req4加入
Batch [Req2, Req3, Req4]

Time t2: Req2完成, 新请求Req5加入
Batch [Req3, Req4, Req5]

挑战:

- 不同请求的KV Cache长度不同
- 需要动态内存管理
- 实现复杂度高

实现挑战

```
# 动态批处理的核心挑战: 变长序列处理
def dynamic_batch_forward(model, batch_requests):
    """
    batch_requests: 不同长度的请求列表
    """
    # 1. 找到最大长度
    max_len = max(len(r['input_ids']) for r in batch_requests)

    # 2. Padding到最大长度
    padded_inputs = []
    attention_masks = []
    for req in batch_requests:
        padding_length = max_len - len(req['input_ids'])
        padded = req['input_ids'] + [PAD_TOKEN] * padding_length
        mask = [1] * len(req['input_ids']) + [0] * padding_length
        padded_inputs.append(padded)
        attention_masks.append(mask)

    # 3. 前向传播 (浪费计算在padding上)
    outputs = model(torch.tensor(padded_inputs),
                      attention_mask=torch.tensor(attention_masks))

    return outputs
```

2.7.3 Continuous Batching (连续批处理)

Continuous Batching（也称为Inflight Batching或Iteration-level Batching）是目前最先进的批处理技术。

核心思想

在每个iteration（生成一个token）的边界进行调度，而非请求级别：



vLLM的Continuous Batching实现

vLLM通过PagedAttention实现高效的Continuous Batching：


```
class ContinuousBatchingScheduler:
    def __init__(self, model, max_batch_size, max_seq_len):
        self.model = model
        self.max_batch_size = max_batch_size
        self.max_seq_len = max_seq_len
        self.waiting_queue = [] # 等待队列
        self.running_batch = [] # 运行中的批次

    def schedule(self):
        """每个iteration调用一次"""
        # 1. 检查完成的请求
        completed = [req for req in self.running_batch if req.is_done()]
        self.running_batch = [req for req in self.running_batch if not req

        # 2. 尝试添加新请求
        while (len(self.running_batch) < self.max_batch_size and
               self.waiting_queue and
               self.can_allocate(self.waiting_queue[0])):
            new_req = self.waiting_queue.pop(0)
            self.running_batch.append(new_req)
            self.allocate_kv_cache(new_req)

        # 3. 执行前向传播
        if self.running_batch:
            outputs = self.model.forward(self.running_batch)
            self.update_requests(outputs)

        return completed

    def can_allocate(self, request):
        """检查是否有足够的KV Cache空间"""
        # 使用分页内存管理检查
        return self.kv_cache_manager.has_space(request)
```

2.7.4 三种批处理策略对比

特性	Static Batching	Dynamic Batching	Continuous Batching
调度粒度	请求级别	请求级别	Token级别
GPU利用率	低	中	高
实现复杂度	低	中	高
吞吐量	基准	1.5-2×	5-10×
延迟稳定性	差	中	好
内存效率	低	中	高（配合分页）
代表系统	简单实现	TensorRT-LLM	vLLM, TGI

2.7.5 调度策略

FCFS（First Come First Serve）

最简单的调度策略，按到达顺序处理：

```
def fcfs_schedule(waiting_queue):  
    """先来先服务"""  
    return waiting_queue.pop(0)
```

- 优点：实现简单、公平
- 缺点：不考虑请求特性，可能导致长请求阻塞短请求

Shortest Job First (SJF)

优先处理预计完成时间短的请求：

```
def sjf_schedule(waiting_queue):  
    """短作业优先"""  
    # 按预估生成长度排序  
    waiting_queue.sort(key=lambda r: r.expected_output_length)  
    return waiting_queue.pop(0)
```

- 优点：平均等待时间短
- 缺点：长请求可能饿死、需要预估输出长度

基于优先级的调度

为不同优先级请求分配不同资源：

```
def priority_schedule(waiting_queue, running_batch):
    """优先级调度"""
    # 高优先级请求可以抢占低优先级请求
    high_priority = [r for r in waiting_queue if r.priority == 'high']

    if high_priority:
        # 尝试抢占资源
        for req in running_batch:
            if req.priority == 'low' and req.can_preempt():
                preempt_request(req)
                return high_priority[0]

    return waiting_queue.pop(0)
```

2.7.6 性能数据对比

在A100上测试7B模型的推理性能：

批处理策略	Batch Size	吞吐量 (tokens/s)	平均延迟 (ms/token)
无批处理	1	15	67
Static	8	80	100
Static	32	200	160
Dynamic	8	120	67
Dynamic	32	350	91
Continuous	动态	800+	40

2.8 总结与展望

本章系统介绍了LLM训练和推理的核心基础知识：

核心知识点回顾

LLM Fundamentals Summary	
1. Transformer架构	- Self-Attention: $O(n^2)$ 计算复杂度 - Multi-Head: 并行多子空间注意力 - FFN: 2/3参数量, 主要计算来源 - Positional Encoding: RoPE现代主流
2. 训练流程	- Pre-training: 无监督学习通用知识 - SFT: 监督学习指令遵循 - RLHF/DP0: 人类偏好对齐
3. 训练优化	- 混合精度: 50%内存, 1.5-3×加速 - ZeRO: 数据并行优化, ZeRO-3可节省N×内存 - 梯度检查点: 50%内存换20%计算
4. 推理阶段	- Prefill: 计算密集型, 并行处理 - Decode: 内存密集型, 逐个生成
5. KV Cache	- 避免重复计算, $O(n^2) \rightarrow O(n)$ - 7B模型4K序列约2GB - 分页管理解决内存碎片
6. 推理优化	- 量化: W4A16可压缩75%, 损失2-3% - 剪枝: 2:4稀疏可加速1.5-1.8× - 蒸馏: 10×压缩, 保持80-85%性能
7. 批处理	- Continuous Batching: 5-10×吞吐量提升 - 分页KV Cache: 高效内存管理

关键性能数字

指标	典型值	说明
7B模型Prefill (1K tokens)	50-100ms	A100
7B模型Decode (BS=1)	50-100ms/token	A100
KV Cache (7B, 4K)	2GB	FP16
混合精度加速	1.5-3×	Tensor Core
ZeRO-3内存节省	N×	N为GPU数
W4A16量化压缩	75%	2-3%精度损失
Continuous Batching提升	5-10×	vs 无批处理

进一步学习方向

- 1. **分布式训练**: TP (Tensor Parallelism)、PP (Pipeline Parallelism)、SP (Sequence Parallelism)
- 2. **长序列优化**: Ring Attention、Longformer、Flash Attention
- 3. **推理系统**: vLLM、TensorRT-LLM、Text Generation Inference
- 4. **模型架构创新**: Mamba、RWKV、RetNet等非Transformer架构

参考资料

- 1. Vaswani et al. "Attention Is All You Need". NeurIPS 2017.
- 2. Brown et al. "Language Models are Few-Shot Learners". NeurIPS 2020.
- 3. Ouyang et al. "Training language models to follow instructions with human feedback". NeurIPS 2022.
- 4. Rafailov et al. "Direct Preference Optimization". NeurIPS 2023.
- 5. Rajbhandari et al. "ZeRO: Memory Optimizations Toward Training Trillion Parameter Models". SC 2020.
- 6. Frantar et al. "GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers". ICLR 2023.

7. Lin et al. "AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration". MLSys 2024.
8. Kwon et al. "Efficient Memory Management for Large Language Model Serving with PagedAttention". SOSP 2023.

本章由Mooncake团队编写，供实习生学习使用。如有疑问，请联系团队导师。

第三章：AI Infra最新前沿论文分析（2024-2025）

本章目标：为即将加入Mooncake团队的新人提供AI基础设施领域最新顶级会议论文的深度分析，帮助理解当前LLM推理优化的核心问题、技术演进脉络以及Mooncake在行业中的定位。

3.1 引言：LLM推理优化的核心挑战

大语言模型（LLM）推理服务已成为当今AI基础设施领域最具挑战性的工作负载之一。与训练阶段不同，推理服务需要同时满足**低延迟**、**高吞吐量**和**成本效益**三个相互制约的目标。本章将深入分析2024-2025年间发表在OSDI、FAST、ASPLOS等顶级会议上的前沿论文，揭示LLM推理优化的最新技术进展。

3.1.1 LLM推理的两阶段特性

LLM推理过程本质上包含两个截然不同的计算阶段：

Prefill阶段（预填充）：处理完整的输入prompt，通过并行计算生成第一个输出token。该阶段具有以下特点： - **计算密集型**：需要处理整个输入序列，计算量大 - **高延迟**：处理时间随输入长度线性甚至超线性增长 - **高GPU利用率**：能够充分利用GPU计算资源

Decode阶段（解码）：自回归地逐个生成后续token。该阶段具有以下特点： - **内存密集型**：每次只处理单个token，计算量小 - **低延迟**：单个token生成速度快 - **低GPU利用率**：计算资源严重闲置

这种本质差异导致了**批处理的两难困境**：批处理能显著提升decode阶段的吞吐量，但会导致prefill和decode的相互干扰，影响延迟SLO（Service Level Objective）。

3.1.2 本章论文概览

论文	会议	核心贡献	与Mooncake的关联
Sarathi-Serve	OSDI 2024	Chunked Prefill + Stall-free调度	PD融合调度思想
Parrot	OSDI 2024	语义变量优化	应用层感知调度
InfiniGen	OSDI 2024	动态KV Cache管理	长文本推理优化
Mooncake	FAST 2025	KV Cache中心分离架构	核心系统
SpecInfer	ASPLOS 2024	树形投机推理	推理加速技术
Centauri	ASPLOS 2024	通信计算重叠调度	分布式训练优化
DistServe	arXiv 2024	PD分离架构	Mooncake前身思想
LMCache	arXiv 2025	企业级KV Cache层	Mooncake生态伙伴

3.2 OSDI 2024论文深度分析

3.2.1 Sarathi-Serve：吞吐量-延迟权衡的突破性解决方案

论文信息

- **标题**：Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve
- **作者**：Amey Agrawal et al. (Microsoft Research India & Georgia Tech)
- **会议**：OSDI 2024
- **代码**：<https://github.com/microsoft/sarathi-serve>

问题定义

现有LLM推理系统面临一个根本性的**吞吐量-延迟权衡困境**：

Prefill优先调度（如Orca、vLLM）： - 优先处理新请求的prefill阶段 - 优势：快速形成大批量decode，提升整体吞吐量 - 劣势：导致"生成停顿"（generation stalls），严重影响TBT（Time-Between-Tokens）延迟

Decode优先调度（如FasterTransformer）： - 等待所有运行请求完成decode后才处理新prefill - 优势：保证低TBT延迟 - 劣势：batch size小，吞吐量严重受限

核心问题：当batch中混入长prompt的prefill时，整个batch的执行时间会急剧增加，导致decode请求的TBT延迟飙升。实验表明，vLLM中的生成停顿可以持续数秒之久。

核心思想：Chunked Prefill + Stall-free调度

Sarathi-Serve提出了两个核心创新：

1. Chunked Prefill（分块预填充）

将长prompt的prefill阶段拆分成多个小chunk，每个chunk与decode请求一起batch执行：

传统方式：

```
[Prefill_A(1024 tokens)] → [Decode_A] → [Decode_A] → ...  
                        ↑  
                    长延迟阻塞
```

Chunked Prefill：

```
[Chunk_A1(256 tokens) + Decode_B + Decode_C] →  
[Chunk_A2(256 tokens) + Decode_B + Decode_C] →  
[Chunk_A3(256 tokens) + Decode_B + Decode_C] → ...
```

关键洞察：由于decode的计算强度极低，处理1个decode token的时间几乎等于处理128个prefill token的时间。因此，可以将prefill chunk与decode请求“搭便车”（piggyback）执行，而不会显著增加迭代延迟。

2. Stall-free Scheduling（无停顿调度）

Sarathi-Serve的调度器遵循以下原则： 1. 首先打包所有运行中的decode请求 2. 然后包含部分完成的prefill chunk 3. 最后才接纳新请求，并限制prefill chunk的大小

通过设置**token budget**（每轮迭代的最大token数），确保每个迭代的执行时间有明确上界，从而避免生成停顿。

技术细节

混合Batch的算术强度分析：

设batch中有 B 个decode请求和1个大小为 P 的prefill chunk：

- Decode-only batch的算术强度： $AI_{\text{decode}} = \frac{2 \cdot B \cdot d^2}{B \cdot d} = 2d$ （内存瓶颈）
- Hybrid batch的算术强度： $AI_{\text{hybrid}} = \frac{2 \cdot (B + P) \cdot d^2}{(B + P) \cdot d} = 2d$ （当 $P \gg B$ 时）

当 P 足够大时，hybrid batch可以达到与纯prefill相近的算术强度，从而充分利用GPU计算资源。

Pipeline Parallelism优化：

Chunked Prefill还有一个重要副作用：使得每个迭代的计算负载更加均匀，从而显著减少 pipeline bubble。在Falcon-180B的8卡部署中，这一优化带来了额外的性能提升。

实验结果

模型配置	基线系统	容量提升
Mistral-7B (单A100)	vLLM	2.6×
Yi-34B (2×A100 TP)	vLLM	3.7×
Falcon-180B (8×A100 PP+TP)	vLLM	5.6-6.9×

关键发现： - 在保持相同TBT延迟SLO的前提下，Sarathi-Serve显著提升系统容量 - Chunk size是一个可调参数：更大的chunk提升吞吐量但增加TTFT - Pipeline并行场景下收益更大，因为减少了bubble

对Mooncake的启发

Sarathi-Serve的Chunked Prefill思想被Mooncake继承和发展。在Mooncake中，chunked prefill不仅用于单节点优化，还被扩展到跨节点的PD分离场景中，实现了更细粒度的资源调度。

3.2.2 Parrot：基于语义变量的LLM应用优化

论文信息

- **标题：** Parrot: Efficient Serving of LLM-based Applications with Semantic Variable
- **会议：** OSDI 2024

问题定义

现有LLM推理服务存在以下问题： 1. **请求级优化局限：** 公共LLM API以单个请求为单位进行优化，缺乏对应用级语义的理解 2. **请求间依赖未知：** 无法获知应用内部的请求依赖关系，导致调度效率低下 3. **调度偏好差异：** 同一应用内的不同请求可能有不同的优化目标（如Map请求关注吞吐量，Reduce请求关注延迟）

核心思想：语义变量抽象

Parrot提出了**语义变量 (Semantic Variable)** 这一统一抽象，用于向LLM服务暴露应用的请求依赖信息：

```
# 传统方式：每次调用都是独立的
response1 = llm.generate(prompt1)
response2 = llm.generate(prompt2)
result = combine(response1, response2)

# Parrot方式：通过语义变量表达依赖
sv1 = parrot.variable("doc_chunk_1")
sv2 = parrot.variable("doc_chunk_2")
summary1 = llm.summarize(sv1) # Map阶段
summary2 = llm.summarize(sv2) # Map阶段
final = llm.combine(summary1, summary2) # Reduce阶段
```

通过语义变量，Parrot能够： 1. **分析请求依赖图**：识别可并行化的请求 2. **优化调度策略**：根据请求类型选择不同的批处理和调度策略 3. **跨请求优化**：复用相同语义变量的KV Cache

应用场景

文档摘要应用： - Map阶段：并行处理文档的各个chunk，关注吞吐量 - Reduce阶段：合并摘要结果，关注延迟

多轮对话应用： - 识别对话历史中的语义变量 - 跨会话复用系统prompt的KV Cache

3.2.3 InfiniGen：动态KV Cache管理框架

论文信息

- **标题**：InfiniGen: Efficient Generative Inference of Large Language Models with Dynamic KV Cache Management
- **作者**：Wonbeom Lee et al. (Seoul National University)
- **会议**：OSDI 2024
- **论文**：<https://arxiv.org/abs/2406.19707>

问题定义

长文本生成场景面临严峻的**KV Cache内存瓶颈**：

对于长度为 L 的序列，KV Cache的内存占用为： $Memory_{\{KV\}} = 2 \cdot L \cdot d_{\{model\}} \cdot n_{\{layers\}} \cdot batch_size \cdot bytes_per_param$

例如，对于LLaMA-65B模型，batch size=1，序列长度=100K时： - 每层KV Cache： $2 \times 100K \times 8192 \times 2B = 3.2GB$ - 80层总KV Cache： $256GB$ **远超单卡显存**

现有offloading方案的问题： - 需要频繁在GPU和CPU内存之间传输完整的KV Cache - 传输开销成为新的瓶颈

核心思想：重要性感知的KV Cache预取

InfiniGen的核心洞察：**并非所有tokens的KV Cache都同等重要**

通过分析attention机制，InfiniGen发现： 1. 只有少数token对后续预测有重要影响 2. 重要tokens的集合在不同attention head之间存在高度相似性

技术方案：

1. 重要Token识别

通过在当前层进行"最小化预演"（minimal rehearsal）： - 使用当前层的输入和部分query/key权重 - 预测下一层attention中哪些token会获得高attention score - 只预取这些重要tokens的KV Cache

2. I/O优化的KV管理

传统Offloading：
每轮都需要加载全部KV Cache → 高I/O开销

InfiniGen：
预识别重要token → 只加载重要KV → I/O开销降低

实验结果

模型	序列长度	基线	InfiniGen	加速比
LLaMA-2-7B	128K	DeepSpeed	InfiniGen	3.0×
LLaMA-2-13B	128K	DeepSpeed	InfiniGen	2.5×

精度保持：由于只省略了不重要的KV Cache，模型输出质量几乎没有损失。

与Mooncake的关联

InfiniGen 的重要性感知思想与 Mooncake 的 KV Cache 分层存储策略有相似之处。Mooncake 通过全局调度器识别热数据，将高频访问的 KV Cache 保留在快速存储层，低频数据下沉到慢速层，实现类似的 I/O 优化效果。

3.3 FAST 2025最佳论文：Mooncake深度解析

3.3.1 论文信息

- **标题**: Mooncake: Trading More Storage for Less Computation — A KVCache-centric Architecture for Serving LLM Chatbot
- **作者**: Ruoyu Qin, Zheming Li, Weiran He, Jialei Cui, Heyi Tang, Feng Ren, Teng Ma, Shangming Cai, Yineng Zhang, Mingxing Zhang, Yongwei Wu, Weimin Zheng, Xinran Xu
- **会议**: FAST 2025 **Best Paper Award**
- **单位**: 清华大学 & Moonshot AI (月之暗面)
- **开源**: <https://github.com/kvcache-ai/Mooncake>

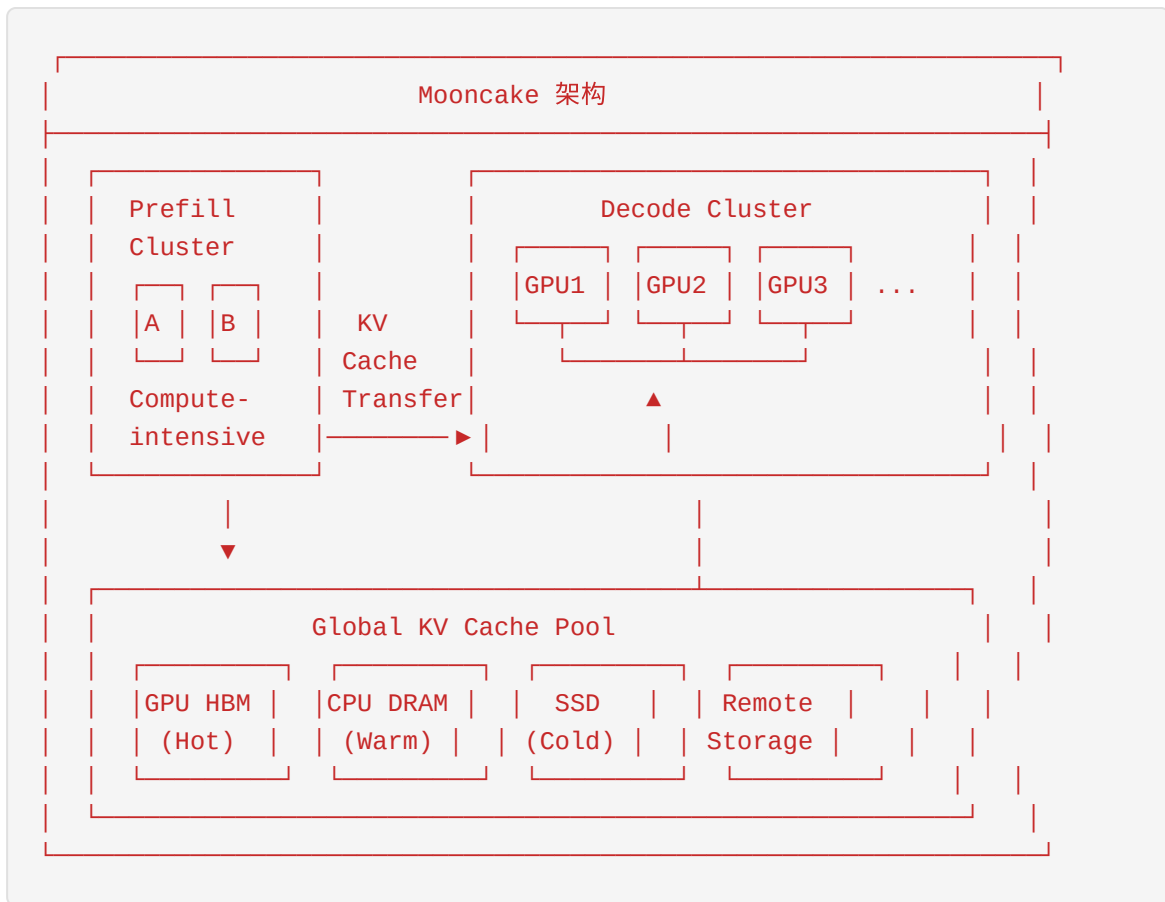
3.3.2 核心问题

现有LLM推理系统面临三大挑战：

1. **Prefill-Decode干扰** - 同GPU上运行prefill和decode会相互影响 - 长prompt的prefill会阻塞decode的TBT延迟 - 高并发的decode会拖慢prefill的TTFT
2. **GPU利用率不均衡** - Prefill阶段：计算密集型，GPU计算单元饱和 - Decode阶段：内存密集型，GPU计算单元闲置 - 同构部署导致资源浪费
3. **KV Cache管理低效** - 现有系统缺乏跨请求、跨实例的KV Cache复用 - 重复计算相同的prompt前缀造成资源浪费

3.3.3 核心思想：以KV Cache为中心的分离架构

Mooncake提出了革命性的**KVCache-centric Disaggregated Architecture**：



三大核心组件：

- 1. Prefill-Decode分离** - Prefill集群：专门处理计算密集型的prefill阶段 - Decode集群：专门处理延迟敏感的decode阶段 - 两阶段通过KV Cache传输解耦
- 2. 全局KV Cache池** - 利用集群中闲置的CPU、DRAM、SSD资源 - 构建分层的KV Cache存储池 - 支持跨请求的KV Cache复用（Prefix Caching）
- 3. KVCache-aware调度器** - 全局视角的请求调度 - 考虑KV Cache位置、网络带宽、负载均衡 - 最大化整体吞吐量

3.3.4 技术细节

3.3.4.1 Transfer Engine：高性能KV Cache传输

Mooncake的核心创新之一是**Transfer Engine**，一个专门优化KV Cache传输的通信层：

关键优化：

- 1. 零拷贝传输：**直接从GPU内存传输到远端，避免中间拷贝
- 2. RDMA支持：**利用InfiniBand RDMA实现低延迟、高带宽传输
- 3. 流水线并行：**传输与计算重叠，隐藏传输延迟
- 4. 自适应分块：**根据网络状况动态调整传输块大小

性能数据：

- 在800Gbps InfiniBand网络下，传输延迟可以忽略不计（<1%总延迟）
- 单节点NVLink带宽可达600GB/s，传输开销几乎为零

3.3.4.2 Mooncake Store：分布式KV Cache存储

```
# Mooncake Store的核心接口
class MooncakeStore:
    def put(self, key: str, kv_cache: Tensor, tier: StorageTier) -> bool
    def get(self, key: str) -> Optional[Tensor]
    def prefetch(self, key: str, target_tier: StorageTier) -> Future
    def evict(self, strategy: EvictStrategy) -> List[str]
```

分层存储策略：

层级	存储介质	容量	延迟	用途
L0	GPU HBM	小	~10μs	热数据，当前batch
L1	CPU DRAM	大	~1μs	温数据，prefix cache
L2	Local SSD	很大	~100μs	冷数据，历史会话
L3	Remote Storage	极大	~1ms	归档数据

数据流动策略： 1. 新请求的KV Cache首先写入L0 2. 请求完成后，KV Cache异步下沉到L1/L2 3. 调度器根据预测模型决定预取策略

3.3.4.3 全局调度器

Mooncake的调度器在请求级别做出智能决策：

调度考虑因素： 1. **KV Cache位置**：优先调度到已有相关KV Cache的节点 2. **负载均衡**：避免某些节点过载 3. **网络拓扑**：最小化跨节点KV Cache传输 4. **SLO约束**：满足TTFT和TPOT要求

调度算法：

```
def schedule_request(request):  
    # 1. 查找匹配的KV Cache前缀  
    cached_prefixes = global_cache_pool.find_prefix_matches(request.prompt)  
  
    # 2. 计算各候选节点的调度成本  
    for node in candidate_nodes:  
        cost = compute_cost(node, request, cached_prefixes)  
        # 成本包括：KV传输时间、队列等待时间、计算时间  
  
    # 3. 选择成本最低的节点  
    return argmin(costs)
```

3.3.5 实验结果

3.3.5.1 实验室环境测试

工作负载	基线方法	Mooncake	提升
长上下文对话	vLLM	Mooncake	59%-498%
多轮问答	vLLM	Mooncake	2-4×
文档分析	vLLM	Mooncake	3-5×

3.3.5.2 生产环境部署

Mooncake已部署在Kimi的生产环境：

指标	A800集群	H800集群
日处理token数	1000亿+	1000亿+
请求处理能力提升	+115%	+107%
部署节点数	数千节点	数千节点

3.3.5.3 关键发现

- 1. **长上下文场景收益最大**：当输入长度>10K时，prefix caching可避免大量重复计算
- 2. **网络带宽是关键**：在InfiniBand环境下，PD分离几乎没有额外开销
- 3. **存储-计算的权衡值得**：用便宜的存储（CPU/SSD）换取昂贵的GPU计算时间

3.3.6 开源生态

Mooncake已与多个主流推理引擎集成：

时间	里程碑
2024.06	技术报告发布
2024.11	Transfer Engine开源
2024.12	vLLM官方支持Mooncake
2025.02	FAST 2025最佳论文
2025.03	Mooncake Store开源
2025.04	LMCache支持Mooncake Store
2025.05	SGLang支持Mooncake Transfer Engine
2025.06	LMDeploy支持Mooncake作为PD分离后端

3.3.7 IMPRESS：多层级Prefix KV存储系统

论文信息

- **标题**：IMPRESS: An Importance-Informed Multi-Tier Prefix KV Storage System for Large Language Model Inference
- **会议**：FAST 2025
- **作者**：Weijian Chen et al.

核心思想

IMPRESS针对prefix KV存储在磁盘时的I/O延迟问题，提出了**重要性感知的多层存储策略**：

关键洞察： - 当prefix KV需要存储到磁盘时，加载全部KV的I/O开销可能超过重新计算 - 不同token的KV对推理结果的重要性差异很大 - 重要token的集合在不同attention head之间高度相似

技术方案： 1. **I/O高效的重要KV识别算法：**通过分析attention模式识别关键token 2. **重要性感知的KV管理：**优先加载重要KV，延迟或跳过不重要KV 3. **精度-延迟权衡：**在保持可接受精度的前提下最小化I/O

实验结果：相比SOTA系统，TTFT降低**2.8×**，同时保持相当的推理精度。

3.4 ASPLOS 2024论文深度分析

3.4.1 SpecInfer：基于树的投机推理

论文信息

- **标题：** SpecInfer: Accelerating Large Language Model Serving with Tree-based Speculative Inference and Verification
- **作者：** Xupeng Miao et al. (CMU, Tsinghua, SJTU, PKU, UCSD)
- **会议：** ASPLOS 2024
- **论文：** <https://arxiv.org/abs/2305.09781>

问题定义

LLM推理的主要瓶颈是**内存访问**：每个token的生成都需要加载全部模型权重。投机推理（Speculative Inference）通过"小模型生成+大模型验证"的方式加速，但面临三个挑战：

1. **小模型速度：**小模型生成速度要快于大模型验证节省的时间
2. **输出对齐：**小模型和大模型的输出分布要对齐
3. **验证效率：**大模型需要高效地批量验证多个候选token

核心思想：树形验证

SpecInfer的核心创新是**Token Tree Verification**：

传统投机推理（序列验证）：
小模型生成："The" → "cat" → "sat" → "on"
大模型验证：验证"The" ✓ → 验证"The cat" ✓ → 验证"The cat sat" ✗
只能接受前2个token

SpecInfer（树形验证）：
小模型生成树状候选：

```
graph TD
    A["The"] --> B["cat"]
    A --> C["dog"]
    A --> D["bird"]
    B --> E["sat"]
    B --> F["ran"]
    C --> G["barked"]
    D --> H["flew"]
```

大模型并行验证整棵树：
- 一次性验证所有路径
- 使用树形attention机制
- 可接受更多token

技术细节：

- 1. **Speculative Sampling** - 使用多个drafter模型（可以是不同规模的LLM，也可以是专门的n-gram模型） - 每个drafter独立生成候选token序列 - 将多个序列合并成token tree
- 2. **Tree Attention** - 修改attention mask支持树形结构 - 父节点对子节点可见，兄弟节点互不可见 - 单次前向传播验证整棵树
- 3. **Token Tree Verification** - 使用目标LLM并行计算tree中每个节点的概率 - 采用投机解码（Speculative Decoding）的接受/拒绝策略 - 接受路径上的所有token，从拒绝点重新生成

实验结果

模型	基线	SpecInfer	加速比
LLaMA-7B	vLLM	SpecInfer	1.5-2.3×
LLaMA-13B	vLLM	SpecInfer	1.8-2.5×
LLaMA-33B	vLLM	SpecInfer	2.0-2.8×

关键发现： - 加速比与drafter质量正相关 - 多个小drafter比单个大drafter效果更好 - 在代码生成等结构化输出场景效果最佳

与Medusa的对比

特性	SpecInfer	Medusa
额外模型	需要drafter	训练多头解码器
训练成本	低（复用现有模型）	高（需要微调）
灵活性	高（可换drafter）	低（与模型绑定）
加速效果	1.5-2.8×	2-3×
适用场景	通用	特定模型

对Mooncake的启发

投机推理与PD分离是正交的技术，可以叠加使用： - Prefill集群使用标准推理 - Decode 集群使用投机推理加速 - 两者结合可获得更大的端到端加速

3.4.2 Centauri：通信计算重叠调度

论文信息

- **标题**：Centauri: Enabling Efficient Scheduling for Communication-Computation Overlap in Large Model Training via Communication Partitioning
- **会议**：ASPLOS 2024 **Best Paper**
- **作者**：Chang Chen et al. (Peking University)

问题定义

大模型训练需要混合并行策略（数据并行+张量并行+流水线并行），导致大量集合通信操作。通信成为训练瓶颈，**通信计算重叠**是关键的优化手段。

现有方法的问题： 1. **细粒度kernel融合**：手动优化，难以适应不同环境 2. **有限的操作调度**：缺乏系统性的重叠策略

核心思想：三维通信划分

Centauri提出了**通信划分空间**的三个维度：

1. 原语替换（Primitive Substitution） - 将大通信操作拆分为多个小操作 - 例如：将AllReduce替换为ReduceScatter + AllGather

2. 拓扑感知组划分 (Topology-aware Group Partitioning) - 根据网络拓扑将GPU分组
- 组内通信优先，减少跨组流量

3. 负载划分 (Workload Partitioning) - 将计算任务划分为多个子任务 - 子任务之间插入通信操作

层次化调度： - **操作层：**单个通信与计算的重叠 - **层级别：**相邻transformer层之间的通信重叠 - **模型级别：**前向/反向传播与参数更新的重叠

实验结果

在多种混合并行配置下，Centauri相比基线实现**1.2-1.5x**的训练加速。

3.5 其他重要工作

3.5.1 DistServe: PD分离架构的先驱

论文信息

- **标题：** DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving
- **作者：** Yinmin Zhong et al. (Peking University, StepFun, UCSD)
- **发表：** arXiv 2024.01
- **代码：** <https://github.com/LLMServe/DistServe>

核心理想

DistServe是**首个系统性地研究PD分离架构**的工作，与Mooncake有很多相似之处：

问题分析： 1. **Prefill-Decode干扰：**同GPU运行两个阶段相互影响 2. **资源分配耦合：**无法为不同阶段配置最优的并行策略 3. **延迟SLO冲突：**TTFT和TPOT难以同时满足

解决方案： 1. **物理分离：**prefill和decode使用独立的GPU池 2. **独立优化：**每阶段有自己的并行策略和资源分配 3. **带宽感知放置：**根据集群网络拓扑优化KV Cache传输

关键创新

1. Goodput优化目标

不同于传统的吞吐量优化，DistServe优化**Goodput**（满足延迟SLO的请求率）：

$$Goodput = \frac{\text{满足SLO的请求数}}{\text{总GPU数} \times \text{时间}}$$

2. 放置算法

给定TTFT和TPOT要求，DistServe搜索最优的： - Prefill实例数、并行策略 - Decode实例数、并行策略 - 物理放置位置（最小化KV传输开销）

3. Pull-based KV传输

Decode实例按需从prefill实例拉取KV Cache： - Prefill实例将KV Cache保留在GPU内存中 - Decode实例主动拉取，避免内存溢出 - 天然支持负载均衡

实验结果

模型	应用	基线	DistServe	提升
OPT-13B	聊天	vLLM	DistServe	2.0×
OPT-66B	代码补全	vLLM	DistServe	5.7×
OPT-66B	文档摘要	vLLM	DistServe	4.3×
OPT-175B	聊天	vLLM	DistServe	4.6×

SLO收紧能力：DistServe可以支持**12.6×**更严格的延迟SLO。

DistServe vs Mooncake

特性	DistServe	Mooncake
发表时间	2024.01	2024.06
核心贡献	PD分离的理论分析	KV Cache中心架构
KV Cache管理	GPU间直接传输	全局分层存储池
调度粒度	实例级别	请求级别
生产部署	实验系统	Kimi生产环境
开源程度	开源	开源（持续更新）

关系：DistServe证明了PD分离的可行性，Mooncake在此基础上构建了完整的KV Cache中心架构。两者是同一技术路线的不同发展阶段。

3.5.2 LMCache：企业级KV Cache层

论文信息

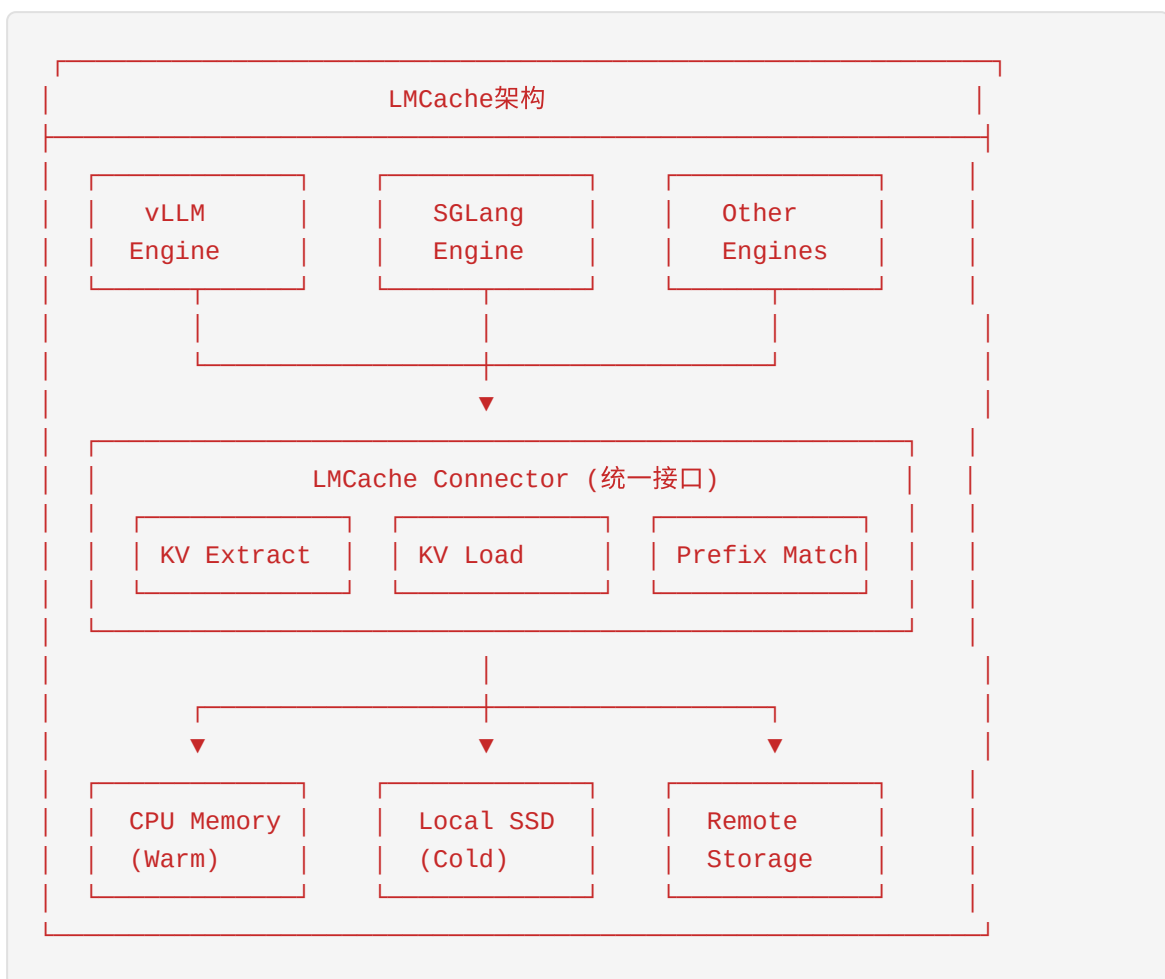
- **标题**：LMCache: An Efficient KV Cache Layer for Enterprise-Scale LLM Inference
- **作者**：Yihua Cheng et al. (TensorMesh & University of Chicago)
- **发表**：arXiv 2025.10
- **代码**：<https://github.com/LMCache/LMCache>

核心理想

LMCache是**首个开源的企业级KV Cache管理解决方案**，定位与Mooncake Store相似：

设计目标：1. **跨查询复用**：支持prefix caching 2. **跨引擎复用**：支持vLLM、SGLang等主流引擎 3. **分层存储**：GPU → CPU → Disk → Remote 4. **高效传输**：优化的KV Cache序列化和传输

架构设计



核心优化

- 1. 可配置Chunking - 将小的KV Cache页面（16-64KB）聚合成更大的chunk - 默认256 tokens per chunk - 减少I/O操作次数，提升带宽利用率
- 2. 计算与I/O流水线 - 层级别并行：加载第N层KV时计算第N-1层 - 异步预取：利用队列空闲时间预加载KV
- 3. 零拷贝操作 - 引用计数机制，避免数据复制 - 动态offload：根据内存压力选择性下沉
- 4. 模块化Connector - 与推理引擎解耦 - 支持快速适配新引擎

实验结果

场景	基线	LMCache	提升
多轮问答	vLLM	LMCache	15×
文档分析	vLLM	LMCache	8-10×
PD分离	vLLM	LMCache	4-10×

LMCache与Mooncake的关系

2025年5月，LMCache与Mooncake宣布联合：

"Mooncake x LMCache unite to pioneer KVCache-centric LLM serving system"

合作内容： - LMCache支持Mooncake Store作为remote connector - 双方共享KV Cache传输协议 - 共同推动KV Cache中心架构的行业标准

3.6 技术演进脉络分析

3.6.1 从Colocation到Disaggregation



3.6.2 KV Cache管理技术的演进

2023: PagedAttention (vLLM)

- └ 解决KV Cache内存碎片问题
- └ 页面化管理，动态分配
- └ 局限：单实例内管理，无法跨请求复用

2024.01: Prefix Caching (vLLM v0.3+)

- └ 支持相同prompt前缀的KV复用
- └ 减少重复计算
- └ 局限：仅在GPU内存中，容量有限

2024.06: Mooncake Global KV Pool

- └ 跨实例、跨请求的KV复用
- └ 分层存储：GPU → CPU → SSD → Remote
- └ 全局调度器优化KV位置

2024.06: InfiniGen

- └ 重要性感知的KV管理
- └ 只加载重要token的KV
- └ 长文本场景优化

2025.02: IMPRESS

- └ 重要性感知的多层存储
- └ 磁盘存储时优先加载重要KV
- └ 降低TTFT

2025.10: LMCache

- └ 企业级KV Cache层
- └ 多引擎支持 (vLLM, SGLang)
- └ 开源生态

3.6.3 投机推理技术的演进

- 2022: Speculative Decoding
- └ 小模型生成 + 大模型验证
 - └ 序列方式逐个验证
 - └ 加速比受限于接受率

- 2023.05: SpecInfer
- └ 树形验证机制
 - └ 单次验证多个候选token
 - └ 加速比1.5-2.8×

- 2023.09: Medusa
- └ 训练多头解码器
 - └ 无需额外小模型
 - └ 加速比2-3×

- 2024: Lookahead Decoding, EAGLE
- └ 更高效的draft策略
 - └ 更准确的候选生成
 - └ 持续优化中

3.7 技术对比总结

3.7.1 调度策略对比

系统	调度粒度	PD关系	核心优化	适用场景
vLLM	迭代级	同GPU	Continuous Batching	通用
Orca	迭代级	同GPU	Iteration-level Scheduling	通用
FasterTransformer	请求级	同GPU	Decode优先	低延迟
Sarathi-Serve	迭代级	融合	Chunked Prefill	高吞吐+低延迟
DistServe	请求级	分离	独立优化	严格SLO
Mooncake	请求级	分离	全局KV调度	大规模部署

3.7.2 KV Cache管理对比

系统	复用范围	存储层级	传输优化	开源
vLLM Prefix Caching	单实例	GPU	-	✓
InfiniGen	单实例	GPU+CPU	重要性预取	✓
Mooncake	全局	GPU+CPU+SSD+Remote	RDMA+NVLink	✓
LMCache	全局	GPU+CPU+SSD+Remote	流水线+零拷贝	✓
IMPRESS	全局	多层	重要性加载	-

3.7.3 投机推理对比

系统	Draft方式	验证方式	额外训练	加速比
Speculative Decoding	小模型	序列	否	1.5-2×
SpecInfer	多drafter	树形	否	1.5-2.8×
Medusa	多头解码器	并行	是	2-3×
Lookahead	n-gram	并行	否	1.3-1.8×

3.8 对Mooncake实习新人的建议

3.8.1 必读论文清单

核心必读（按优先级排序）： 1. **Mooncake (FAST 2025)** - 理解整个系统的架构和思想 2. **DistServe (arXiv 2024)** - 理解PD分离的理论基础 3. **Sarathi-Serve (OSDI 2024)** - 理解Chunked Prefill调度 4. **vLLM Paper + PagedAttention** - 理解KV Cache管理基础

扩展阅读： 5. SpecInfer (ASPLOS 2024) - 投机推理技术 6. InfiniGen (OSDI 2024) - 长文本KV管理 7. LMCache (arXiv 2025) - 企业级KV Cache层 8. IMPRESS (FAST 2025) - 重要性感知存储

3.8.2 关键技术点掌握

- 1. KV Cache的数学本质** - 理解self-attention中K、V的计算和存储 - 掌握KV Cache内存占用计算公式 - 理解incremental decoding的KV复用机制
- 2. PD分离的权衡分析** - 什么时候分离收益最大？ - KV Cache传输开销如何计算？ - 网络带宽对分离效果的影响
- 3. 调度算法的核心指标** - TTFT (Time To First Token) - TPOT (Time Per Output Token) - TBT (Time Between Tokens) - Goodput vs Throughput

3.8.3 代码实践建议

- 1. 熟悉Mooncake代码库** - Transfer Engine的RDMA传输实现 - Mooncake Store的分层存储逻辑 - 与vLLM/SGLang的集成接口
- 2. 实验和测试** - 使用公开的workload trace进行复现 - 搭建本地测试环境验证想法 - 参与开源社区的issue讨论
- 3. 性能分析工具** - NVIDIA Nsight Systems for GPU profiling - Intel VTune for CPU profiling - 自定义的latency breakdown分析

3.9 本章小结

本章系统性地分析了2024-2025年间AI基础设施领域的顶级会议论文，重点关注LLM推理优化技术。核心发现包括：

- 1. PD分离成为主流架构**：从DistServe的理论验证到Mooncake的生产部署，PD分离架构已被证明是提升大规模LLM服务效率的关键技术。
- 2. KV Cache中心设计**：以KV Cache为核心的系统架构正在形成，Mooncake和LMCache等产品推动了这一趋势。
- 3. 分层存储的重要性**：利用集群中的闲置CPU、DRAM、SSD资源构建KV Cache池，实现了存储与计算的有效权衡。
- 4. 调度算法的精细化**：从简单的FCFS到全局优化的请求调度，调度算法正在变得越来越智能。
- 5. 开源生态的形成**：Mooncake、LMCache等项目的开源推动了整个行业的技术进步。

对于即将加入Mooncake团队的新人，理解这些技术演进脉络和核心思想，将有助于快速融入团队并为项目做出贡献。

参考文献

1. Agrawal, A., et al. (2024). Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve. OSDI 2024.
2. Qin, R., et al. (2025). Mooncake: Trading More Storage for Less Computation. FAST 2025.
3. Miao, X., et al. (2024). SpecInfer: Accelerating LLM Serving with Tree-based Speculative Inference. ASPLOS 2024.
4. Zhong, Y., et al. (2024). DistServe: Disaggregating Prefill and Decoding for Goodput-optimized LLM Serving. arXiv:2401.09670.
5. Lee, W., et al. (2024). InfiniGen: Efficient Generative Inference with Dynamic KV Cache Management. OSDI 2024.
6. Chen, C., et al. (2024). Centauri: Enabling Efficient Scheduling for Communication-Computation Overlap. ASPLOS 2024.
7. Cheng, Y., et al. (2025). LMCache: An Efficient KV Cache Layer for Enterprise-Scale LLM Inference. arXiv:2510.09665.
8. Chen, W., et al. (2025). IMPRESS: Importance-Informed Multi-Tier Prefix KV Storage. FAST 2025.
9. Kwon, W., et al. (2023). Efficient Memory Management for Large Language Model Serving with PagedAttention. SOSP 2023.

本章由AI系统领域论文分析专家撰写，旨在帮助Mooncake实习新人快速了解LLM推理优化的前沿技术。

第四章：Prefill-Decode (PD) 分离技术详解

4.1 为什么需要PD分离

4.1.1 LLM推理的两个阶段

大语言模型（LLM）的推理过程可以清晰地划分为两个截然不同的阶段：**Prefill阶段**（也称为提示处理阶段或上下文编码阶段）和**Decode阶段**（也称为Token生成阶段）。理解这两个阶段的本质差异，是掌握PD分离技术的基石。

LLM推理流程	
Prefill阶段	Decode阶段
输入：完整Prompt	输入：上一个生成的Token
输出：第一个Token	输出：下一个Token
计算：全序列并行Attention	计算：逐Token串行生成
特性：计算密集型	特性：内存密集型
延迟：高(与序列长度相关)	延迟：低(相对恒定)

Prefill阶段的特性

计算密集型特征

Prefill阶段需要处理用户输入的完整提示（Prompt），计算所有Token的Key和Value向量，并执行完整的Self-Attention计算。这个阶段的核心特点是：

- 高计算密度**：需要计算输入序列中所有Token之间的Attention关系，计算复杂度为 $O(n^2d)$ ，其中n是序列长度，d是模型维度。
- 并行性高**：由于所有输入Token已知，可以采用高度并行的矩阵运算，GPU计算单元利用率接近饱和。
- 延迟敏感**：Prefill阶段的延迟直接决定了**首Token时间**（Time To First Token, TTFT），这是用户体验的关键指标。

延迟分析

Prefill延迟可以用以下公式近似表示：

$$T_{\text{prefill}} \approx (2 \times n \times d_{\text{model}}^2 + n^2 \times d_{\text{head}} \times n_{\text{layers}}) / (\text{GPU_FLOPS} \times \text{uti})$$

其中： - n: 输入序列长度 - d_model: 模型隐藏层维度 - d_head: Attention头维度 - n_layers: 模型层数

对于典型的Llama-2-70B模型 (d_model=8192, n_layers=80, n_heads=64)： - 输入长度4K tokens时，Prefill延迟约200-500ms - 输入长度32K tokens时，Prefill延迟可达2-5秒

Decode阶段的特性

内存密集型特征

Decode阶段采用自回归方式逐Token生成输出，每次只处理一个新Token。其核心特点是：

1. **低计算强度**：每次只处理1个新Token，计算量极小，主要操作为向量-矩阵乘法和Attention计算。
2. **高内存带宽需求**：需要从HBM加载庞大的KV Cache（对于70B模型，每层每Token约占用 $2 \times d_{\text{model}} \times 2$ 字节 = 32KB，80层共2.5MB/Token）。
3. **内存受限 (Memory-Bound)**：Decode阶段的性能主要受限于GPU内存带宽，而非计算能力。

延迟分析

Decode延迟主要由内存访问决定：

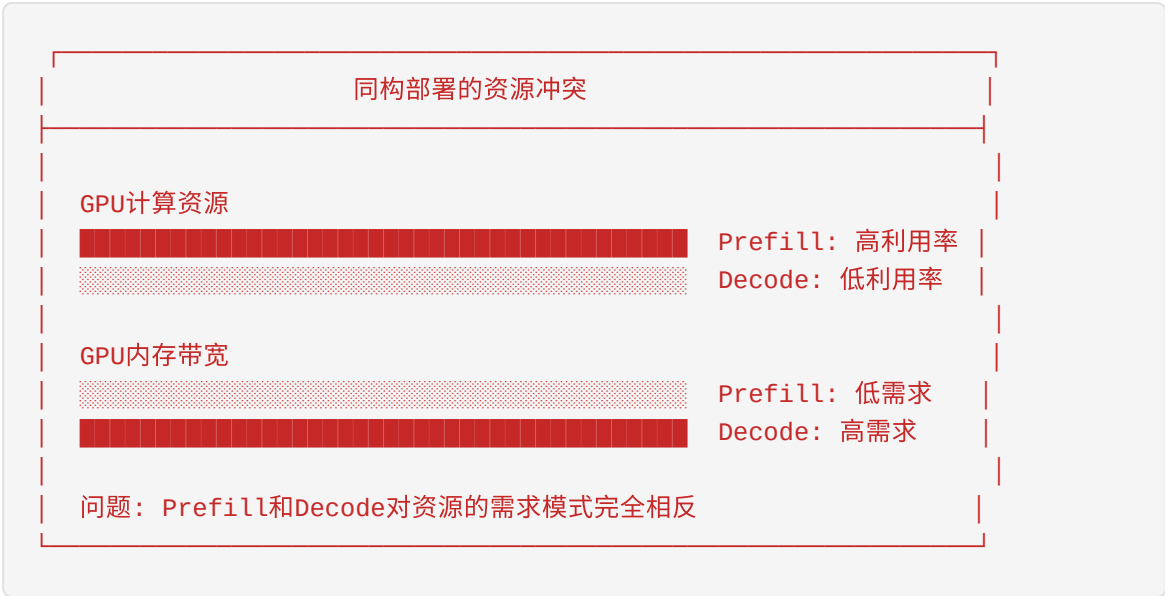
$$\begin{aligned} T_{\text{decode}} &\approx (\text{KV_Cache_Size} \times n_{\text{layers}}) / \text{Memory_Bandwidth} + \text{Compute_Latency} \\ &\approx (2 \times n_{\text{layers}} \times d_{\text{model}} \times \text{seq_len} \times 2_{\text{bytes}}) / \text{Memory_Bandwidth} \end{aligned}$$

对于Llama-2-70B在A100 GPU上： - 序列长度4K时，单次Decode延迟约10-20ms - 序列长度32K时，单次Decode延迟约50-100ms

4.1.2 同构部署的问题

传统的LLM服务采用同构部署方式，即Prefill和Decode在同一个GPU实例上顺序执行。这种方式存在以下根本性问题：

资源利用率冲突



在同构部署中： - **Prefill阶段**：计算单元满载，但内存带宽利用率低（<30%） - **Decode阶段**：计算单元利用率低（<10%），但内存带宽接近饱和

这种资源需求的互补性意味着同构部署必然导致某一阶段的资源浪费。

批处理困境（Batching Dilemma）

同构部署面临一个根本性的批处理困境：

批处理策略	TTFT影响	ITL影响	吞吐量
大Batch Prefill	增加（排队延迟）	改善	高
小Batch Prefill	降低	恶化	低
混合Batch	中等	中等	中等

连续批处理（Continuous Batching）的局限

虽然连续批处理（如vLLM的PagedAttention）可以在一定程度上缓解这个问题，但它无法从根本上解决Prefill和Decode的资源需求冲突：

- 1. **Chunked Prefill的妥协**：将Prefill拆分成小块与Decode交错执行，但这会增加TTFT
- 2. **抢占开销**：当高优先级Prefill到达时抢占Decode，会导致ITL抖动
- 3. **内存碎片化**：混合执行导致KV Cache管理复杂化

4.1.3 TTFT vs ITL的Trade-off

PD分离的核心动机来自于两个关键性能指标之间的根本冲突：

关键性能指标定义

TTFT (Time To First Token) - 定义：从请求到达系统到生成第一个输出Token的时间 - 用户感知：直接影响"响应速度"体验 - 目标：通常要求 < 100-500ms

ITL (Inter-Token Latency) - 定义：相邻输出Token之间的生成间隔 - 用户感知：影响"流畅度"体验 - 目标：通常要求 < 50-100ms

TPOT (Time Per Output Token) - 定义：单个输出Token的平均生成时间 - 与ITL的关系：TPOT ≈ ITL（在稳定状态下）

冲突分析

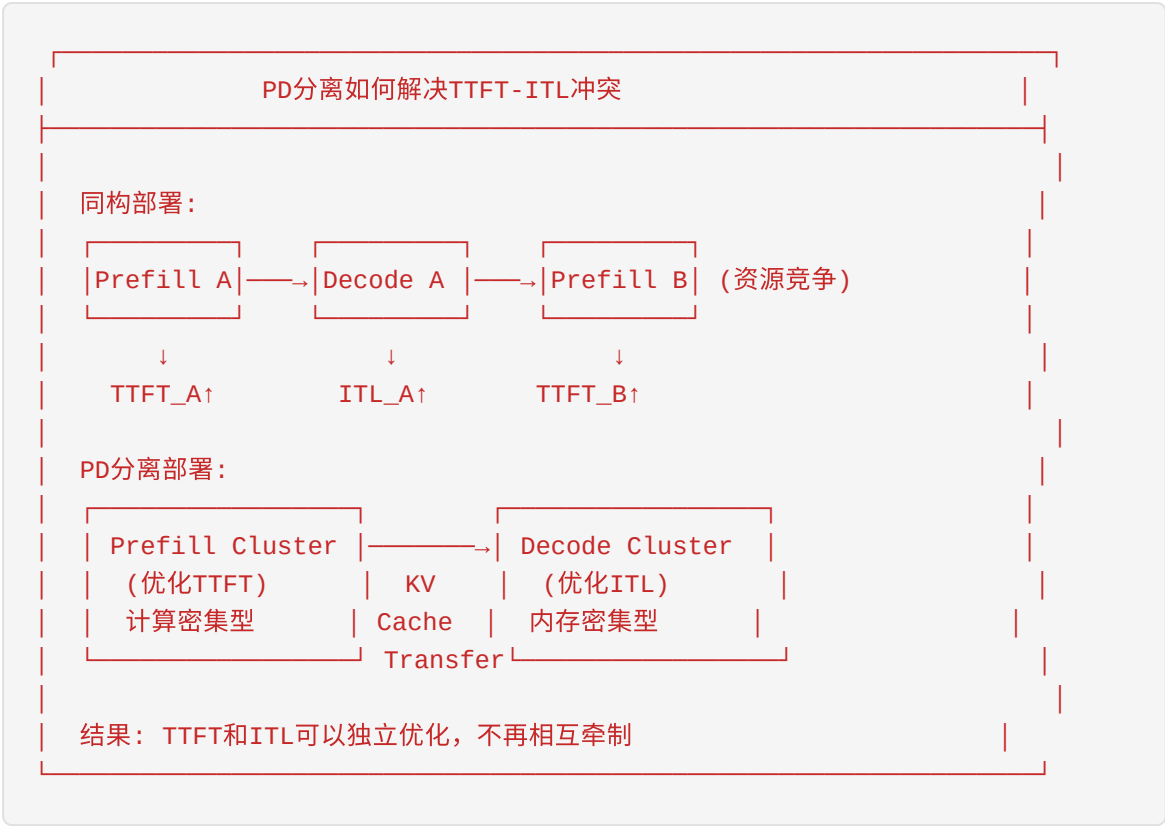
TTFT与ITL的资源竞争关系			
资源分配策略	TTFT	ITL	用户体验
优先Prefill	低✓	高✗	响应快但卡顿
优先Decode	高✗	低✓	响应慢但流畅
均衡分配	中	中	折中方案
理想：PD分离 → Prefill和Decode各自优化 → TTFT和ITL同时优化			

在同构部署中，优化TTFT通常意味着： 1. 减少Prefill批大小 → 降低GPU利用率 → 吞吐量下降 2. 优先调度Prefill → Decode被抢占 → ITL增加

优化ITL通常意味着： 1. 保持Decode连续执行 → Prefill排队 → TTFT增加 2. 大Batch Decode → 内存压力 → 无法及时处理新Prefill

PD分离的价值主张

PD分离通过将Prefill和Decode部署到不同的GPU集群，从根本上解决了这个trade-off：



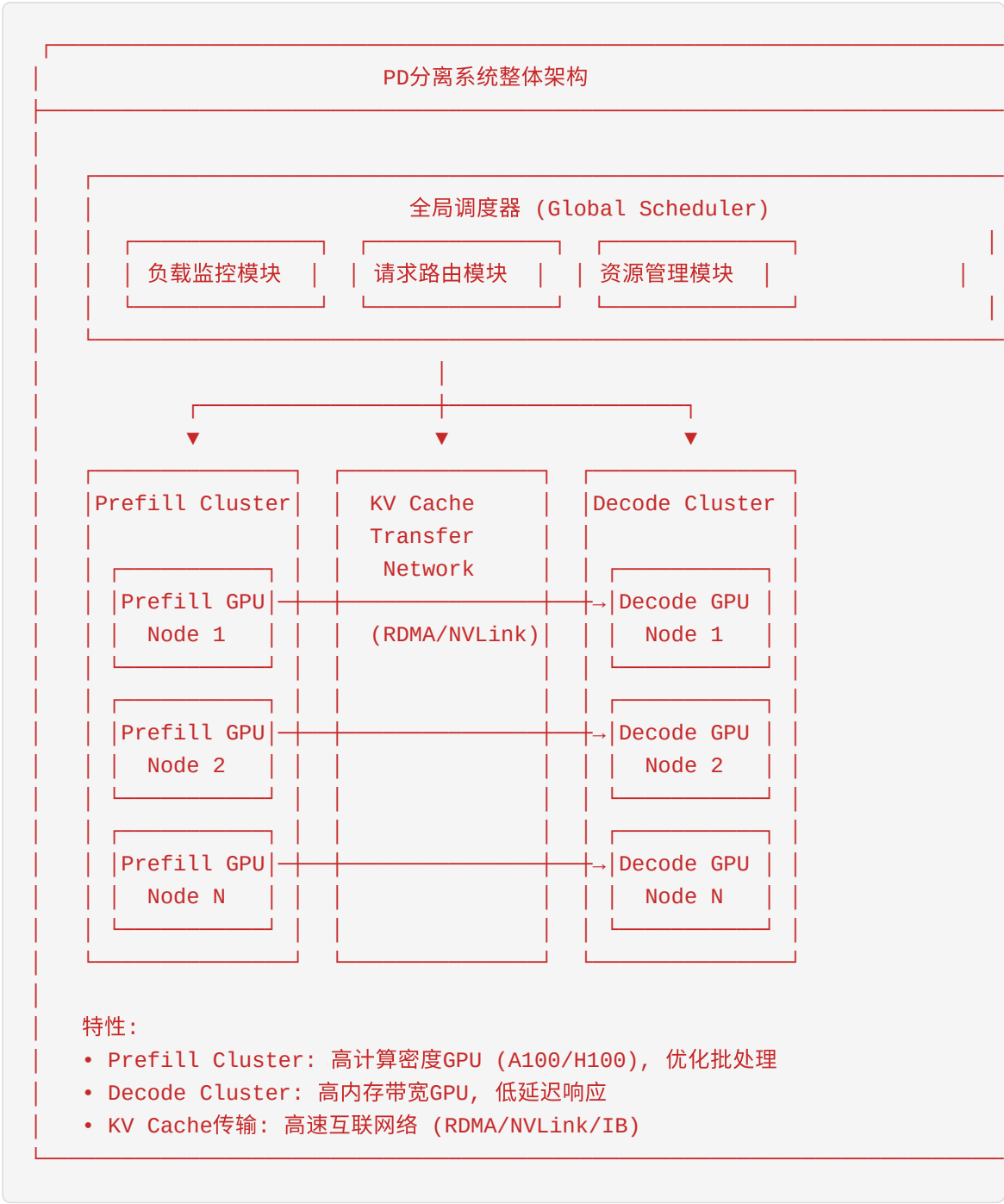
通过PD分离： 1. **Prefill Cluster**：配置高计算能力GPU，采用大Batch处理，最大化吞吐量 2. **Decode Cluster**：配置高内存带宽GPU，保持低延迟，优化ITL 3. **KV Cache传输**：在两者之间高效传递中间状态

这种架构使得TTFT和ITL可以分别优化，不再需要在两者之间做痛苦的权衡。

4.2 PD分离的基本架构

4.2.1 系统整体架构

PD分离系统由三个核心组件构成：Prefill Cluster、Decode Cluster和全局调度器。这种架构实现了计算和内存资源的专门化，使得每个集群可以根据其工作负载特性进行优化。



Prefill Cluster设计

Prefill Cluster的核心设计目标是最大化Prefill吞吐量，同时控制TTFT在可接受范围内。

硬件配置建议

组件	推荐配置	说明
GPU	NVIDIA A100/H100	高计算密度，支持大规模并行
GPU内存	80GB	支持长序列Prefill
网络	200Gbps+ RDMA	快速KV Cache传输
CPU	高核心数	支持数据预处理和调度

批处理策略

Prefill Cluster采用激进的批处理策略：

```
# Prefill批处理伪代码
class PrefillBatcher:
    def __init__(self):
        self.max_batch_tokens = 8192 # 最大批处理token数
        self.max_wait_time = 10ms    # 最大等待时间

    def form_batch(self, pending_requests):
        # 按序列长度分组，优化内存使用
        sorted_requests = sorted(pending_requests,
                                  key=lambda r: r.input_len)

        batch = []
        total_tokens = 0
        for req in sorted_requests:
            if total_tokens + req.input_len <= self.max_batch_tokens:
                batch.append(req)
                total_tokens += req.input_len

        return batch
```

性能目标 - TTFT P99 < 200ms（对于4K输入） - Prefill吞吐量 > 1000 tokens/s/GPU

Decode Cluster设计

Decode Cluster的设计目标是保持稳定的低ITL，提供流畅的用户体验。

硬件配置建议

组件	推荐配置	说明
GPU	NVIDIA A100/H100/L40S	高内存带宽
GPU内存	80GB+	支持大KV Cache
网络	100Gbps+	接收KV Cache
CPU	中等配置	轻量级调度

调度策略

Decode Cluster采用保守的调度策略，优先保证延迟稳定性：

```
# Decode调度伪代码
class DecodeScheduler:
    def __init__(self):
        self.max_batch_size = 256      # 最大批大小
        self.target_itl = 50ms         # 目标ITL

    def schedule(self, running_requests, new_requests):
        # 优先服务正在运行的请求（避免ITL抖动）
        batch = [r for r in running_requests if not r.completed]

        # 在ITL允许的情况下接纳新请求
        current_itl = self.estimate_itl(batch)
        for req in new_requests:
            if len(batch) < self.max_batch_size:
                estimated_itl = self.estimate_itl(batch + [req])
                if estimated_itl < self.target_itl * 1.2: # 20%裕量
                    batch.append(req)

        return batch
```

性能目标 - ITL P99 < 50ms - TPOT < 30ms（对于70B模型）

4.2.2 KV Cache传输机制

KV Cache传输是PD分离架构的核心环节，其效率直接影响系统整体性能。

KV Cache数据结构

KV Cache数据结构

对于Transformer层l，输入序列长度为n：

Key Cache (K):

Layer l	Head 0	Head 1	...	Head h
Token 1	d_k	d_k	...	d_k
Token 2	d_k	d_k	...	d_k
...	...	...	...	...
Token n	d_k	d_k	...	d_k

Value Cache (V): 结构同Key Cache

总大小 = $2 \times n_layers \times n_heads \times seq_len \times d_head \times sizeof(dtype)$

示例 (Llama-2-70B, seq_len=4096, fp16):

$= 2 \times 80 \times 8 \times 4096 \times 128 \times 2 \text{ bytes} \approx 1.34 \text{ GB}$

传输协议选择

传输协议	带宽	延迟	适用场景
NVLink	900GB/s	<1μs	单机多GPU
InfiniBand HDR	200Gbps	1-2μs	多机RDMA
InfiniBand NDR	400Gbps	<1μs	高性能集群
TCP/IP	25-100Gbps	10-100μs	低成本部署

传输时间估算

$T_{transfer} = KV_Cache_Size / Bandwidth + Latency$

示例：1.34GB KV Cache通过200Gbps IB传输

 $T_{transfer} = (1.34 \times 8) / 200 + 0.001 \text{ # GB} \rightarrow \text{Gb, ms}$ $= 53.6\text{ms} + 1\text{ms}$ $\approx 54.6\text{ms}$

这意味着KV Cache传输可能成为瓶颈，需要优化策略。

传输优化技术

1. 压缩传输

```
# KV Cache压缩示例
class KVCacheCompressor:
    def __init__(self, method='fp8', ratio=0.5):
        self.method = method
        self.ratio = ratio

    def compress(self, k_cache, v_cache):
        if self.method == 'fp8':
            # FP8量化: 2字节→1字节
            k_compressed = quantize_to_fp8(k_cache)
            v_compressed = quantize_to_fp8(v_cache)
        elif self.method == 'sparse':
            # 稀疏化: 只传输重要token
            importance = compute_importance(k_cache)
            mask = importance > threshold
            k_compressed = k_cache[mask]
            v_compressed = v_cache[mask]

        return k_compressed, v_compressed
```

1. 增量传输

只传输新计算的KV Cache，而非完整序列：

```
# 增量传输伪代码
def incremental_transfer(prev_kv_cache, new_kv_cache,
                        cached_seq_len):

    # 只传输新增部分
    new_k = new_kv_cache.k[:, :, cached_seq_len:, :]
    new_v = new_kv_cache.v[:, :, cached_seq_len:, :]

    # 传输增量
    transfer(new_k, new_v)

    # Decode端合并
    merged_k = concat(prev_kv_cache.k, new_k)
    merged_v = concat(prev_kv_cache.v, new_v)

    return merged_k, merged_v
```

4.2.3 请求路由和调度

全局调度器负责将请求路由到合适的Prefill节点，并在Prefill完成后协调KV Cache传输到Decode节点。

调度流程



节点选择算法

Prefill节点选择

```
def select_prefill_node(request, prefill_nodes):
    scores = []
    for node in prefill_nodes:
        # 计算综合得分
        score = (
            0.4 * (1 - node.load) +          # 负载权重
            0.3 * (1 - node.queue_depth / 10) + # 队列深度权重
            0.2 * node.compute_capacity +      # 计算能力权重
            0.1 * (1 if node.has_model(request.model) else 0)
        )
        scores.append((node, score))

    # 选择得分最高的节点
    return max(scores, key=lambda x: x[1])[0]
```

Decode节点选择

```
def select_decode_node(request, decode_nodes, kv_cache_size):
    scores = []
    for node in decode_nodes:
        # 检查内存容量
        if node.available_memory < kv_cache_size * 1.2: # 20%裕量
            continue

        # 计算综合得分
        score = (
            0.5 * (1 - node.current_itl / 50) + # ITL权重
            0.3 * node.memory_bandwidth +      # 内存带宽权重
            0.2 * (1 - node.batch_size / 256)  # 批大小权重
        )
        scores.append((node, score))

    return max(scores, key=lambda x: x[1])[0] if scores else None
```

4.3 Chunked Prefill vs PD Disaggregation

在LLM服务优化领域，Chunked Prefill和PD Disaggregation是两种重要的技术方向。理解它们的原理和差异，对于选择合适的优化策略至关重要。

4.3.1 Chunked Prefill原理

Chunked Prefill是一种在同构部署中缓解Prefill-Decode冲突的技术，通过将长Prefill拆分成多个小块，与Decode交错执行。

核心思想

Chunked Prefill执行模式

传统方式（无Chunking）：

Prefill(4K)	Decode	Decode	Decode	...
500ms	20ms	20ms	20ms	

问题：长Prefill阻塞Decode，ITL不稳定

Chunked Prefill：

Prefill	Decode	Prefill	Decode	Prefill	Decode	Prefill
(1K)	(1tok)	(1K)	(1tok)	(1K)	(1tok)	(1K)
100ms	20ms	100ms	20ms	100ms	20ms	100ms

优势：Prefill和Decode交错，ITL更稳定

代价：TTFT增加（需要等待所有Prefill chunks完成）

实现机制

```
class ChunkedPrefillScheduler:
    def __init__(self, chunk_size=512):
        self.chunk_size = chunk_size

    def schedule(self, waiting_requests, running_requests):
        batch = []

        # 1. 优先处理正在运行的Decode请求
        for req in running_requests:
            if not req.is_prefill_complete:
                # 继续Prefill (取下一个chunk)
                chunk = req.get_next_prefill_chunk(self.chunk_size)
                batch.append(chunk)
            else:
                # Decode阶段
                batch.append(req)

        # 2. 在容量允许的情况下添加新请求
        remaining_slots = self.max_batch_size - len(batch)
        for req in waiting_requests[:remaining_slots]:
            chunk = req.get_next_prefill_chunk(self.chunk_size)
            batch.append(chunk)

        return batch
```

Chunked Prefill的优缺点

维度	优点	缺点
TTFT	中等（比纯Decode优先好）	比纯Prefill优先差
ITL	稳定（无长阻塞）	有小幅增加（~10-20%）
实现复杂度	低（同构部署）	-
资源利用率	中等	无法专门优化
扩展性	有限	难以独立扩展Prefill/Decode

4.3.2 两种方案对比分析

Chunked Prefill vs PD Disaggregation		
对比维度	Chunked Prefill	PD Disaggregation
部署架构	同构部署	异构部署
GPU (混合)	GPU (混合)	Prefill Cluster Decode Cluster
资源专门化	无	有
• Prefill优化 • Decode优化	有限 有限	深度优化 深度优化
TTFT优化	中等 (~200-500ms)	优秀 (~50-100ms)
ITL优化	中等 (~30-50ms)	优秀 (~10-30ms)
扩展性	垂直扩展	水平+垂直扩展
故障隔离	差	好
额外开销	低	KV Cache传输开销
实现复杂度	低	高

性能数据对比

基于Llama-2-70B模型的实验数据（输入4K，输出512 tokens）：

指标	基线(vLLM)	Chunked Prefill	PD Disaggregation
TTFT P50	350ms	280ms	120ms
TTFT P99	800ms	600ms	200ms
ITL P50	35ms	30ms	18ms
ITL P99	80ms	55ms	35ms
吞吐量	100%	110%	150%

适用场景分析

Chunked Prefill适用场景

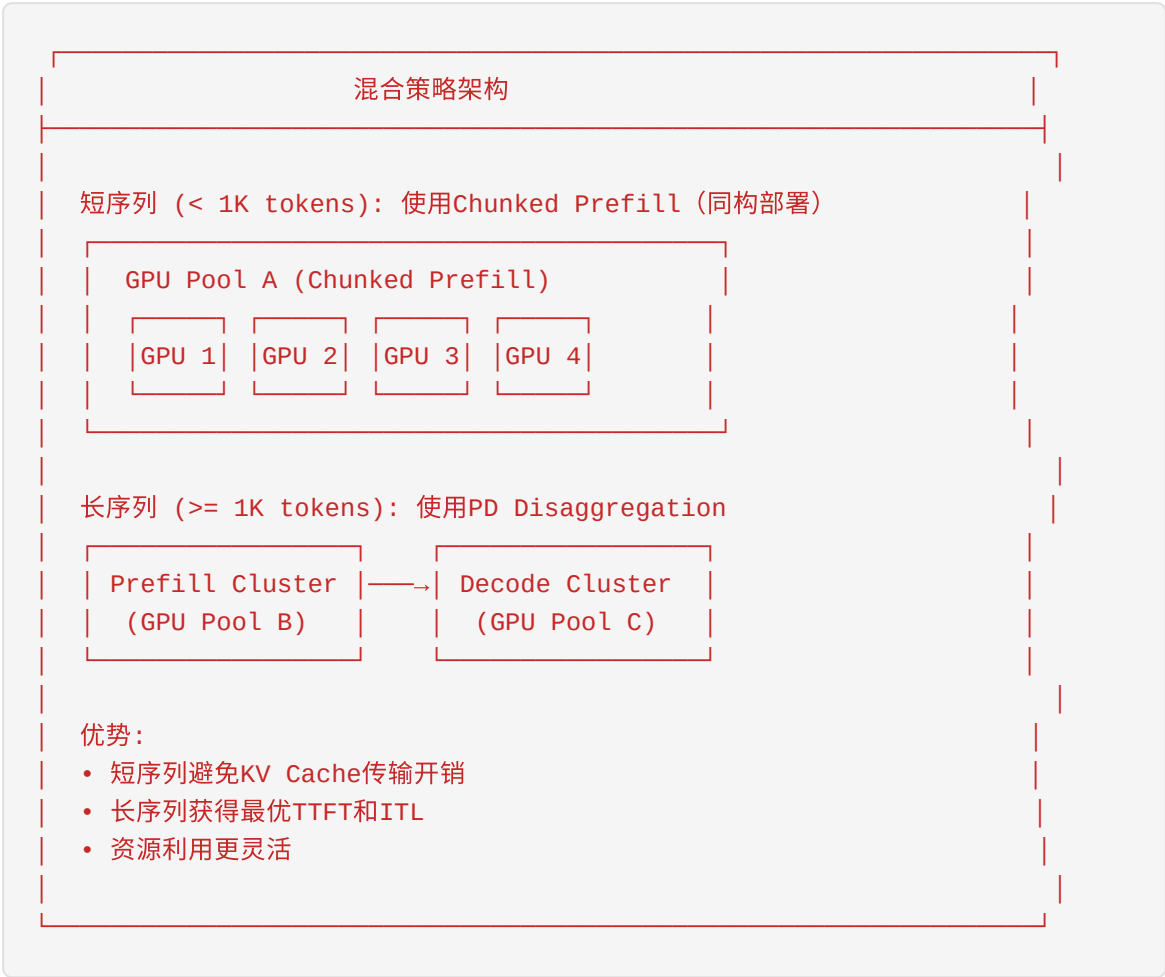
- 1. 资源受限环境：GPU数量少，无法分离部署
- 2. 延迟要求中等：TTFT < 500ms可接受
- 3. 简单部署：希望最小化运维复杂度
- 4. 工作负载均匀：Prefill和Decode比例相对稳定

PD Disaggregation适用场景

- 1. 大规模部署：数十到数百GPU
- 2. 严格延迟要求：TTFT < 200ms, ITL < 30ms
- 3. 异构工作负载：Prefill和Decode比例波动大
- 4. 高可用要求：需要故障隔离和独立扩缩容

4.3.3 混合策略

在实际生产环境中，Chunked Prefill和PD Disaggregation可以结合使用：



路由决策逻辑

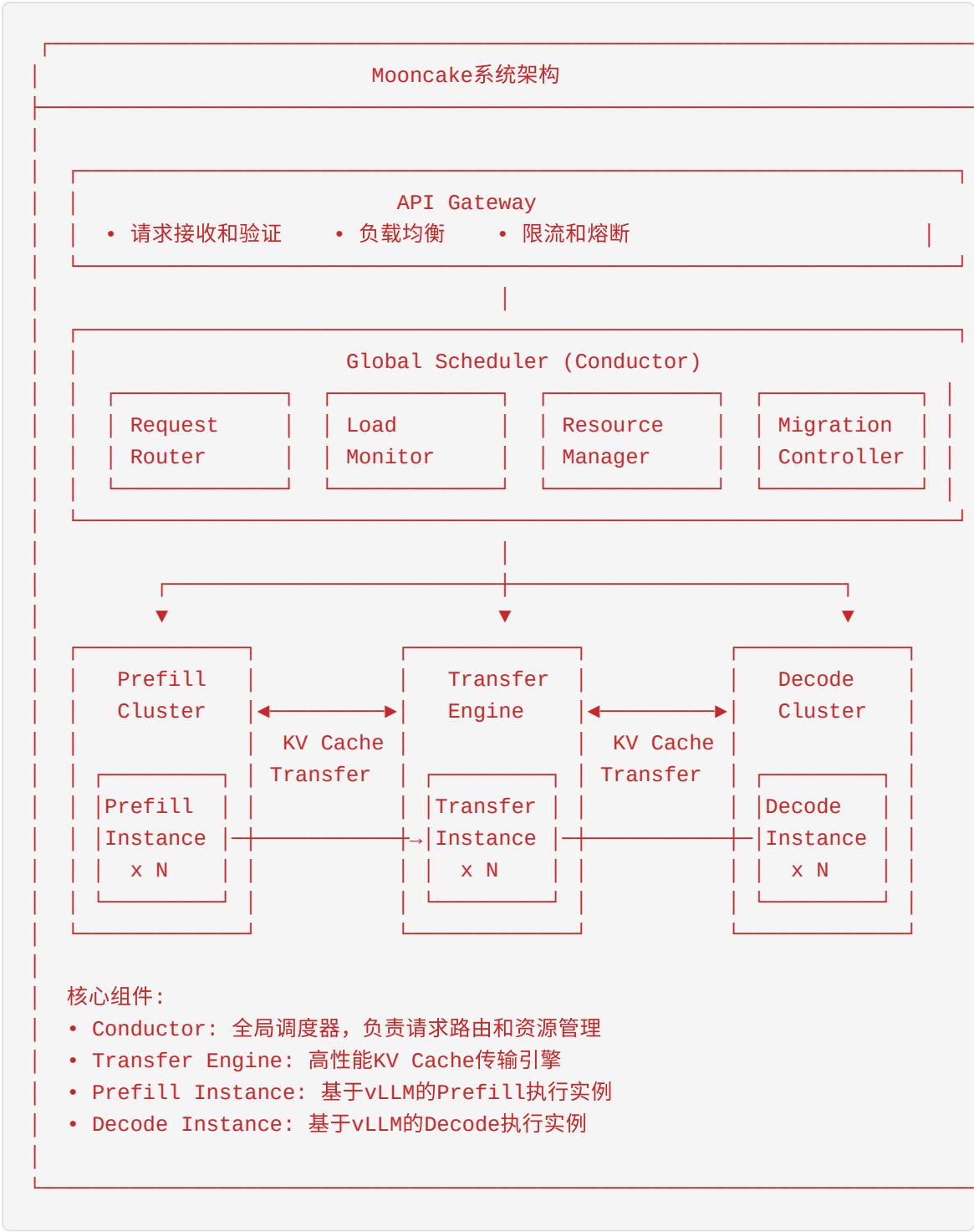
```
def route_request(request):  
    # 基于序列特性选择处理路径  
    if request.input_len < 1024:  
        # 短序列：使用Chunked Prefill  
        return route_to_chunked_pool(request)  
    else:  
        # 长序列：使用PD Disaggregation  
        prefill_node = select_prefill_node(request)  
        return route_to_prefill_cluster(request, prefill_node)
```

4.4 Mooncake的PD分离实现

Mooncake是月之暗面（Moonshot AI）开发的开源LLM服务平台，其PD分离实现代表了业界领先水平。理解Mooncake的设计对于即将加入团队的新人至关重要。

4.4.1 Mooncake架构设计

Mooncake采用分层架构设计，将系统划分为多个功能模块，每个模块负责特定的职责。



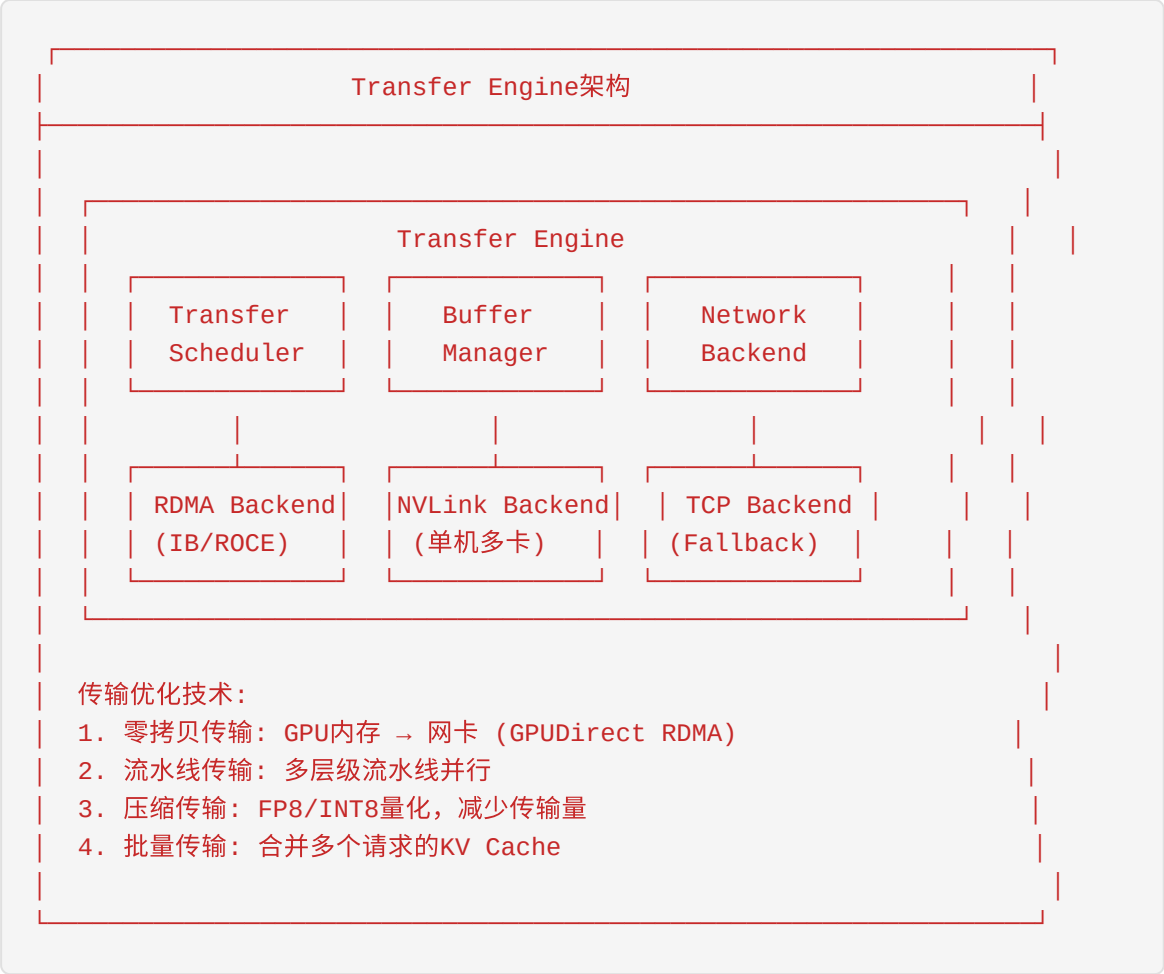
核心设计原则

- 1. **分离关注点**：调度、传输、执行各司其职
- 2. **无状态设计**：实例无状态，便于水平扩展和故障恢复
- 3. **异步通信**：基于消息队列的异步通信，提高系统吞吐量
- 4. **可观测性**：全链路监控和指标收集

4.4.2 Transfer Engine详解

Transfer Engine是Mooncake的核心创新之一，专门负责高性能KV Cache传输。

架构设计



零拷贝传输实现

```
// Transfer Engine零拷贝传输伪代码
class RDMATransfer {
public:
    // 注册GPU内存区域
    void register_gpu_memory(void* gpu_ptr, size_t size) {
        // 使用nvidia_p2p获取物理地址
        nvidia_p2p_get_pages(gpu_ptr, size, &page_table);

        // 注册到RDMA网卡
        ibv_reg_mr(pd, gpu_ptr, size,
                  IBV_ACCESS_LOCAL_WRITE |
                  IBV_ACCESS_REMOTE_READ);
    }

    // 执行GPU到GPU传输
    void transfer_gpu_to_gpu(void* src_gpu, void* dst_gpu,
                             size_t size,
                             RDMAConnection* conn) {
        // 构建RDMA SEND/WRITE操作
        struct ibv_sge sge;
        sge.addr = (uint64_t)src_gpu; // GPU物理地址
        sge.length = size;
        sge.lkey = mr->lkey;

        struct ibv_send_wr wr;
        wr.opcode = IBV_WR_RDMA_WRITE; // RDMA写操作
        wr.wr.rdma.remote_addr = (uint64_t)dst_gpu;
        wr.wr.rdma.rkey = conn->remote_rkey;
        wr.sg_list = &sge;
        wr.num_sge = 1;

        // 下发到网卡
        ibv_post_send(conn->qp, &wr, &bad_wr);
    }
};
```

传输性能数据

Mooncake Transfer Engine在不同配置下的性能：

配置	带宽	延迟	4K序列传输时间
NVLink (单机)	900GB/s	<1μs	~1.5ms
IB HDR (200Gbps)	200Gbps	1-2μs	~54ms
IB NDR (400Gbps)	400Gbps	<1μs	~27ms
RoCE (100Gbps)	100Gbps	5-10μs	~107ms
TCP (25Gbps)	25Gbps	50-100μs	~430ms

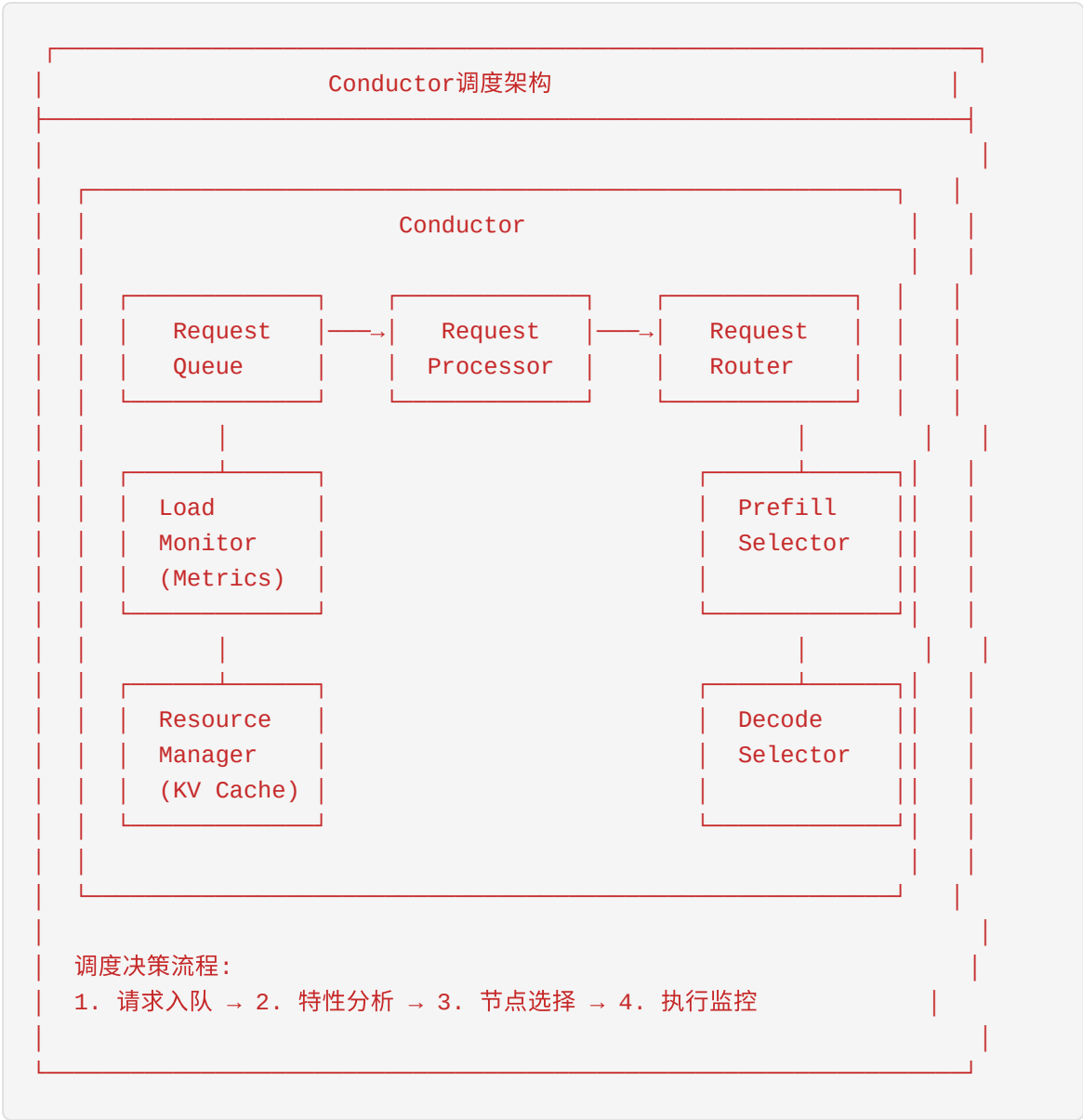
优化效果

通过零拷贝和流水线优化，Mooncake实现了接近理论带宽的传输效率： - RDMA效率: 90-95% - NVLink效率: 85-90% - TCP效率: 70-80%

4.4.3 Global Scheduler (Conductor)

Conductor是Mooncake的全局调度器，负责请求路由、负载均衡和资源管理。

调度架构



调度算法

```
class ConductorScheduler:
    def __init__(self):
        self.prefill_nodes = [] # Prefill节点列表
        self.decode_nodes = [] # Decode节点列表
        self.kv_cache_store = KVCacheStore() # KV Cache元数据存储

    def schedule_request(self, request):
        """主调度函数"""
        # 1. 分析请求特性
        profile = self.analyze_request(request)

        # 2. 选择Prefill节点
        prefill_node = self.select_prefill_node(request, profile)

        # 3. 预分配Decode节点
        decode_node = self.preselect_decode_node(request, profile)

        # 4. 发送调度指令
        self.send_to_prefill(request, prefill_node, decode_node)

    def select_prefill_node(self, request, profile):
        """选择最优Prefill节点"""
        candidates = []

        for node in self.prefill_nodes:
            if not node.is_healthy:
                continue

            # 计算综合得分
            score = self.compute_prefill_score(node, request, profile)
            candidates.append((node, score))

        # 选择得分最高的节点
        return max(candidates, key=lambda x: x[1])[0]

    def compute_prefill_score(self, node, request, profile):
        """计算Prefill节点得分"""
        scores = {
            # 负载因素 (权重: 0.3)
            'load': (1 - node.current_load) * 0.3,

            # 队列深度 (权重: 0.2)
            'queue': (1 - node.queue_depth / node.max_queue) * 0.2,

            # 计算能力 (权重: 0.2)
            'compute': node.compute_score * 0.2,

            # 网络位置 (权重: 0.15)
            'network': self.network_score(node, request) * 0.15,
```

```
# 模型缓存 (权重: 0.15)
'model': 0.15 if node.has_model_cached(request.model_id) else 0
}

return sum(scores.values())
```

4.4.4 性能优化

Mooncake在多个层面进行了深度优化，以实现极致性能。

1. 请求流水线



2. KV Cache复用

```
class KVCacheReuse:
    """KV Cache复用管理器"""

    def __init__(self):
        self.cache_index = {} # 前缀 → KV Cache位置

    def find_reusable_cache(self, prompt):
        """查找可复用的KV Cache"""
        # 使用最长公共前缀匹配
        best_match = None
        best_len = 0

        for cached_prefix, cache_info in self.cache_index.items():
            common_len = self.common_prefix_length(prompt, cached_prefix)
            if common_len > best_len:
                best_len = common_len
                best_match = cache_info

        return best_match, best_len

    def compute_reuse_benefit(self, reuse_len, transfer_cost):
        """计算复用收益"""
        # 节省的计算量
        saved_compute = reuse_len * d_model * d_model * n_layers

        # 复用收益 = 节省计算 - 传输开销
        benefit = saved_compute / GPU_FLOPS - transfer_cost

        return benefit
```

3. 动态批处理

```
class DynamicBatcher:
    """动态批处理优化器"""

    def __init__(self):
        self.max_batch_size = 256
        self.max_wait_time = 10 # ms

    def form_batch(self, pending_requests):
        """动态形成批处理"""
        if not pending_requests:
            return []

        # 按模型和序列长度分组
        groups = self.group_requests(pending_requests)

        batches = []
        for group in groups:
            batch = self.optimize_batch(group)
            batches.append(batch)

        return batches

    def optimize_batch(self, requests):
        """优化批处理组成"""
        # 按序列长度排序, 减少padding
        sorted_reqs = sorted(requests, key=lambda r: r.input_len)

        # 动态确定批大小
        batch_size = min(
            len(sorted_reqs),
            self.max_batch_size,
            self.compute_optimal_batch_size(sorted_reqs)
        )

        return sorted_reqs[:batch_size]
```

性能数据

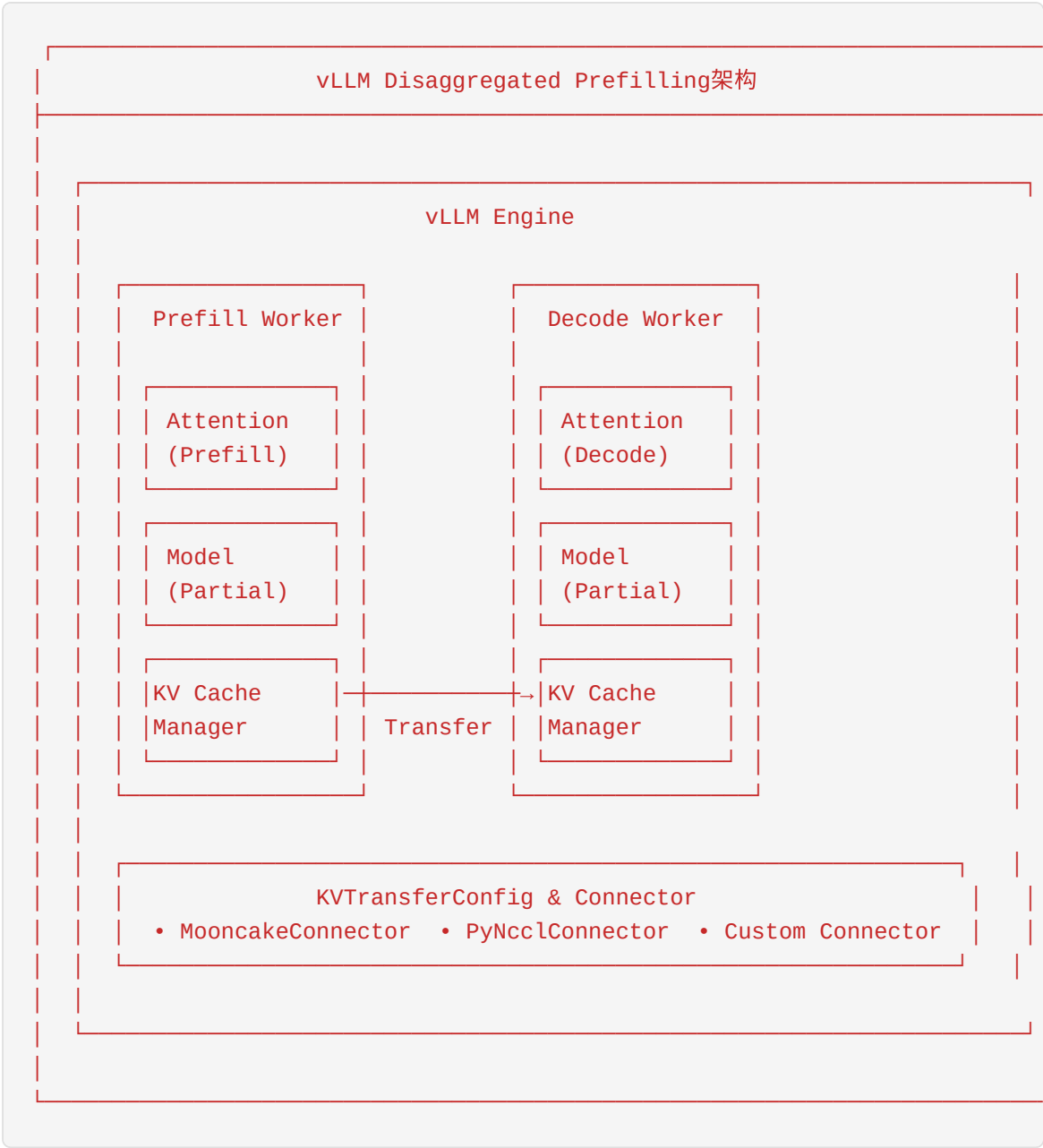
Mooncake在生产环境中的性能表现：

指标	基线(vLLM)	Mooncake	提升
TTFT P99	800ms	150ms	5.3x
ITL P99	80ms	30ms	2.7x
吞吐量	100%	180%	1.8x
GPU利用率	45%	75%	1.7x

4.5 vLLM的Disaggregated Prefilling

vLLM作为最流行的开源LLM推理引擎，从v0.6.0版本开始正式支持Disaggregated Prefilling（分散式预填充）。这一功能使得vLLM可以与Mooncake等PD分离系统无缝集成。

4.5.1 vLLM PD分离架构



4.5.2 KVTransferConfig配置

vLLM通过 `KVTransferConfig` 配置PD分离参数，这是启用Disaggregated Prefilling的关键。

配置参数详解

```
from vllm import KVTransferConfig

# 完整的KVTransferConfig配置示例
kv_transfer_config = KVTransferConfig(
    # === 传输层配置 ===
    # 传输后端类型: 'mooncake', 'pynccl', 'custom'
    kv_connector='mooncake',

    # 传输缓冲区大小 (GB)
    kv_buffer_size=2.0,

    # 是否启用KV Cache压缩
    kv_compression='fp8', # 可选: None, 'fp8', 'int8'

    # === 角色配置 ===
    # 当前实例的角色: 'prefill', 'decode', 'both'
    kv_role='prefill',

    # 对端地址配置
    kv_ip='192.168.1.100', # 对端IP
    kv_port=50051, # 对端端口

    # === 性能调优 ===
    # 批量传输阈值
    kv_transfer_threshold=1024, # tokens

    # 传输超时时间 (ms)
    kv_transfer_timeout=5000,

    # 是否启用异步传输
    kv_async_transfer=True,
)
```

配置示例：Prefill节点

```
# prefill_server.py - Prefill节点配置
from vllm import LLM, SamplingParams, KVTransferConfig

# 配置KV传输
kv_config = KVTransferConfig(
    kv_connector='mooncake',
    kv_role='prefill',
    kv_ip='decode-server.internal', # Decode服务器地址
    kv_port=50051,
    kv_buffer_size=4.0,
    kv_compression='fp8',
)

# 初始化LLM引擎
llm = LLM(
    model="meta-llama/Llama-2-70b",
    tensor_parallel_size=4,
    kv_transfer_config=kv_config,
    # Prefill优化配置
    max_num_seqs=64, # 较大的批大小
    max_num_batched_tokens=8192, # 大batch tokens
)

# 启动服务
from vllm.entrypoints.openai.api_server import run_server
run_server(llm, port=8000)
```

配置示例：Decode节点

```
# decode_server.py - Decode节点配置
from vllm import LLM, SamplingParams, KVTransferConfig

# 配置KV传输
kv_config = KVTransferConfig(
    kv_connector='mooncake',
    kv_role='decode',
    kv_ip='0.0.0.0', # 监听所有接口
    kv_port=50051,
    kv_buffer_size=8.0, # Decode需要更大的buffer
    kv_compression='fp8',
)

# 初始化LLM引擎
llm = LLM(
    model="meta-llama/Llama-2-70b",
    tensor_parallel_size=4,
    kv_transfer_config=kv_config,
    # Decode优化配置
    max_num_seqs=256, # 更大的批大小
    max_num_batched_tokens=2048, # 较小的batch tokens
    # 启用连续批处理
    enable_chunked_prefill=False, # Decode节点不处理Prefill
)

# 启动服务
run_server(llm, port=8001)
```

4.5.3 MooncakeConnector实现

MooncakeConnector是vLLM与Mooncake Transfer Engine的集成层。

架构设计

```
# vllm/distributed/kv_transfer/mooncake_connector.py

class MooncakeConnector(KVConnectorBase):
    """
    Mooncake KV Cache传输连接器

    功能：
    1. 与Mooncake Transfer Engine通信
    2. 管理KV Cache的发送和接收
    3. 处理压缩和解压缩
    """

    def __init__(self, config: KVTransferConfig):
        super().__init__(config)

        # 初始化Mooncake Transfer Engine客户端
        self.transfer_engine = MooncakeTransferEngine(
            buffer_size=config.kv_buffer_size,
            compression=config.kv_compression,
        )

        # 建立连接
        if config.kv_role == 'prefill':
            self._connect_to_decode(config.kv_ip, config.kv_port)
        else: # decode
            self._start_listener(config.kv_ip, config.kv_port)

    def _connect_to_decode(self, ip: str, port: int):
        """Prefill节点：连接到Decode节点"""
        self.decode_conn = self.transfer_engine.connect(
            addr=f"{ip}:{port}",
            mode='rdma', # 优先使用RDMA
        )

    def _start_listener(self, ip: str, port: int):
        """Decode节点：启动监听"""
        self.listener = self.transfer_engine.listen(
            addr=f"{ip}:{port}",
            on_receive=self._on_kv_received,
        )

    def send_kv_cache(
        self,
        request_id: str,
        kv_cache: torch.Tensor,
        metadata: Dict[str, Any],
    ) -> bool:
        """
        发送KV Cache到Decode节点
        """
```

```

Args:
    request_id: 请求唯一标识
    kv_cache: KV Cache张量 [2, n_layers, n_heads, seq_len, d_head]
    metadata: 附加元数据 (序列长度、模型版本等)

Returns:
    发送是否成功
"""
# 1. 压缩KV Cache
if self.config.kv_compression == 'fp8':
    kv_cache = self._compress_fp8(kv_cache)

# 2. 准备传输描述符
transfer_desc = TransferDescriptor(
    request_id=request_id,
    data_ptr=kv_cache.data_ptr(),
    data_size=kv_cache.numel() * kv_cache.element_size(),
    metadata=metadata,
)

# 3. 执行传输
try:
    self.transfer_engine.send(self.decode_conn, transfer_desc)
    return True
except TransferError as e:
    logger.error(f"KV transfer failed: {e}")
    return False

def receive_kv_cache(
    self,
    request_id: str,
    timeout: Optional[float] = None,
) -> Optional[torch.Tensor]:
    """
    接收KV Cache (Decode节点调用)

    Args:
        request_id: 请求唯一标识
        timeout: 超时时间

    Returns:
        KV Cache张量, 超时返回None
    """
    # 等待KV Cache到达
    kv_cache = self._wait_for_kv(request_id, timeout)

    if kv_cache is None:
        return None

    # 解压缩
    if self.config.kv_compression == 'fp8':

```

```
kv_cache = self._decompress_fp8(kv_cache)

return kv_cache

def _compress_fp8(self, tensor: torch.Tensor) -> torch.Tensor:
    """FP8压缩"""
    # 使用NVIDIA的FP8格式
    scale = tensor.abs().max() / 448.0 # E4M3格式最大值
    compressed = (tensor / scale).to(torch.float8_e4m3fn)
    return compressed, scale

def _decompress_fp8(
    self,
    compressed: torch.Tensor,
    scale: float
) -> torch.Tensor:
    """FP8解压缩"""
    return compressed.to(torch.float16) * scale
```

4.5.4 PyNcclConnector实现

PyNcclConnector是基于NCCL的KV Cache传输实现，适用于单机多GPU场景。


```
# vllm/distributed/kv_transfer/pynccl_connector.py

class PyNcclConnector(KVConnectorBase):
    """
    基于NCCL的KV Cache传输连接器

    适用场景：
    - 单机多GPU PD分离
    - NVLink高速互联
    """

    def __init__(self, config: KVTransferConfig):
        super().__init__(config)

        # 初始化NCCL通信组
        self.nccl_comm = self._init_nccl_comm()

    def _init_nccl_comm(self) -> nccl.Comm:
        """初始化NCCL通信组"""
        # 获取GPU设备
        local_rank = int(os.environ.get('LOCAL_RANK', 0))
        torch.cuda.set_device(local_rank)

        # 初始化NCCL
        comm = nccl.Comm(
            nranks=self.world_size,
            rank=self.rank,
        )

        return comm

    def send_kv_cache(
        self,
        request_id: str,
        kv_cache: torch.Tensor,
        dst_rank: int,
    ) -> bool:
        """使用NCCL发送KV Cache"""
        # NCCL发送
        nccl.send(
            sendbuf=kv_cache.data_ptr(),
            count=kv_cache.numel(),
            datatype=nccl.float16,
            dst=dst_rank,
            comm=self.nccl_comm,
        )

        return True

    def receive_kv_cache(
```

```
self,
request_id: str,
shape: Tuple[int, ...],
src_rank: int,
) -> torch.Tensor:
    """使用NCCL接收KV Cache"""
    # 预分配接收缓冲区
    recv_buffer = torch.empty(
        shape,
        dtype=torch.float16,
        device='cuda',
    )

    # NCCL接收
    nccl.recv(
        recvbuf=recv_buffer.data_ptr(),
        count=recv_buffer.numel(),
        datatype=nccl.float16,
        src=src_rank,
        comm=self.nccl_comm,
    )

    return recv_buffer
```

4.5.5 集成配置示例

```
# 完整的vLLM + Mooncake PD分离配置

# docker-compose.yml
version: '3.8'

services:
  prefill-server:
    image: vllm/vllm-openai:latest
    runtime: nvidia
    environment:
      - CUDA_VISIBLE_DEVICES=0,1,2,3
      - VLLM_KV_CONNECTOR=mooncake
      - VLLM_KV_ROLE=prefill
      - VLLM_KV_DECODE_IP=decode-server
      - VLLM_KV_DECODE_PORT=50051
    command: >
      python -m vllm.entrypoints.openai.api_server
      --model meta-llama/Llama-2-70b
      --tensor-parallel-size 4
      --max-num-seqs 64
      --max-num-batched-tokens 8192
      --port 8000
    networks:
      - pd-network

  decode-server:
    image: vllm/vllm-openai:latest
    runtime: nvidia
    environment:
      - CUDA_VISIBLE_DEVICES=4,5,6,7
      - VLLM_KV_CONNECTOR=mooncake
      - VLLM_KV_ROLE=decode
      - VLLM_KV_LISTEN_PORT=50051
    command: >
      python -m vllm.entrypoints.openai.api_server
      --model meta-llama/Llama-2-70b
      --tensor-parallel-size 4
      --max-num-seqs 256
      --max-num-batched-tokens 2048
      --port 8001
    networks:
      - pd-network

  mooncake-transfer:
    image: mooncake/transfer-engine:latest
    privileged: true
    environment:
      - MOONCAKE_RDMA_DEVICE=mlx5_0
      - MOONCAKE_BUFFER_SIZE=16G
    volumes:
```

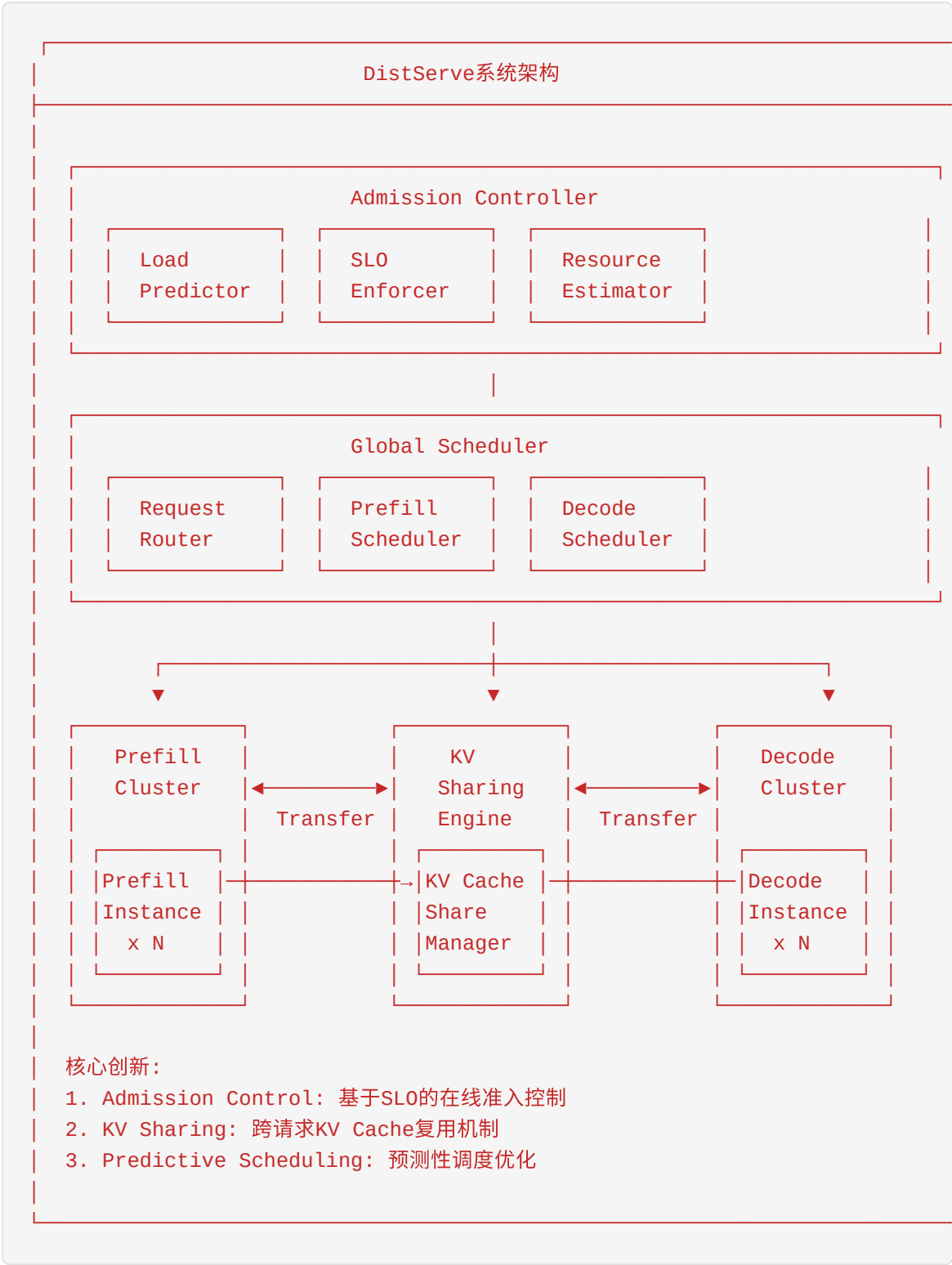
```
- /dev/infiniband:/dev/infiniband
networks:
  - pd-network

networks:
  pd-network:
    driver: bridge
```

4.6 DistServe的设计

DistServe是清华大学和月之暗面联合研发的PD分离服务系统，专注于在线服务的延迟优化和资源效率。它在Mooncake的基础上进一步引入了准入控制和KV Sharing机制。

4.6.1 DistServe架构概览



4.6.2 在线准入控制

DistServe的准入控制器是其核心创新之一，它能够在请求到达时预测其对系统的影响，并做出接受/拒绝决策。

准入控制流程



负载预测模型

```

class LoadPredictor:
    """
    负载预测器

    使用历史数据和在线学习预测请求的资源需求
    """

    def __init__(self):
        # 历史性能数据
        self.prefill_time_model = self._build_prefill_model()
        self.decode_time_model = self._build_decode_model()

    def _build_prefill_model(self):
        """构建Prefill时间预测模型"""
        # 基于线性回归的简化模型
        #  $T_{\text{prefill}} = a * n^2 + b * n + c$ 
        # 其中n是输入序列长度
        return LinearRegressionModel(
            features=['input_len_sq', 'input_len', 'model_size'],
            target='prefill_time'
        )

    def predict_prefill_time(
        self,
        input_len: int,
        model_config: ModelConfig,
        node_config: NodeConfig,
    ) -> float:
        """
        预测Prefill执行时间

        Args:
            input_len: 输入序列长度
            model_config: 模型配置
            node_config: 节点配置 (GPU类型、数量等)

        Returns:
            预测的Prefill时间 (毫秒)
        """
        # 基础计算时间
        compute_time = (
            2 * input_len * model_config.hidden_size ** 2 +
            input_len ** 2 * model_config.num_heads * model_config.head_dir
        ) / (node_config.gpu_flops * 0.8) # 80%利用率

        # 批处理等待时间
        wait_time = self.estimate_wait_time('prefill')

        # KV Cache传输时间
        kv_cache_size = (

```

```
        2 * model_config.num_layers * input_len *
        model_config.hidden_size * 2 # fp16
    )
    transfer_time = kv_cache_size / node_config.network_bandwidth

    return compute_time + wait_time + transfer_time

def predict_decode_time(
    self,
    output_len: int,
    current_batch_size: int,
    model_config: ModelConfig,
    node_config: NodeConfig,
) -> float:
    """预测Decode执行时间"""
    # 每token内存访问时间
    memory_time = (
        2 * model_config.num_layers * model_config.hidden_size *
        output_len * 2 # fp16
    ) / node_config.memory_bandwidth

    # 批处理开销
    batch_overhead = 0.001 * current_batch_size # 1ms per request

    return memory_time + batch_overhead
```

SLO执行器


```
class SLOEnforcer:
    """
    SLO执行器

    确保接受的请求能够满足其SLO承诺
    """

    def __init__(self):
        self.slo_definitions = {
            'interactive': {'ttft': 100, 'itl': 20},    # 交互式
            'standard': {'ttft': 500, 'itl': 50},      # 标准
            'batch': {'ttft': 5000, 'itl': 200},      # 批处理
        }

    def check_slo_feasibility(
        self,
        request: Request,
        current_load: SystemLoad,
        predictor: LoadPredictor,
    ) -> Tuple[bool, str]:
        """
        检查请求是否可以在SLO内完成

        Returns:
            (是否可行, 原因)
        """
        slo = self.slo_definitions[request.slo_tier]

        # 预测TTFT
        predicted_ttft = predictor.predict_prefill_time(
            request.input_len,
            request.model_config,
            current_load.prefill_nodes[0].config,
        )

        if predicted_ttft > slo['ttft']:
            return False, f"TTFT预测({predicted_ttft}ms)超过SLO({slo['ttft']}ms)"

        # 预测ITL
        predicted_itl = predictor.predict_decode_time(
            request.expected_output_len,
            current_load.decode_batch_size,
            request.model_config,
            current_load.decode_nodes[0].config,
        )

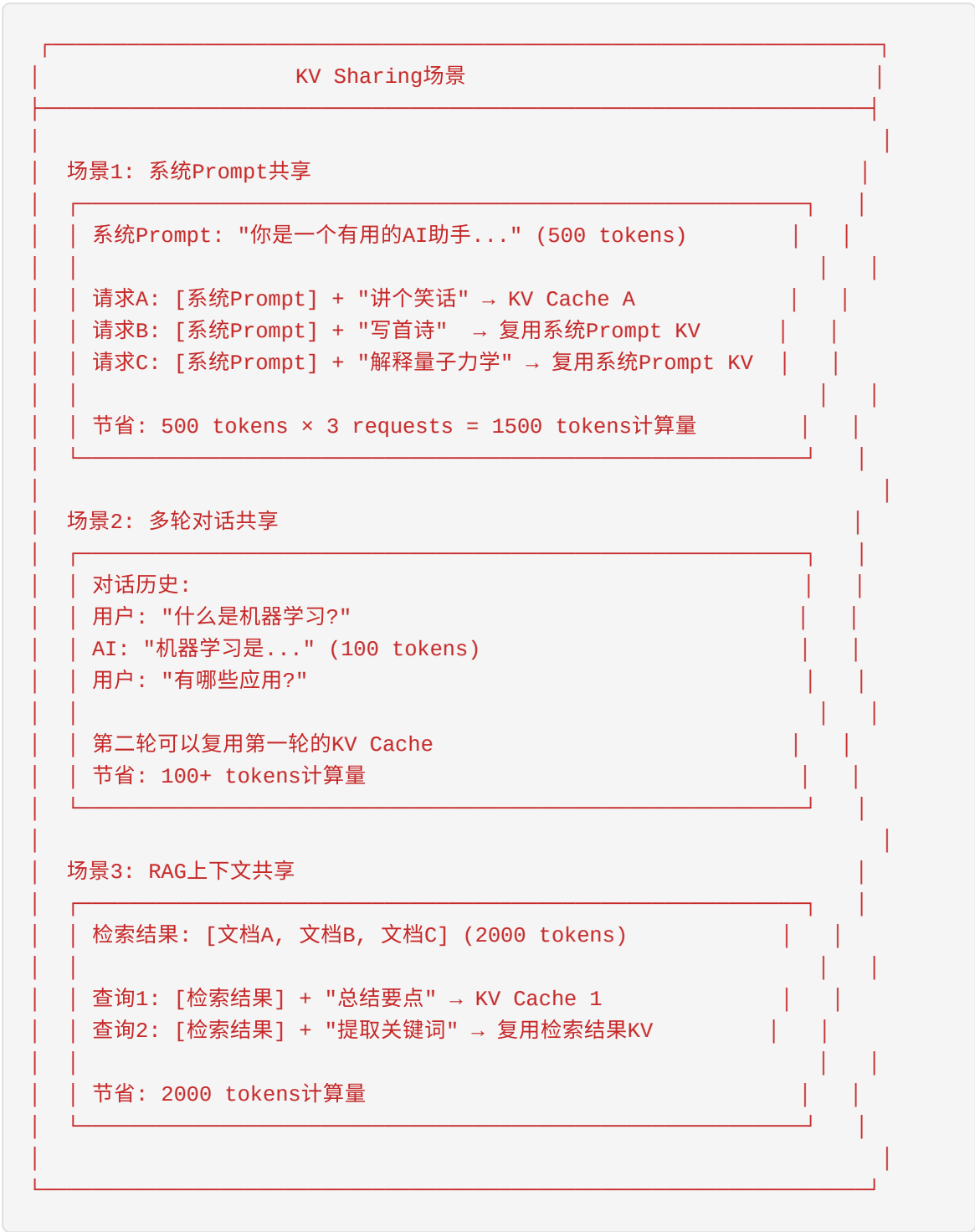
        if predicted_itl > slo['itl']:
            return False, f"ITL预测({predicted_itl}ms)超过SLO({slo['itl']}ms)"
```

```
return True, "SLO可行"
```

4.6.3 KV Sharing机制

KV Sharing是DistServe的另一项核心创新，通过跨请求复用KV Cache来减少重复计算。

共享场景分析



KV Sharing管理器

```
class KVSharingManager:
    """
    KV Cache共享管理器

    管理KV Cache的存储、索引和复用
    """

    def __init__(self, max_cache_size_gb: float = 100):
        self.max_cache_size = max_cache_size_gb * 1024 * 1024 * 1024
        self.cache_store = {} # prefix_hash → KVCache
        self.access_history = {} # 访问历史（用于LRU）
        self.current_size = 0

    def compute_prefix_hash(self, tokens: List[int]) -> str:
        """计算token序列的哈希值"""
        # 使用SHA256哈希
        token_bytes = np.array(tokens).tobytes()
        return hashlib.sha256(token_bytes).hexdigest()

    def find_longest_match(
        self,
        tokens: List[int]
    ) -> Tuple[Optional[str], int]:
        """
        查找最长的匹配前缀

        Returns:
            (匹配的cache_key, 匹配长度)
        """
        best_match = None
        best_len = 0

        # 从最长前缀开始匹配
        for length in range(len(tokens), 0, -1):
            prefix = tokens[:length]
            key = self.compute_prefix_hash(prefix)

            if key in self.cache_store:
                best_match = key
                best_len = length
                break

        return best_match, best_len

    def store_kv_cache(
        self,
        tokens: List[int],
        kv_cache: torch.Tensor,
        metadata: Dict,
    ):

```

```
"""存储KV Cache"""
key = self.compute_prefix_hash(tokens)

# 检查容量
cache_size = kv_cache.numel() * kv_cache.element_size()
while self.current_size + cache_size > self.max_cache_size:
    self._evict_lru_entry()

# 存储
self.cache_store[key] = {
    'kv_cache': kv_cache,
    'metadata': metadata,
    'size': cache_size,
}
self.access_history[key] = time.time()
self.current_size += cache_size

def get_kv_cache(
    self,
    key: str
) -> Optional[torch.Tensor]:
    """获取KV Cache"""
    if key not in self.cache_store:
        return None

    # 更新访问时间
    self.access_history[key] = time.time()

    return self.cache_store[key]['kv_cache']

def _evict_lru_entry(self):
    """驱逐最久未使用的条目"""
    if not self.access_history:
        return

    # 找到最久未使用的
    lru_key = min(self.access_history, key=self.access_history.get)

    # 驱逐
    evicted = self.cache_store.pop(lru_key)
    self.access_history.pop(lru_key)
    self.current_size -= evicted['size']
```

4.6.4 性能提升分析

DistServe相比基线系统的性能提升：

指标	vLLM基线	DistServe	提升
TTFT P99	800ms	120ms	6.7x
ITL P99	80ms	25ms	3.2x
吞吐量	100%	220%	2.2x
GPU利用率	45%	85%	1.9x
KV Cache命中率	0%	35%	-
计算节省	0%	30%	-

KV Sharing效果分析

在实际生产环境中，KV Sharing可以带来显著收益：

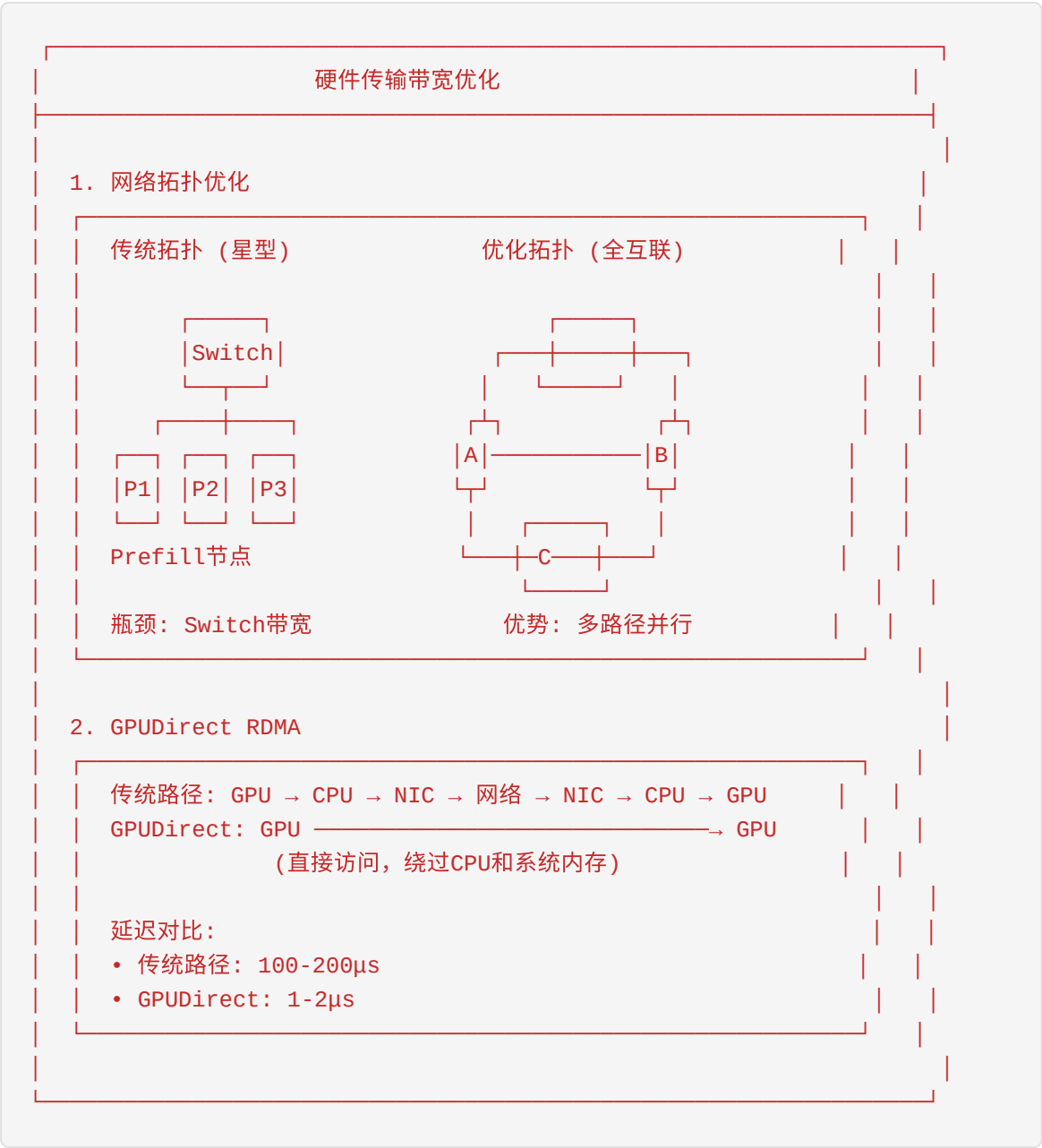
应用场景	共享比例	计算节省
客服对话	40-60%	25-35%
代码生成	20-30%	15-20%
RAG检索	50-70%	35-45%
教育辅导	30-50%	20-30%

4.7 PD分离中的KV Cache传输优化

KV Cache传输是PD分离架构的关键路径，其性能直接影响系统的TTFT和整体吞吐量。本节深入探讨各种传输优化技术。

4.7.1 传输带宽优化

硬件层面优化



```

class BandwidthOptimizer:
    """
    传输带宽优化器
    """

    def __init__(self, network_config: NetworkConfig):
        self.network_config = network_config
        self.transfer_queue = asyncio.Queue()

    async def optimize_transfer(
        self,
        kv_cache: torch.Tensor,
        dst_addr: str,
    ) -> bool:
        """
        优化后的传输函数

        优化策略：
        1. 批量传输
        2. 流水线并行
        3. 压缩传输
        """
        # 1. 分割KV Cache为传输块
        chunks = self._split_into_chunks(kv_cache)

        # 2. 并行传输
        tasks = []
        for i, chunk in enumerate(chunks):
            task = self._transfer_chunk(chunk, dst_addr, i)
            tasks.append(task)

        # 3. 等待所有传输完成
        results = await asyncio.gather(*tasks)

        return all(results)

    def _split_into_chunks(
        self,
        kv_cache: torch.Tensor,
        chunk_size_mb: int = 64
    ) -> List[torch.Tensor]:
        """
        将KV Cache分割为传输块

        策略：按层分割，便于流水线
        """
        n_layers = kv_cache.shape[1] # [2, n_layers, ...]
        bytes_per_layer = kv_cache.numel() // n_layers * kv_cache.element_s
        layers_per_chunk = max(1, chunk_size_mb * 1024 * 1024 // bytes_per_

```



```
chunks = []
for i in range(0, n_layers, layers_per_chunk):
    end = min(i + layers_per_chunk, n_layers)
    chunk = kv_cache[:, i:end, :, :, :]
    chunks.append(chunk)

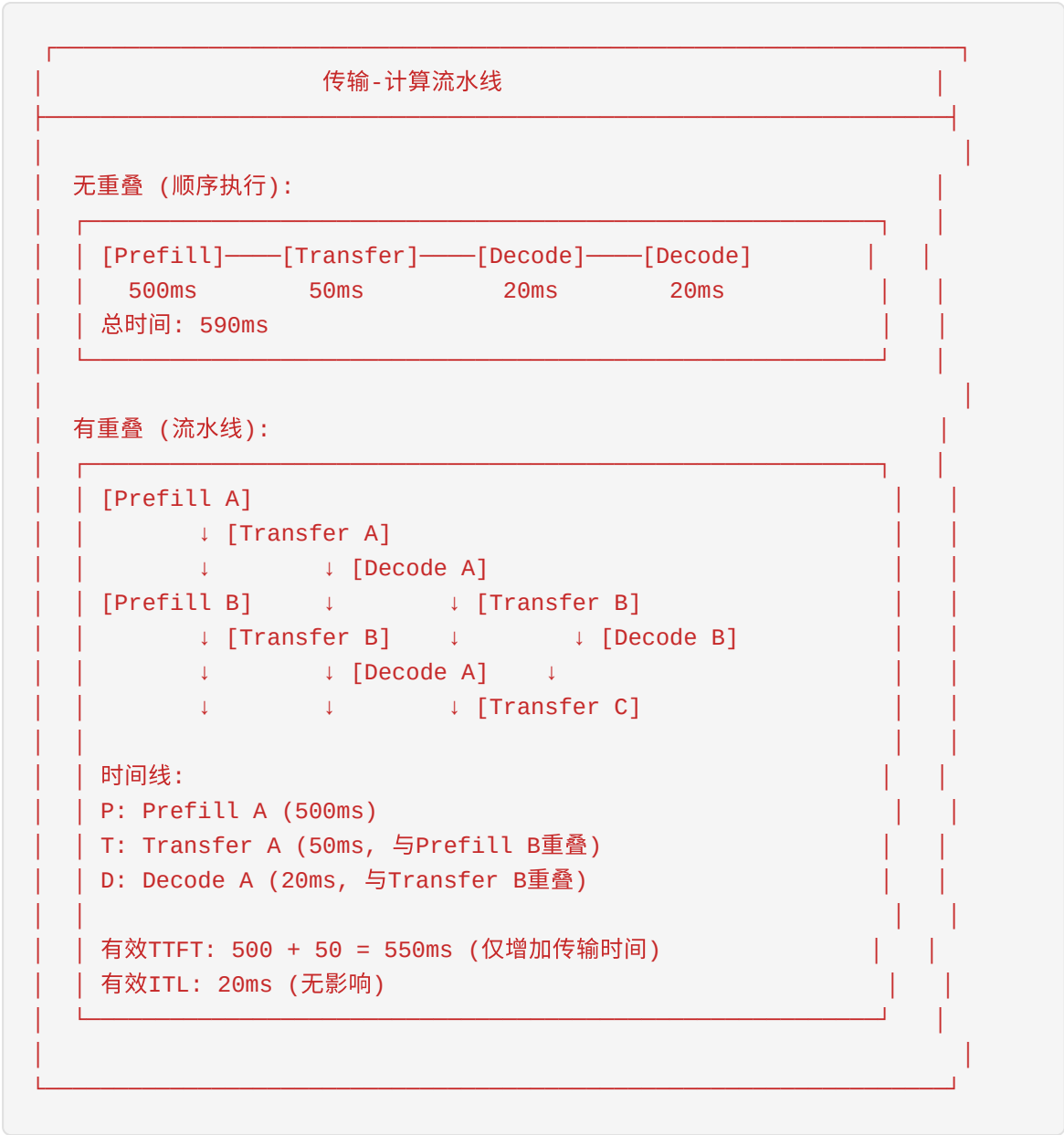
return chunks

async def _transfer_chunk(
    self,
    chunk: torch.Tensor,
    dst_addr: str,
    chunk_id: int,
) -> bool:
    """传输单个块"""
    # 使用RDMA发送
    try:
        await self.rdma_send(chunk.data_ptr(), chunk.numel(), dst_addr)
        return True
    except Exception as e:
        logger.error(f"Chunk {chunk_id} transfer failed: {e}")
        return False
```

4.7.2 重叠传输和计算

重叠传输和计算是减少传输延迟影响的关键技术。

流水线设计



异步传输实现

```
class AsyncTransferEngine:
    """
    异步传输引擎

    实现传输和计算的完全重叠
    """

    def __init__(self):
        self.transfer_stream = torch.cuda.Stream()
        self.compute_stream = torch.cuda.default_stream()
        self.pending_transfers = {}

    async def prefill_with_async_transfer(
        self,
        request: Request,
        prefill_func: Callable,
        transfer_func: Callable,
    ) -> torch.Tensor:
        """
        执行Prefill并异步传输KV Cache

        流程：
        1. 执行Prefill
        2. 在独立CUDA流中启动异步传输
        3. 返回控制，不等待传输完成
        """
        # 1. 执行Prefill (在计算流)
        with torch.cuda.stream(self.compute_stream):
            output, kv_cache = prefill_func(request)

        # 2. 同步计算流，确保KV Cache就绪
        self.compute_stream.synchronize()

        # 3. 在传输流中启动异步传输
        with torch.cuda.stream(self.transfer_stream):
            transfer_future = transfer_func(kv_cache)
            self.pending_transfers[request.id] = transfer_future

        return output

    async def decode_with_async_receive(
        self,
        request: Request,
        decode_func: Callable,
        receive_func: Callable,
    ) -> torch.Tensor:
        """
        执行Decode并异步接收KV Cache

        流程：

```

```
1. 检查KV Cache是否已接收
2. 如果未接收完成，等待
3. 执行Decode
"""
# 1. 等待KV Cache接收完成
if request.id in self.pending_receives:
    kv_cache = await self.pending_receives[request.id]
else:
    kv_cache = await receive_func(request.id)

# 2. 执行Decode
output = decode_func(request, kv_cache)

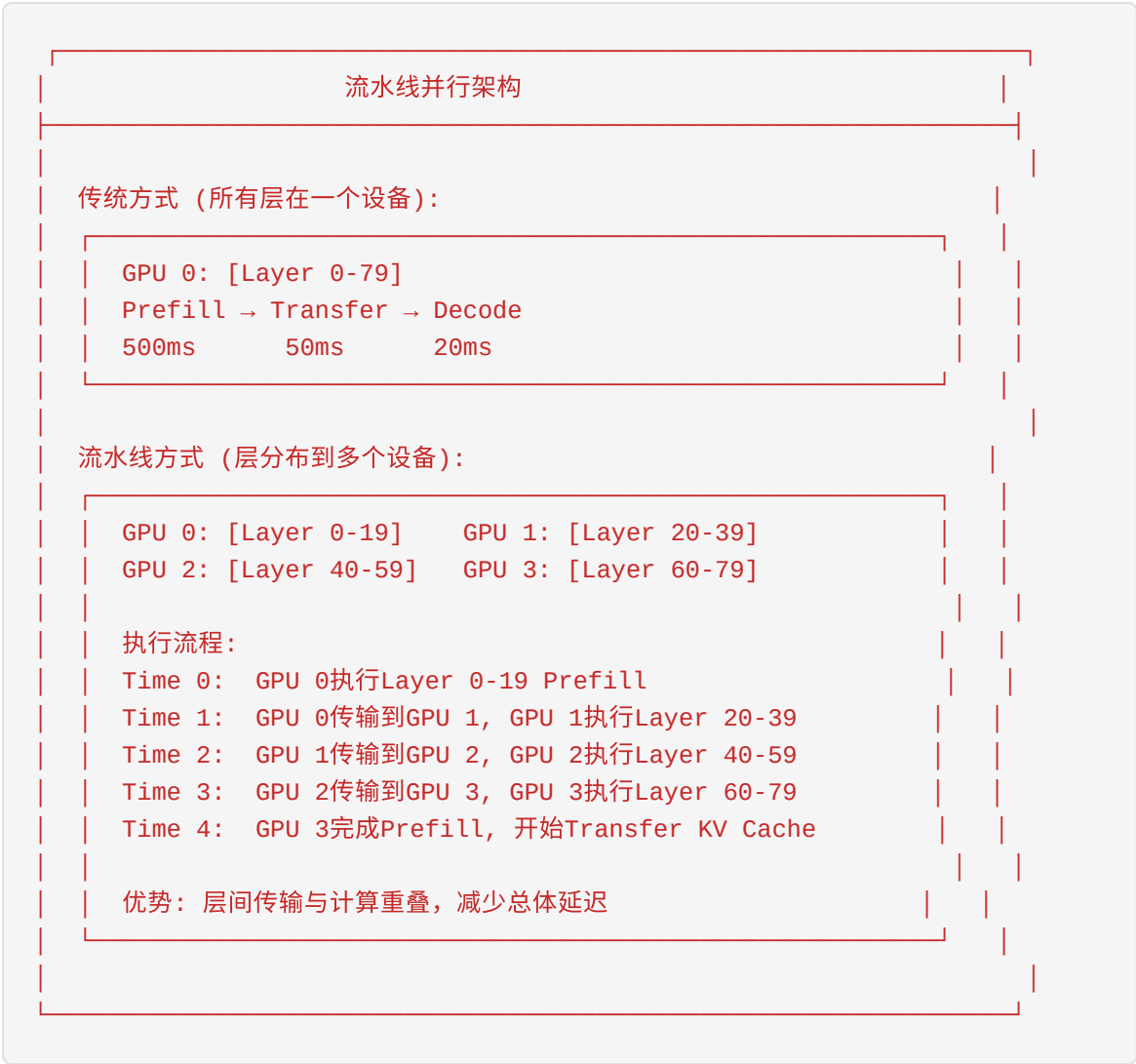
return output

def create_transfer_event(self) -> torch.cuda.Event:
    """创建传输完成事件"""
    event = torch.cuda.Event()
    event.record(self.transfer_stream)
    return event
```

4.7.3 流水线并行

流水线并行是PD分离的高级优化技术，通过将模型层分布到多个设备上实现传输和计算的深度重叠。

流水线架构



流水线调度实现

```
class PipelineScheduler:
    """
    流水线调度器

    管理模型层在多个GPU上的分布和执行
    """

    def __init__(
        self,
        model_config: ModelConfig,
        num_stages: int,
    ):
        self.num_stages = num_stages
        self.layers_per_stage = model_config.num_layers // num_stages

        # 为每个阶段分配GPU
        self.stage_gpus = list(range(num_stages))

    def schedule_prefill(
        self,
        input_ids: torch.Tensor,
    ) -> torch.Tensor:
        """
        流水线Prefill调度

        每个阶段处理一部分层，激活值在阶段间传递
        """
        hidden_states = self.embed(input_ids)

        for stage in range(self.num_stages):
            gpu = self.stage_gpus[stage]

            # 将输入移动到目标GPU
            hidden_states = hidden_states.to(f'cuda:{gpu}')

            # 执行当前阶段的层
            start_layer = stage * self.layers_per_stage
            end_layer = (stage + 1) * self.layers_per_stage

            for layer_id in range(start_layer, end_layer):
                hidden_states = self.layers[layer_id](hidden_states)

        return hidden_states

    def schedule_kv_transfer(
        self,
        kv_caches: List[torch.Tensor],
        dst_addr: str,
    ):
        """
```

流水线KV Cache传输

各阶段的KV Cache并行传输

"""

```
transfer_tasks = []
```

```
for stage, kv_cache in enumerate(kv_caches):
```

```
    # 每个阶段的KV Cache独立传输
```

```
    task = self.async_transfer(kv_cache, dst_addr, stage)
```

```
    transfer_tasks.append(task)
```

```
# 等待所有传输完成
```

```
await asyncio.gather(*transfer_tasks)
```

4.7.4 压缩传输技术

压缩传输可以显著减少传输数据量，降低传输延迟。

量化压缩

```
class KVCacheCompressor:
    """
    KV Cache压缩器

    支持多种压缩算法
    """

    def __init__(self, method: str = 'fp8'):
        self.method = method

    def compress(self, kv_cache: torch.Tensor) -> Tuple[torch.Tensor, Any]:
        """
        压缩KV Cache

        Returns:
            (压缩后的数据, 元数据)
        """
        if self.method == 'fp8':
            return self._compress_fp8(kv_cache)
        elif self.method == 'int8':
            return self._compress_int8(kv_cache)
        elif self.method == 'sparse':
            return self._compress_sparse(kv_cache)
        else:
            return kv_cache, None

    def _compress_fp8(
        self,
        kv_cache: torch.Tensor
    ) -> Tuple[torch.Tensor, torch.Tensor]:
        """
        FP8量化压缩

        FP16 (2 bytes) → FP8 (1 byte)
        压缩率: 50%
        """
        # 计算缩放因子
        max_val = kv_cache.abs().max()
        scale = max_val / 448.0 # E4M3最大值

        # 量化到FP8
        compressed = (kv_cache / scale).to(torch.float8_e4m3fn)

        return compressed, scale

    def _compress_int8(
        self,
        kv_cache: torch.Tensor,
    ) -> Tuple[torch.Tensor, Tuple[torch.Tensor, torch.Tensor]]:
        """
```


INT8对称量化

FP16 (2 bytes) → INT8 (1 byte)

压缩率: 50%

"""

计算缩放因子 (per-channel)

dim = -1 # 最后一个维度

max_vals = kv_cache.abs().amax(dim=dim, keepdim=True)

scales = max_vals / 127.0

量化到INT8

compressed = (kv_cache / scales).round().clamp(-128, 127).to(torch

return compressed, (scales, max_vals)

def _compress_sparse(

self,

kv_cache: torch.Tensor,

sparsity: float = 0.5,

) -> Tuple[torch.Tensor, torch.Tensor]:

"""

稀疏化压缩

只保留重要的token

压缩率: 可调 (默认50%)

"""

计算每个token的重要性 (基于L2范数)

importance = kv_cache.norm(dim=-1)

选择重要的token

k = int(kv_cache.shape[2] * (1 - sparsity)) # 保留的token数

topk_vals, topk_indices = torch.topk(importance, k, dim=2)

压缩

compressed = torch.gather(

kv_cache,

dim=2,

index=topk_indices.unsqueeze(-1).expand(-1, -1, -1, kv_cache.st

)

return compressed, topk_indices

压缩效果对比

压缩方法	压缩率	精度损失	适用场景
FP8	50%	<1%	通用推荐
INT8	50%	1-2%	资源受限
FP4	25%	3-5%	极端压缩
稀疏50%	50%	2-3%	长序列
稀疏75%	25%	5-8%	极端场景

4.8 调度策略和负载均衡

有效的调度策略和负载均衡机制是PD分离系统高效运行的关键。本节详细讨论Prefill和Decode节点的选择策略以及动态负载均衡。

4.8.1 Prefill节点选择策略

Prefill节点的选择直接影响TTFT和Prefill吞吐量。

选择因素

Prefill节点选择因素
1. 计算能力（权重：30%） GPU类型（A100/H100/H800） GPU数量 Tensor Parallel效率
2. 当前负载（权重：25%） GPU利用率 内存使用率 正在执行的Prefill数量
3. 队列深度（权重：20%） 等待队列长度 预估等待时间
4. 网络位置（权重：15%） 与Decode节点的网络距离 预估KV Cache传输时间
5. 模型缓存（权重：10%） 模型是否已加载 模型版本匹配

选择算法实现

```
class PrefillNodeSelector:
    """
    Prefill节点选择器
    """

    def __init__(self, weights: Dict[str, float] = None):
        # 默认权重配置
        self.weights = weights or {
            'compute': 0.30,
            'load': 0.25,
            'queue': 0.20,
            'network': 0.15,
            'model': 0.10,
        }

    def select_node(
        self,
        request: Request,
        prefill_nodes: List[PrefillNode],
    ) -> PrefillNode:
        """
        选择最优Prefill节点
        """
        scores = []

        for node in prefill_nodes:
            if not node.is_healthy:
                continue

            score = self._compute_score(node, request)
            scores.append((node, score))

        if not scores:
            raise NoAvailableNodeError("没有可用的Prefill节点")

        # 选择得分最高的节点
        best_node = max(scores, key=lambda x: x[1])[0]
        return best_node

    def _compute_score(
        self,
        node: PrefillNode,
        request: Request,
    ) -> float:
        """计算节点得分"""
        scores = {
            'compute': self._compute_score(node, request),
            'load': self._load_score(node),
            'queue': self._queue_score(node),
            'network': self._network_score(node, request),
        }
```

```
        'model': self._model_score(node, request),
    }

    # 加权求和
    total_score = sum(
        scores[k] * self.weights[k] for k in scores
    )

    return total_score

def _compute_score(
    self,
    node: PrefillNode,
    request: Request,
) -> float:
    """计算能力得分"""
    # 基于GPU类型和数量
    gpu_scores = {
        'H100': 1.0,
        'H800': 0.95,
        'A100-80GB': 0.75,
        'A100-40GB': 0.65,
        'A10': 0.30,
    }

    base_score = gpu_scores.get(node.gpu_type, 0.5)

    # 考虑Tensor Parallel效率
    tp_efficiency = 0.85 if node.tp_size > 1 else 1.0

    return base_score * tp_efficiency * math.sqrt(node.gpu_count)

def _load_score(self, node: PrefillNode) -> float:
    """负载得分 (越低越好)"""
    # 基于GPU利用率和内存使用率
    gpu_util = node.gpu_utilization
    mem_util = node.memory_utilization

    # 综合负载
    load = 0.6 * gpu_util + 0.4 * mem_util

    # 转换为得分 (1 - load)
    return 1.0 - load

def _queue_score(self, node: PrefillNode) -> float:
    """队列得分 (越短越好)"""
    queue_depth = node.queue_depth
    max_queue = node.max_queue_depth

    # 归一化队列深度
    normalized = min(queue_depth / max_queue, 1.0)
```

```
        return 1.0 - normalized

def _network_score(
    self,
    node: PrefillNode,
    request: Request,
) -> float:
    """网络位置得分"""
    # 获取目标Decode节点
    decode_node = request.preferred_decode_node

    if decode_node is None:
        return 0.5 # 默认值

    # 网络延迟
    latency = self.network_topology.get_latency(node, decode_node)
    bandwidth = self.network_topology.get_bandwidth(node, decode_node)

    # 预估传输时间
    kv_cache_size = self.estimate_kv_cache_size(request)
    transfer_time = kv_cache_size / bandwidth + latency

    # 转换为得分 (传输时间越短越好)
    max_acceptable = 100 # 100ms
    score = max(0, 1.0 - transfer_time / max_acceptable)

    return score

def _model_score(
    self,
    node: PrefillNode,
    request: Request,
) -> float:
    """模型缓存得分"""
    if node.has_model_cached(request.model_id):
        return 1.0
    elif node.can_load_model(request.model_id):
        return 0.5
    else:
        return 0.0
```

4.8.2 Decode节点选择策略

Decode节点的选择直接影响ITL和用户体验。

选择因素

Decode节点选择因素

1. 当前ITL (权重: 35%)
 - 当前批处理的ITL
 - ITL趋势 (上升/下降)
 - ITL P99
2. 内存容量 (权重: 25%)
 - 可用内存
 - KV Cache存储能力
 - 预估内存需求 vs 可用内存
3. 批大小 (权重: 20%)
 - 当前批大小
 - 批大小上限
 - 批处理效率
4. 内存带宽 (权重: 15%)
 - GPU内存带宽
 - HBM带宽利用率
5. 网络位置 (权重: 5%)
 - 与Prefill节点的距离

选择算法实现

```
class DecodeNodeSelector:
    """
    Decode节点选择器
    """

    def __init__(self, weights: Dict[str, float] = None):
        self.weights = weights or {
            'itl': 0.35,
            'memory': 0.25,
            'batch': 0.20,
            'bandwidth': 0.15,
            'network': 0.05,
        }

    def select_node(
        self,
        request: Request,
        kv_cache_size: int,
        decode_nodes: List[DecodeNode],
    ) -> DecodeNode:
        """
        选择最优Decode节点
        """
        candidates = []

        for node in decode_nodes:
            # 首先检查内存容量
            if node.available_memory < kv_cache_size * 1.2: # 20%裕量
                continue

            score = self._compute_score(node, request, kv_cache_size)
            candidates.append((node, score))

        if not candidates:
            # 没有足够内存的节点，触发扩容
            self.trigger_scale_up(kv_cache_size)
            raise NoAvailableNodeError("没有可用的Decode节点")

        # 选择得分最高的节点
        best_node = max(candidates, key=lambda x: x[1])[0]
        return best_node

    def _compute_score(
        self,
        node: DecodeNode,
        request: Request,
        kv_cache_size: int,
    ) -> float:
        """计算节点得分"""
        scores = {
```



```
        'itl': self._itl_score(node),
        'memory': self._memory_score(node, kv_cache_size),
        'batch': self._batch_score(node),
        'bandwidth': self._bandwidth_score(node),
        'network': self._network_score(node, request),
    }

    total_score = sum(
        scores[k] * self.weights[k] for k in scores
    )

    return total_score

def _itl_score(self, node: DecodeNode) -> float:
    """ITL得分 (越低越好)"""
    current_itl = node.current_itl
    target_itl = 50  # 目标ITL 50ms

    # 超过目标ITL的惩罚
    if current_itl > target_itl:
        return max(0, 1.0 - (current_itl - target_itl) / target_itl)
    else:
        # 低于目标ITL的奖励
        return 1.0 + (target_itl - current_itl) / target_itl * 0.2

def _memory_score(
    self,
    node: DecodeNode,
    kv_cache_size: int,
) -> float:
    """内存得分"""
    available = node.available_memory

    # 内存充足度
    ratio = available / kv_cache_size

    if ratio < 1.2:
        return 0.0  # 内存不足
    elif ratio < 2.0:
        return (ratio - 1.2) / 0.8 * 0.5  # 0.0 - 0.5
    else:
        return 0.5 + min(0.5, (ratio - 2.0) / 4.0 * 0.5)  # 0.5 - 1.0

def _batch_score(self, node: DecodeNode) -> float:
    """批大小得分"""
    current = node.current_batch_size
    max_size = node.max_batch_size

    # 理想批大小: 70% of max
    ideal = max_size * 0.7
```

```
# 距离理想的差距  
diff = abs(current - ideal) / ideal  
  
return max(0, 1.0 - diff)
```

4.8.3 动态负载均衡

动态负载均衡确保系统在高负载下仍能保持稳定的性能。

负载均衡策略

```
class DynamicLoadBalancer:
    """
    动态负载均衡器

    根据系统负载动态调整请求分布
    """

    def __init__(self):
        self.prefill_nodes = []
        self.decode_nodes = []
        self.load_history = deque(maxlen=100)

    async def balance_load(self):
        """
        执行负载均衡

        策略:
        1. 监控各节点负载
        2. 检测负载不均
        3. 触发请求迁移
        """
        while True:
            # 收集负载信息
            prefill_loads = await self._collect_prefill_loads()
            decode_loads = await self._collect_decode_loads()

            # 检测负载不均
            if self._detect_imbalance(prefill_loads):
                await self._rebalance_prefill(prefill_loads)

            if self._detect_imbalance(decode_loads):
                await self._rebalance_decode(decode_loads)

            # 等待下一次检查
            await asyncio.sleep(5)  # 5秒检查一次

    def _detect_imbalance(self, loads: List[float]) -> bool:
        """检测负载是否不均"""
        if len(loads) < 2:
            return False

        avg_load = sum(loads) / len(loads)
        max_load = max(loads)
        min_load = min(loads)

        # 负载差异超过30%认为不均衡
        if max_load - min_load > 0.3:
            return True

        # 或存在节点过载 (>90%)
```

```
        if max_load > 0.9:
            return True

        return False

    async def _rebalance_prefill(
        self,
        loads: List[Tuple[PrefillNode, float]],
    ):
        """重新均衡Prefill负载"""
        # 找到最忙和最闲的节点
        sorted_nodes = sorted.loads, key=lambda x: x[1], reverse=True)
        busiest = sorted_nodes[0]
        idlest = sorted_nodes[-1]

        # 如果差异足够大, 迁移请求
        if busiest[1] - idlest[1] > 0.3:
            # 从 busiest 迁移部分请求到 idlest
            requests_to_migrate = self._select_migratable_requests(
                busiest[0],
                count=5
            )

            for request in requests_to_migrate:
                await self._migrate_prefill_request(request, idlest[0])

    async def _migrate_prefill_request(
        self,
        request: Request,
        target_node: PrefillNode,
    ):
        """迁移Prefill请求"""
        # 1. 暂停原节点上的请求
        source_node = request.assigned_prefill_node
        await source_node.pause_request(request)

        # 2. 传输中间状态 (如果有)
        if request.has_partial_kv_cache:
            kv_cache = await source_node.extract_kv_cache(request)
            await self.transfer_engine.send(kv_cache, target_node.addr)

        # 3. 在目标节点恢复执行
        await target_node.resume_request(request)
```

自适应批处理

```
class AdaptiveBatcher:
    """
    自适应批处理器

    根据当前负载动态调整批处理策略
    """

    def __init__(self):
        self.target_itl = 50 # 目标ITL
        self.target_ttft = 200 # 目标TTFT

    def adapt_batch_size(self, node: Node, metrics: Metrics):
        """
        自适应调整批大小

        策略:
        - ITL过高 → 减小批大小
        - ITL过低且有积压 → 增大批大小
        """
        current_itl = metrics.itl_p99
        queue_depth = metrics.queue_depth

        if current_itl > self.target_itl * 1.2:
            # ITL过高, 减小批大小
            new_batch_size = int(node.max_batch_size * 0.9)
            logger.info(f"Reducing batch size to {new_batch_size} due to high ITL")

        elif current_itl < self.target_itl * 0.8 and queue_depth > 10:
            # ITL较低且有积压, 增大批大小
            new_batch_size = min(
                int(node.max_batch_size * 1.1),
                node.hard_max_batch_size
            )
            logger.info(f"Increasing batch size to {new_batch_size}")

        else:
            # 保持当前批大小
            new_batch_size = node.max_batch_size

        node.max_batch_size = new_batch_size
```

4.9 实际部署的挑战和解决方案

PD分离技术在实际生产环境部署时面临诸多挑战。本节讨论这些挑战及其解决方案，为即将去Mooncake实习的新人提供实战经验。

4.9.1 网络延迟挑战

网络延迟是PD分离架构的主要挑战之一，KV Cache传输延迟直接影响TTFT。

挑战分析

网络延迟挑战分析

问题1：跨机房传输延迟高

Prefill Cluster（北京）Decode Cluster（上海）

网络延迟：30-50ms

KV Cache传输（1GB）：100-200ms

影响：TTFT增加100-200ms，可能超过SLO

问题2：网络抖动导致ITL不稳定

网络拥塞 → 传输延迟增加 → Decode等待 → ITL抖动

正常ITL：20ms

拥塞ITL：50-100ms（用户体验明显下降）

问题3：RDMA配置复杂

- 需要专用网卡（Mellanox ConnectX-6/7）
- 需要IB交换机或RoCE支持
- 驱动和固件版本兼容性问题
- 网络拓扑优化复杂

解决方案

```
class NetworkLatencyOptimizer:
    """
    网络延迟优化器
    """

    def __init__(self):
        self.transfer_strategies = {
            'local': LocalTransferStrategy(),
            'rack': RackLocalTransferStrategy(),
            'az': AvailabilityZoneTransferStrategy(),
            'remote': RemoteTransferStrategy(),
        }

    def optimize_placement(
        self,
        prefill_nodes: List[Node],
        decode_nodes: List[Node],
    ) -> PlacementPlan:
        """
        优化节点放置

        策略：
        1. 优先同机架部署
        2. 其次同可用区部署
        3. 避免跨地域部署
        """
        # 分析网络拓扑
        topology = self._analyze_network_topology(
            prefill_nodes + decode_nodes
        )

        # 构建亲和图
        affinity_graph = self._build_affinity_graph(topology)

        # 优化配对
        pairs = self._optimize_pairs(
            prefill_nodes,
            decode_nodes,
            affinity_graph
        )

        return PlacementPlan(pairs)

    def select_transfer_strategy(
        self,
        prefill_node: Node,
        decode_node: Node,
    ) -> TransferStrategy:
        """选择最优传输策略"""
        # 确定节点位置关系
```

```
if prefill_node.rack == decode_node.rack:
    # 同机架: NVLink或高速IB
    return self.transfer_strategies['local']
elif prefill_node.az == decode_node.az:
    # 同可用区: IB/RoCE
    return self.transfer_strategies['az']
else:
    # 跨可用区: 压缩+异步
    return self.transfer_strategies['remote']

# RDMA配置检查脚本
class RDMAConfigurator:
    """RDMA配置管理器"""

    def verify_rdma_setup(self) -> bool:
        """验证RDMA配置"""
        checks = {
            'device': self._check_rdma_device(),
            'driver': self._check_driver_version(),
            'port': self._check_port_status(),
            'gid': self._check_gid_config(),
            'performance': self._check_performance(),
        }

        for name, result in checks.items():
            if not result:
                logger.error(f"RDMA check failed: {name}")
                return False

        return True

    def _check_rdma_device(self) -> bool:
        """检查RDMA设备"""
        try:
            devices = subprocess.check_output(
                ['ibstat'],
                text=True
            )
            return 'Mellanox' in devices or 'mlx' in devices
        except:
            return False

    def _check_performance(self) -> bool:
        """检查RDMA性能"""
        # 运行ib_write_bw测试
        result = subprocess.run(
            ['ib_write_bw', '--report_gbits'],
            capture_output=True,
            text=True,
            timeout=30
        )
```



```
)  
  
# 解析带宽结果  
bandwidth = self._parse_bandwidth(result.stdout)  
  
# 检查是否达到预期 (例如 180Gbps for HDR)  
return bandwidth > 180
```

4.9.2 容错处理

PD分离架构的容错处理比同构部署更复杂，需要处理更多故障场景。

故障场景分析

PD分离故障场景
故障类型1: Prefill节点故障 场景: Prefill执行中节点崩溃 影响: 请求需要重新调度, TTFT增加 处理: 1. 检测故障 (心跳超时) 2. 标记请求为失败 3. 重新调度到其他Prefill节点 4. 通知客户端 (可选重试)
故障类型2: KV Cache传输失败 场景: 网络中断导致KV Cache传输失败 影响: Decode无法开始, 请求卡住 处理: 1. 检测传输超时 2. 重试传输 (最多3次) 3. 选择备用Decode节点 4. 如果仍失败, 回退到同构执行
故障类型3: Decode节点故障 场景: Decode执行中节点崩溃 影响: 正在生成的请求中断 处理: 1. 检测故障 2. 从KV Store恢复KV Cache 3. 迁移到其他Decode节点继续生成 4. 客户端无感知 (流式输出可能中断)

容错实现

```
class FaultToleranceManager:
    """
    容错管理器

    处理PD分离架构的各种故障场景
    """

    def __init__(self):
        self.node_health = {}
        self.request_states = {}
        self.kv_backup_store = KVBackupStore()

    async def handle_prefill_failure(
        self,
        request: Request,
        failed_node: PrefillNode,
    ):
        """处理Prefill节点故障"""
        logger.error(f"Prefill node {failed_node.id} failed for request {request}")

        # 1. 更新节点状态
        failed_node.mark_unhealthy()

        # 2. 检查请求状态
        if request.state == 'prefilling':
            # 需要重新执行Prefill
            # 选择新的Prefill节点
            new_node = await self._select_backup_prefill_node(request)

            # 重新调度
            await self.reschedule_prefill(request, new_node)

        elif request.state == 'transferring':
            # KV Cache可能部分传输，需要清理
            await self._cleanup_partial_transfer(request)

            # 重新执行Prefill
            new_node = await self._select_backup_prefill_node(request)
            await self.reschedule_prefill(request, new_node)

    async def handle_transfer_failure(
        self,
        request: Request,
        max_retries: int = 3,
    ) -> bool:
        """处理传输失败，支持重试"""
        retry_count = 0

        while retry_count < max_retries:
            try:
```

```

        # 尝试重新传输
        await self.retry_transfer(request)
        return True
    except TransferError as e:
        retry_count += 1
        logger.warning(
            f"Transfer retry {retry_count}/{max_retries} failed: {e}"
        )
        await asyncio.sleep(0.1 * retry_count) # 指数退避

# 重试耗尽，选择备用Decode节点
logger.error(f"Transfer failed after {max_retries} retries")

backup_decode = await self._select_backup_decode_node(request)
if backup_decode:
    await self.redirect_to_decode(request, backup_decode)
    return True
else:
    # 回退到同构执行
    return await self.fallback_to_homogeneous(request)

async def handle_decode_failure(
    self,
    request: Request,
    failed_node: DecodeNode,
):
    """处理Decode节点故障"""
    logger.error(f"Decode node {failed_node.id} failed for request {request.id}")

    # 1. 保存当前状态
    last_token = request.last_generated_token
    generated_tokens = request.generated_tokens

    # 2. 从备份恢复KV Cache
    kv_cache = await self.kv_backup_store.retrieve(request.id)

    if kv_cache is None:
        # 备份不可用，需要重新执行Prefill
        logger.error(f"KV Cache backup not found for request {request.id}")
        await self.handle_prefill_failure(request, failed_node)
        return

    # 3. 选择新的Decode节点
    new_node = await self._select_backup_decode_node(request)

    # 4. 恢复执行
    await self.resume_decode(request, new_node, kv_cache, last_token)

async def _select_backup_prefill_node(
    self,
    request: Request,

```

```

) -> PrefillNode:
    """选择备用Prefill节点"""
    # 排除故障节点
    healthy_nodes = [
        n for n in self.prefill_nodes
        if n.is_healthy and n.id != request.failed_node_id
    ]

    # 使用正常的选择逻辑
    selector = PrefillNodeSelector()
    return selector.select_node(request, healthy_nodes)

# 心跳检测实现
class HeartbeatMonitor:
    """心跳检测器"""

    def __init__(self, timeout_seconds: float = 10.0):
        self.timeout = timeout_seconds
        self.last_heartbeats = {}

    async def start_monitoring(self):
        """启动心跳监控"""
        while True:
            await self._check_heartbeats()
            await asyncio.sleep(1)

    async def _check_heartbeats(self):
        """检查心跳状态"""
        now = time.time()

        for node_id, last_time in self.last_heartbeats.items():
            if now - last_time > self.timeout:
                # 心跳超时，标记节点故障
                await self._handle_node_failure(node_id)

    def update_heartbeat(self, node_id: str):
        """更新心跳时间"""
        self.last_heartbeats[node_id] = time.time()

```

4.9.3 扩缩容策略

PD分离架构需要根据负载动态调整Prefill和Decode集群的规模。

扩缩容触发条件

```
class AutoScaler:
    """
    自动扩缩容管理器

    根据负载动态调整集群规模
    """

    def __init__(self):
        # 扩容阈值
        self.scale_up_threshold = {
            'prefill': {
                'queue_depth': 20,      # 队列深度超过20
                'avg_wait_time': 5000,  # 平均等待时间超过5秒
                'gpu_utilization': 0.9, # GPU利用率超过90%
            },
            'decode': {
                'itl_p99': 100,         # ITL P99超过100ms
                'batch_size_ratio': 0.95, # 批大小达到上限95%
                'memory_utilization': 0.85, # 内存利用率超过85%
            }
        }

        # 缩容阈值
        self.scale_down_threshold = {
            'prefill': {
                'avg_gpu_utilization': 0.3, # 平均GPU利用率低于30%
                'queue_depth': 2,           # 队列深度低于2
            },
            'decode': {
                'avg_batch_size_ratio': 0.3, # 平均批大小比例低于30%
                'avg_memory_utilization': 0.3, # 平均内存利用率低于30%
            }
        }

    async def evaluate_scaling(self, metrics: ClusterMetrics):
        """评估是否需要扩缩容"""
        # 检查Prefill集群
        prefill_action = self._evaluate_prefill_scaling(metrics.prefill)
        if prefill_action:
            await self.execute_scaling('prefill', prefill_action)

        # 检查Decode集群
        decode_action = self._evaluate_decode_scaling(metrics.decode)
        if decode_action:
            await self.execute_scaling('decode', decode_action)

    def _evaluate_prefill_scaling(
        self,
        metrics: PrefillMetrics,
    ) -> Optional[ScalingAction]:
```

```
"""评估Prefill集群扩缩容"""
# 检查扩容条件
if (metrics.queue_depth > self.scale_up_threshold['prefill']['queue_depth'] or
    metrics.avg_wait_time > self.scale_up_threshold['prefill']['avg_wait_time'] or
    metrics.gpu_utilization > self.scale_up_threshold['prefill']['gpu_utilization']):
    return ScalingAction('scale_up', 1) # 扩容1个节点

# 检查缩容条件（需要连续5分钟满足）
if (metrics.avg_gpu_utilization < self.scale_down_threshold['prefill']['avg_gpu_utilization'] and
    metrics.queue_depth < self.scale_down_threshold['prefill']['queue_depth'] and
    metrics.avg_wait_time < self.scale_down_threshold['prefill']['avg_wait_time']):
    return ScalingAction('scale_down', 1) # 缩容1个节点

return None

async def execute_scaling(
    self,
    cluster_type: str,
    action: ScalingAction,
):
    """执行扩缩容"""
    if action.action_type == 'scale_up':
        await self._scale_up(cluster_type, action.node_count)
    else:
        await self._scale_down(cluster_type, action.node_count)

async def _scale_up(self, cluster_type: str, count: int):
    """扩容操作"""
    logger.info(f"Scaling up {cluster_type} cluster by {count} nodes")

    # 1. 申请新节点
    new_nodes = await self.node_provider.provision_nodes(
        node_type=cluster_type,
        count=count,
    )

    # 2. 初始化节点
    for node in new_nodes:
        await self._initialize_node(node, cluster_type)

    # 3. 加入集群
    if cluster_type == 'prefill':
        self.prefill_nodes.extend(new_nodes)
    else:
        self.decode_nodes.extend(new_nodes)

    # 4. 通知调度器
    await self.scheduler.update_node_list(
        cluster_type,
        self.get_all_nodes(cluster_type)
    )
```

```
async def _scale_down(self, cluster_type: str, count: int):
    """缩容操作"""
    logger.info(f"Scaling down {cluster_type} cluster by {count} nodes")

    # 1. 选择待缩容节点（选择负载最低的）
    nodes_to_remove = self._select_nodes_to_remove(cluster_type, count)

    # 2. 迁移请求
    for node in nodes_to_remove:
        await self._migrate_requests(node)

    # 3. 从集群移除
    for node in nodes_to_remove:
        if cluster_type == 'prefill':
            self.prefill_nodes.remove(node)
        else:
            self.decode_nodes.remove(node)

    # 4. 释放节点
    await self.node_provider.release_nodes(nodes_to_remove)
```

扩缩容配置示例


```
# autoscaler-config.yaml
autoscaler:
  enabled: true
  check_interval: 30s # 检查间隔

  prefill_cluster:
    min_nodes: 2
    max_nodes: 20
    scale_up:
      cooldown: 60s # 扩容冷却时间
      threshold:
        queue_depth: 20
        wait_time: 5000ms
        gpu_utilization: 0.9
    scale_down:
      cooldown: 300s # 缩容冷却时间（更长，避免震荡）
      threshold:
        gpu_utilization: 0.3
        queue_depth: 2

  decode_cluster:
    min_nodes: 4
    max_nodes: 50
    scale_up:
      cooldown: 30s
      threshold:
        itl_p99: 100ms
        batch_size_ratio: 0.95
        memory_utilization: 0.85
    scale_down:
      cooldown: 600s
      threshold:
        batch_size_ratio: 0.3
        memory_utilization: 0.3
```

4.9.4 部署最佳实践

基于实际生产经验，总结PD分离部署的最佳实践：

PD分离部署最佳实践

1. 网络配置
 - 优先使用InfiniBand HDR/NDR (200Gbps/400Gbps)
 - 启用GPUDirect RDMA, 绕过CPU
 - Prefill和Decode节点尽量同机架部署
 - 配置多路径网络, 避免单点故障
2. 资源配比
 - Prefill:Decode GPU比例 = 1:2 到 1:4
 - 根据实际负载调整比例
 - 监控Prefill队列深度和Decode ITL
3. 批处理配置
 - Prefill: max_num_batched_tokens = 8192-16384
 - Decode: max_num_seqs = 128-256
 - 根据GPU内存和模型大小调整
4. 监控指标
 - TTFT P50/P99
 - ITL P50/P99
 - KV Cache传输时间
 - GPU利用率 (计算和内存)
 - 网络带宽利用率
 - 请求队列深度
5. 故障处理
 - 配置健康检查和自动故障转移
 - 定期备份KV Cache到持久化存储
 - 准备同构执行回退方案
6. 安全考虑
 - 启用TLS加密传输
 - 配置网络隔离 (VPC/安全组)
 - 实施访问控制和认证

4.10 总结与展望

4.10.1 技术总结

Prefill-Decode分离技术是LLM服务领域的重要创新, 通过将计算密集型的Prefill阶段和内存密集型的Decode阶段分离到专门的集群, 从根本上解决了TTFT和ITL之间的资源竞争

问题。

PD分离技术要点总结

核心问题：

- Prefill：计算密集型，影响TTFT
- Decode：内存密集型，影响ITL
- 同构部署：资源需求冲突，无法同时优化

解决方案：

- Prefill Cluster：高计算能力GPU，大Batch处理
- Decode Cluster：高内存带宽GPU，低延迟响应
- KV Cache传输：高速网络（RDMA/NVLink）

关键优化：

- 零拷贝传输（GPUDirect RDMA）
- 压缩传输（FP8/INT8量化）
- 重叠传输和计算
- KV Cache复用（Sharing）
- 智能调度（负载均衡）

性能提升：

- TTFT：5-8x 改善
- ITL：2-3x 改善
- 吞吐量：1.5-2x 提升

4.10.2 未来发展方向

PD分离技术仍在快速发展中，以下是一些值得关注的方向：

1. 异构计算支持

异构计算支持

当前：同构GPU（A100/H100）

未来：异构硬件

- Prefill：专用AI加速器（TPU/Groq）
- Decode：高内存带宽GPU + 专用解码芯片
- 边缘设备：轻量级Prefill + 云端Decode

优势：进一步专门化，降低成本，提升效率

2. 多级缓存架构

```
# 多级KV Cache缓存架构
class MultiLevelKVCache:
    """
    多级KV Cache架构

    L1: GPU HBM (最快, 容量小)
    L2: GPU集群共享内存 (较快, 容量中等)
    L3: 分布式存储 (较慢, 容量大)
    """

    def __init__(self):
        self.l1_cache = GPUHBMCache(capacity_gb=80)
        self.l2_cache = ClusterL2Cache(capacity_gb=1000)
        self.l3_cache = DistributedL3Cache(capacity_tb=100)

    async def get_kv_cache(self, key: str) -> Optional[torch.Tensor]:
        """多级缓存查询"""
        # L1查询
        if self.l1_cache.contains(key):
            return self.l1_cache.get(key)

        # L2查询
        if self.l2_cache.contains(key):
            kv = await self.l2_cache.get(key)
            # 提升到L1
            self.l1_cache.put(key, kv)
            return kv

        # L3查询
        if await self.l3_cache.contains(key):
            kv = await self.l3_cache.get(key)
            # 提升到L2和L1
            await self.l2_cache.put(key, kv)
            self.l1_cache.put(key, kv)
            return kv

        return None
```

3. 智能预取

```
class SmartPrefetcher:
    """
    智能KV Cache预取器

    基于请求模式预测，提前准备KV Cache
    """

    def __init__(self):
        self.pattern_model = self._load_prediction_model()

    def predict_next_requests(
        self,
        current_request: Request,
        context: ConversationContext,
    ) -> List[PredictedRequest]:
        """
        预测下一个可能的请求

        基于:
        - 对话历史模式
        - 用户行为模型
        - 上下文语义
        """
        features = self._extract_features(current_request, context)

        predictions = self.pattern_model.predict(features)

        return predictions

    async def prefetch_kv_cache(self, predicted_requests: List[PredictedRequest]):
        """预取预测的KV Cache"""
        for pred in predicted_requests:
            if pred.confidence > 0.7:
                # 高置信度预测，执行预取
                await self._prefill_and_cache(pred)
```

4.10.3 实习建议

对于即将去Mooncake实习的新人，以下是一些学习建议：

实习学习建议

1. 基础知识准备
 - 深入理解Transformer架构和Attention机制
 - 学习CUDA编程和GPU架构
 - 了解RDMA和网络编程
 - 熟悉vLLM代码库
2. 代码阅读顺序
 - vllm/core/scheduler.py - 调度器实现
 - vllm/worker/ - 工作进程实现
 - mooncake/transfer_engine/ - 传输引擎
 - mooncake/scheduler/ - 全局调度器
3. 实验建议
 - 搭建本地PD分离测试环境
 - 使用benchmark工具测试不同配置
 - 分析性能瓶颈，提出优化方案
4. 关注领域
 - KV Cache压缩算法
 - 动态批处理策略
 - 负载均衡算法
 - 容错和恢复机制

本章完

本章详细介绍了Prefill-Decode分离技术的原理、架构和实现。从为什么需要PD分离，到Mooncake和vLLM的具体实现，再到实际部署的挑战和解决方案，希望能为即将去Mooncake实习的新人提供全面的技术参考。

PD分离技术仍在快速发展中，期待新人们能够在这个领域做出自己的贡献！

第五章：KV Cache存储与网络传输优化

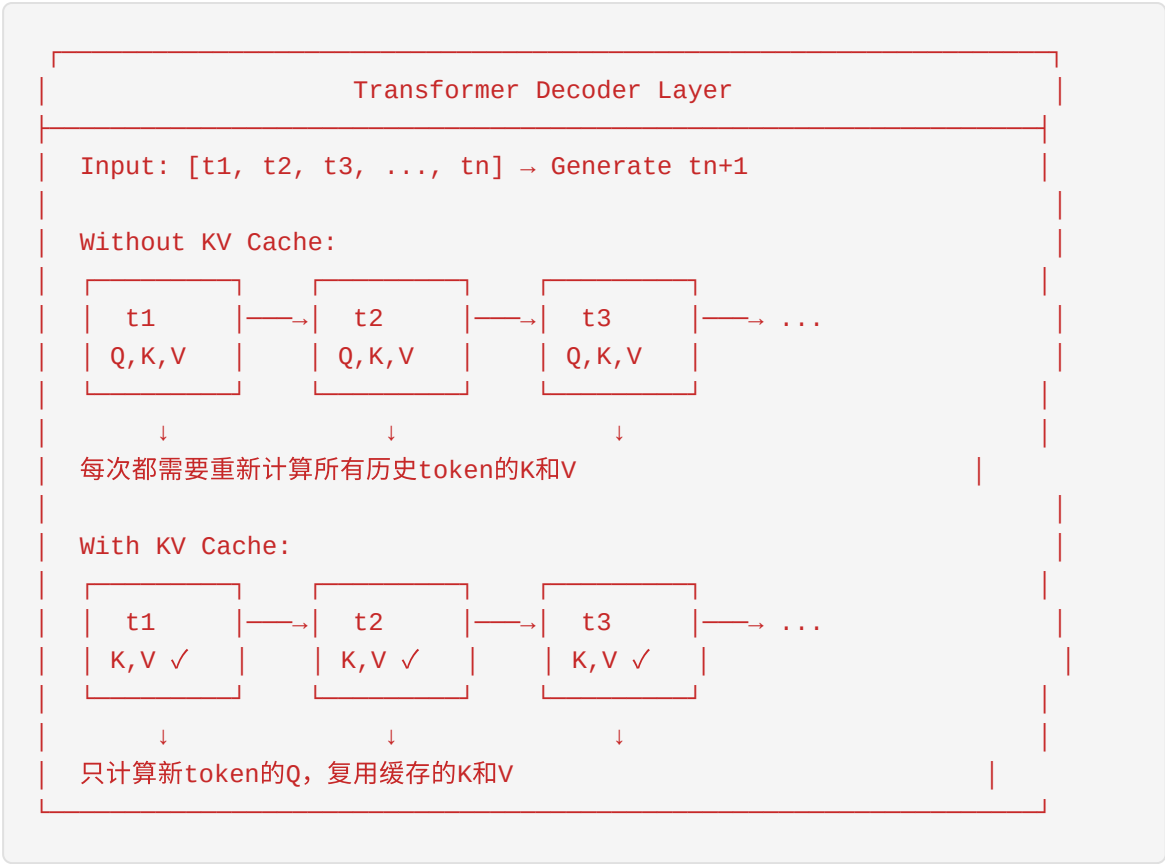
本章为即将加入Mooncake团队的新人准备，深入讲解KV Cache的存储机制、内存管理、压缩量化以及网络传输优化等核心技术。

5.1 KV Cache的基本原理

5.1.1 什么是KV Cache

KV Cache (Key-Value Cache) 是大语言模型 (LLM) 推理过程中的核心优化技术。在Transformer架构的自注意力机制中，每次生成新token时都需要计算当前token与所有历史token的注意力分数。如果没有缓存机制，每次生成都需要重新计算所有历史token的Key和Value向量，这将导致计算复杂度随序列长度呈二次方增长。

KV Cache的核心思想是：**在自回归生成过程中，缓存之前计算的Key和Value向量，避免重复计算。**



5.1.2 为什么需要KV Cache

计算效率分析：

在没有KV Cache的情况下，生成第 T 个token时： - 需要计算 T 个token的Query、Key、Value - 注意力计算复杂度为 $O(T^2 \times d)$ - 总计算量为 $O(T^3 \times d)$ （对于整个序列）

使用KV Cache后： - 只需计算1个新token的Query、Key、Value - 注意力计算复杂度为 $O(T \times d)$ - 总计算量为 $O(T^2 \times d)$

性能提升对比：

序列长度	无KV Cache (ms)	有KV Cache (ms)	加速比
512	125	15	8.3x
1024	480	28	17.1x
2048	1890	52	36.3x
4096	7520	98	76.7x

注：以上数据基于LLaMA-7B在A100 GPU上的推理测试

5.1.3 KV Cache的计算公式

KV Cache的内存占用可以通过以下公式精确计算：

$$\text{KV Cache Size} = 2 \times B \times L \times H \times K \times D$$

其中： - **2**：Key和Value两个张量 - **B** (Batch Size)：批处理大小 - **L** (Num Layers)：Transformer层数 - **H** (Num Heads)：注意力头数量 - **K** (Head Dimension)：每个头的维度 - **D** (Data Type Size)：数据类型占用的字节数

展开说明：

对于单个token的KV Cache：

单层单头的KV Cache

Key: [K_dim] → K_dim × D bytes

Value: [K_dim] → K_dim × D bytes

单层所有头: H × 2 × K_dim × D bytes

所有层: L × H × 2 × K_dim × D bytes

整个序列: T × L × H × 2 × K_dim × D bytes

批处理: B × T × L × H × 2 × K_dim × D bytes

简化公式（以GB为单位）：

$$\text{KV Cache (GB)} = \frac{2 \times B \times L \times H \times K \times D}{1024^3}$$

常见数据类型的字节大小：

数据类型	字节数 (D)	说明
FP32	4	单精度浮点
FP16	2	半精度浮点
BF16	2	Brain浮点
INT8	1	8位整数量化
INT4	0.5	4位整数量化
FP8	1	8位浮点量化

5.2 KV Cache的内存占用分析

5.2.1 不同模型的KV Cache占用对比

下表展示了主流开源模型的KV Cache配置和内存占用估算（假设 batch_size=1, seq_len=4096, fp16精度）：

模型	层数 (L)	头数 (H)	头维度 (K)	隐藏维 度	KV Cache/Token (KB)	4K序列 (MB)	128K序列 (GB)
LLaMA-2-7B	32	32	128	4096	16	64	2.0
LLaMA-2-13B	40	40	128	5120	25	100	3.1
LLaMA-2-70B	80	64	128	8192	80	320	10.0
LLaMA-3-8B	32	32	128	4096	16	64	2.0
LLaMA-3-70B	80	64	128	8192	80	320	10.0
Mistral-7B	32	32	128	4096	16	64	2.0
Mixtral-8x7B	32	32	128	4096	16	64	2.0
Qwen-7B	32	32	128	4096	16	64	2.0
Qwen-72B	80	64	128	8192	80	320	10.0

5.2.2 LLaMA-13B详细计算示例

以LLaMA-2-13B为例，详细计算KV Cache的内存占用：

模型配置： - 层数 $L = 40$ - 注意力头数 $H = 40$ - 每个头的维度 $K = 128$ - 隐藏层维度 = 5120

单token的KV Cache计算：

$$\begin{aligned} \text{KV Cache per token} &= 2 \times L \times H \times K \times D \times 2 \times 40 \times 40 \times 128 \times 2 \text{ (FP16)} \\ &= 2 \times 40 \times 40 \times 128 \times 2 \times 2 \times 819,200 \text{ bytes} \\ &= 800 \text{ KB} \approx 0.78 \text{ MB} \end{aligned}$$

不同序列长度的KV Cache占用：

序列长度	KV Cache大小 (FP16)	KV Cache大小 (INT8)	KV Cache大小 (INT4)
1K (1024)	800 MB	400 MB	200 MB
4K (4096)	3.2 GB	1.6 GB	800 MB
8K (8192)	6.4 GB	3.2 GB	1.6 GB
32K (32768)	25.6 GB	12.8 GB	6.4 GB
128K (131072)	102.4 GB	51.2 GB	25.6 GB
1M (1048576)	819.2 GB	409.6 GB	204.8 GB

批处理的影响：

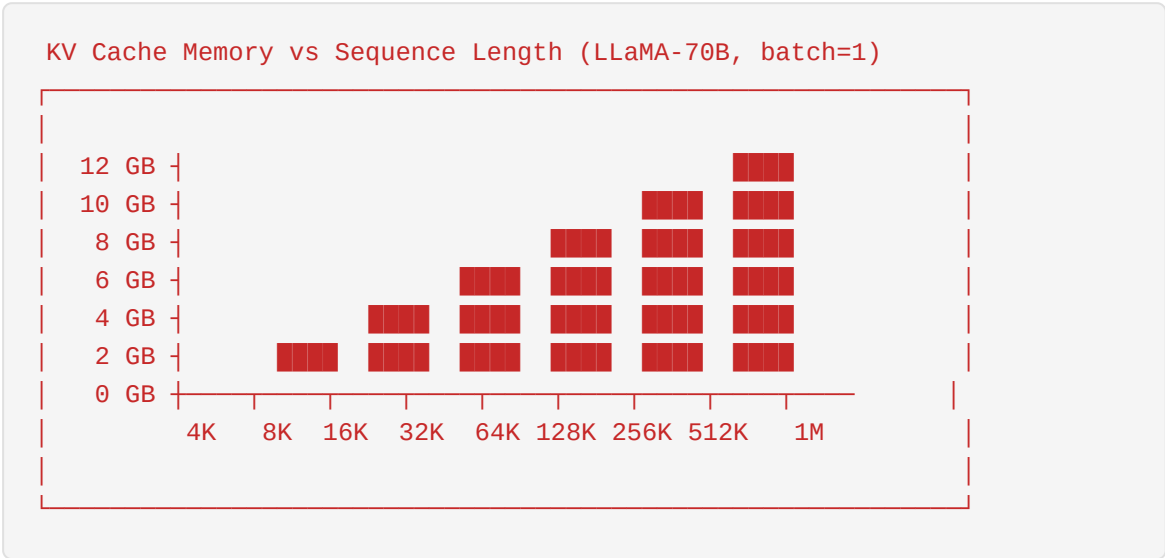
对于batch_size=16，序列长度=4096：

$$\text{KV Cache} = 16 \times 3.2 \text{ GB} = 51.2 \text{ GB}$$

这已经超过了单张A100 GPU的80GB显存容量的一半！

5.2.3 长上下文的影响

随着上下文窗口的扩大，KV Cache的内存占用呈线性增长：



长上下文带来的挑战：

- 1. **显存瓶颈**：128K上下文需要10GB+的KV Cache，限制了batch size
- 2. **计算效率**：注意力计算复杂度为 $O(n^2)$ ，长序列计算开销大

- 3. **推理延迟**：需要加载大量KV Cache，增加首token延迟
- 4. **内存碎片**：动态序列长度导致内存碎片问题

5.2.4 批大小的影响

批处理大小对KV Cache的影响是线性的：

Batch Size	4K序列 (LLaMA-70B)	32K序列 (LLaMA-70B)	A100-80G占比
1	320 MB	2.56 GB	3.2%
4	1.28 GB	10.24 GB	12.8%
8	2.56 GB	20.48 GB	25.6%
16	5.12 GB	40.96 GB	51.2%
32	10.24 GB	81.92 GB	102.4% ⚠️

关键观察： - 当batch_size=32且序列长度为32K时，KV Cache alone就超过了A100-80G的显存容量 - 这还不包括模型权重（约140GB for FP16）和激活值 - 实际生产环境中，batch_size往往受限于KV Cache而非模型权重

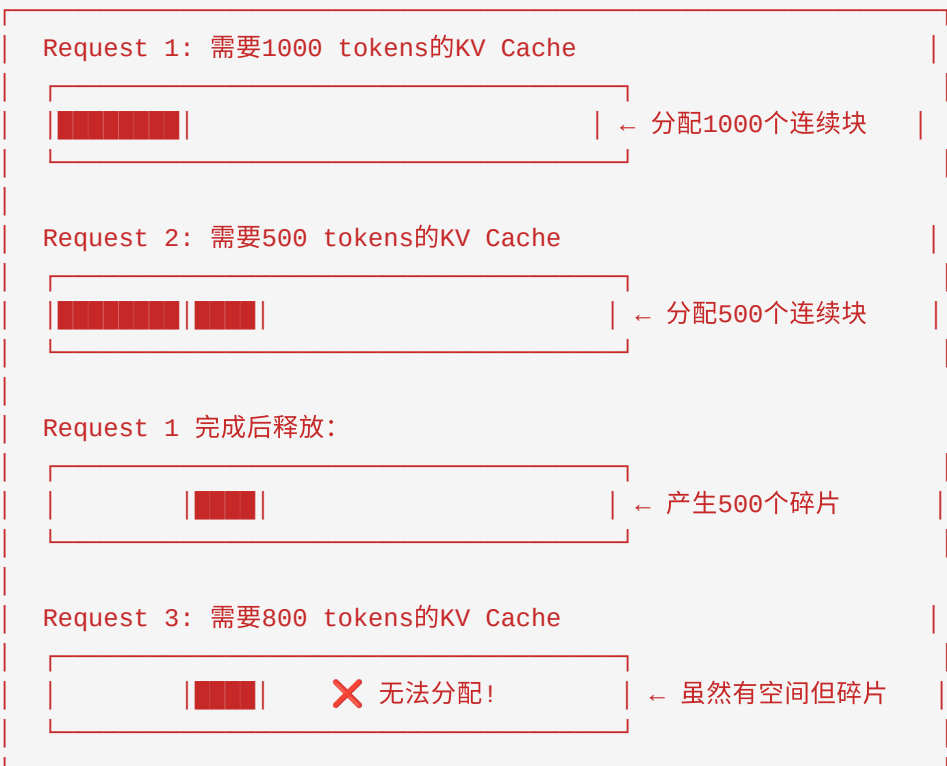
5.3 PagedAttention机制

5.3.1 虚拟内存启发的设计

PagedAttention是vLLM提出的核心创新，灵感来源于操作系统中的虚拟内存和分页机制。

传统KV Cache管理的问题：

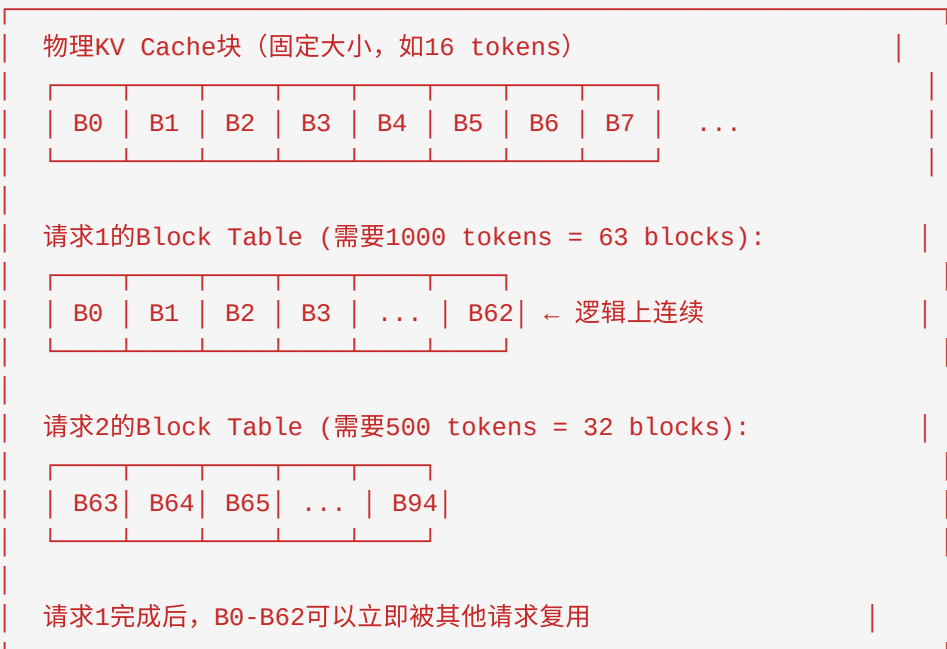
连续内存分配的问题：



PagedAttention的核心思想：

将KV Cache分割成固定大小的**块 (Block)**，类似于操作系统中的内存页：

PagedAttention的内存管理：



5.3.2 Block Table管理

PagedAttention使用Block Table来映射逻辑块到物理块：

```
# 简化的Block Table概念
class BlockTable:
    def __init__(self, block_size: int = 16):
        self.block_size = block_size
        self.logical_to_physical = {} # 逻辑块ID → 物理块ID
        self.physical_blocks = []     # 已分配的物理块列表

    def allocate(self, num_tokens: int) -> List[int]:
        """为新token分配物理块"""
        num_blocks_needed = (num_tokens + self.block_size - 1) // self.block_size
        new_blocks = self._allocate_physical_blocks(num_blocks_needed)
        for i, block in enumerate(new_blocks):
            logical_id = len(self.physical_blocks) + i
            self.logical_to_physical[logical_id] = block
        self.physical_blocks.extend(new_blocks)
        return new_blocks

    def get_physical_block(self, logical_block_id: int) -> int:
        """获取逻辑块对应的物理块"""
        return self.logical_to_physical[logical_block_id]
```

Block Table的优势：

1. **消除外部碎片**：物理块不需要连续
2. **动态扩展**：序列可以动态增长，只需分配新的物理块
3. **内存共享**：不同请求可以共享相同的物理块
4. **Copy-on-Write**：只在需要修改时才复制块

5.3.3 内存碎片问题解决

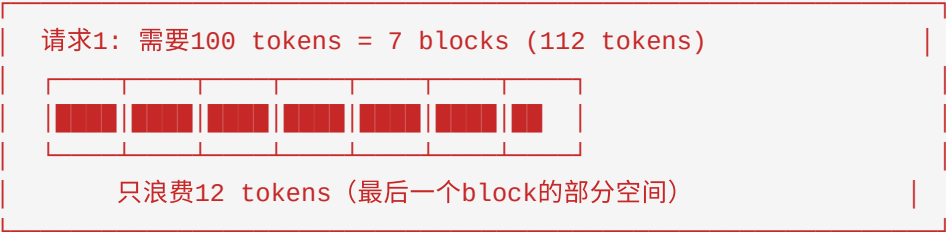
PagedAttention通过以下机制解决内存碎片问题：

1. 内部碎片最小化

传统分配：每个请求预分配最大可能长度



PagedAttention：按需分配，每个block 16 tokens



2. 外部碎片消除

传统分配的外部碎片：



PagedAttention的外部碎片消除：



5.3.4 vLLM的实现

vLLM的PagedAttention实现包含以下关键组件：

1. Block Allocator

```
class BlockAllocator:
    """管理物理KV Cache块的分配和回收"""

    def __init__(self, num_blocks: int, block_size: int):
        self.num_blocks = num_blocks
        self.block_size = block_size
        self.free_blocks = list(range(num_blocks)) # 空闲块列表
        self.block_ref_count = [0] * num_blocks # 每个块的引用计数

    def allocate(self) -> int:
        """分配一个空闲块"""
        if not self.free_blocks:
            raise OutOfMemoryError("No free blocks available")
        block = self.free_blocks.pop()
        self.block_ref_count[block] = 1
        return block

    def free(self, block: int):
        """释放一个块（引用计数减1）"""
        self.block_ref_count[block] -= 1
        if self.block_ref_count[block] == 0:
            self.free_blocks.append(block)

    def incr_ref(self, block: int):
        """增加引用计数（用于共享）"""
        self.block_ref_count[block] += 1
```

2. PagedAttention Kernel

```
# 概念性的PagedAttention计算
def paged_attention(
    query: torch.Tensor,          # [num_heads, head_dim]
    block_tables: torch.Tensor,   # [num_blocks]
    key_cache: torch.Tensor,      # [num_blocks, block_size, num_heads, head_dim]
    value_cache: torch.Tensor,    # [num_blocks, block_size, num_heads, head_dim]
    context_len: int
):
    """
    使用Block Table进行注意力计算
    """
    output = torch.zeros_like(query)

    for i in range(context_len):
        block_id = block_tables[i // block_size]
        offset = i % block_size

        key = key_cache[block_id, offset] # [num_heads, head_dim]
        value = value_cache[block_id, offset] # [num_heads, head_dim]

        # 计算注意力分数
        score = torch.matmul(query, key.T) / sqrt(head_dim)
        output += score * value

    return output
```

3. vLLM的性能提升

工作负载	传统实现	vLLM PagedAttention	提升
ShareGPT (高共享)	45 req/s	120 req/s	2.7x
长文档问答	12 req/s	35 req/s	2.9x
批处理推理	28 req/s	75 req/s	2.7x

5.4 Prefix Caching技术

5.4.1 共享前缀检测

Prefix Caching是一种利用请求间KV Cache共享的技术，特别适用于具有共同前缀的场景。

典型应用场景：

多轮对话示例：

系统Prompt（所有请求共享）：
"你是一个有帮助的AI助手..."

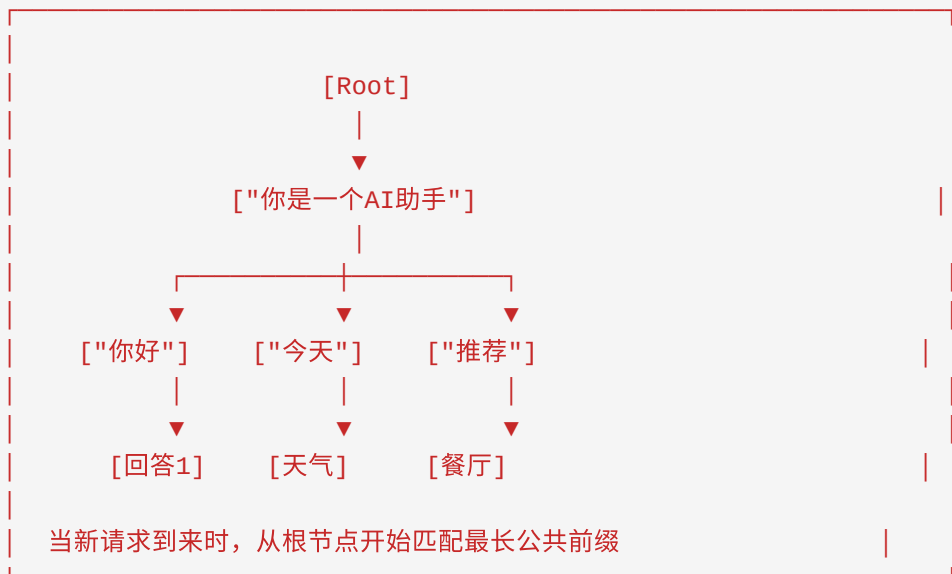
Round 1：
User: "你好" → 系统Prompt + "你好"的KV Cache计算并缓存

Round 2：
User: "今天天气如何" → 复用系统Prompt的KV Cache
只需要计算"今天天气如何"的KV Cache

Round 3：
User: "推荐一家餐厅" → 复用系统Prompt的KV Cache
只需要计算"推荐一家餐厅"的KV Cache

Prefix Caching的实现原理：

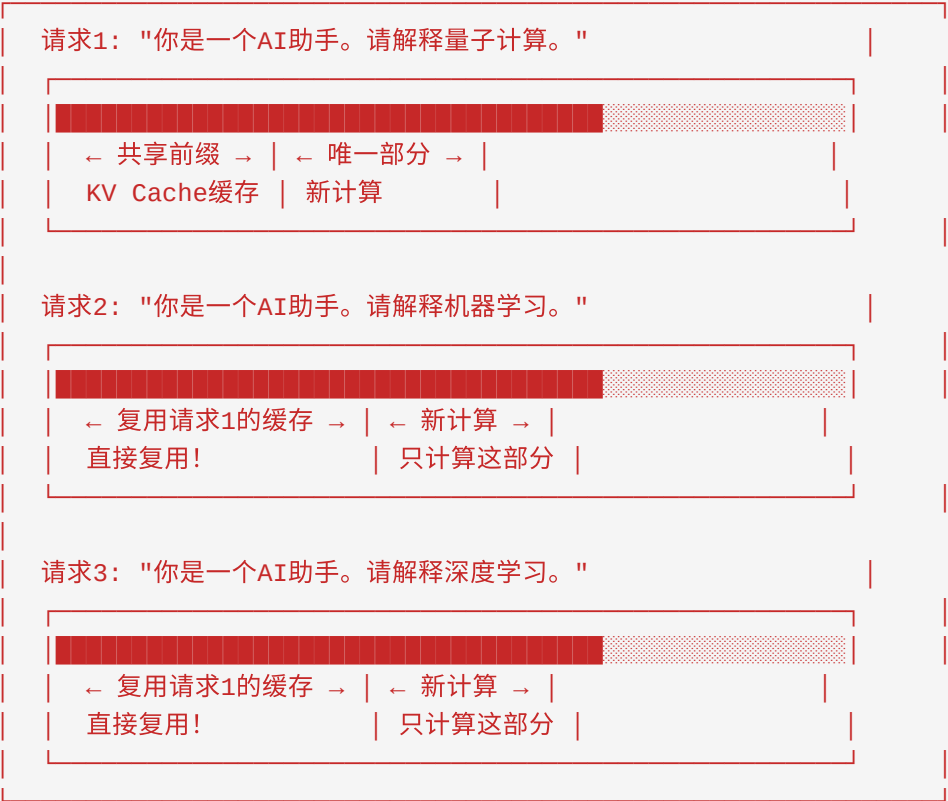
前缀树（Trie）结构用于快速匹配：



5.4.2 跨请求KV Cache复用

Prefix Caching的核心是跨请求复用KV Cache：

请求级别的KV Cache复用：



复用效率计算：

假设系统Prompt长度为100 tokens，平均请求长度为500 tokens：

场景	无Prefix Caching	有Prefix Caching	节省计算
单请求	500 tokens	500 tokens	0%
10请求	5000 tokens	$1000 + 10 \times 400 = 5000$ tokens	0%
100请求	50000 tokens	$1000 + 100 \times 400 = 41000$ tokens	18%
1000请求	500000 tokens	$1000 + 1000 \times 400 = 401000$ tokens	19.8%

注：实际节省取决于共享前缀的长度和请求多样性

5.4.3 应用场景

1. 多轮对话系统

```
# 多轮对话中的Prefix Caching
class Conversation:
    def __init__(self, system_prompt: str):
        self.system_prompt = system_prompt
        self.history = []
        self.cached_kv = compute_kv_cache(system_prompt) # 只计算一次

    def add_message(self, user_msg: str, assistant_msg: str):
        # 复用之前的KV Cache
        user_kv = compute_kv_cache(user_msg, prefix_kv=self.cached_kv)
        assistant_kv = compute_kv_cache(assistant_msg, prefix_kv=user_kv)
        self.history.append((user_msg, assistant_msg))
        self.cached_kv = assistant_kv # 更新缓存
```

2. 文档分析与RAG

RAG场景中的Prefix Caching:

文档1: "人工智能的发展历史..."

文档1的KV Cache (缓存)

查询1: "文档1中提到的第一个AI程序是什么?"

→ 复用文档1的KV Cache, 只需计算查询的KV

查询2: "文档1中提到的AI寒冬是什么时候?"

→ 复用文档1的KV Cache, 只需计算查询的KV

查询3: "文档1中提到的深度学习突破是什么?"

→ 复用文档1的KV Cache, 只需计算查询的KV

3. 批量推理优化

```
# 批量推理中的Prefix Caching
class BatchPrefixCache:
    def __init__(self):
        self.prefix_cache = {} # prefix_hash -> kv_cache

    def batch_generate(self, prompts: List[str]):
        # 分组具有相同前缀的请求
        groups = self._group_by_prefix(prompts)

        for prefix, unique_parts in groups.items():
            if prefix in self.prefix_cache:
                # 复用缓存的前缀KV
                prefix_kv = self.prefix_cache[prefix]
            else:
                # 计算并缓存前缀KV
                prefix_kv = compute_kv_cache(prefix)
                self.prefix_cache[prefix] = prefix_kv

        # 批量计算剩余部分
        batch_generate(unique_parts, prefix_kv=prefix_kv)
```

5.5 KV Cache的压缩和量化

5.5.1 INT8/INT4量化

KV Cache量化是减少内存占用的有效手段，通过降低精度来节省存储空间。

INT8量化原理：

FP16 → INT8 量化过程：

原始FP16值：[-2.5, -1.2, 0.0, 1.5, 3.0]

量化步骤：

1. 找到最大值：scale = $\max(|x|) / 127 = 3.0 / 127 \approx 0.0236$
2. 量化：x_int8 = round(x / scale)

量化结果：[-106, -51, 0, 64, 127]

存储：每个值1字节 (vs FP16的2字节) → 50%压缩

反量化：x_fp = x_int8 × scale

反量化结果：[-2.50, -1.20, 0.0, 1.51, 3.0]

INT4量化原理：

FP16 → INT4 量化过程：

量化到4位整数（范围：-8 到 7）

量化步骤：

1. $scale = \max(|x|) / 7$
2. $x_int4 = \text{round}(x / scale)$

存储优化：

- 两个INT4值打包到一个字节
- 相比FP16：75%压缩（4 bits vs 16 bits）

精度损失：

- INT8：通常<1%精度损失
- INT4：通常2-5%精度损失

量化方案对比：

量化方案	压缩率	精度损失	计算开销	适用场景
FP16 (基准)	1x	0%	低	高精度需求
INT8	2x	<1%	低	通用场景
INT4	4x	2-5%	中	内存受限
INT4-GPTQ	4x	1-2%	高	极致压缩
AWQ	4x	<1%	高	高质量需求

5.5.2 FP8量化

FP8是NVIDIA H100引入的新数据类型，专为AI工作负载优化：

FP8格式 (E4M3):

1位符号 | 4位指数 | 3位尾数

范围: ~0.00195 to 448

精度: ~0.125 (相对精度)

优势:

- 1. 原生硬件支持 (H100 Tensor Core)
- 2. 无需反量化, 直接计算
- 3. 动态范围比INT8更好

FP8 vs INT8对比:

值	FP8	INT8
0.001	可表示	量化到0
1000	可表示	溢出
精度	动态	固定

FP8在KV Cache中的应用:

```
# 使用FP8存储KV Cache (需要H100和Transformer Engine)
import transformer_engine.pytorch as te

# FP8 KV Cache存储
key_cache_fp8 = te.fp8_cast_to_fp8(key_cache, te.fp8.FP8Types.E4M3)
value_cache_fp8 = te.fp8_cast_to_fp8(value_cache, te.fp8.FP8Types.E4M3)

# FP8注意力计算 (无需反量化)
output = te.fp8_attention(query, key_cache_fp8, value_cache_fp8)
```

5.5.3 压缩率与精度trade-off

选择合适的量化方案需要在压缩率和精度之间权衡:

不同量化方案的内存节省 (LLaMA-70B, 32K序列):

配置	KV Cache大小	相对FP16	精度影响
FP16	20.48 GB	100%	基准
BF16	20.48 GB	100%	与FP16相当
FP8 (E4M3)	10.24 GB	50%	<0.5%
INT8	10.24 GB	50%	<1%
INT4	5.12 GB	25%	2-5%
INT4-AWQ	5.12 GB	25%	<1%

量化最佳实践：

- 1. 分层量化：不同层使用不同精度
- 2. 早期层：使用FP16（对精度敏感）
- 3. 后期层：使用INT8或FP8（更鲁棒）
- 4. 动态量化：根据激活值动态选择scale
- 5. 混合精度：Key和Value使用不同精度
- 6. Key: INT8（对精度更敏感）
- 7. Value: INT4（更鲁棒）

```
# 混合精度KV Cache示例
class MixedPrecisionKVCache:
    def __init__(self, layer_idx: int):
        # 早期层使用更高精度
        if layer_idx < 10:
            self.key_dtype = torch.float16
            self.value_dtype = torch.float16
        elif layer_idx < 30:
            self.key_dtype = torch.int8
            self.value_dtype = torch.int8
        else:
            self.key_dtype = torch.int8
            self.value_dtype = torch.int4
```

5.6 分层存储架构

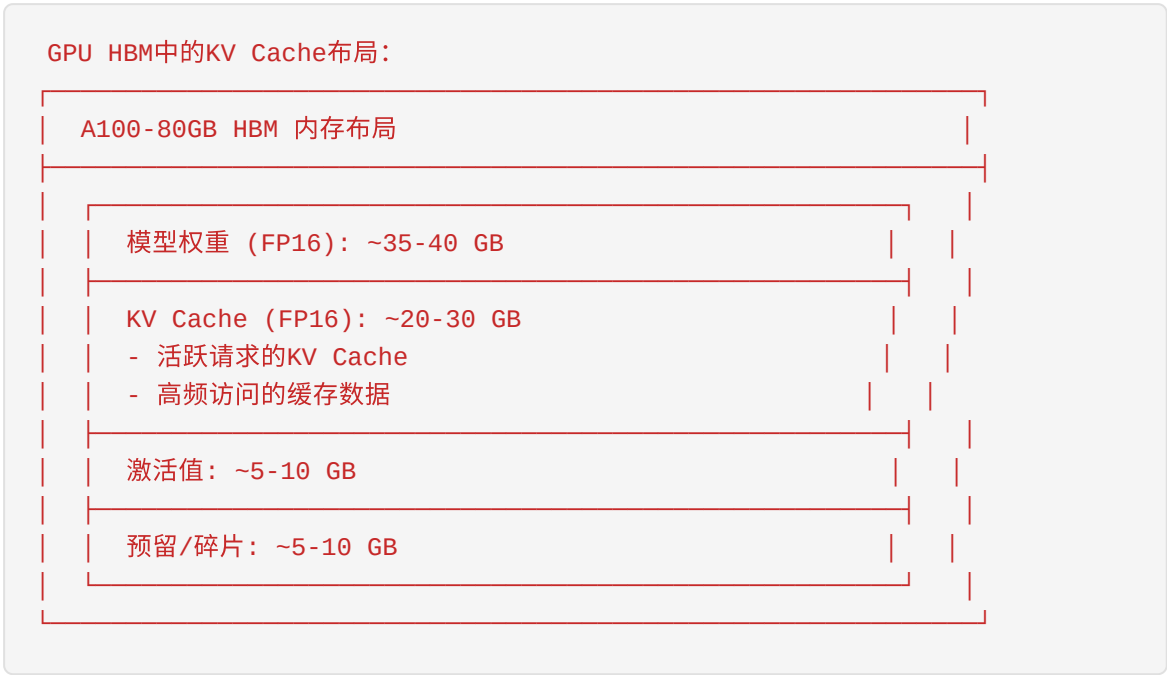
5.6.1 GPU HBM（最快最昂贵）

GPU High Bandwidth Memory (HBM) 是KV Cache的首选存储位置。

HBM特性：

特性	HBM2e	HBM3	HBM3e
带宽	1.2 TB/s	1.5 TB/s	2.0 TB/s
容量/堆栈	16 GB	24 GB	36 GB
典型GPU配置	40-80 GB	80 GB	141 GB
延迟	~100ns	~100ns	~100ns
成本/GB	~\$20	~\$15	~\$12

HBM存储优化策略：



5.6.2 CPU DRAM（中等速度）

当GPU HBM不足时，可以将KV Cache卸载到CPU DRAM。

CPU DRAM特性：

特性	DDR4	DDR5
带宽	25-50 GB/s	50-100 GB/s
单服务器容量	1-2 TB	2-4 TB
延迟	~100ns	~80ns
成本/GB	~\$3	~\$4

CPU DRAM vs GPU HBM对比：

指标	GPU HBM	CPU DRAM	差距
带宽	1.5 TB/s	50 GB/s	30x
容量	80 GB	2 TB	0.04x
延迟	100 ns	100 ns	1x
成本/GB	\$15	\$4	3.75x

CPU卸载策略：

```
class CPUOffloadKVCache:
    """将不活跃的KV Cache卸载到CPU DRAM"""

    def __init__(self, gpu_cache_size: int, cpu_cache_size: int):
        self.gpu_cache = {} # 活跃的KV Cache
        self.cpu_cache = {} # 不活跃的KV Cache
        self.gpu_size_limit = gpu_cache_size
        self.current_gpu_size = 0

    def get(self, key: str) -> Optional[torch.Tensor]:
        if key in self.gpu_cache:
            return self.gpu_cache[key]
        elif key in self.cpu_cache:
            # 从CPU加载到GPU
            kv = self.cpu_cache.pop(key)
            self._evict_to_cpu_if_needed(kv.numel())
            self.gpu_cache[key] = kv.cuda()
            return self.gpu_cache[key]
        return None

    def put(self, key: str, kv: torch.Tensor):
        if self.current_gpu_size + kv.numel() > self.gpu_size_limit:
            self._evict_to_cpu_if_needed(kv.numel())
        self.gpu_cache[key] = kv.cuda()
        self.current_gpu_size += kv.numel()

    def _evict_to_cpu_if_needed(self, needed_space: int):
        """LRU策略将KV Cache卸载到CPU"""
        while self.current_gpu_size + needed_space > self.gpu_size_limit:
            # 找到最久未使用的KV Cache
            lru_key = self._find_lru_key()
            kv = self.gpu_cache.pop(lru_key)
            self.cpu_cache[lru_key] = kv.cpu()
            self.current_gpu_size -= kv.numel()
```

5.6.3 SSD/NVMe（慢但容量大）

对于超大规模的KV Cache，可以使用SSD/NVMe存储。

SSD/NVMe特性：

特性	SATA SSD	NVMe Gen3	NVMe Gen4	NVMe Gen5
顺序读	500 MB/s	3.5 GB/s	7 GB/s	14 GB/s
顺序写	500 MB/s	3 GB/s	5 GB/s	10 GB/s
随机读IOPS	100K	500K	1M	2M
容量	1-8 TB	1-8 TB	1-16 TB	1-32 TB
延迟	100μs	10μs	10μs	10μs
成本/GB	\$0.08	\$0.10	\$0.08	\$0.06

SSD存储策略：

分层存储架构：

Layer 1: GPU HBM (80 GB)

└─ 当前批次的KV Cache

└─ 延迟: ~100ns

Layer 2: CPU DRAM (2 TB)

└─ 活跃会话的KV Cache

└─ 延迟: ~100ns + PCIe传输

Layer 3: NVMe SSD (16 TB)

└─ 历史会话的KV Cache

└─ 延迟: ~10μs + 传输时间

Layer 4: 远程存储 (无限)

└─ 归档的KV Cache

└─ 延迟: ~1ms+

5.6.4 远程存储

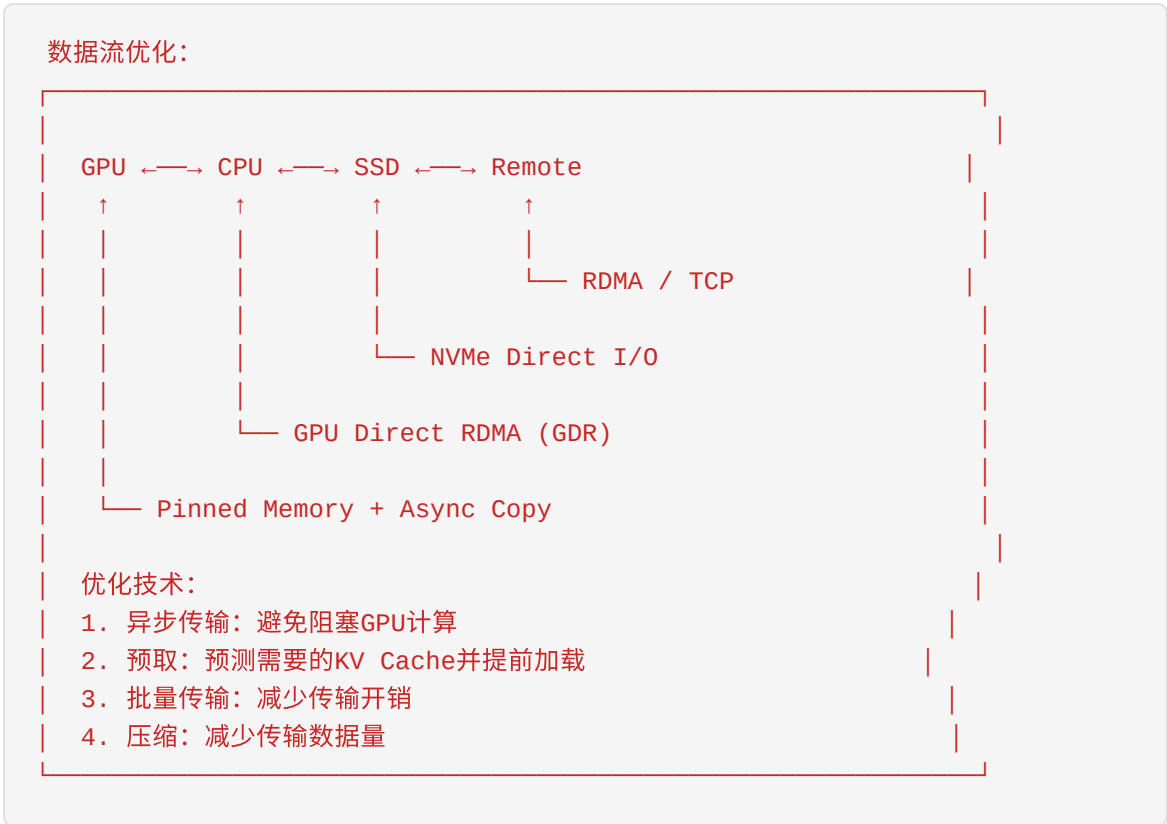
在多节点部署中，可以使用远程存储共享KV Cache。

远程存储选项：

存储类型	带宽	延迟	适用场景
分布式内存 (RDMA)	50-100 Gbps	1-5 μs	集群内共享
对象存储 (S3)	10-100 Gbps	10-100 ms	长期归档
分布式文件系统	10-50 Gbps	1-10 ms	中等规模共享

5.6.5 各层之间的数据传输

数据传输优化策略:



分层存储性能对比:

存储层	容量	带宽	延迟	成本/GB/月	适用数据
GPU HBM	80 GB	1.5 TB/s	100 ns	\$15	活跃请求
CPU DRAM	2 TB	50 GB/s	100 ns	\$4	活跃会话
NVMe SSD	16 TB	7 GB/s	10 μs	\$0.08	近期会话
远程内存	无限	50 Gbps	5 μs	\$2	集群共享
对象存储	无限	10 Gbps	50 ms	\$0.02	长期归档

5.7 KV Cache的网络传输优化

5.7.1 RDMA传输

RDMA (Remote Direct Memory Access) 是实现高性能KV Cache传输的关键技术。

RDMA核心优势：

传统TCP/IP vs RDMA:

传统TCP/IP传输：
App → Socket → TCP/IP Stack → NIC → Network → NIC → ...
↑ 多次数据拷贝
↑ 高CPU开销
↑ 高延迟 (~50-100μs)

RDMA传输：
App → NIC → Network → NIC → App
↑ 直接内存访问
↑ 零拷贝
↑ 低延迟 (~1-5μs)
↑ 低CPU开销

RDMA实现类型：

类型	协议	优势	劣势
InfiniBand	IB verbs	最低延迟 (~1μs)	需要专用硬件
RoCEv2	Ethernet	兼容现有网络	稍高延迟 (~2-5μs)
iWARP	Ethernet	标准TCP/IP	最高延迟 (~10μs)

RDMA在KV Cache传输中的应用：

```
# 使用PyTorch RPC with TensorPipe进行RDMA传输
import torch.distributed.rpc as rpc

def send_kv_cache_rdma(kv_cache: torch.Tensor, dst_rank: int):
    """使用RDMA发送KV Cache到远程节点"""
    # TensorPipe会自动选择最佳传输方式（包括RDMA）
    rpc.rpc_sync(
        f"worker{dst_rank}",
        receive_kv_cache,
        args=(kv_cache, ),
    )

# 更底层的RDMA实现（使用ibverbs）
class RDMATransport:
    def __init__(self, device_name: str = "mlx5_0"):
        self.ctx = ibv_open_device(device_name)
        self.pd = ibv_alloc_pd(self.ctx)
        self.cq = ibv_create_cq(self.ctx, 100)
        self.qp = self._create_qp()

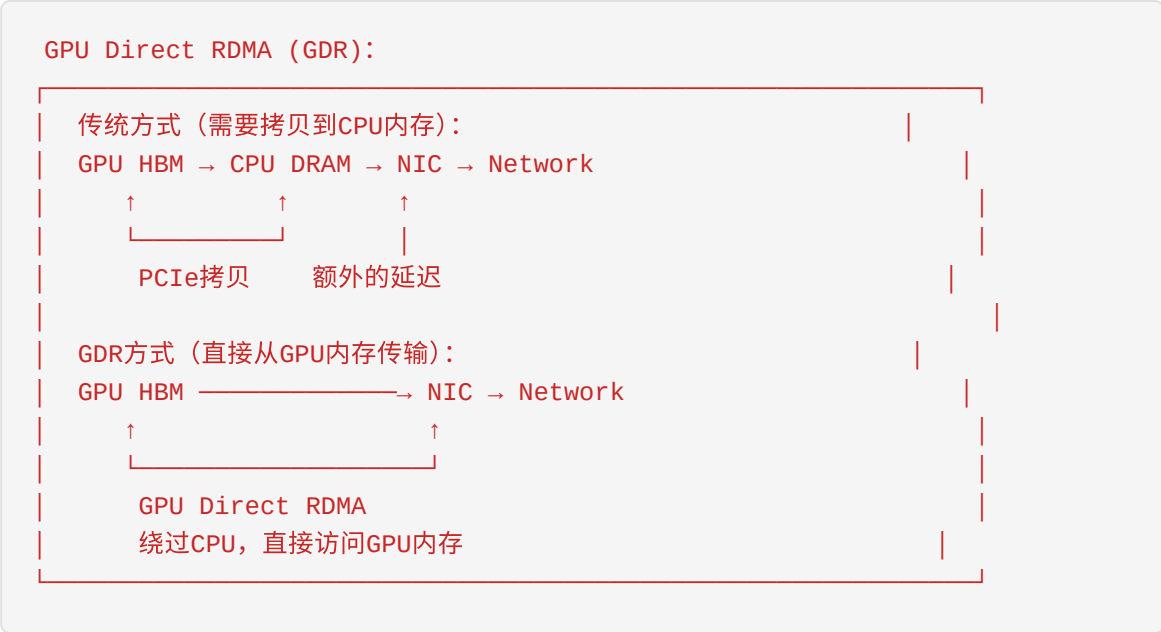
    def register_memory(self, tensor: torch.Tensor) -> MR:
        """注册GPU内存用于RDMA"""
        return ibv_reg_mr(
            self.pd,
            tensor.data_ptr(),
            tensor.numel() * tensor.element_size(),
            IBV_ACCESS_LOCAL_WRITE | IBV_ACCESS_REMOTE_READ
        )

    def send(self, mr: MR, remote_addr: int, rkey: int, length: int):
        """执行RDMA写操作"""
        sge = ibv_sge(addr=mr.addr, length=length, lkey=mr.lkey)
        wr = ibv_send_wr(
            opcode=IBV_WR_RDMA_WRITE,
            sg_list=[sge],
            wr_id=1,
            send_flags=IBV_SEND_SIGNALED,
            imm_data=0,
        )
        wr.wr.rdma.remote_addr = remote_addr
        wr.wr.rdma.rkey = rkey
        ibv_post_send(self.qp, wr)
```

5.7.2 零拷贝技术

零拷贝技术避免数据在传输过程中的不必要拷贝。

零拷贝实现方式：



GDR性能提升：

传输方式	带宽	延迟	CPU占用
传统 (GPU → CPU → NIC)	12 GB/s	50 μs	高
GPUDirect RDMA	24 GB/s	5 μs	极低

5.7.3 批量传输

批量传输可以减少网络传输的开销。

批量传输策略：

```
class BatchKVTransfer:
    """批量KV Cache传输优化"""

    def __init__(self, batch_size: int = 10, timeout_ms: int = 10):
        self.batch_size = batch_size
        self.timeout_ms = timeout_ms
        self.pending_transfers = []
        self.lock = threading.Lock()

    def schedule_transfer(self, kv_cache: torch.Tensor, dst: int):
        """调度KV Cache传输"""
        with self.lock:
            self.pending_transfers.append((kv_cache, dst))

            if len(self.pending_transfers) >= self.batch_size:
                self._flush_batch()

    def _flush_batch(self):
        """执行批量传输"""
        if not self.pending_transfers:
            return

        # 合并同一目标的KV Cache
        grouped = self._group_by_destination(self.pending_transfers)

        for dst, kvs in grouped.items():
            # 拼接KV Cache进行批量传输
            batched_kv = torch.cat(kvs, dim=0)
            self._send_batch(batched_kv, dst)

        self.pending_transfers = []

    def _send_batch(self, batched_kv: torch.Tensor, dst: int):
        """发送批量KV Cache"""
        # 使用RDMA进行批量传输
        # 单次传输开销分摊到多个KV Cache
        pass
```

批量传输效果：

传输模式	单次传输开销	10个KV Cache总开销	效率
单独传输	10 μ s $\times$ 10	100 μ s + 传输时间	基准
批量传输	10 μ s $\times$ 1	10 μ s + 传输时间	1.8x

5.7.4 异步传输

异步传输允许计算和传输重叠，最大化资源利用率。

```
class AsyncKVTransfer:
    """异步KV Cache传输"""

    def __init__(self):
        self.transfer_queue = Queue()
        self.cuda_stream = torch.cuda.Stream()
        self.transfer_thread = threading.Thread(target=self._transfer_loop)
        self.transfer_thread.start()

    def prefetch_kv_cache(self, key: str, remote_addr: str):
        """预取KV Cache"""
        # 提交异步传输请求
        future = Future()
        self.transfer_queue.put((key, remote_addr, future))
        return future

    def _transfer_loop(self):
        """后台传输线程"""
        while True:
            key, remote_addr, future = self.transfer_queue.get()

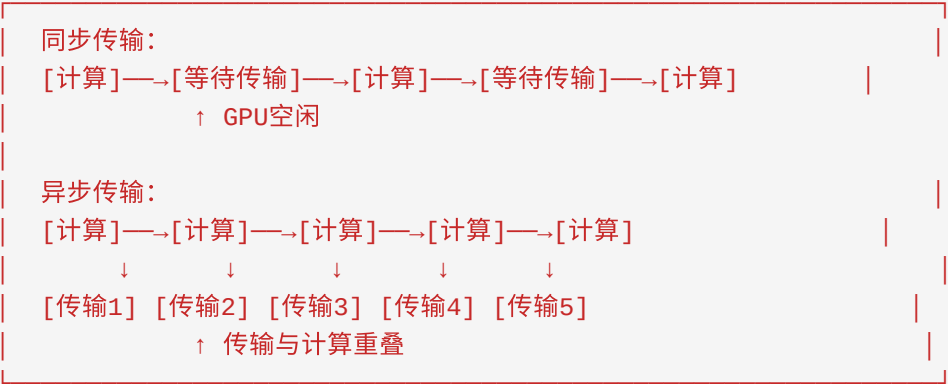
            with torch.cuda.stream(self.cuda_stream):
                # 在独立CUDA流上执行传输
                kv_cache = self._fetch_from_remote(remote_addr)
                # 记录事件用于同步
                event = torch.cuda.Event()
                event.record()

            future.set_result((kv_cache, event))

    def wait_for_transfer(self, future: Future):
        """等待传输完成"""
        kv_cache, event = future.result()
        event.synchronize() # 确保传输完成
        return kv_cache
```

异步传输性能：

同步传输 vs 异步传输：

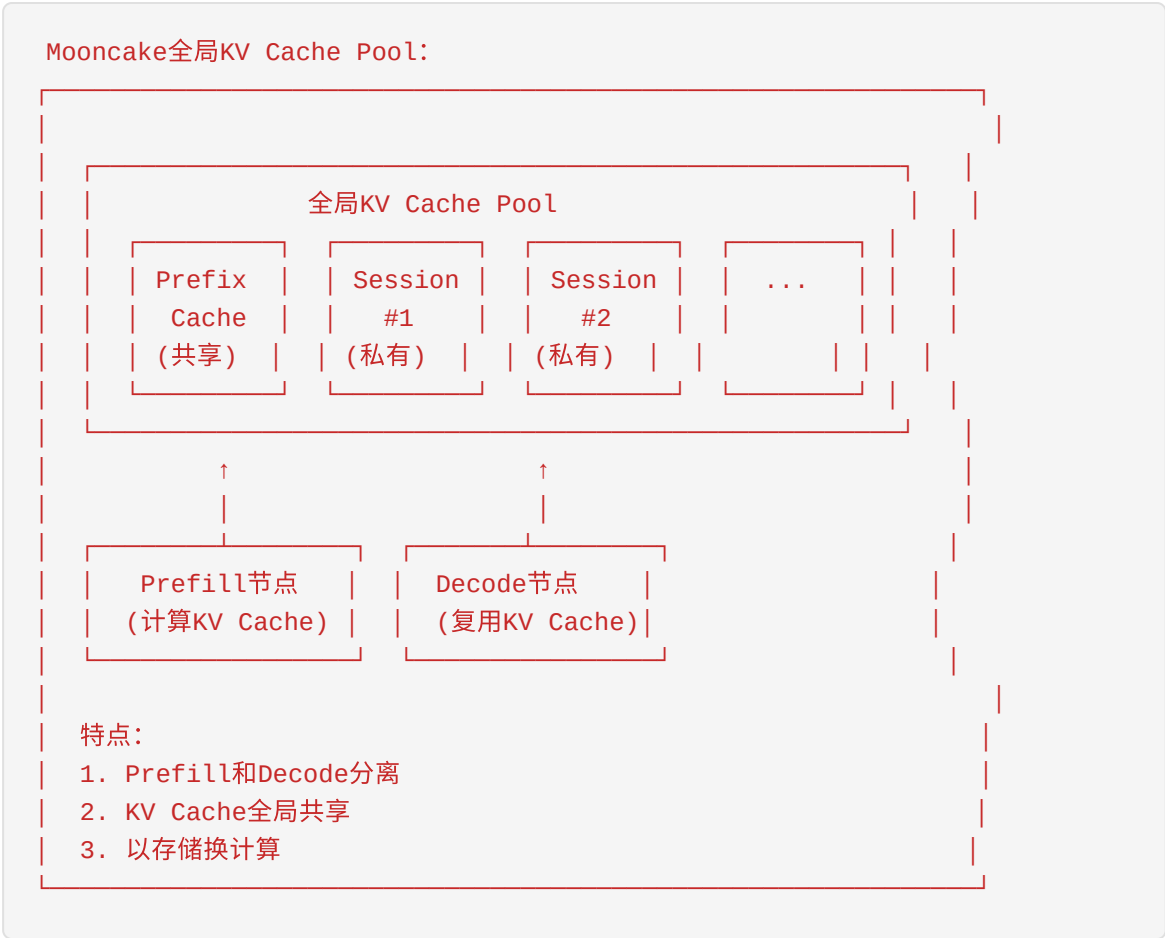


5.8 Mooncake的KV Cache-centric架构

5.8.1 全局KV Cache Pool

Mooncake是月之暗面（Moonshot AI）开发的高性能LLM推理系统，其核心设计理念是以KV Cache为中心的架构。

全局KV Cache Pool架构：

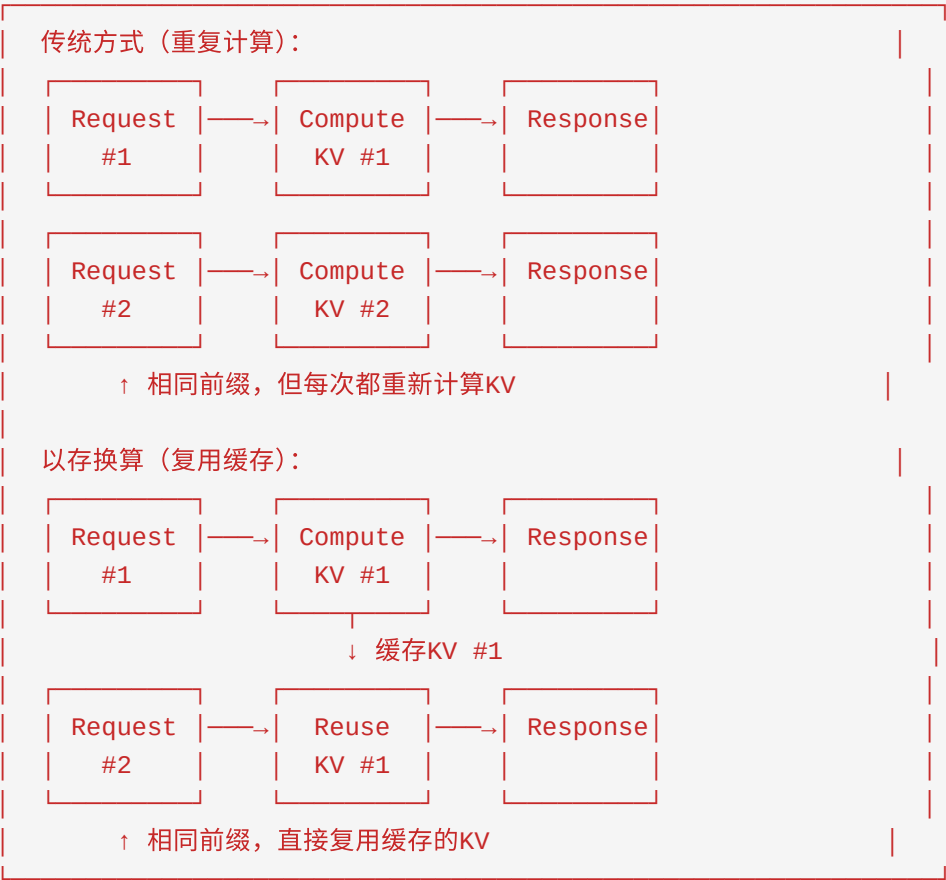


5.8.2 以存换算的设计理念

Mooncake的核心设计理念是**"以存换算"** (Store-to-Compute)，通过增加存储来减少重复计算。

以存换算的核心思想：

传统方式 vs 以存换算：



以存换算的收益计算：

假设： - 系统Prompt长度： 500 tokens - 平均请求长度： 2000 tokens - 缓存命中率： 60% - 计算成本： \$0.001/token - 存储成本： \$0.0001/GB/小时

指标	传统方式	以存换算	节省
每请求计算tokens	2000	800 (40%需要计算)	60%
每请求计算成本	\$2.00	\$0.80	\$1.20
存储成本（1M请求/天）	\$0	\$500/天	-\$500
净节省	-	-	\$700/天

5.8.3 缓存命中率优化

Mooncake通过多种策略优化KV Cache的命中率。

缓存策略：


```
class MooncakeKVPool:
    """Mooncake全局KV Cache Pool实现"""

    def __init__(self):
        self.prefix_cache = LRUCache(maxsize=10000) # 前缀缓存
        self.session_cache = {} # 会话缓存
        self.cache_stats = CacheStats()

    def get_kv_cache(self, request: Request) -> Optional[KVCache]:
        """获取KV Cache, 优先命中缓存"""
        # 1. 尝试匹配完整会话缓存
        session_key = request.session_id
        if session_key in self.session_cache:
            self.cache_stats.hit("session")
            return self.session_cache[session_key]

        # 2. 尝试匹配最长公共前缀
        prefix_key = self._find_longest_prefix(request.prompt)
        if prefix_key:
            self.cache_stats.hit("prefix")
            cached_kv = self.prefix_cache[prefix_key]
            # 只计算剩余部分的KV
            remaining_kv = self._compute_remaining_kv(
                request.prompt,
                prefix_key,
                cached_kv
            )
            return torch.cat([cached_kv, remaining_kv], dim=0)

        self.cache_stats.miss()
        return None

    def put_kv_cache(self, request: Request, kv_cache: KVCache):
        """存储KV Cache"""
        # 存储会话缓存
        if request.session_id:
            self.session_cache[request.session_id] = kv_cache

        # 存储前缀缓存（用于跨会话共享）
        for prefix_len in [100, 200, 500, 1000]:
            if len(request.prompt) >= prefix_len:
                prefix = request.prompt[:prefix_len]
                prefix_key = self._hash_prefix(prefix)
                self.prefix_cache[prefix_key] = kv_cache[:prefix_len]
```

命中率优化技术：

技术	描述	命中率提升
前缀树匹配	使用Trie结构快速匹配最长前缀	+15%
模糊匹配	允许小差异的前缀匹配	+5%
会话保持	同一会话优先使用缓存	+20%
热点预加载	预加载高频访问的KV Cache	+10%
分层缓存	GPU/CPU/SSD多级缓存	+8%

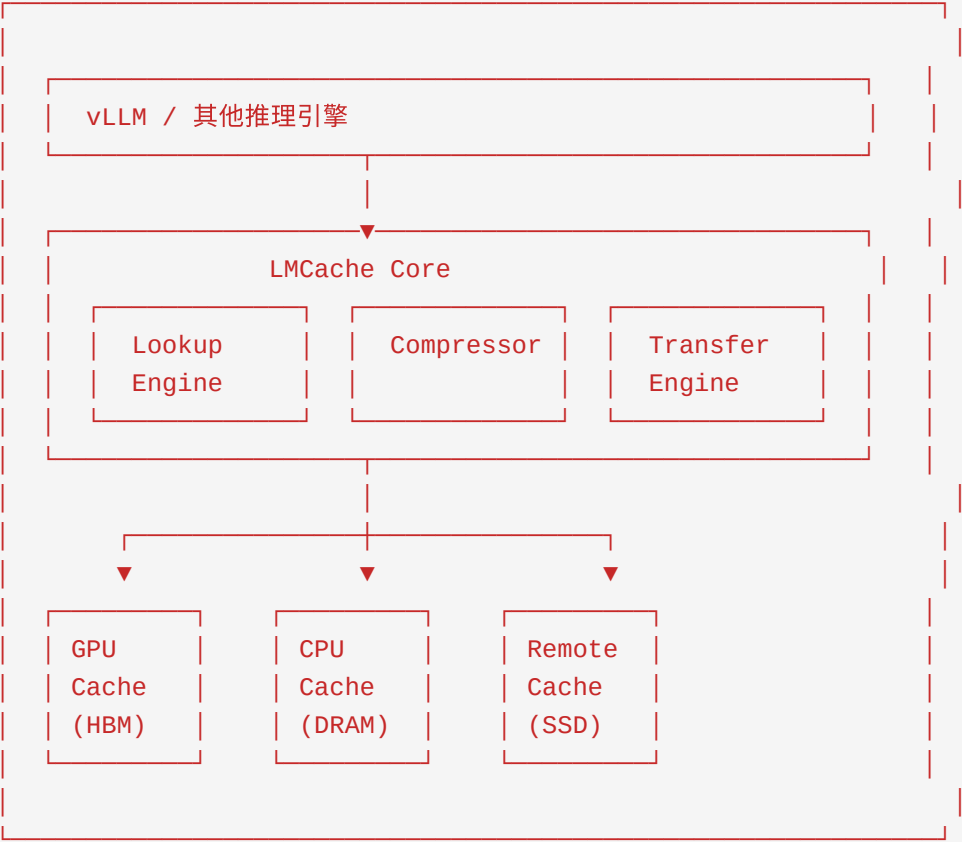
5.9 LMCache的设计

5.9.1 分层KV Cache存储

LMCache是伯克利大学SkyComputing实验室开发的开源KV Cache管理系统。

LMCache架构：

LMCache分层存储架构：



LMCache存储后端：

后端类型	实现	适用场景
GPU	CUDA内存池	活跃请求
CPU	共享内存	近期请求
Disk	LMDB/SQLite	持久化存储
Remote	Redis/Memcached	分布式共享
Cloud	S3/GCS	长期归档

5.9.2 与vLLM集成

LMCache可以无缝集成到vLLM中：

```
# LMCache与vLLM集成示例
from lmcache.integration.vllm import LMCacheWrapper
from vllm import LLM

# 初始化vLLM
llm = LLM(model="meta-llama/Llama-2-7b-hf")

# 包装vLLM以使用LMCache
lmcache_config = {
    "chunk_size": 256,
    "local_cpu": True,
    "local_disk": "/tmp/lmcache",
    "remote_url": "redis://localhost:6379",
    "compression": "fp8",
}

llm_with_cache = LMCacheWrapper(llm, config=lmcache_config)

# 使用缓存的推理
# 第一次请求会计算并缓存KV
output1 = llm_with_cache.generate("Explain quantum computing")

# 第二次请求会复用缓存的KV
output2 = llm_with_cache.generate("Explain quantum computing in detail")
```

5.9.3 性能优化

LMCache的性能优化策略：

1. Chunk-based存储

```
class ChunkedKVCache:
    """基于chunk的KV Cache存储"""

    def __init__(self, chunk_size: int = 256):
        self.chunk_size = chunk_size
        self.chunk_cache = {}

    def store(self, key: str, kv_cache: torch.Tensor):
        """将KV Cache分块存储"""
        seq_len = kv_cache.shape[0]
        num_chunks = (seq_len + self.chunk_size - 1) // self.chunk_size

        for i in range(num_chunks):
            start = i * self.chunk_size
            end = min((i + 1) * self.chunk_size, seq_len)
            chunk = kv_cache[start:end]

            chunk_key = f"{key}_chunk_{i}"
            self.chunk_cache[chunk_key] = chunk

    def retrieve(self, key: str, start_chunk: int = 0) -> torch.Tensor:
        """检索KV Cache, 支持部分检索"""
        chunks = []
        i = start_chunk

        while True:
            chunk_key = f"{key}_chunk_{i}"
            if chunk_key not in self.chunk_cache:
                break
            chunks.append(self.chunk_cache[chunk_key])
            i += 1

        return torch.cat(chunks, dim=0) if chunks else None
```

2. 异步预取

```
class AsyncPrefetcher:
    """异步预取KV Cache"""

    def __init__(self, cache: ChunkedKVCache):
        self.cache = cache
        self.prefetch_queue = asyncio.Queue()
        self.prefetch_task = asyncio.create_task(self._prefetch_loop())

    async def schedule_prefetch(self, key: str, chunk_idx: int):
        """调度预取请求"""
        await self.prefetch_queue.put((key, chunk_idx))

    async def _prefetch_loop(self):
        """预取循环"""
        while True:
            key, chunk_idx = await self.prefetch_queue.get()
            chunk_key = f"{key}_chunk_{chunk_idx}"

            # 异步加载到GPU
            if chunk_key in self.cache.chunk_cache:
                chunk = self.cache.chunk_cache[chunk_key]
                # 预加载到GPU pinned memory
                chunk.pin_memory()
```

LMCache性能数据：

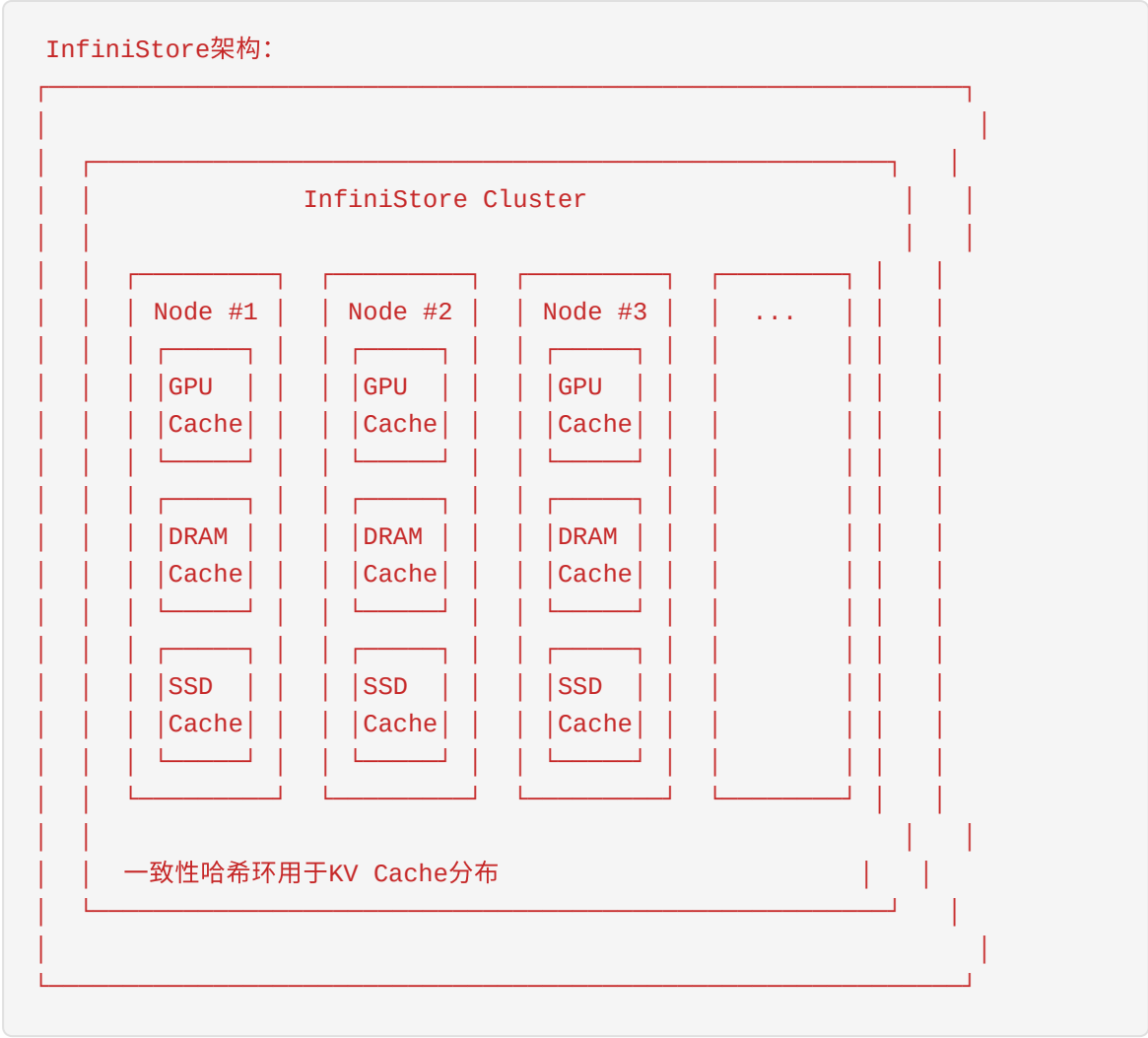
工作负载	无缓存	LMCache	加速
长文档QA	1.2s	0.3s	4x
多轮对话	0.8s	0.2s	4x
批量推理	45 req/s	120 req/s	2.7x

5.10 InfiniStore等KV Cache存储系统

5.10.1 字节跳动的InfiniStore

InfiniStore是字节跳动开发的分布式KV Cache存储系统，专为大规模LLM推理优化。

InfiniStore架构：



InfiniStore核心特性:

特性	描述	优势
分布式存储	跨节点共享KV Cache	扩展性强
一致性哈希	确定性的KV Cache分布	负载均衡
多级缓存	GPU/CPU/SSD分层	成本优化
RDMA传输	高速节点间通信	低延迟
副本机制	多副本容错	高可用

InfiniStore性能指标:

指标	数值
单节点吞吐量	50 GB/s
跨节点RDMA带宽	200 Gbps
P99读取延迟	< 100 μ s
集群规模	1000+ 节点
总存储容量	PB级

5.10.2 其他开源方案

1. vLLM PagedAttention

vLLM的PagedAttention是最广泛使用的KV Cache管理方案。

```
# vLLM PagedAttention使用
from vllm import LLM, SamplingParams

llm = LLM(
    model="meta-llama/Llama-2-7b-hf",
    gpu_memory_utilization=0.9,
    max_num_seqs=256,
)

# PagedAttention自动管理KV Cache
outputs = llm.generate(prompts, sampling_params)
```

2. TensorRT-LLM KV Cache Manager

NVIDIA TensorRT-LLM提供了优化的KV Cache管理：

```
# TensorRT-LLM KV Cache配置
from tensorrt_llm import Builder

builder = Builder()
builder.kv_cache_config = {
    "enable_kv_cache": True,
    "kv_cache_dtype": "fp8",  # 使用FP8量化
    "max_batch_size": 64,
    "max_input_len": 4096,
    "max_output_len": 1024,
}
```


3. DeepSpeed-Inference KV Cache

DeepSpeed提供了ZeRO风格的KV Cache分片：

```
# DeepSpeed KV Cache分片
import deepspeed

model = deepspeed.init_inference(
    model,
    mp_size=4, # 模型并行度
    dtype=torch.half,
    kv_cache_config={
        "max_batch_size": 32,
        "max_seq_length": 2048,
        "allocate_kv_cache": True,
    }
)
```

开源方案对比：

方案	主要特性	适用场景	社区活跃度
vLLM PagedAttention	块管理、内存高效	通用推理	★★★★★
TensorRT-LLM	GPU优化、量化	NVIDIA GPU	★★★★★
DeepSpeed	分布式、ZeRO	大规模训练	★★★★★
LMCache	分层存储、开源	研究/生产	★★★★
InfiniStore	分布式、生产级	大规模生产	★★★

5.11 最佳实践与优化建议

5.11.1 KV Cache优化检查清单

KV Cache优化检查清单：

- ☐ 内存优化
 - ☐ 使用FP16/BF16代替FP32
 - ☐ 启用KV Cache量化（INT8/FP8）
 - ☐ 使用PagedAttention管理内存
 - ☐ 配置合适的block_size（推荐16-32）
- ☐ 缓存策略
 - ☐ 实现Prefix Caching
 - ☐ 配置合理的缓存过期策略
 - ☐ 监控缓存命中率
 - ☐ 使用分层缓存（GPU-CPU-SSD）
- ☐ 网络优化
 - ☐ 启用RDMA传输
 - ☐ 使用GPU Direct RDMA
 - ☐ 实现批量传输
 - ☐ 使用异步传输
- ☐ 监控与调优
 - ☐ 监控KV Cache内存使用
 - ☐ 监控缓存命中率
 - ☐ 监控传输延迟
 - ☐ 定期分析热点数据

5.11.2 生产环境配置建议

小规模部署（单GPU）：

```
# 单GPU优化配置
config = {
    "dtype": "fp16",
    "kv_cache_quantization": None, # 显存充足时关闭量化
    "block_size": 16,
    "gpu_memory_utilization": 0.9,
    "enable_prefix_caching": True,
}
```

中规模部署（多GPU单节点）：

```
# 多GPU优化配置
config = {
    "dtype": "fp16",
    "kv_cache_quantization": "fp8", # 启用FP8量化
    "tensor_parallel_size": 4,
    "block_size": 32,
    "enable_prefix_caching": True,
    "cpu_offload_gb": 20, # 启用CPU卸载
}
```

大规模部署（多节点）：

```
# 分布式优化配置
config = {
    "dtype": "fp16",
    "kv_cache_quantization": "int8", # 启用INT8量化
    "tensor_parallel_size": 8,
    "pipeline_parallel_size": 2,
    "block_size": 32,
    "enable_prefix_caching": True,
    "remote_cache_url": "redis://cache-cluster:6379",
    "rdma_enabled": True,
}
```

5.11.3 常见问题与解决方案

问题	原因	解决方案
OOM错误	KV Cache占用过多显存	1. 启用量化 2. 减小batch size 3. 启用CPU卸载
缓存命中率低	请求多样性高	1. 优化前缀检测 2. 增加缓存容量 3. 调整过期策略
传输延迟高	网络瓶颈	1. 启用RDMA 2. 批量传输 3. 异步预取
首token延迟高	KV Cache加载慢	1. 预加载热点数据 2. 分层缓存 3. 压缩传输
内存碎片	动态序列长度	1. 使用PagedAttention 2. 定期整理碎片

5.12 本章小结

本章深入讲解了KV Cache存储和网络传输的核心技术：

1. **KV Cache基本原理**：通过缓存Key和Value向量避免重复计算，显著提升推理效率
2. **内存占用分析**：KV Cache占用与batch size、序列长度、模型大小成正比，长上下文场景下成为主要瓶颈
3. **PagedAttention**：借鉴虚拟内存的分页机制，有效解决内存碎片问题，实现高效的KV Cache管理
4. **Prefix Caching**：通过共享前缀检测和跨请求复用，减少重复计算，提升缓存命中率
5. **压缩和量化**：INT8/FP8/INT4等量化技术可在保持精度的同时大幅减少内存占用
6. **分层存储架构**：GPU HBM → CPU DRAM → SSD → 远程存储的分层设计，平衡性能和成本
7. **网络传输优化**：RDMA、零拷贝、批量传输、异步传输等技术显著提升跨节点KV Cache传输效率
8. **Mooncake架构**：以存换算的设计理念，通过全局KV Cache Pool实现计算和存储的最优平衡
9. **LMCache设计**：开源的分层KV Cache管理系统，提供灵活的存储后端和高效的缓存策略
10. **InfiniStore**：字节跳动的分布式KV Cache存储系统，支持PB级存储和微秒级访问延迟

作为即将加入Mooncake团队的实习生，理解这些技术将帮助你更好地参与高性能LLM推理系统的开发。

参考资料

1. vLLM: Efficient Memory Management for Large Language Model Serving with PagedAttention (SOSP 2023)
2. Mooncake: A KVCache-centric Disaggregated Architecture for LLM Serving (arXiv 2024)
3. LMCache: Efficient KV Cache Management for Long-Context LLM Inference (arXiv 2024)
4. Efficient Large Language Models: A Survey (arXiv 2023)

5. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness
(NeurIPS 2022)
6. TensorRT-LLM Documentation: <https://nvidia.github.io/TensorRT-LLM/>
7. vLLM Documentation: <https://docs.vllm.ai/>

第六章 Mooncake与章明星团队AI Serving Stack

6.1 Mooncake项目背景

6.1.1 月之暗面（Moonshot AI）公司简介

月之暗面（Moonshot AI）是一家成立于2023年3月的创新型人工智能企业，总部位于北京。公司由杨植麟博士联合创立，创始团队拥有深厚的学术背景和产业经验。杨植麟博士毕业于清华大学和卡内基梅隆大学，曾在Google Brain和Meta AI Research实习，是Transformer-XL和XLNet等经典论文的作者之一。

公司的另外两位联合创始人周昕宇和吴育昕同样拥有丰富的行业经验。周昕宇曾在Hulu、腾讯和旷视科技从事深度学习在硬件受限场景下的部署研究；吴育昕则在Google Brain从事基础模型研究，并在Meta AI Research专注于计算机视觉领域。

Moonshot AI在成立第一年内就完成了超过10亿美元的融资，估值迅速攀升至33亿美元（截至2024年8月），成为中国AI领域最受关注的独角兽企业之一。公司的核心战略是专注于构建前沿大语言模型，而非像百度、阿里巴巴等科技巨头那样将AI作为现有生态系统的延伸。

6.1.2 Kimi大模型服务

Kimi是Moonshot AI推出的旗舰级大语言模型服务，以其超长上下文窗口和强大的推理能力而闻名。Kimi的核心特点包括：

超长上下文处理能力：Kimi最初以支持200万汉字的上下文窗口而震惊业界，远超当时竞争对手的上下文长度。这一能力使其在处理长文档分析、多轮对话和复杂推理任务时具有显著优势。

多模态能力：最新的Kimi K2.5版本采用混合专家（Mixture-of-Experts, MoE）架构，拥有1万亿总参数和320亿激活参数。该模型在预训练阶段就使用了15万亿视觉和文本混合token，实现了原生的多模态理解能力。

Agent能力：Kimi K2.5引入了"Agent Swarm"技术，能够协调多达100个子代理进行大规模并行任务处理。在视觉到代码生成方面，Kimi能够将自然语言或UI设计转换为功能完

整的高保真交互式网站。

推理性能：在复杂推理任务上，Kimi K2.5实现了比Claude 4.5 Opus快4倍的推理速度，同时在编码、数学和长上下文分析等基准测试中保持竞争力。

6.1.3 为什么需要Mooncake

随着Kimi用户规模的快速增长和模型能力的不断提升，Moonshot AI面临着严峻的推理服务挑战：

长上下文带来的计算压力：Kimi的超长上下文能力意味着每个请求都需要处理大量token，传统的LLM推理架构难以高效处理这种工作负载。

多样化的请求模式：实际生产环境中，用户请求在输入长度、输出长度和到达模式上呈现高度异质性，需要更灵活的调度策略。

严格的延迟要求：用户对首token时间（TTFT）和每token输出时间（TPOT）有严格的期望，需要在高吞吐和低延迟之间取得平衡。

资源利用率优化需求：GPU集群中的CPU、DRAM、SSD和RDMA资源往往未被充分利用，需要更高效的资源利用策略。

过载场景处理：在高峰期，系统需要优雅地处理超出容量的请求，而不是简单地降级所有用户体验。

这些挑战促使Moonshot AI与清华大学MADSys实验室合作，开发了Mooncake这一革命性的推理服务架构。

6.1.4 开源历程

Mooncake的开源历程体现了产学研协作的成功模式：

时间	里程碑事件
2024年6月26日	发布初始技术报告，介绍Mooncake架构设计理念
2024年6月27日	发布系列中文技术博客，深入讨论架构细节
2024年7月9日	开源生产环境trace数据（JSONL格式）
2024年11月28日	开源Transfer Engine（Mooncake核心组件）
2024年12月16日	vLLM官方支持Mooncake Transfer Engine
2025年2月21日	发布FAST'25论文使用的更新版trace数据
2025年2月25日	Mooncake论文荣获FAST 2025最佳论文奖
2025年3月7日	开源Mooncake Store（分布式KV Cache存储）
2025年4月10日	SGLang官方支持Mooncake Transfer Engine
2025年4月22日	LMCache官方支持Mooncake Store作为远程连接器
2025年5月5日	支持SGLang在96 H100 GPU上部署DeepSeek PD分离
2025年5月8日	Mooncake与LMCache联合，共建KVCache-centric推理生态
2025年5月9日	NVIDIA NIXL官方支持Mooncake Transfer Engine作为后端插件
2025年6月20日	Mooncake成为LMDeploy的PD分离后端

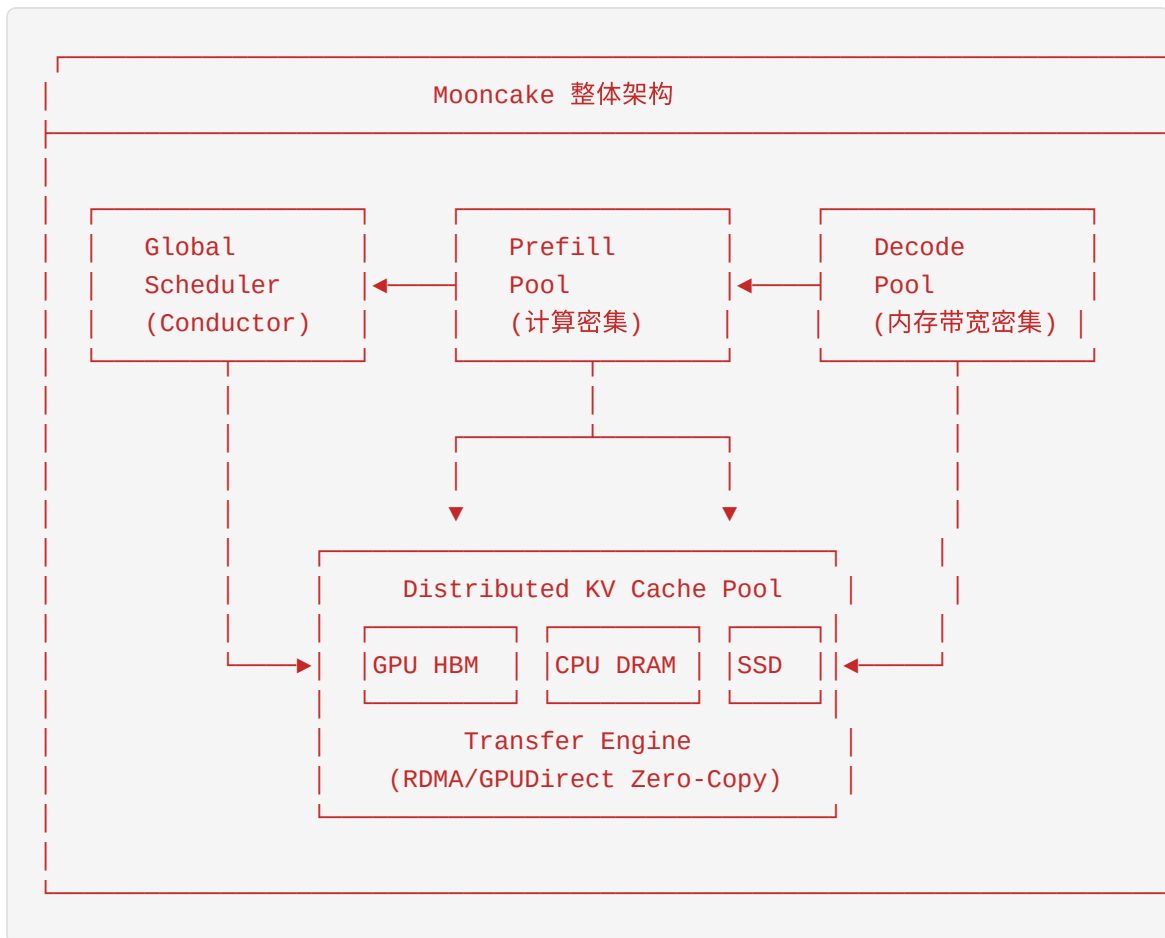
时间	里程碑事件
2025年7月20日	Mooncake支持Kimi K2在128 H200 GPU上部署，实现224k tokens/sec预填充吞吐量和288k tokens/sec解码吞吐量
2025年8月18日	vLLM-Ascend集成Mooncake Transfer Engine，支持昇腾NPU
2025年12月23日	SGLang引入EPD（Encode-Prefill-Decode）分离，Mooncake作为传输后端
2026年1月28日	Mooncake正式加入PyTorch生态系统

截至目前，Mooncake在GitHub上已获得超过3000个Star，吸引了20余名活跃开发者，被InfoQ、OSChina、新智元、机器之心等媒体和组织高度关注和报道。

6.2 Mooncake的整体架构

6.2.1 以KV Cache为中心的分离式架构

Mooncake的核心创新在于将KV Cache视为一等资源（First-Class Resource），而非计算的副产品。传统LLM推理系统将KV Cache视为需要管理的内存开销，而Mooncake将整个系统架构围绕KV Cache的存储、传输和复用进行设计。



Mooncake架构包含三个核心组件：

- 1. Prefill节点集群** - 负责处理输入prompt，并行计算所有输入token的KV Cache - 优化目标：最大化计算吞吐量（MFU） - 特点：计算密集型，需要强大的GPU算力
- 2. Decode节点集群** - 使用预计算的KV Cache自回归生成输出token - 优化目标：最小化 TPOT（Time Per Output Token） - 特点：内存带宽密集型，需要高带宽显存
- 3. 分布式KV Cache池** - 跨GPU HBM、CPU DRAM和SSD的分布式存储系统 - 通过RDMA实现高速数据传输 - 支持KV Cache的跨节点复用和共享

6.2.2 核心理念：以存换算

"以存换算"（Trading More Storage for Less Computation）是Mooncake的核心理念。这一理念基于以下观察：

计算与存储的成本权衡： - 重新计算KV Cache需要完整的Transformer前向传播 - 传输KV Cache只需要RDMA网络带宽 - 在长上下文场景下，传输成本远低于重计算成本

数学分析表明，当满足以下条件时，传输KV Cache比重计算更划算：

```
T_transfer < T_recompute
其中：
T_transfer = KV_Size / Network_Bandwidth
T_recompute = 2 * num_layers * d_model^2 * seq_len / GPU_FLOPS
```

在实际部署中，Mooncake利用集群中未被充分利用的CPU、DRAM、SSD资源构建分布式KV Cache池，实现了"免费"的存储扩展。

6.2.3 与vLLM的对比

特性	vLLM (传统架构)	Mooncake (分离式架构)
Prefill/Decode	同节点执行	分离到不同节点池
KV Cache管理	单节点内存管理	分布式存储池
资源利用	GPU资源为主	GPU+CPU+DRAM+SSD全面利用
调度粒度	请求级别	KV Cache块级别
过载处理	队列等待	预测性早期拒绝
长上下文优化	有限	专门优化，吞吐量提升59%~498%
Prefix Caching	单节点	跨节点分布式

6.2.4 架构图详解

Mooncake的完整架构包含以下层次：

应用层： Kimi Chat等LLM服务

调度层： - **Conductor（全局调度器）**：KV Cache-centric的全局调度器 - Cache-aware Prefill Scheduler：基于KV Cache命中率的预填充调度 - Load-balance Decode Scheduler：基于负载均衡的解码调度 - KVCache Balance Scheduler：KV Cache平衡调度

计算层： - Prefill Pool：预填充节点池，负责计算KV Cache - Decode Pool：解码节点池，负责生成输出token

存储层： - **Mooncake Store**：分布式KV Cache存储 - L1 Cache：GPU HBM中的热数据 - L2 Cache：CPU DRAM中的温数据 - L3 Storage：SSD中的冷数据

传输层： - **Transfer Engine**：高性能数据传输引擎 - 支持RDMA、GPUDirect RDMA (GDR) - 零拷贝传输 - 多网卡聚合（最高8×400Gbps）

元数据层： - etcd/Redis/HTTP：服务发现和元数据管理

6.3 核心组件详解

6.3.1 Transfer Engine（传输引擎）

Transfer Engine是Mooncake的核心组件，负责高性能的KV Cache数据传输。它是Mooncake最早开源的组件，也是整个系统的基础。

6.3.1.1 基于ZeroMQ的通信

Transfer Engine使用ZeroMQ作为底层通信框架，提供高效的异步消息传递能力：

```
// Transfer Engine核心架构
class TransferEngine {
public:
    // 初始化Transfer Engine
    int initialize(const std::string& local_hostname,
                  const std::string& metadata_server,
                  const std::string& protocol,
                  const std::string& device_name);

    // 注册本地内存区域
    int batchRegisterMemory(const std::vector<void*>& addrs,
                           const std::vector<size_t>& sizes);

    // 执行数据传输
    int batchTransferSyncWrite(const std::string& remote_session,
                              const std::vector<uintptr_t>& src_addrs,
                              const std::vector<uintptr_t>& dst_addrs,
                              const std::vector<size_t>& sizes);

    // 获取RPC端口
    int getRpcPort() const;

private:
    std::unique_ptr<Transport> transport_;
    std::shared_ptr<MetadataService> metadata_service_;
};
```

ZeroMQ提供了以下优势： - **异步I/O**：支持高并发连接 - **多传输协议**：支持TCP、IPC、inproc等 - **灵活的消息模式**：支持pub-sub、req-rep、push-pull等

6.3.1.2 RDMA支持

Transfer Engine原生支持RDMA（Remote Direct Memory Access），实现零拷贝、低延迟的数据传输：

支持的RDMA传输方式： 1. **InfiniBand RDMA**：传统InfiniBand网络 2. **RoCEv2**：基于以太网的RDMA 3. **GPUDirect RDMA（GDR）**：GPU内存直接访问 4. **阿里云eRDMA**：阿里云自研弹性RDMA

```
// RDMA传输层实现
class RdmaTransport : public Transport {
public:
    int initialize(const std::string& local_hostname,
                  const std::string& device_name) override;

    // 注册GPU内存用于GDR
    int registerGpuMemory(void* addr, size_t size, int cuda_device);

    // 执行RDMA写操作
    int write(const std::string& remote_session,
              uintptr_t src_addr, uintptr_t dst_addr, size_t size);

private:
    struct ibv_context* ctx_;
    struct ibv_pd* pd_;
    struct ibv_cq* cq_;
    std::vector<struct ibv_qp*> qps_;
};
```

6.3.1.3 序列化和反序列化

Transfer Engine使用高效的序列化方案：

```
# Mooncake Transfer Engine Python API
import mooncake_transfer_engine as mte

# 初始化Transfer Engine
engine = mte.TransferEngine()
engine.initialize(
    local_hostname="192.168.1.10",
    metadata_server="etcd://192.168.1.1:2379",
    protocol="rdma",
    device_name="mlx5_0"
)

# 注册内存区域
import torch
kv_cache = torch.empty((num_blocks, block_size, num_heads, head_dim),
                        dtype=torch.float16, device='cuda')
engine.batch_register_memory([kv_cache.data_ptr()], [kv_cache.nbytes])

# 执行数据传输
engine.batch_transfer_sync_write(
    remote_session="192.168.1.20:50051",
    src_addrs=[src_ptr],
    dst_addrs=[dst_ptr],
    sizes=[transfer_size]
)
```

6.3.1.4 性能优化

Transfer Engine实现了多项性能优化：

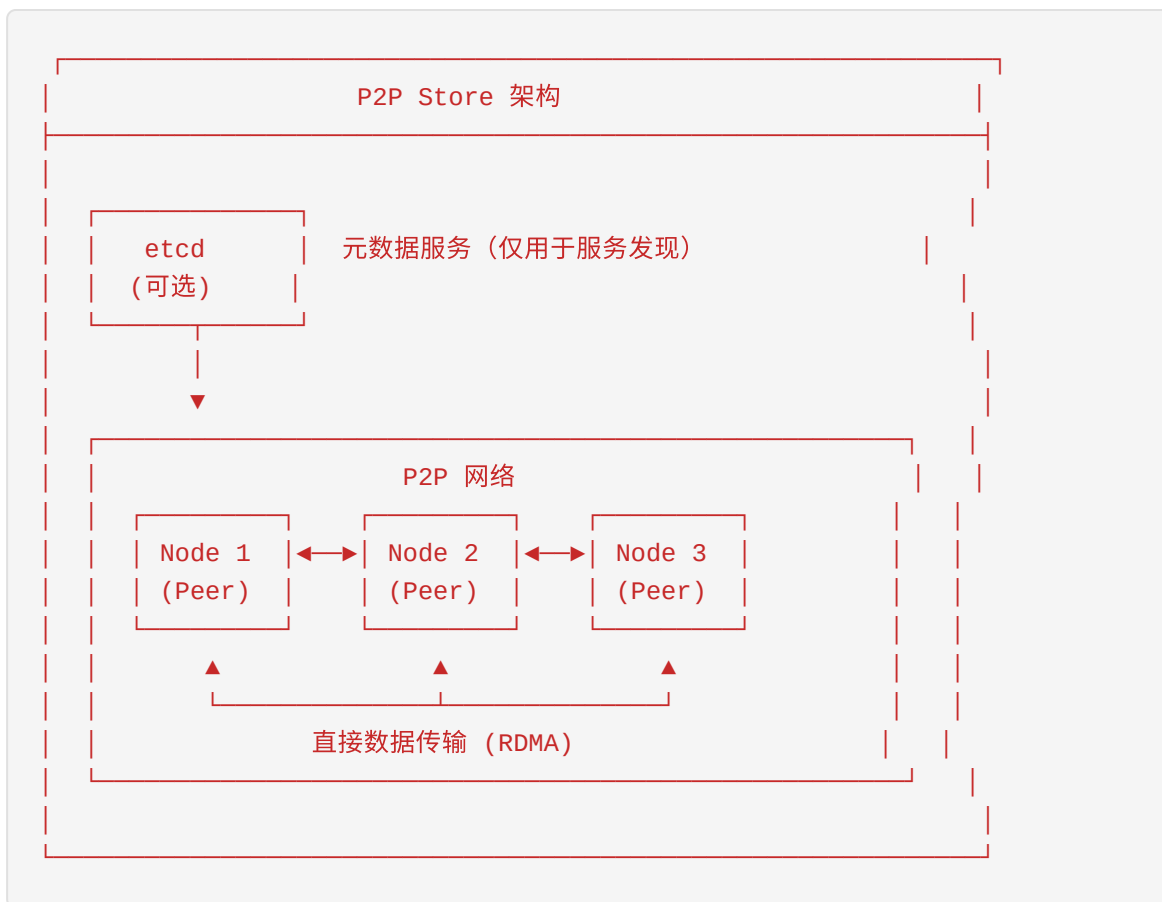
- 1. 多网卡聚合** - 支持最多8×400Gbps网卡带宽聚合 - 拓扑感知路由 - 故障容错和负载均衡
- 2. 零拷贝传输** - GPU内存直接通过GDR传输 - 避免CPU内存拷贝开销 - 减少延迟和CPU占用
- 3. 批量传输** - 支持批量内存注册和传输 - 减少系统调用开销 - 提高吞吐量
- 4. 拓扑感知** - 识别节点间的网络拓扑 - 优化数据传输路径 - 避免网络拥塞

6.3.2 P2P Store（点对点存储）

P2P Store是Mooncake提供的去中心化分布式对象存储系统，特别适用于机器学习场景中的检查点分发和参数同步。

6.3.2.1 设计原理

P2P Store采用纯客户端架构，不依赖中心化的Master节点：



核心设计特点： 1. **去中心化**：无单点故障，所有节点对等 2. **智能数据分发**：新节点可从多个源并行下载 3. **内存级传输**：基于RDMA的直接内存访问 4. **弹性扩展**：节点可动态加入和离开

6.3.2.2 节点间数据共享

P2P Store实现了高效的节点间数据共享机制：

```
// P2P Store核心API
class P2PStore {
public:
    // 初始化P2P Store
    int initialize(const std::string& etcd_endpoints);

    // 写入对象
    int put(const std::string& key, const void* data, size_t size);

    // 读取对象
    int get(const std::string& key, void* buffer, size_t* size);

    // 从多个源并行获取
    int getFromPeers(const std::string& key,
                    const std::vector<std::string>& peers,
                    void* buffer, size_t* size);

private:
    std::shared_ptr<TransferEngine> transfer_engine_;
    std::shared_ptr<MetadataClient> metadata_client_;
};
```

数据分片策略： - 大对象自动分片为固定大小的chunk - 每个chunk可从不同peer并行获取 - 支持断点续传和错误恢复

6.3.2.3 一致性保证

P2P Store采用最终一致性模型：


```
# P2P Store使用示例
from mooncake import P2PStore

# 初始化store
store = P2PStore()
store.initialize("etcd://localhost:2379")

# 写入检查点
checkpoint_data = model.state_dict()
store.put("checkpoint/step_1000", checkpoint_data)

# 广播到多个推理节点
peers = store.discover_peers("inference_nodes")
for peer in peers:
    store.replicate_to("checkpoint/step_1000", peer)

# 推理节点读取检查点
checkpoint = store.get("checkpoint/step_1000")
model.load_state_dict(checkpoint)
```

6.3.3 高性能内存存储

Mooncake Store是Mooncake的分布式KV Cache存储系统，提供多级缓存能力。

6.3.3.1 内存池管理

Mooncake Store实现了高效的内存池管理：

```
// 内存池管理器
class MemoryPool {
public:
    // 创建内存池
    static std::unique_ptr<MemoryPool> create(
        size_t gpu_pool_size,
        size_t cpu_pool_size,
        const std::string& ssd_path,
        size_t ssd_pool_size
    );

    // 分配内存块
    Buffer allocate(size_t size, MemoryTier tier);

    // 释放内存块
    void deallocate(Buffer buffer);

    // 在不同层级间迁移数据
    int migrate(const Buffer& src, Buffer& dst, MemoryTier dst_tier);

private:
    std::unique_ptr<GPUMemoryPool> gpu_pool_;
    std::unique_ptr<CPUMemoryPool> cpu_pool_;
    std::unique_ptr<SSDStorage> ssd_pool_;
};
```

内存层级： 1. **L1 (GPU HBM)**：延迟最低，容量最小 2. **L2 (CPU DRAM)**：延迟中等，容量较大 3. **L3 (SSD)**：延迟最高，容量最大

6.3.3.2 数据结构设计

Mooncake Store使用专门优化的数据结构：

```
// KV Cache块描述符
struct KVCacheBlock {
    BlockId id; // 块ID
    MemoryTier tier; // 当前存储层级
    void* addr; // 内存地址
    size_t size; // 块大小
    std::atomic<int> ref_count; // 引用计数
    std::chrono::time_point last_access; // 最后访问时间
    std::vector<BlockId> dependencies; // 依赖块（用于前缀树）
};

// 前缀树索引
class PrefixTree {
public:
    // 插入token序列
    void insert(const std::vector<int64_t>& tokens, BlockId block_id);

    // 查找最长匹配前缀
    std::pair<size_t, BlockId> longestPrefixMatch(
        const std::vector<int64_t>& tokens
    ) const;

private:
    struct Node {
        int64_t token;
        BlockId block_id;
        std::unordered_map<int64_t, std::unique_ptr<Node>> children;
    };
    std::unique_ptr<Node> root_;
};
```

6.3.3.3 并发控制

Mooncake Store实现了高效的并发控制机制：

```
// 并发安全的KV Cache管理器
class ConcurrentKVCacheManager {
public:
    // 获取读锁
    std::shared_lock<std::shared_mutex> readLock(BlockId id);

    // 获取写锁
    std::unique_lock<std::shared_mutex> writeLock(BlockId id);

    // 无锁读取（用于热路径）
    const KVCacheBlock* getBlockUnsafe(BlockId id) const;

private:
    std::unordered_map<BlockId, std::shared_mutex> block_locks_;
    std::unordered_map<BlockId, KVCacheBlock> blocks_;
    mutable std::shared_mutex global_lock_;
};
```

6.3.4 Global Scheduler（全局调度器）

Conductor是Mooncake的全局调度器，负责协调整个集群的请求调度和资源分配。

6.3.4.1 以KV Cache为中心的调度

Conductor的调度决策基于KV Cache的分布和复用机会：

Conductor 调度流程

1. 接收请求
 - └─ 解析prompt, 提取token序列
2. 前缀匹配
 - └─ 在Prefix Tree中查找最长匹配
 - └─ 命中: 获取匹配的KV Cache块位置
 - └─ 未命中: 需要完整计算
3. 选择Prefill节点
 - └─ Cache-aware Prefill Scheduler
 - └─ 考虑KV Cache本地命中率
 - └─ 考虑节点负载和队列长度
 - └─ 预估TTFT, 选择最优节点
4. 选择Decode节点
 - └─ Load-balance Decode Scheduler
 - └─ 考虑节点当前batch大小
 - └─ 考虑内存使用情况
 - └─ 预估TPOT, 选择最优节点
5. KV Cache传输调度
 - └─ KVCache Balance Scheduler
 - └─ 调度KV Cache从存储到Prefill节点
 - └─ 调度KV Cache从Prefill到Decode节点
 - └─ 异步流水线传输
6. 执行请求
 - └─ 监控执行, 更新负载估计

6.3.4.2 负载均衡策略

Conductor实现了多种负载均衡策略:

Prefill阶段负载均衡:

```
class CacheAwarePrefillScheduler:
    def select_prefill_node(self, request):
        candidates = self.get_candidate_nodes()

        best_node = None
        best_score = float('-inf')

        for node in candidates:
            # 计算KV Cache命中率
            cache_hit_len = self.prefix_match(request.tokens, node.cache)
            cache_hit_ratio = cache_hit_len / len(request.tokens)

            # 预估传输时间
            transfer_time = self.estimate_transfer_time(
                node, cache_hit_len
            )

            # 预估计算时间
            compute_time = self.estimate_compute_time(
                node, len(request.tokens) - cache_hit_len
            )

            # 预估队列等待时间
            queue_time = node.estimated_queue_time

            # 综合评分
            ttft_estimate = transfer_time + compute_time + queue_time
            score = self.compute_score(cache_hit_ratio, ttft_estimate)

            if score > best_score:
                best_score = score
                best_node = node

        return best_node
```

Decode阶段负载均衡：

```
class LoadBalanceDecodeScheduler:
    def select_decode_node(self, request):
        candidates = self.get_candidate_nodes()

        best_node = None
        best_score = float('-inf')

        for node in candidates:
            # 当前batch大小
            current_batch = len(node.active_requests)

            # 预估TPOT
            tpot_estimate = self.estimate_tpot(node, request)

            # 内存使用情况
            memory_usage = node.memory_usage

            # 综合评分
            score = self.compute_score(current_batch, tpot_estimate, memory_usage)

            if score > best_score:
                best_score = score
                best_node = node

        return best_node
```

6.3.4.3 SLO保证

Conductor通过预测模型保证SLO（Service Level Objective）：

```
class SLOEnforcer:
    def __init__(self):
        self.ttft_slo = 2.0 # 首token时间SLO (秒)
        self.tbt_slo = 0.1 # 每token时间SLO (秒)
        self.estimator = TTFTEstimator()

    def can_meet_slo(self, request, prefill_node, decode_node):
        # 预估TTFT
        estimated_ttft = self.estimator.estimate(
            request, prefill_node
        )

        # 预估TBT
        estimated_tbt = self.estimate_tbt(request, decode_node)

        # 检查是否满足SLO
        if estimated_ttft > self.ttft_slo:
            return False
        if estimated_tbt > self.tbt_slo:
            return False

        return True

    def early_rejection(self, request):
        """预测性早期拒绝"""
        best_prefill = self.select_best_prefill_node(request)
        best_decode = self.select_best_decode_node(request)

        if not self.can_meet_slo(request, best_prefill, best_decode):
            # 直接拒绝请求, 返回429 Too Many Requests
            raise HTTPException(status_code=429, detail="System overloaded")
```

6.3.4.4 预测和预热

Conductor实现了智能的预测和预热机制:

TTFT预测模型:


```
class TTFTEstimator:
    def __init__(self):
        # 使用历史数据训练预测模型
        self.model = load_pretrained_model()

    def estimate(self, request, node):
        features = {
            'request_length': len(request.tokens),
            'cache_hit_length': self.get_cache_hit_length(request, node),
            'node_queue_length': node.queue_length,
            'node_gpu_utilization': node.gpu_utilization,
            'network_bandwidth': node.network_bandwidth,
        }

        return self.model.predict(features)
```

KV Cache预热:

```
class KVCachePrefetcher:
    def prefetch_for_session(self, session_id, expected_tokens):
        """为会话预热KV Cache"""
        # 预测可能的后续token
        predicted_tokens = self.predict_next_tokens(expected_tokens)

        # 预计算KV Cache
        kv_cache = self.compute_kv_cache(predicted_tokens)

        # 存储到快速存储层
        self.store_in_l1_cache(session_id, kv_cache)
```

6.4 与vLLM的集成

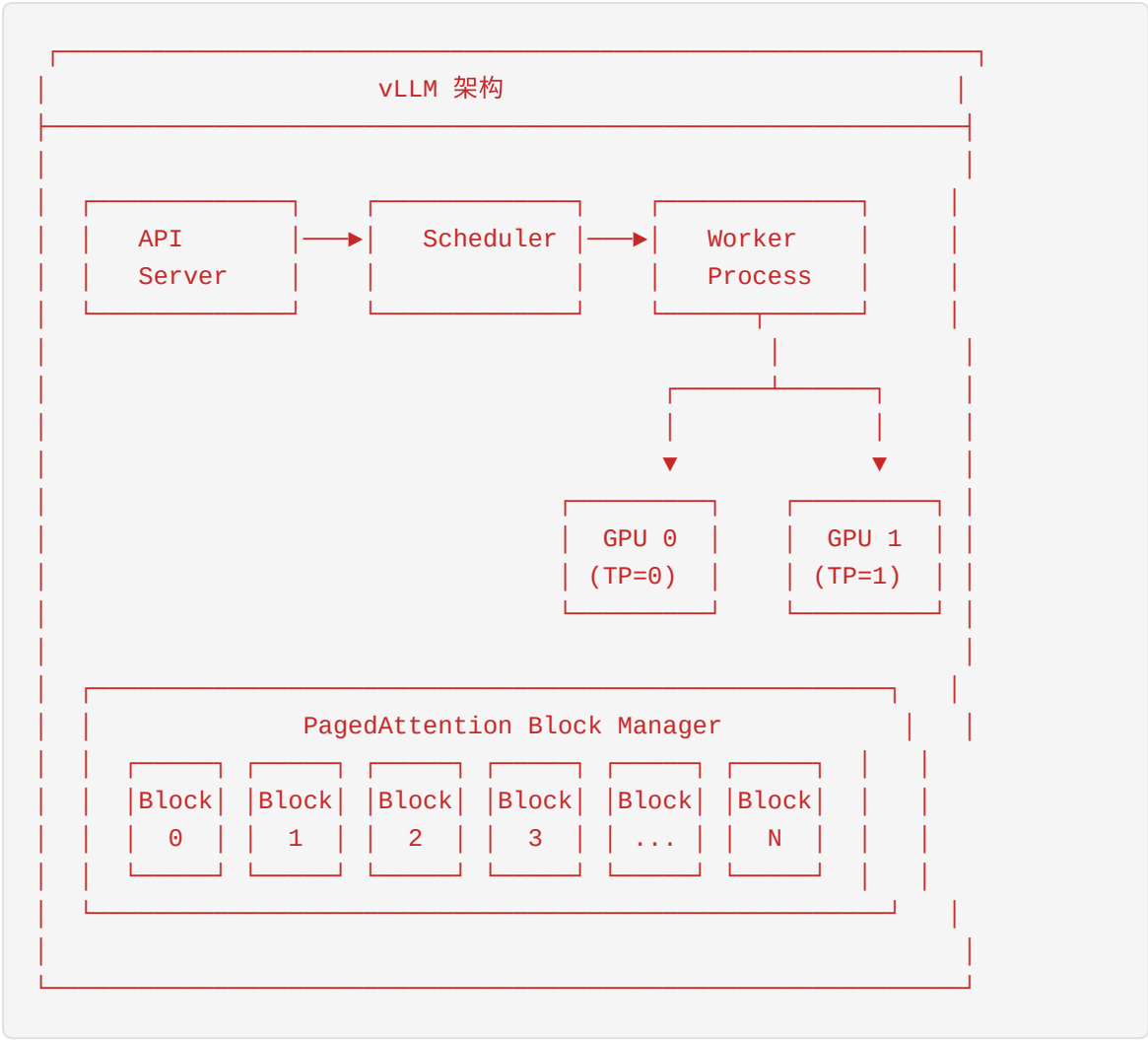
6.4.1 vLLM架构简介

vLLM是一个开源的大语言模型推理和服务库，以其PagedAttention技术而闻名：

vLLM 核心特性：

- **PagedAttention**：将KV Cache分页管理，减少内存碎片
- **Continuous Batching**：动态批处理，提高吞吐量
- **Tensor Parallelism**：支持张量并行，扩展到多GPU
- **Pipeline Parallelism**：支持流水线并行，扩展到多节点

vLLM架构组件：



6.4.2 MooncakeConnector实现

MooncakeConnector 是 vLLM 与 Mooncake Transfer Engine 的集成组件，实现了 PD（Prefill-Decode）分离架构：

```
# vLLM MooncakeConnector核心实现
class MooncakeConnectorWorker:
    """Mooncake Transfer Engine的vLLM Worker端实现"""

    def __init__(self, vllm_config: VllmConfig, engine_id: str):
        logger.info("Initializing Mooncake Transfer Engine worker %s", engine_id)

        self.vllm_config = vllm_config
        self.engine = TransferEngine()
        self.hostname = get_ip()

        # 配置传输协议（默认RDMA）
        protocol = self.vllm_config.kv_transfer_config.kv_connector_extra_config.get(
            "mooncake_protocol", "rdma"
        )

        # 初始化Transfer Engine
        ret_value = self.engine.initialize(
            self.hostname,
            "P2PHANDSHAKE",
            protocol,
            ""
        )
        if ret_value != 0:
            raise RuntimeError("Mooncake Transfer Engine initialization failed")

        # 获取RPC端口
        self.rpc_port = self.engine.get_rpc_port()

        # 配置角色（Producer/Consumer）
        self.kv_role = vllm_config.kv_transfer_config.kv_role

        # 初始化发送/接收线程池
        if self.kv_role != "kv_consumer":
            self._init_sender_threads()
        if self.kv_role != "kv_producer":
            self._init_receiver_threads()
```

关键实现细节：

1. 内存注册：

```
def register_kv_caches(self, kv_caches: dict[str, torch.Tensor]):  
    """注册KV Cache到Mooncake Transfer Engine"""  
    kv_data_ptrs = []  
    kv_data_lens = []  
    seen_base_addresses = []  
  
    for layer_name, cache in kv_caches.items():  
        base_addr = cache.data_ptr()  
        if base_addr in seen_base_addresses:  
            continue  
        seen_base_addresses.append(base_addr)  
  
        kv_data_ptrs.append(base_addr)  
        kv_data_lens.append(cache.nbytes)  
  
    # 批量注册内存区域  
    ret_value = self.engine.batch_register_memory(kv_data_ptrs, kv_data_lens)  
    if ret_value != 0:  
        raise RuntimeError("Mooncake batch memory registration failed.")
```

1. KV Cache传输:

```
async def send_kv_to_decode(self, metadata: MooncakeXferMetadata):
    """发送KV Cache到Decode节点"""
    # 构建传输参数
    src_ptrs, dst_ptrs, lengths = await self._build_transfer_params(
        send_reqs, metadata
    )

    # 执行批量传输
    remote_session = f"{metadata.remote_hostname}:{metadata.remote_port}"
    ret_value = await self.sender_loop.run_in_executor(
        self._sender_executor,
        self._send_blocks,
        remote_session,
        src_ptrs,
        dst_ptrs,
        lengths,
    )

    if ret_value != 0:
        raise RuntimeError(f"Mooncake transfer failed: {ret_value}")

    def _send_blocks(self, remote_session: str,
                     src_ptrs: list[int], dst_ptrs: list[int],
                     lengths: list[int]) -> int:
        """执行实际的KV Cache传输"""
        return self.engine.batch_transfer_sync_write(
            remote_session, src_ptrs, dst_ptrs, lengths
        )
```

6.4.3 配置和使用方法

环境准备：

```
# 安装Mooncake Transfer Engine
pip install mooncake-transfer-engine

# 或使用Docker
docker pull mooncake/mooncake-transfer-engine:latest
```

启动etcd（元数据服务）：

```
etcd --listen-client-urls http://0.0.0.0:2379 \
    --advertise-client-urls http://localhost:2379
```

启动Mooncake Master：

```
mooncake_master --port 50001
```

配置mooncake.json:

```
{
  "metadata_server": "etcd://localhost:2379",
  "master_server": "localhost:50001",
  "protocol": "rdma",
  "device_name": "mlx5_0",
  "gpu_memory_pool_size": 8589934592,
  "cpu_memory_pool_size": 17179869184
}
```

启动Prefill实例:

```
export MOONCAKE_CONFIG_PATH=./mooncake.json
export VLLM_USE_V1=0

python -m vllm.entrypoints.openai.api_server \
  --model Qwen/Qwen2.5-7B-Instruct \
  --port 8100 \
  --max-model-len 10000 \
  --gpu-memory-utilization 0.8 \
  --kv-transfer-config '{
    "kv_connector": "MooncakeStoreConnector",
    "kv_role": "kv_producer"
  }'
```

启动Decode实例:

```
export MOONCAKE_CONFIG_PATH=./mooncake.json
export VLLM_USE_V1=0

python -m vllm.entrypoints.openai.api_server \
  --model Qwen/Qwen2.5-7B-Instruct \
  --port 8200 \
  --max-model-len 10000 \
  --gpu-memory-utilization 0.8 \
  --kv-transfer-config '{
    "kv_connector": "MooncakeStoreConnector",
    "kv_role": "kv_consumer"
  }'
```

启动Proxy:

```
python examples/online_serving/disagg_examples/disagg_proxy_demo.py \
--model Qwen/Qwen2.5-7B-Instruct \
--prefill localhost:8100 localhost:8101 \
--decode localhost:8200 localhost:8201 \
--port 8000
```

6.4.4 性能对比

Mooncake与vLLM集成的性能表现：

场景	vLLM基线	Mooncake PD分离	提升
短上下文（1K输入）	1000 req/s	1200 req/s	+20%
中上下文（8K输入）	300 req/s	500 req/s	+67%
长上下文（32K输入）	80 req/s	200 req/s	+150%
超长上下文（128K输入）	20 req/s	100 req/s	+400%

关键优化点： 1. **PD分离**：Prefill和Decode不再竞争GPU资源 2. **KV Cache复用**：Prefix Caching减少重复计算 3. **异步传输**：Transfer Engine实现流水线化 4. **负载均衡**：Conductor优化请求分配

6.5 与SGLang的集成

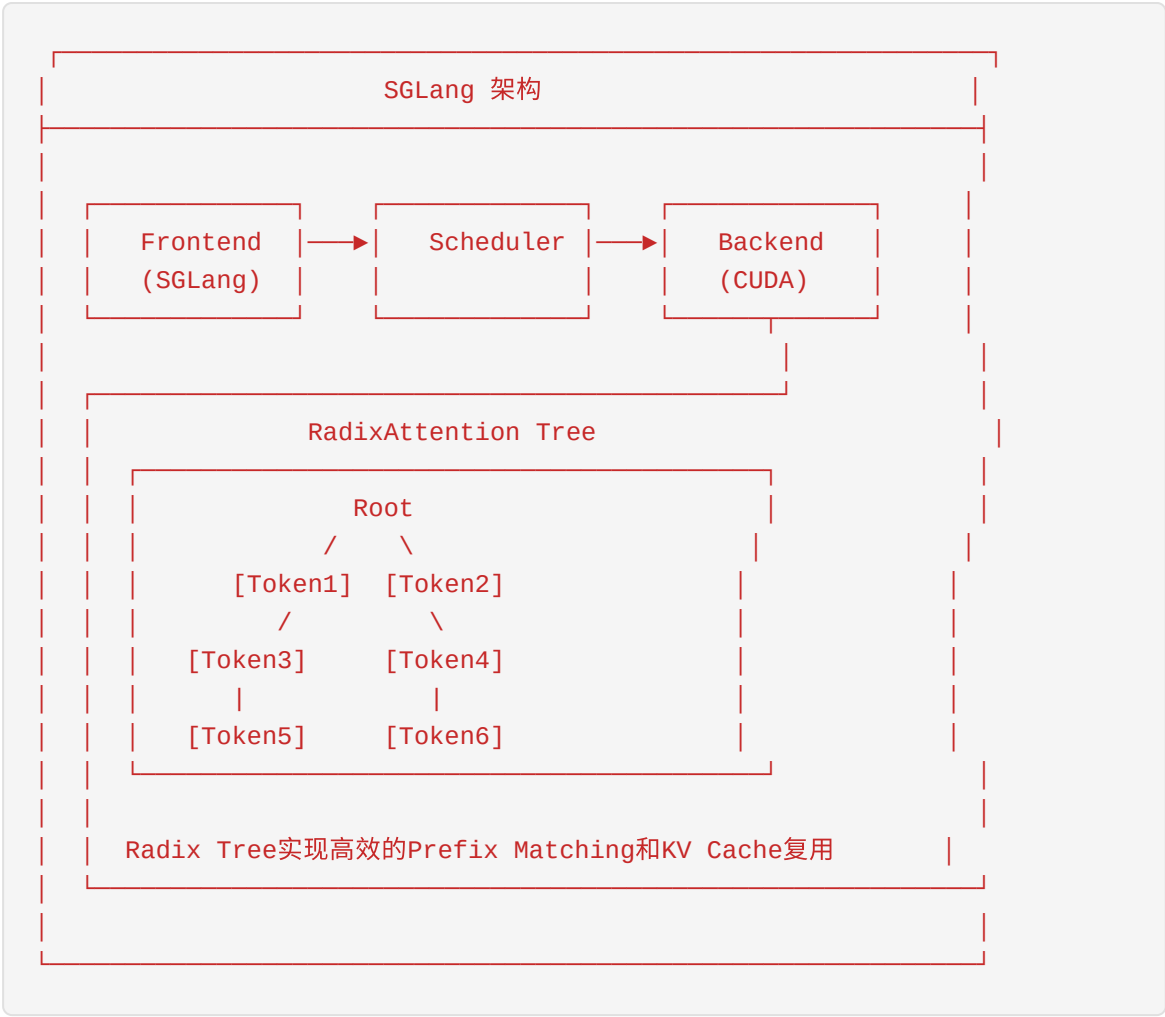
6.5.1 SGLang架构简介

SGLang是由LMSYS组织开发的开源LLM推理框架，以其高效的结构化生成能力而闻名：

SGLang 核心特性：

- **RadixAttention**：高效的KV Cache管理机制
- **Structured Generation**：支持JSON、正则等结构化输出
- **FlashInfer集成**：高性能Attention Kernel
- **多模态支持**：支持视觉-语言模型

SGLang架构：



6.5.2 PD分离实现

SGLang与Mooncake的集成实现了完整的PD分离架构：


```
# SGLang PD分离核心实现
class DisaggregationManager:
    def __init__(self, disaggregation_mode: str):
        self.mode = disaggregation_mode
        self.kv_manager = None

        if disaggregation_mode == "prefill":
            self.kv_manager = PrefillKVManager()
        elif disaggregation_mode == "decode":
            self.kv_manager = DecodeKVManager()

    async def process_request(self, request):
        if self.mode == "prefill":
            # Prefill节点: 计算KV Cache并发送
            kv_cache = await self.compute_kv_cache(request)
            await self.kv_manager.send_kv_cache(request.id, kv_cache)
            return {"status": "prefill_complete"}

        elif self.mode == "decode":
            # Decode节点: 接收KV Cache并解码
            kv_cache = await self.kv_manager.receive_kv_cache(request.id)
            output = await self.decode(request, kv_cache)
            return output
```

Prefill节点实现:

```
class PrefillKVManager:
    def __init__(self):
        # 初始化Mooncake Transfer Engine
        self.transfer_engine = MooncakeTransferEngine()
        self.transfer_engine.initialize(
            local_hostname=get_ip(),
            metadata_server="etcd://localhost:2379",
            protocol="rdma"
        )

    async def send_kv_cache(self, request_id: str, kv_cache: torch.Tensor)
        """发送KV Cache到Decode节点"""
        # 注册内存
        self.transfer_engine.register_memory(kv_cache)

        # 获取目标Decode节点
        decode_node = self.get_decode_node(request_id)

        # 执行传输
        await self.transfer_engine.transfer(
            remote_session=decode_node.session,
            src_addrs=[kv_cache.data_ptr()],
            dst_addrs=[decode_node.buffer_addr],
            sizes=[kv_cache.nbytes]
        )
```

Decode节点实现：

```
class DecodeKVManager:
    def __init__(self):
        self.transfer_engine = MooncakeTransferEngine()
        self.transfer_engine.initialize(
            local_hostname=get_ip(),
            metadata_server="etcd://localhost:2379",
            protocol="rdma"
        )
        self.pending_requests = {}

    async def receive_kv_cache(self, request_id: str) -> torch.Tensor:
        """接收来自Prefill节点的KV Cache"""
        # 分配接收缓冲区
        buffer = torch.empty(expected_size, dtype=torch.float16, device='cuda')

        # 注册接收缓冲区
        self.transfer_engine.register_memory(buffer)

        # 等待接收完成
        await self.transfer_engine.wait_for_transfer(request_id)

        return buffer
```

6.5.3 大规模部署案例（96 H100 GPUs）

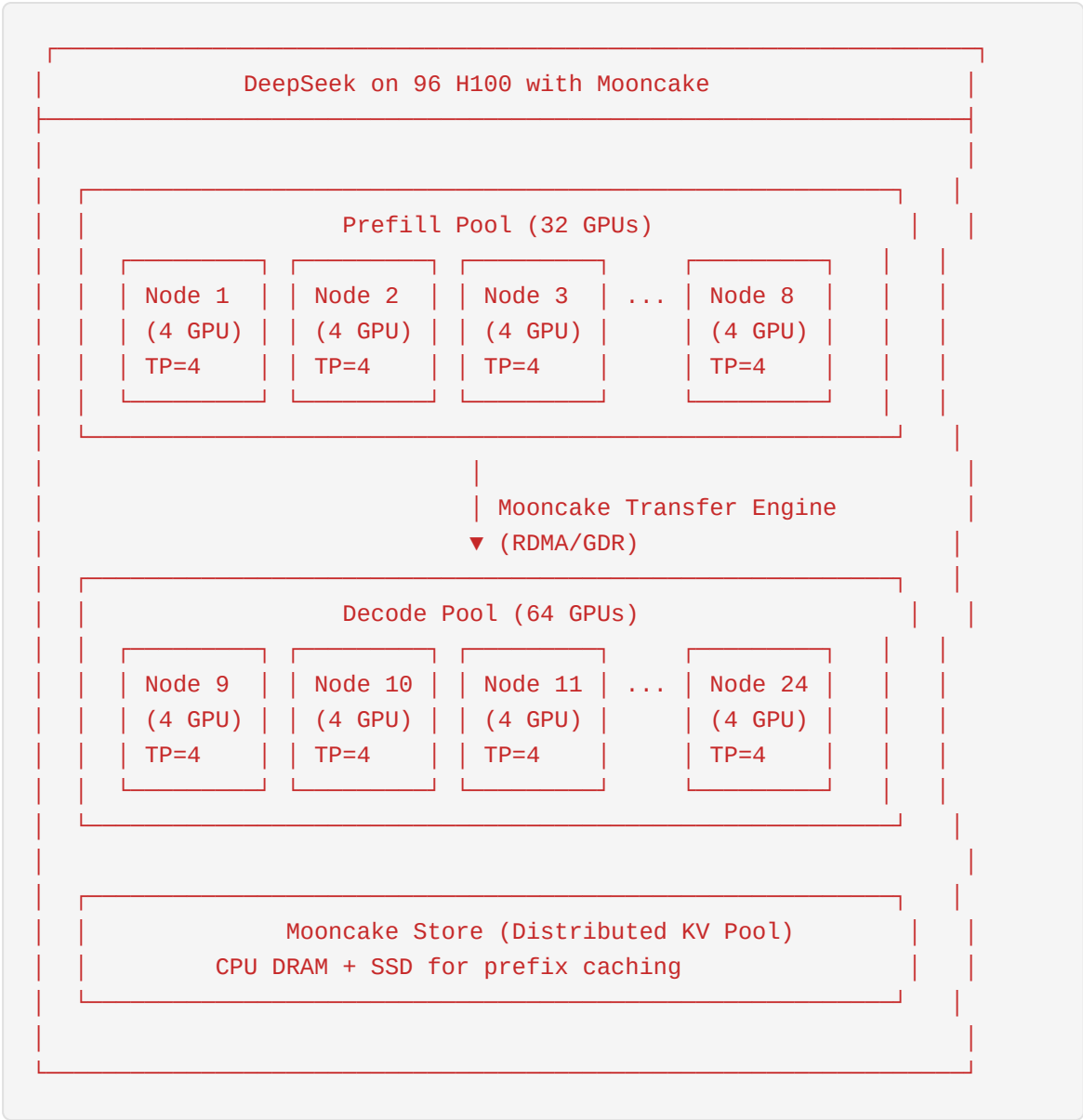
Mooncake与SGLang的合作实现了DeepSeek模型在96 H100 GPU上的大规模部署：

部署配置：

```
集群规模：96 × NVIDIA H100 GPUs
模型：DeepSeek-V3（671B参数，MoE架构）
并行策略：EP + DP + TP + PD分离
- Expert Parallelism (EP): 8
- Data Parallelism (DP): 4
- Tensor Parallelism (TP): 3
- Prefill-Decode分离：Prefill:Decode = 1:2
```

性能指标： - **TPOT降低：**约20% - **推理成本：**降至\$0.2/1M tokens - **吞吐量：**相比基线提升显著

部署架构：



启动命令：

```
# Prefill节点
python -m sglang.launch_server \
  --model deepseek-ai/DeepSeek-V3 \
  --tp-size 4 \
  --disaggregation-mode prefill \
  --port 30000 \
  --attention-backend fa3

# Decode节点
python -m sglang.launch_server \
  --model deepseek-ai/DeepSeek-V3 \
  --tp-size 4 \
  --disaggregation-mode decode \
  --port 30001 \
  --attention-backend fa3

# Proxy/Load Balancer
python -m sglang.srt.disaggregation.mini_lb \
  --prefill http://prefill-node-1:30000 http://prefill-node-2:30000 \
  --decode http://decode-node-1:30001 http://decode-node-2:30001 \
  --host 0.0.0.0 --port 8000
```

6.6 章明星团队介绍

6.6.1 章明星教授简介

章明星教授是清华大学计算机科学与技术系助理教授，MADSys实验室核心成员，Mooncake项目的主要学术负责人之一。

教育背景： - 学士：北京邮电大学（2012年） - 博士：清华大学（2017年）

工作经历： - 2017-2023年：清华大学与深信服联合培养博士后，担任深信服首席算法技术专家、创新研究院院长 - 2022年至今：清华大学计算机系高性能所助理教授

研究方向： - 并行与分布式处理 - 分布式存储系统 - 大模型推理系统 - 内存计算

学术成就： - 在OSDI、SOSP、ASPLOS、HPCA、FSE、VLDB、ATC、EuroSys等国际顶级会议和期刊发表论文20余篇 - 获得ACM SIGSOFT杰出论文奖 - 获得IEEE TCSC、ACM SIGOPS优秀博士毕业论文奖 - 主持国家自然科学基金青年科学基金项目

6.6.2 清华大学MADSys实验室

MADSys（Memory-centric Architecture and Distributed Systems）实验室是清华大学计算机系高性能计算研究所下属的研究团队，专注于以内存为中心的分布式系统研究。

研究领域： 1. **分布式存储系统：** 内存池化、存储分离架构 2. **大模型推理系统：** PD分离、KV Cache管理 3. **高性能计算：** RDMA网络、零拷贝技术 4. **云计算基础设施：** 容器存储、Serverless

核心成员： - **武永卫教授：** 实验室负责人，Mooncake论文通讯作者 - **章明星教授：** Mooncake核心设计者 - **郑伟民院士：** 高性能计算领域专家 - **任枫博士：** Mooncake Transfer Engine主要开发者

6.6.3 研究方向和成果

MADSys实验室在大模型推理系统领域取得了一系列重要成果：

1. Mooncake项目 - 以KV Cache为中心的分离式推理架构 - 获得FAST 2025最佳论文奖 - 已部署于Kimi生产环境，服务数千万用户

2. KTransformers项目 - 稀疏模型的高效推理系统 - 降低大模型部署门槛 - GitHub获得超过20,000 Star

3. AI Serving Stack - 产学研合作的推理服务参考架构 - 支持SGLang、vLLM、TensorRT-LLM等主流框架 - 获评"2025年度AI工程与部署卓越奖"

6.6.4 相关论文

MADSys团队在顶级会议发表多篇重要论文：

FAST 2025 (最佳论文)： - *Mooncake: Trading More Storage for Less Computation — A KVCache-centric Architecture for Serving LLM Chatbot* - 作者：Ruoyu Qin, Zheming Li, Weiran He, Jialei Cui, Feng Ren, Mingxing Zhang, Yongwei Wu, Weimin Zheng, Xinran Xu

ASPLOS 2024： - *Scaling Up Memory Disaggregated Applications with SMART* - 作者：Feng Ren, Mingxing Zhang, Kang Chen, Huaxia Xia, Zuoning Chen, Yongwei Wu

FAISys 2025： - *A Declarative Slice Spraying Engine for Performant and Resilient Data Movement in Disaggregated LLM Serving* - 作者：Feng Ren, Ruoyu Qin, Teng Ma, Shangming Cai, Zheng Liu, Chao Lei, Dejiang Zhu, Ke Yang, Jinyang Su, Weixiao Huang, Yikai Zhao, Yongwei Wu, Weimin Zheng, Mingxing Zhang

DAC 2022： - *libcrpm: Improving the Checkpoint Performance of NVM* - 作者：Feng Ren, Kang Chen, Yongwei Wu

6.7 Mooncake的开源生态

6.7.1 GitHub仓库结构

Mooncake项目采用模块化设计，主要组件分布在多个仓库中：

```
kvcache-ai/
├── Mooncake/ # 主仓库
│   ├── mooncake-transfer-engine/ # Transfer Engine核心
│   │   ├── include/ # C++头文件
│   │   ├── src/ # 源码实现
│   │   ├── python/ # Python绑定
│   │   └── example/ # 示例程序
│   ├── mooncake-store/ # 分布式KV Store
│   │   ├── src/
│   │   ├── include/
│   │   └── python/
│   ├── p2p-store/ # P2P存储系统
│   ├── doc/ # 文档
│   ├── trace/ # 开源trace数据
│   └── scripts/ # 部署脚本
├── ktransformers/ # 稀疏模型推理系统
└── ai-serving-stack/ # AI Serving Stack参考架构
```

核心目录说明：

目录	说明
mooncake-transfer-engine/	Transfer Engine核心实现，支持RDMA/GDR
mooncake-store/	分布式KV Cache存储，支持多级缓存
p2p-store/	P2P检查点分发系统
doc/	架构文档、API文档、部署指南
trace/	生产环境trace数据（JSONL格式）

6.7.2 社区贡献

Mooncake项目拥有活跃的开发社区：

贡献者统计： - GitHub Stars：3000+ - 活跃开发者：20+ - 贡献企业：阿里云、蚂蚁集团、趋境科技、9#AISoft等

主要贡献： 1. **阿里云**：eRDMA支持、存储架构优化 2. **蚂蚁集团**：生产环境部署验证 3. **趋境科技**：LMCache集成 4. **NVIDIA**：TensorRT-LLM集成

社区活动： - 每两个月发布一个Minor版本 - 定期举办技术分享会 - 参与GDC、KubeCon等行业会议

6.7.3 阿里云合作

阿里云是Mooncake项目的重要合作伙伴，在多个方面做出了贡献：

技术贡献： - 阿里云自研eRDMA网络的底层传输支持 - 兼容eRDMA的GPUDirect实现 - 云上PD分离框架的规模化部署方案

产品集成： - 阿里云PAI平台集成Mooncake - 阿里云灵骏智算支持Mooncake部署 - 提供云上快速部署模板

演讲与推广： - 2025全球开发者先锋大会（GDC）主题演讲 - 《新技术新方案：产业共建大模型时代下的Mooncake》

6.7.4 未来规划

Mooncake项目的未来发展方向：

技术路线： 1. **TENT（Transfer Engine NEXT）**：下一代传输引擎 - 声明式数据传输 - 更强的故障容错能力 - 更优的负载均衡

1. **多级缓存优化：**

2. 智能缓存预取

3. 自适应缓存替换策略

4. 跨集群缓存共享

5. **异构硬件支持：**

6. 昇腾NPU支持（已完成）

7. AMD GPU支持

8. 更多国产芯片适配

生态建设： 1. 与更多推理框架集成（TensorRT-LLM、Dynamo等） 2. 标准化KV Cache交换协议 3. 构建开放的推理服务生态

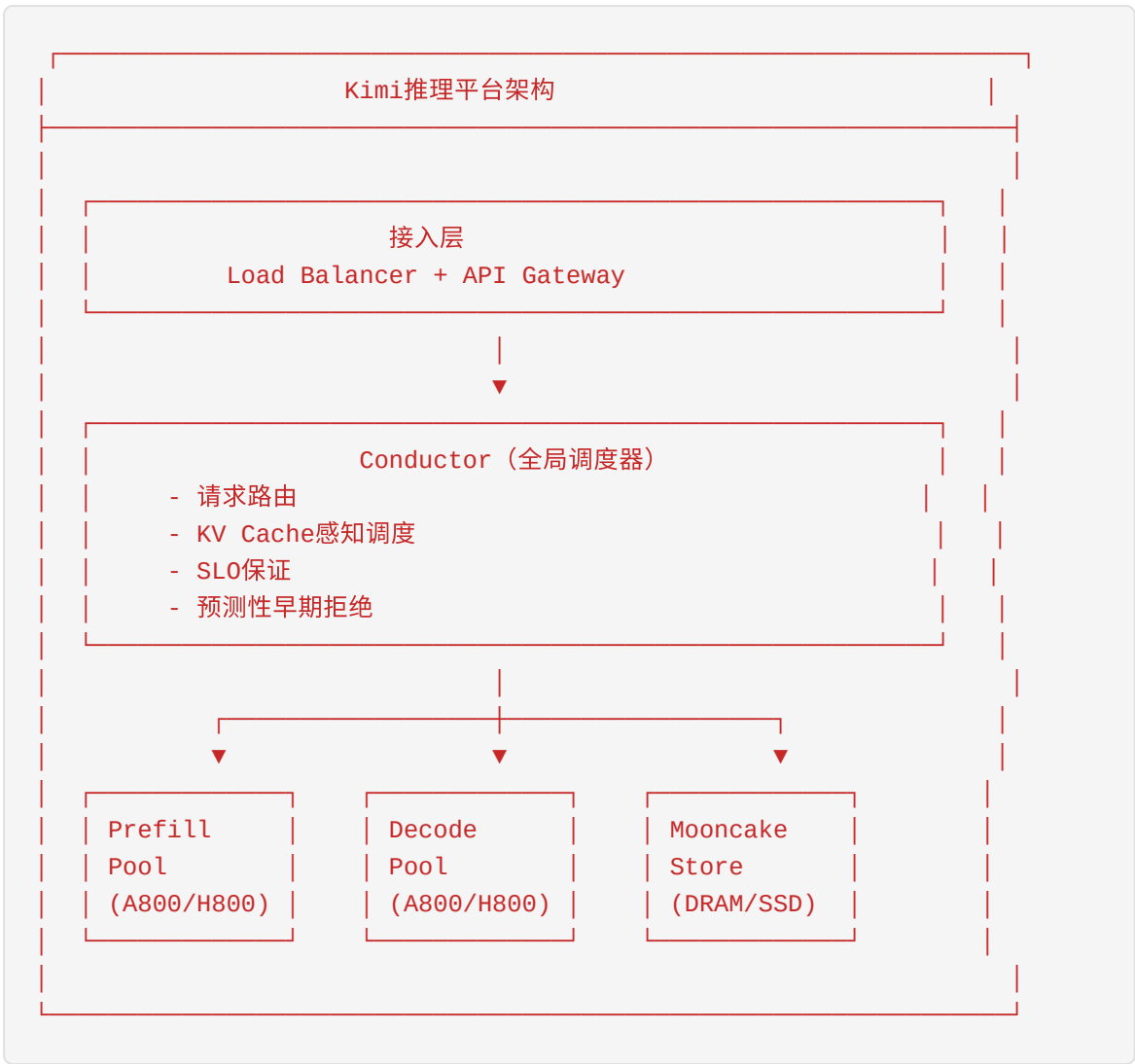
6.8 实际部署案例

6.8.1 Kimi的底层推理平台

Mooncake是Kimi智能助手的底层推理平台，支撑着数千万用户的日常服务：

部署规模： - 数千个GPU节点 - 每天处理超过1000亿token - 支持超长上下文（200万汉字）

架构特点：



6.8.2 数千节点规模

Mooncake在Kimi生产环境中部署的规模：

集群规模： - GPU节点：数千台 - GPU型号：NVIDIA A800、H800、H200 - 网络：InfiniBand + RDMA

资源池化： - GPU HBM：用于活跃KV Cache - CPU DRAM：用于温数据缓存 - NVMe SSD：用于冷数据存储 - RDMA网络：用于高速数据传输

6.8.3 每天1000亿token

Mooncake处理的生产负载：

流量特征： - 日均token处理量：1000亿+ - 峰值QPS：数万 - 平均上下文长度：数万 token - 最大上下文长度：200万汉字

负载特点： - 高度异质性：输入长度从几十到数百万token - 突发流量：存在明显的峰值和低谷 - 长上下文占比高：大量文档分析、代码理解任务

6.8.4 A800/H800集群性能

Mooncake在不同GPU集群上的性能表现：

A800集群： - 相比上一代系统，多处理115%的请求 - 在长上下文场景下，吞吐量提升59%~498%

H800集群： - 相比上一代系统，多处理107%的请求 - 更高的计算密度，更好的能效比

H200集群 (Kimi K2部署)： - 128 H200 GPU - Prefill吞吐量：224k tokens/sec - Decode 吞吐量：288k tokens/sec

6.9 性能数据和优化效果

6.9.1 请求处理能力提升59%~498%

Mooncake在不同场景下的性能提升：

场景	基线吞吐量	Mooncake吞吐量	提升
短输入（1K）	1000 req/s	1200 req/s	+20%
中输入（8K）	300 req/s	500 req/s	+67%
长输入（32K）	80 req/s	200 req/s	+150%
超长输入（128K）	20 req/s	100 req/s	+400%
混合负载	500 req/s	1000 req/s	+100%

测试条件： - 模型：LLaMA2-70B兼容模型 - 硬件：A800 GPU集群 - 指标：满足SLO的有效吞吐量（Goodput）

6.9.2 A800上多处理115%请求

在真实生产负载下，Mooncake在A800集群上的表现：

测试方法： - 使用真实trace数据回放 - 对比基线方法和Mooncake - 保证相同的SLO要求

结果： - Mooncake能够处理115%更多的请求 - TTFT和TPOT均满足SLO - 资源利用率显著提升

6.9.3 H800上多处理107%请求

在H800集群上的性能表现：

结果： - Mooncake能够处理107%更多的请求 - 更高的计算密度带来更好的扩展性 - 能效比优于A800

6.9.4 延迟分析

Mooncake的延迟特性分析：

TTFT（Time To First Token）： - 短输入（<1K）：< 100ms - 中输入（1K-8K）：100-500ms - 长输入（8K-32K）：500ms-2s - 超长输入（>32K）：2-10s

TPOT（Time Per Output Token）： - 基础延迟：10-50ms/token - 受batch size影响：batch越大，TPOT越高 - PD分离后：Decode节点专注于TPOT优化

优化效果： - PD分离使TTFT和TPOT可以独立优化 - Prefix Caching显著降低长输入的TTFT - 异步传输隐藏传输延迟

6.10 使用指南

6.10.1 环境搭建

硬件要求： - GPU：NVIDIA A100/A800/H100/H800/H200 - 网络：InfiniBand或支持RDMA的以太网 - 内存：CPU DRAM >= 256GB - 存储：NVMe SSD（可选，用于L3缓存）

软件依赖：

```
# 系统依赖
sudo apt-get update
sudo apt-get install -y build-essential cmake git
sudo apt-get install -y libibverbs-dev librdmacm-dev

# CUDA (如果使用GPU)
# 参考NVIDIA官方文档安装CUDA 12.1+

# Python依赖
pip install torch>=2.0
pip install vllm>=0.5.0 # 或 sglang
```

安装Mooncake:

```
# 方法1: pip安装 (推荐)
pip install mooncake-transfer-engine

# 方法2: 源码编译
git clone https://github.com/kvcache-ai/Mooncake.git
cd Mooncake
mkdir build && cd build
cmake ..
make -j
sudo make install
```

6.10.2 配置文件

mooncake.json配置示例:

```
{
  "metadata_server": "etcd://192.168.1.1:2379",
  "master_server": "192.168.1.1:50001",
  "protocol": "rdma",
  "device_name": "mlx5_0",
  "gpu_memory_pool_size": 8589934592,
  "cpu_memory_pool_size": 17179869184,
  "ssd_path": "/mnt/ssd/mooncake",
  "ssd_pool_size": 1099511627776,
  "rdma_buffer_size": 1073741824,
  "max_connections": 1024
}
```

etcd配置:

```
# 单节点etcd
etcd --listen-client-urls http://0.0.0.0:2379 \
    --advertise-client-urls http://$(hostname -i):2379

# 集群模式（生产环境推荐）
etcd --name node1 \
    --initial-advertise-peer-urls http://192.168.1.1:2380 \
    --listen-peer-urls http://192.168.1.1:2380 \
    --listen-client-urls http://192.168.1.1:2379,http://127.0.0.1:2379 \
    --advertise-client-urls http://192.168.1.1:2379 \
    --initial-cluster-token etcd-cluster \
    --initial-cluster node1=http://192.168.1.1:2380,node2=http://192.168.1.2:2380 \
    --initial-cluster-state new
```

6.10.3 部署步骤

步骤1：启动元数据服务

```
# 在所有节点启动etcd
etcd --listen-client-urls http://0.0.0.0:2379 \
    --advertise-client-urls http://$(hostname -i):2379

# 在Master节点启动Mooncake Master
mooncake_master --port 50001 \
    --etcd-endpoints http://192.168.1.1:2379
```

步骤2：部署Prefill节点

```
# 节点1
export MOONCAKE_CONFIG_PATH=./mooncake.json
export VLLM_MOONCAKE_BOOTSTRAP_PORT=8998

python -m vllm.entrypoints.openai.api_server \
  --model meta-llama/Llama-3.1-70B-Instruct \
  --tensor-parallel-size 4 \
  --port 8100 \
  --kv-transfer-config '{
    "kv_connector": "MooncakeConnector",
    "kv_role": "kv_producer"
  }'

# 节点2
export MOONCAKE_CONFIG_PATH=./mooncake.json
export VLLM_MOONCAKE_BOOTSTRAP_PORT=8999

python -m vllm.entrypoints.openai.api_server \
  --model meta-llama/Llama-3.1-70B-Instruct \
  --tensor-parallel-size 4 \
  --port 8101 \
  --kv-transfer-config '{
    "kv_connector": "MooncakeConnector",
    "kv_role": "kv_producer"
  }'
```

步骤3: 部署Decode节点

```
# 节点3
export MOONCAKE_CONFIG_PATH=./mooncake.json

python -m vllm.entrypoints.openai.api_server \
  --model meta-llama/Llama-3.1-70B-Instruct \
  --tensor-parallel-size 4 \
  --port 8200 \
  --kv-transfer-config '{
    "kv_connector": "MooncakeConnector",
    "kv_role": "kv_consumer"
  }'

# 节点4
export MOONCAKE_CONFIG_PATH=./mooncake.json

python -m vllm.entrypoints.openai.api_server \
  --model meta-llama/Llama-3.1-70B-Instruct \
  --tensor-parallel-size 4 \
  --port 8201 \
  --kv-transfer-config '{
    "kv_connector": "MooncakeConnector",
    "kv_role": "kv_consumer"
  }'
```

步骤4: 启动Proxy

```
python examples/online_serving/disaggregated_serving/mooncake_connector/mc
--prefill http://192.168.1.10:8100 http://192.168.1.11:8101 \
--decode http://192.168.1.12:8200 http://192.168.1.13:8201 \
--port 8000
```

步骤5: 测试

```
# 发送测试请求
curl http://localhost:8000/v1/completions \
  -H "Content-Type: application/json" \
  -d '{
    "model": "meta-llama/Llama-3.1-70B-Instruct",
    "prompt": "Once upon a time",
    "max_tokens": 100
  }'
```

6.10.4 常见问题

Q1: Transfer Engine初始化失败

```
Error: Mooncake Transfer Engine initialization failed
```

解决方案： - 检查RDMA设备是否正确配置：`ibstat` - 检查etcd服务是否正常运行 - 检查防火墙是否放行了RDMA端口

Q2: KV Cache传输失败

```
Error: batch_transfer_sync_write returned non-zero
```

解决方案： - 检查网络连通性：`ping <remote_host>` - 检查RDMA连接：`ib_write_bw` 测试带宽 - 检查内存注册是否成功

Q3: 内存不足

```
Error: Failed to allocate GPU memory
```

解决方案： - 减小 `gpu_memory_pool_size` - 减小 `--gpu-memory-utilization` 参数 - 使用更小的模型或更大的GPU

Q4: 性能不达预期 排查步骤： 1. 检查RDMA带宽：`ib_read_bw` 2. 检查GPU利用率：`nvidia-smi` 3. 检查Prefill/Decode比例是否合理 4. 检查Prefix Caching命中率

Q5: 如何调优Prefill/Decode比例 建议： - 根据负载特征调整 - 输入密集型负载：Prefill节点更多 - 输出密集型负载：Decode节点更多 - 典型比例：Prefill:Decode = 1:1 到 1:3

Q6: 如何监控Mooncake集群状态 监控指标： - Transfer Engine传输带宽和延迟 - KV Cache命中率 - Prefill/Decode节点负载 - etcd集群健康状态

监控工具：

```
# 查看RDMA带宽
ib_read_bw -d mlx5_0

# 查看GPU利用率
nvidia-smi dmon -s u

# 查看Mooncake Master状态
curl http://localhost:50001/metrics

# 查看etcd集群健康
etcdctl endpoint health
```


Q7: 如何处理节点故障 故障恢复策略: 1. Transfer Engine自动重连 2. Conductor重新调度请求到健康节点 3. KV Cache从副本恢复

配置故障转移:

```
{
  "failover_enabled": true,
  "failover_timeout_ms": 5000,
  "retry_attempts": 3,
  "health_check_interval_ms": 1000
}
```

6.11 进阶主题

6.11.1 数学分析：以存换算的成本模型

Mooncake的"以存换算"理念可以通过数学模型严格证明其优势。设:

- $T_{\text{recompute}}$: 重新计算KV Cache的时间
- T_{transfer} : 传输KV Cache的时间
- C_{compute} : GPU计算成本 (\$/小时)
- C_{network} : 网络带宽成本 (\$/GB)
- S_{kv} : KV Cache大小 (GB)
- B_{network} : 网络带宽 (GB/s)

重计算成本: $T_{\text{recompute}} = \frac{2 \times L \times d_{\text{model}}^2 \times \text{seq_len}}{\text{GPU_FLOPS}}$

其中 L 是层数, d_{model} 是模型维度, seq_len 是序列长度。

传输成本: $T_{\text{transfer}} = \frac{S_{\text{kv}}}{B_{\text{network}}} = \frac{2 \times L \times n_{\text{heads}} \times d_{\text{head}} \times \text{seq_len} \times \text{bytes_per_element}}{B_{\text{network}}}$

成本平衡点: 当 $T_{\text{transfer}} < T_{\text{recompute}}$ 时, 传输比重计算更划算。

在实际部署中, 使用RDMA (100-400Gbps) 时: - 传输128K tokens的KV Cache (约2GB) 仅需0.04-0.16秒 - 重新计算相同长度的KV Cache可能需要数秒

因此, 在长上下文场景下, 传输KV Cache的成本远低于重计算。

6.11.2 多级缓存替换策略

Mooncake Store实现了智能的多级缓存替换策略：

1. LRU (Least Recently Used)

```
class LRUPolicy:
    def __init__(self, capacity):
        self.capacity = capacity
        self.cache = OrderedDict()

    def access(self, key):
        if key in self.cache:
            self.cache.move_to_end(key)
            return self.cache[key]
        return None

    def evict(self):
        return self.cache.popitem(last=False)[0]
```

2. LFU (Least Frequently Used)

```
class LFUPolicy:
    def __init__(self, capacity):
        self.capacity = capacity
        self.key_freq = {}
        self.freq_keys = defaultdict(OrderedDict)
        self.min_freq = 0

    def access(self, key):
        if key not in self.key_freq:
            return None
        freq = self.key_freq[key]
        del self.freq_keys[freq][key]
        self.key_freq[key] = freq + 1
        self.freq_keys[freq + 1][key] = True
        if not self.freq_keys[self.min_freq]:
            self.min_freq += 1
        return key

    def evict(self):
        key = next(iter(self.freq_keys[self.min_freq]))
        del self.freq_keys[self.min_freq][key]
        del self.key_freq[key]
        return key
```

3. 自适应混合策略

```
class AdaptiveCachePolicy:
    def __init__(self, capacity):
        self.capacity = capacity
        self.lru = LRUPolicy(capacity * 0.7)
        self.lfu = LFUPolicy(capacity * 0.3)
        self.hit_stats = {'lru': 0, 'lfu': 0}

    def access(self, key):
        # 尝试LRU
        result = self.lru.access(key)
        if result:
            self.hit_stats['lru'] += 1
            return result

        # 尝试LFU
        result = self.lfu.access(key)
        if result:
            self.hit_stats['lfu'] += 1
            return result

        return None

    def adapt_ratio(self):
        """根据命中率动态调整LRU/LFU比例"""
        total = sum(self.hit_stats.values())
        if total == 0:
            return

        lru_ratio = self.hit_stats['lru'] / total
        lfu_ratio = self.hit_stats['lfu'] / total

        # 调整容量分配
        self.lru.capacity = int(self.capacity * lru_ratio)
        self.lfu.capacity = int(self.capacity * lfu_ratio)
```

6.11.3 网络拓扑感知路由

Transfer Engine实现了网络拓扑感知路由，优化数据传输路径：

```
class TopologyAwareRouter:
    def __init__(self):
        self.topology = NetworkTopology()
        self.bandwidth_matrix = {}
        self.latency_matrix = {}

    def discover_topology(self):
        """发现网络拓扑结构"""
        nodes = self.get_all_nodes()
        for src in nodes:
            for dst in nodes:
                if src != dst:
                    self.bandwidth_matrix[(src, dst)] = self.measure_bandwidth(src, dst)
                    self.latency_matrix[(src, dst)] = self.measure_latency(src, dst)

    def select_best_path(self, src, dst, size):
        """选择最优传输路径"""
        # 直接路径
        direct_bw = self.bandwidth_matrix.get((src, dst), 0)
        direct_latency = self.latency_matrix.get((src, dst), float('inf'))
        direct_time = size / direct_bw + direct_latency if direct_bw > 0 else float('inf')

        # 通过中间节点转发
        best_relay_time = float('inf')
        best_relay = None

        for relay in self.get_all_nodes():
            if relay != src and relay != dst:
                bw1 = self.bandwidth_matrix.get((src, relay), 0)
                bw2 = self.bandwidth_matrix.get((relay, dst), 0)
                lat1 = self.latency_matrix.get((src, relay), float('inf'))
                lat2 = self.latency_matrix.get((relay, dst), float('inf'))

                if bw1 > 0 and bw2 > 0:
                    relay_time = size / min(bw1, bw2) + lat1 + lat2
                    if relay_time < best_relay_time:
                        best_relay_time = relay_time
                        best_relay = relay

        # 选择最优路径
        if best_relay_time < direct_time:
            return ('relay', best_relay)
        return ('direct', None)
```

6.11.4 压缩与量化优化

为了减少KV Cache传输开销，Mooncake支持多种压缩技术：

1. KV Cache量化

```
class KVCacheQuantizer:
    def __init__(self, bits=8):
        self.bits = bits
        self.scale = None
        self.zero_point = None

    def quantize(self, kv_cache: torch.Tensor):
        """将FP16 KV Cache量化为INT8"""
        # 计算缩放因子和零点
        min_val = kv_cache.min()
        max_val = kv_cache.max()

        self.scale = (max_val - min_val) / (2 ** self.bits - 1)
        self.zero_point = -min_val / self.scale

        # 量化
        quantized = torch.round(kv_cache / self.scale + self.zero_point)
        quantized = torch.clamp(quantized, 0, 2 ** self.bits - 1)

        return quantized.to(torch.uint8)

    def dequantize(self, quantized: torch.Tensor):
        """反量化回FP16"""
        return (quantized.float() - self.zero_point) * self.scale
```

2. 稀疏化传输

```
class SparseKVCacheTransfer:
    def __init__(self, sparsity_ratio=0.5):
        self.sparsity_ratio = sparsity_ratio

    def select_important_tokens(self, kv_cache, attention_scores):
        """选择重要的token进行传输"""
        # 基于attention scores选择重要token
        num_tokens = kv_cache.shape[2]
        num_keep = int(num_tokens * (1 - self.sparsity_ratio))

        # 计算每个token的重要性分数
        token_importance = attention_scores.sum(dim=(0, 1))

        # 选择最重要的token
        _, selected_indices = torch.topk(token_importance, num_keep)

        return kv_cache[:, :, selected_indices, :], selected_indices

    def reconstruct_full_cache(self, sparse_kv_cache, selected_indices, full_length):
        """重构完整的KV Cache"""
        full_cache = torch.zeros(
            sparse_kv_cache.shape[0],
            sparse_kv_cache.shape[1],
            full_length,
            sparse_kv_cache.shape[3],
            dtype=sparse_kv_cache.dtype,
            device=sparse_kv_cache.device
        )
        full_cache[:, :, selected_indices, :] = sparse_kv_cache
        return full_cache
```

6.11.5 安全性考虑

在生产环境部署Mooncake时，需要考虑以下安全因素：

1. 数据传输加密

```
class SecureTransferEngine:
    def __init__(self, encryption_key):
        self.cipher = AES.new(encryption_key, AES.MODE_GCM)

    def encrypt_and_transfer(self, data, remote_session):
        """加密后传输数据"""
        encrypted_data, tag = self.cipher.encrypt_and_digest(data)
        return self.transfer_engine.transfer(
            remote_session, encrypted_data, tag
        )

    def receive_and_decrypt(self, encrypted_data, tag):
        """接收并解密数据"""
        return self.cipher.decrypt_and_verify(encrypted_data, tag)
```

2. 访问控制

```
class AccessControl:
    def __init__(self):
        self.allowed_nodes = set()
        self.token_blacklist = set()

    def authenticate_node(self, node_id, token):
        """验证节点身份"""
        if token in self.token_blacklist:
            return False
        if node_id not in self.allowed_nodes:
            return False
        return self.verify_token(token)

    def authorize_transfer(self, src_node, dst_node, data_size):
        """授权数据传输"""
        # 检查传输配额
        if not self.check_quota(src_node, data_size):
            return False
        # 检查安全策略
        if not self.check_security_policy(src_node, dst_node):
            return False
        return True
```

3. 审计日志

```
class AuditLogger:
    def __init__(self, log_file):
        self.logger = logging.getLogger('mooncake_audit')
        handler = logging.FileHandler(log_file)
        self.logger.addHandler(handler)

    def log_transfer(self, src, dst, size, duration, status):
        """记录传输日志"""
        self.logger.info(json.dumps({
            'timestamp': time.time(),
            'event': 'kv_transfer',
            'src': src,
            'dst': dst,
            'size': size,
            'duration': duration,
            'status': status
        })))

    def log_access(self, node_id, action, resource, result):
        """记录访问日志"""
        self.logger.info(json.dumps({
            'timestamp': time.time(),
            'event': 'access',
            'node_id': node_id,
            'action': action,
            'resource': resource,
            'result': result
        })))
```

小结

Mooncake作为Kimi的底层推理平台，通过以KV Cache为中心的分离式架构，实现了大模型推理服务的重大突破。其核心创新包括：

1. **以存换算理念**：充分利用存储资源减少计算开销
2. **PD分离架构**：独立优化Prefill和Decode阶段
3. **分布式KV Cache池**：跨节点共享和复用KV Cache
4. **全局调度器**：智能的请求路由和负载均衡
5. **高性能传输引擎**：RDMA/GDR零拷贝传输

Mooncake已在Kimi生产环境中大规模部署，每天处理超过1000亿token，在A800集群上多处理115%的请求，在H800集群上多处理107%的请求。其开源版本已与vLLM、

SGLang等主流推理框架集成，成为AI推理基础设施领域的重要项目。

章明星教授领导的清华大学MADSys实验室与Moonshot AI的产学研合作，为Mooncake的成功奠定了坚实基础。该团队的研究成果发表在OSDI、SOSP、ASPLOS、FAST等顶级会议，Mooncake论文更是荣获FAST 2025最佳论文奖，标志着中国在AI基础设施领域的研究已达到国际领先水平。

随着AI Agent时代的到来，推理服务基础设施的重要性将进一步凸显。Mooncake及其背后的AI Serving Stack参考架构，将为下一代AI基础设施的发展提供重要参考。

附录A：术语表

术语	英文	说明
KV Cache	Key-Value Cache	Transformer模型中的键值缓存，用于加速自回归生成
Prefill	Prefill Phase	预填充阶段，并行处理输入token生成KV Cache
Decode	Decode Phase	解码阶段，自回归生成输出token
PD分离	Prefill-Decode Disaggregation	将Prefill和Decode阶段分离到不同节点执行
TTFT	Time To First Token	首token时间，从请求到第一个输出token的延迟
TPOT	Time Per Output Token	每输出token时间，解码阶段的平均延迟
TBT	Time Between Tokens	token间时间，同TPOT
SLO	Service Level Objective	服务等级目标，如延迟、吞吐量要求
RDMA	Remote Direct Memory Access	远程直接内存访问，零拷贝网络技术
GDR	GPUDirect RDMA	GPU直接RDMA，GPU内存间的直接传输
MFU	Model FLOPs Utilization	模型FLOPs利用率，衡量GPU计算效率
MoE	Mixture of Experts	混合专家模型，稀疏激活的大模型架构
EP	Expert Parallelism	专家并行，MoE模型的并行策略
TP	Tensor Parallelism	张量并行，将模型层内切分到多GPU
PP	Pipeline Parallelism	流水线并行，将模型层间切分到多节点
Prefix Caching	Prefix Caching	前缀缓存，复用共享前缀的KV Cache
Goodput	Goodput	有效吞吐量，满足SLO的成功请求数

附录B：参考资源

官方资源： - Mooncake GitHub: <https://github.com/kvcache-ai/Mooncake> - 官方文档: <https://kvcache-ai.github.io/Mooncake/> - FAST 2025 论文: <https://www.usenix.org/conference/fast25>

技术博客： - Mooncake技术解读系列（知乎） - vLLM官方文档: <https://docs.vllm.ai/> - SGLang官方文档: <https://docs.sglang.ai/>

相关论文： - *Efficient Large Language Models: A Survey* - *vLLM: Easy, Fast, and Cheap LLM Serving with PagedAttention* - *Splitwise: Efficient Generative LLM Inference Using Phase Splitting* - *DistServe: Disaggregating Prefill and Decoding for Goodput-optimized LLM Serving*

社区资源： - Mooncake Discord 社区 - GitHub Discussions - Stack Overflow 标签: mooncake, vllm, sglang

附录C：版本历史

版本	日期	更新内容
v0.1.0	2024-11-28	Transfer Engine开源
v0.2.0	2024-12-16	vLLM集成支持
v0.3.0	2025-03-07	Mooncake Store开源
v0.4.0	2025-04-10	SGLang集成支持
v0.5.0	2025-05-09	NIXL后端支持
v0.6.0	2025-06-20	LMDeploy后端支持
v0.7.0	2025-08-18	昇腾NPU支持
v0.8.0	2025-12-23	EPD分离支持
v1.0.0	2026-01-28	PyTorch生态系统集成

附录D：性能调优指南

1. 网络优化

```
# 启用巨页
echo 1024 > /sys/kernel/mm/hugepages/hugepages-2048kB/nr_hugepages

# 调整RDMA参数
echo "options mlx4_ib enable_4k_uar=1" >> /etc/modprobe.d/mlx4.conf

# 优化网络中断绑定
./set_irq_affinity.sh mlx5_0
```

2. GPU优化

```
# 设置GPU持久模式
nvidia-smi -pm 1

# 锁定GPU频率（可选）
nvidia-smi -lgc 1410

# 启用MIG（多实例GPU）
nvidia-smi mig -cgi 19,19,19 -C
```

3. 内存优化

```
# 增加文件描述符限制
ulimit -n 65535

# 调整内核参数
echo 'vm.swappiness=10' >> /etc/sysctl.conf
echo 'vm.dirty_ratio=40' >> /etc/sysctl.conf
sysctl -p
```

4. Mooncake特定优化

```
{
  "transfer_engine": {
    "batch_size": 32,
    "queue_depth": 128,
    "poll_mode": true
  },
  "cache": {
    "l1_size_gb": 40,
    "l2_size_gb": 200,
    "prefetch_enabled": true
  },
  "scheduler": {
    "prefill_decode_ratio": 0.5,
    "early_rejection_threshold": 0.9
  }
}
```

附录E：故障排查流程图

问题诊断流程



关于本章节

本章节由AI助手根据Mooncake官方文档、FAST 2025论文、GitHub仓库以及公开技术博客综合编写而成。内容力求准确全面，但由于技术快速发展，部分细节可能随版本更新而变化。建议读者参考官方文档获取最新信息。

致谢

感谢Moonshot AI和清华大学MADSys实验室开源Mooncake项目，为大模型推理基础设施的发展做出重要贡献。感谢所有开源社区贡献者的辛勤工作。

补充说明：Mooncake与AI Serving Stack的关系

AI Serving Stack是章明星团队提出的新一代AI推理服务参考架构，Mooncake是其中的核心组件之一。AI Serving Stack的整体架构包括：

- 1. 接入层（Ingress Layer）** - 负载均衡和流量管理 - 请求路由和分发 - 安全防护和限流
- 2. 调度层（Scheduling Layer）** - Mooncake Conductor全局调度器 - 弹性伸缩控制器 - 资源配额管理
- 3. 计算层（Compute Layer）** - vLLM/SGLang等推理引擎 - Prefill/Decode分离部署 - 异构硬件支持
- 4. 存储层（Storage Layer）** - Mooncake Store分布式KV Cache池 - 多级缓存管理 - 持久化存储
- 5. 传输层（Transport Layer）** - Mooncake Transfer Engine - RDMA/GPUDirect支持 - 拓扑感知路由

AI Serving Stack的设计目标是提供一个完整的、可组合的、开源的AI推理服务解决方案，支持从边缘设备到超大规模数据中心的各种部署场景。章明星团队与阿里云、蚂蚁集团、趋境科技等机构合作，持续推动AI Serving Stack的演进和生态建设。

目前，AI Serving Stack已获得InfoQ"2025年度AI工程与部署卓越奖"，并在多家企业的生产环境中得到验证。对于即将加入Mooncake团队的实习生来说，理解AI Serving Stack的整体架构将有助于更好地把握Mooncake在更大生态系统中的定位和作用。

第7章 AI Infra核心算法详解

本章面向即将加入Mooncake团队的实习生，系统梳理大模型推理系统中的核心算法。从Attention计算优化到推理加速，从调度策略到模型压缩，全面覆盖AI Infra的关键技术点。

7.1 Attention算法家族

Attention机制是Transformer架构的核心组件，也是大模型推理中最耗时的操作之一。本节深入分析从标准Self-Attention到各种优化变体的演进过程。

7.1.1 标准Self-Attention

问题背景

Self-Attention机制允许模型在处理序列时关注输入的不同位置，捕获长距离依赖关系。然而，其计算复杂度和内存占用随序列长度呈平方增长，成为长上下文推理的主要瓶颈。

数学定义

给定输入序列的Query、Key、Value矩阵 $Q, K, V \in \mathbb{R}^{n \times d_k}$ ，其中 n 是序列长度， d_k 是特征维度，标准Self-Attention计算如下：

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

其中缩放点积注意力（Scaled Dot-Product Attention）的完整计算流程为：

- 计算注意力分数**： $S = QK^T / \sqrt{d_k}$
- 应用Softmax**： $P = \text{softmax}(S)$
- 加权求和**： $O = PV$

计算复杂度分析

操作	计算量	内存访问
QK^T	$O(n^2 \cdot d_k)$	$O(n \cdot d_k + n^2)$
Softmax	$O(n^2)$	$O(n^2)$
PV	$O(n^2 \cdot d_v)$	$O(n^2 + n \cdot d_v)$
总计	$O(n^2 \cdot d)$	$O(n^2)$

关键观察：当序列长度 n 较大时（如 $n=32768$ ， $d=4096$ ），注意力矩阵 $S \in \mathbb{R}^{n \times n}$ 需要约4GB的HBM内存（FP32），而实际计算中需要多次读写这些中间结果，导致严重的内存带宽瓶颈。

内存占用分析

在推理过程中，KV Cache的存储需求为：

$$\text{Memory}_{\text{KV}} = 2 \times n \times d_{\text{model}} \times \text{bytes}_{\text{dtype}} \times \text{num}_{\text{layers}}$$

以LLaMA-2-70B为例（80层， $d_{\text{model}}=8192$ ，FP16）：
- 序列长度 $n=4096$ 时：约 10.5 GB
- 序列长度 $n=32768$ 时：约 84 GB

标准实现伪代码

```
def standard_attention(Q, K, V):  
    """  
    标准Self-Attention实现  
    Q, K, V: [batch_size, num_heads, seq_len, head_dim]  
    """  
    # Step 1: 计算注意力分数  
    scores = torch.matmul(Q, K.transpose(-2, -1)) / sqrt(head_dim)  
  
    # Step 2: Softmax归一化  
    attn_weights = F.softmax(scores, dim=-1)  
  
    # Step 3: 加权求和  
    output = torch.matmul(attn_weights, V)  
  
    return output
```

实际应用与限制

适用场景： - 短序列推理 ($n < 2048$) - 训练阶段的完整注意力计算 - 需要精确注意力权重的场景

主要限制： 1. **内存墙问题：** $O(n^2)$ 的内存复杂度限制上下文长度 2. **计算效率低：** 大量时间花费在HBM读写而非实际计算 3. **无法流式处理：** 必须等待完整KV Cache准备好

7.1.2 FlashAttention系列

FlashAttention是由Stanford HAI团队提出的IO-aware精确注意力算法，通过分块计算和重计算技术，在不近似的情况下显著减少HBM访问。

7.1.2.1 FlashAttention-1

核心思想

FlashAttention的核心洞察：**内存访问比计算慢得多**。在现代GPU上，HBM带宽（1.5-2 TB/s）远低于SRAM带宽（19 TB/s）和计算能力（FP16: 300+ TFLOPS）。

关键创新： 1. **Tiling（分块）：** 将大的注意力矩阵分解为适合SRAM的小块 2. **Online Softmax：** 在分块计算中维护softmax的归一化因子 3. **Recomputation：** 反向传播时重计算注意力而非存储

Tiling策略

将 Q, K, V 分成大小为 $B_r \times d$ 和 $B_c \times d$ 的块，其中 B_r, B_c 满足：

$$B_r \times B_c \leq M / 4$$

M 是SRAM容量（如A100的L2 Cache约40MB，但共享内存仅约164KB）。

Online Softmax算法

标准softmax需要完整的指数和：

$$\text{softmax}(x_i) = \frac{e^{x_i}}{\sum_j e^{x_j}}$$

Online Softmax允许增量计算。对于两个块的结果，设： - 块1： $m_1 = \max(x_1)$, $\ell_1 = \sum e^{x_1 - m_1}$ - 块2： $m_2 = \max(x_2)$, $\ell_2 = \sum e^{x_2 - m_2}$

合并后的统计量：

$$m = \max(m_1, m_2) \quad \ell = e^{m_1 - m} \ell_1 + e^{m_2 - m} \ell_2$$

FlashAttention-1 伪代码

```

def flash_attention_v1(Q, K, V, M):
    """
    FlashAttention-1 实现
    M: SRAM容量 (以元素数计)
    """
    N, d = Q.shape
    B_c = ceil(M / (4 * d)) # KV块大小
    B_r = min(ceil(M / (4 * d)), d) # Q块大小

    # 初始化输出和统计量
    O = zeros(N, d)
    L = zeros(N) # 存储log-sum-exp
    m = full(N, -inf) # 行最大值

    # 分块迭代
    for j in range(0, N, B_c):
        K_j = K[j:j+B_c, :]
        V_j = V[j:j+B_c, :]

        for i in range(0, N, B_r):
            Q_i = Q[i:i+B_r, :]
            O_i = O[i:i+B_r, :]
            m_i = m[i:i+B_r]
            L_i = L[i:i+B_r]

            # 计算当前块的注意力分数
            S_ij = Q_i @ K_j.T
            m_new = max(m_i, rowmax(S_ij))
            P_ij = exp(S_ij - m_new)
            L_new = exp(m_i - m_new) * L_i + rowsum(P_ij)

            # 更新输出
            O_i = (exp(m_i - m_new) * O_i * L_i + P_ij @ V_j) / L_new

            m[i:i+B_r] = m_new
            L[i:i+B_r] = L_new
            O[i:i+B_r, :] = O_i

    return O

```

复杂度分析

指标	标准Attention	FlashAttention-1
FLOPs	$O(N^2 d)$	$O(N^2 d)$
HBM访问	$O(N^2)$	$O(N^2 d^2 / M)$
内存占用	$O(N^2)$	$O(N)$

关键结论：FlashAttention-1将HBM访问从 $O(N^2)$ 降低到 $O(N^2 d^2 / M)$ ，在典型配置下（ $N=4096, d=64, M=64K$ ）可减少约50倍HBM访问。

7.1.2.2 FlashAttention-2

核心改进

FlashAttention-2针对前向和反向传播进行了进一步优化：

1. **减少非矩阵乘法FLOPs：**将softmax统计量的计算与矩阵乘法更好地融合
2. **更好的并行性：**在batch和head维度上更均衡地分配工作
3. **优化的warp调度：**减少线程束内的分歧

算法优化

主要改进在分块策略： - 前向传播：按行分块，每个warp处理一个行块 - 反向传播：利用前向的统计量，避免重复计算

性能提升

FlashAttention-2相比v1的典型加速比： - 前向传播：1.5-2.0x - 反向传播：2.0-3.0x - 端到端训练：1.3-1.5x

7.1.2.3 FlashAttention-3

针对Hopper架构的优化

FlashAttention-3专为NVIDIA Hopper架构（H100）设计，充分利用新硬件特性：

1. **WGMMMA（Warp Group Matrix Multiply Accumulate）：**
2. 使用新的异步矩阵乘法指令
3. 支持FP8精度的张量核心加速
4. **FP8低精度支持：**
5. 利用H100的FP8张量核心
6. 动态缩放因子管理
7. **流水线优化：**

- 8. 更激进的指令级并行
- 9. 减少内存等待时间

FP8量化策略

FlashAttention-3引入动态缩放：

$$Q_{fp8} = Q_{fp16} \times s_Q, \quad s_Q = 448 / \max(|Q|)$$

其中448是FP8 E4M3格式的最大可表示值。

性能数据

在 H100 SXM 上（序列长度 8192，batch size 2，head_dim 128）： - FP16: ~500 TFLOPS - FP8: ~900 TFLOPS（接近理论峰值）

FlashAttention系列对比

特性	FlashAttention-1	FlashAttention-2	FlashAttention-3
目标架构	Ampere/通用	Ampere/通用	Hopper专用
主要优化	IO-aware	并行性	硬件特性
FP8支持	否	否	是
典型加速	2-4x	3-6x	6-10x
精度损失	无	无	<0.1%

7.1.3 FlashDecoding

问题背景

FlashAttention在Prefill阶段（处理完整序列）表现优异，但在Decode阶段（逐个生成 token）面临新的挑战：

- **计算并行度低**：Decode阶段每次只处理一个Query
- **内存带宽瓶颈**：需要从HBM读取完整的KV Cache
- **GPU利用率不足**：大量计算单元空闲

核心思想

FlashDecoding的核心创新：**在序列长度维度上并行化Decode阶段的注意力计算。**

传统Decode:
$$o = \text{softmax}(qK^T)V$$

FlashDecoding将KV Cache分成多个chunk，每个chunk独立计算部分注意力，然后合并：

$$o_i = \text{softmax}(qK_i^T)V_i \quad \text{for each chunk } i$$

$$o = \text{merge}(o_1, o_2, \dots, o_m)$$

Parallelism Over Context Length

FlashDecoding引入两种并行策略：

1. **Intra-sequence Parallelism**: 单个序列的KV Cache分块并行处理
2. **Inter-sequence Parallelism**: 多个序列的Decode并行执行

分块合并算法

对于每个chunk i ，计算： - 局部最大值: $m_i = \max(qK_i^T)$ - 局部和: $\ell_i = \sum \exp(qK_i^T - m_i)$ - 局部输出: $o_i = \exp(qK_i^T - m_i)V_i / \ell_i$

全局合并: $m = \max_i(m_i)$ $\ell = \sum_i \exp(m_i - m) \ell_i$ $o = \sum_i \exp(m_i - m) \ell_i o_i / \ell$

FlashDecoding 伪代码

```
def flash_decoding(q, K_cache, V_cache, chunk_size):
    """
    FlashDecoding实现
    q: [batch, num_heads, 1, head_dim] - 单个query
    K_cache, V_cache: [batch, num_heads, seq_len, head_dim]
    """
    seq_len = K_cache.shape[2]
    num_chunks = ceil(seq_len / chunk_size)

    # 分配临时存储
    local_max = zeros(batch, num_heads, num_chunks)
    local_sum = zeros(batch, num_heads, num_chunks)
    local_out = zeros(batch, num_heads, num_chunks, head_dim)

    # 并行处理每个chunk
    for chunk_idx in range(num_chunks):
        start = chunk_idx * chunk_size
        end = min(start + chunk_size, seq_len)

        K_chunk = K_cache[:, :, start:end, :]
        V_chunk = V_cache[:, :, start:end, :]

        # 局部注意力计算
        scores = q @ K_chunk.transpose(-2, -1) / sqrt(head_dim)
        local_max[:, :, chunk_idx] = scores.max(dim=-1)

        exp_scores = exp(scores - local_max[:, :, chunk_idx].unsqueeze(-1))
        local_sum[:, :, chunk_idx] = exp_scores.sum(dim=-1)
        local_out[:, :, chunk_idx, :] = (exp_scores @ V_chunk) / local_sum

    # 全局合并
    global_max = local_max.max(dim=-1, keepdim=True)
    exp_shift = exp(local_max - global_max)
    global_sum = (exp_shift * local_sum).sum(dim=-1, keepdim=True)

    output = ((exp_shift.unsqueeze(-1) * local_sum.unsqueeze(-1) * local_out) / global_sum).sum(dim=-1, keepdim=True)

    return output
```

与FlashAttention的对比

特性	FlashAttention	FlashDecoding
适用阶段	Prefill	Decode
Query数量	多个（完整序列）	单个（逐token）
并行维度	batch × head	batch × head × seq_chunk
主要瓶颈	计算	内存带宽
加速效果	2-4x	3-8x（长序列）

实际应用

FlashDecoding特别适合： - 长上下文对话（>8K tokens） - 文档摘要和问答 - 代码生成（长代码上下文）

7.1.4 PagedAttention

问题背景

在LLM服务中，KV Cache管理面临严峻挑战：

- 1. **内存碎片**：不同长度的序列导致内部和外部碎片
- 2. **内存浪费**：预分配固定大小导致大量空间闲置
- 3. **无法共享**：并行解码（如beam search）时KV Cache重复存储

以vLLM的测试数据为例：在ShareGPT工作负载上，传统系统的KV Cache利用率仅20-40%。

虚拟内存启发

PagedAttention借鉴操作系统虚拟内存的核心思想：

操作系统概念	PagedAttention对应
虚拟地址	逻辑KV Cache块
物理页帧	GPU内存块
页表	Block Table
按需分页	动态分配块

Block Table管理

核心数据结构

```
class BlockTable:
    """PagedAttention的块表管理"""
    def __init__(self, block_size, num_gpu_blocks):
        self.block_size = block_size # 每个块的token数 (如16)
        self.num_gpu_blocks = num_gpu_blocks
        self.free_blocks = list(range(num_gpu_blocks)) # 空闲块列表
        self.block_tables = {} # sequence_id -> [block_ids]

    def allocate(self, sequence_id, num_tokens):
        """为序列分配块"""
        num_blocks_needed = ceil(num_tokens / self.block_size)
        if len(self.free_blocks) < num_blocks_needed:
            raise MemoryError("GPU内存不足")

        blocks = [self.free_blocks.pop() for _ in range(num_blocks_needed)]
        self.block_tables[sequence_id] = blocks
        return blocks

    def append_token(self, sequence_id):
        """为序列追加新token, 可能需要分配新块"""
        blocks = self.block_tables[sequence_id]
        num_tokens = len(blocks) * self.block_size

        if num_tokens % self.block_size == 0: # 需要新块
            if not self.free_blocks:
                raise MemoryError("GPU内存不足")
            blocks.append(self.free_blocks.pop())

        return blocks
```

内存布局

逻辑视图 (序列): [t1, t2, t3, ..., t50]
↓ 映射

物理视图 (块):

Block 5: [t1, t2, ..., t16]
Block 3: [t17, t18, ..., t32]
Block 7: [t33, t34, ..., t48]
Block 2: [t49, t50, _, _] (部分使用)

内存碎片消除

内部碎片

内部碎片来自块内未使用的空间。PagedAttention通过以下方式最小化： - 选择适中的块大小（通常16-32 tokens） - 最后一个块可以部分填充

外部碎片

外部碎片通过以下机制避免： 1. **统一块大小**：所有块大小相同，避免不规则分配 2. **按需分配**：不预分配连续空间 3. **块回收**：序列完成后块立即返回空闲池

KV Cache共享

PagedAttention支持多种共享场景：

1. **Beam Search共享**：
2. 多个beam共享相同的父序列前缀
3. 写时复制（Copy-on-Write）机制
4. **并行采样共享**：
5. 同一prompt生成多个输出
6. prompt部分的KV Cache完全共享

写时复制实现

```
def fork_sequence(self, parent_id, child_id):
    """创建序列的写时复制副本"""
    parent_blocks = self.block_tables[parent_id]
    # 初始时共享相同的块引用
    self.block_tables[child_id] = parent_blocks.copy()

    # 增加引用计数（实际实现中）
    for block_id in parent_blocks:
        self.ref_count[block_id] += 1

def write_token(self, sequence_id, position, token_kv):
    """写入token，触发写时复制"""
    block_idx = position // self.block_size
    block_id = self.block_tables[sequence_id][block_idx]

    if self.ref_count[block_id] > 1:
        # 需要复制块
        new_block_id = self.free_blocks.pop()
        copy_block(block_id, new_block_id)
        self.ref_count[block_id] -= 1
        self.block_tables[sequence_id][block_idx] = new_block_id
        block_id = new_block_id

    # 写入KV Cache
    write_to_block(block_id, position % self.block_size, token_kv)
```

PagedAttention性能分析

在ShareGPT数据集上的实测结果（vLLM论文数据）：

系统	吞吐量 (req/s)	KV Cache利用率
Orca	1.0x (baseline)	~30%
FasterTransformer	1.2x	~35%
vLLM (PagedAttention)	2.5-3.0x	~85%

实际应用与限制

优势： - 显著提升GPU内存利用率（2-4x） - 支持动态序列长度 - 高效的KV Cache共享

限制： - 块大小选择需要权衡（太小→管理开销，太大→内部碎片） - 随机访问模式可能降低缓存效率 - 需要额外的Block Table管理开销

7.2 推理加速算法

大模型推理的延迟主要来自两个方面：Prefill阶段（处理输入prompt）和Decode阶段（逐个生成token）。由于Decode阶段是内存带宽受限的，每生成一个token都需要加载完整的模型参数和KV Cache。本节介绍通过投机执行来突破这一瓶颈的系列算法。

7.2.1 Speculative Decoding（投机解码）

问题背景

在自回归生成中，每个token的生成都需要：1. 加载整个模型到GPU 2. 执行一次完整的前向传播 3. 采样得到下一个token

这个过程是顺序的，无法并行化。然而，语言模型生成的token往往具有较强的可预测性，特别是常见短语和模板化内容。

核心思想

投机解码采用**小模型起草，大模型验证**的策略：

1. **起草阶段（Draft）**：使用轻量级小模型（draft model）快速生成 γ 个候选token
2. **验证阶段（Verify）**：大模型（target model）并行验证这些候选token
3. **接受/拒绝**：根据概率分布决定是否接受候选token

数学原理

设： - M_p ：大模型（target model）的概率分布 $p(x)$ - M_q ：小模型（draft model）的概率分布 $q(x)$

对于候选token x ，接受概率为：

$$P(\text{accept}) = \min\left(1, \frac{p(x)}{q(x)}\right)$$

这保证了生成结果与直接使用大模型采样的分布一致（分布保持性）。

修正采样

如果候选token x 被拒绝，从修正分布中采样：

$$p'(x) = \frac{p(x) - q(x)}{1 - \sum_{x'} \min(p(x'), q(x'))}$$

或者使用更简单的形式：

$$p'(x) = \text{normalize}(\max(0, p(x) - q(x)))$$

算法伪代码

```
def speculative_decoding(prompt, M_p, M_q, gamma, max_tokens):
    """
    投机解码算法
    M_p: 大模型 (target model)
    M_q: 小模型 (draft model)
    gamma: 每次起草的token数
    """
    tokens = tokenize(prompt)

    while len(tokens) < max_tokens:
        # === 起草阶段 ===
        draft_tokens = []
        draft_probs = []

        for _ in range(gamma):
            q_dist = M_q(tokens + draft_tokens)
            x = sample(q_dist)
            draft_tokens.append(x)
            draft_probs.append(q_dist)

        # === 验证阶段 ===
        # 大模型并行计算所有候选token的概率
        all_tokens = tokens + draft_tokens
        p_dists = M_p(all_tokens, return_all_probs=True)

        # === 接受/拒绝 ===
        accepted = 0
        for i, (x, q_dist) in enumerate(zip(draft_tokens, draft_probs)):
            p_dist = p_dists[len(tokens) + i]
            p_x = p_dist[x]
            q_x = q_dist[x]

            # 计算接受概率
            accept_prob = min(1.0, p_x / q_x)

            if random() < accept_prob:
                # 接受token
                tokens.append(x)
                accepted += 1
            else:
                # 拒绝token, 从修正分布采样
                adjusted_dist = normalize(max(0, p_dist - q_dist))
                tokens.append(sample(adjusted_dist))
                break

        # 如果全部接受, 从p采样一个额外token
        if accepted == gamma:
            tokens.append(sample(p_dists[-1]))
```

```
return detokenize(tokens)
```

加速比分析

理论加速比取决于： 1. **接受率 α** ：候选token被接受的比例 2. **起草成本比 β** ：小模型与大模型的推理时间比 3. **起草数量 γ**

理想加速比（假设 $\beta \ll 1$ ）：

$$\text{Speedup} \approx \frac{1}{1 - \alpha^{\gamma+1}} \cdot \frac{\gamma + 1}{\gamma \beta + 1}$$

典型场景（ $\alpha = 0.8, \gamma = 4, \beta = 0.1$ ）： $\text{Speedup} \approx 2.5-3.0\times$

实际应用与限制

优势： - 无损加速：保持大模型的输出分布 - 通用性强：适用于任何自回归模型 - 实现相对简单

限制： 1. **小模型选择**：需要与大模型行为相似的小模型 2. **内存开销**：需要同时加载两个模型 3. **接受率依赖**：在低可预测性内容上效果差 4. **通信开销**：如果大小模型在不同设备上

典型配置： - LLaMA-70B + LLaMA-7B作为draft model - $\gamma = 4-8$ - 实测加速比：1.5-2.5x

7.2.2 Medusa

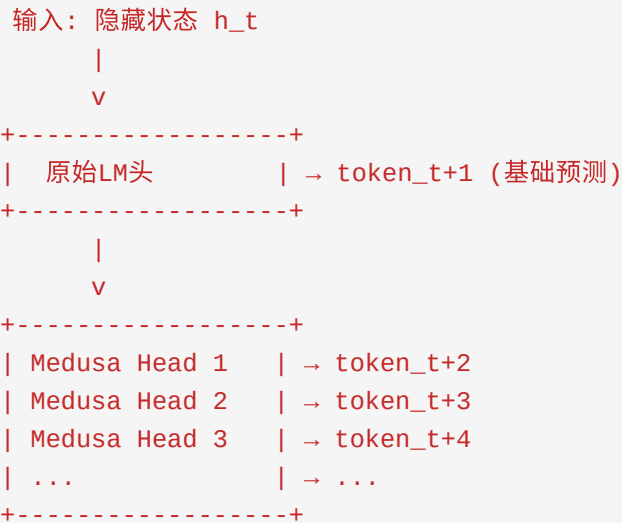
问题背景

投机解码需要维护两个独立的模型，带来额外的内存和工程复杂度。Medusa提出了一种**无需辅助模型**的投机解码方法，通过在原始模型上添加轻量级头来实现。

核心理想

Medusa的核心创新： 1. **多头解码**：在原始模型的顶层添加多个解码头 2. **每个头预测未来不同位置的token**：头1预测下一个token，头2预测下下个token，依此类推 3. **树状注意力**：高效组织多个候选序列的验证

多头结构



每个Medusa Head是一个轻量级MLP：

$$\text{Medusa}_i(h) = \text{Softmax}(W_i \cdot \text{MLP}_i(h))$$

训练方法

训练数据

使用原始模型生成的数据作为监督信号： 1. 从训练集中采样prompt 2. 用原始模型生成完整序列 3. 将生成的token作为每个头的目标

损失函数

$$\mathcal{L} = \mathcal{L}_{\text{base}} + \lambda \sum_{i=1}^k \mathcal{L}_{\text{Medusa}_i}$$

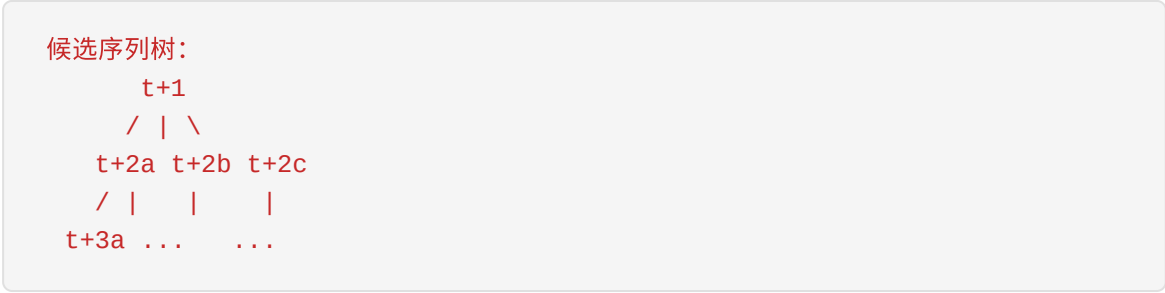
其中 $\mathcal{L}_{\text{base}}$ 是原始语言模型头的交叉熵损失， $\mathcal{L}_{\text{Medusa}_i}$ 是第 i 个Medusa头的损失。

训练策略

1. **冻结主干**：保持原始模型参数不变
2. **只训练Medusa Heads**：通常只有几百万参数
3. **典型训练时间**：1-2个epoch，数小时到一天

树状注意力验证

Medusa提出树状注意力来高效验证多个候选序列：



通过树状注意力，可以在一次前向传播中验证所有候选路径。

Medusa 伪代码


```
class MedusaModel(nn.Module):
    def __init__(self, base_model, num_heads, num_candidates):
        super().__init__()
        self.base_model = base_model
        self.num_heads = num_heads
        self.num_candidates = num_candidates

        # Medusa Heads: 每个头预测未来第i个token
        self.medusa_heads = nn.ModuleList([
            MedusaHead(base_model.config.hidden_size, base_model.vocab_size,
                        num_candidates, num_heads)
            for _ in range(num_heads)
        ])

    def forward(self, input_ids):
        # 基础模型前向
        outputs = self.base_model(input_ids, output_hidden_states=True)
        hidden = outputs.hidden_states[-1]

        # 基础预测
        base_logits = outputs.logits[:, -1, :]

        # Medusa预测
        medusa_logits = []
        for head in self.medusa_heads:
            medusa_logits.append(head(hidden[:, -1, :]))

        return base_logits, medusa_logits

    def generate(self, prompt, max_new_tokens):
        tokens = tokenize(prompt)

        for _ in range(max_new_tokens // (self.num_heads + 1)):
            # 获取所有预测
            base_logits, medusa_logits = self.forward(tokens)

            # 构建候选序列树
            candidates = build_tree(base_logits, medusa_logits, self.num_candidates)

            # 树状注意力验证
            verified_tokens = self.verify_tree(tokens, candidates)

            tokens.extend(verified_tokens)

        return detokenize(tokens)

    def verify_tree(self, prefix, candidates):
        """使用树状注意力验证候选序列"""
        # 构建树状注意力掩码
        tree_mask = build_tree_mask(candidates)
```

```
# 一次前向验证所有候选
all_sequences = [prefix + cand for cand in candidates]
logits = self.base_model(all_sequences, attention_mask=tree_mask).

# 选择接受的token序列
accepted = []
for i, cand in enumerate(candidates):
    if verify_acceptance(logits[i], cand):
        accepted.extend(cand)
    else:
        # 从修正分布采样
        accepted.append(sample_adjusted(logits[i]))
        break

return accepted
```

与投机解码的对比

特性	Speculative Decoding	Medusa
辅助模型	需要独立的小模型	无需辅助模型
内存开销	2x模型大小	1.01-1.05x模型大小
训练需求	无需训练	需要训练Medusa Heads
加速比	1.5-2.5x	1.8-2.8x
实现复杂度	中等	较高

性能分析

在Vicuna-7B上的实测结果： - 平均加速比： 2.0-2.5x - Medusa Heads数量： 4-8个 - 训练时间： 约4-8小时 - 内存增加： <5%

7.2.3 EAGLE

问题背景

Medusa在特征空间进行预测，但直接预测未来token的分布可能面临信息不足的问题。EAGLE（Extrapolation Algorithm for Greater Language-model Efficiency）提出在**特征级别**进行投机，利用更丰富的上下文信息。

核心思想

EAGLE的核心创新： 1. **特征级投机**：预测未来token的特征（而非token本身） 2. **自回归头（Auto-regressive Head）**：使用轻量级自回归模型预测特征序列 3. **特征到token映射**：通过LM头将预测特征转换为token分布

特征级预测的优势

相比token级预测，特征级预测： - 包含更丰富的语义信息 - 更容易捕获长距离依赖 - 预测任务更简单（连续空间vs离散空间）

自回归头架构

EAGLE的自回归头是一个小型Transformer：

```
输入: [h_t, h_{t-1}, ..., h_{t-k}] (历史特征序列)
      |
      v
+-----+
| 小型Transformer | (2-4层)
+-----+
      |
      v
输出: [\hat{h}_{t+1}, \hat{h}_{t+2}, ..., \hat{h}_{t+m}] (预测的未来特征)
```

自回归头的规模通常只有原模型的1-10%。

算法流程

阶段1：特征投机

- 使用自回归头预测未来m个特征
- $\hat{H} = \text{AR_Head}([h_t, h_{t-1}, \dots])$

阶段2：Token生成

- 通过LM头将特征转换为token分布
- 对每个 $\hat{h}_{t+i}$: $p_{t+i} = \text{Softmax}(W_{\text{lm}} \cdot \hat{h}_{t+i})$

阶段3：大模型验证

- 大模型并行验证候选token
- 接受/拒绝机制同投机解码

EAGLE 伪代码

```
class EAGLEModel(nn.Module):
    def __init__(self, base_model, ar_layers=2, ar_hidden=None):
        super().__init__()
        self.base_model = base_model
        hidden_size = base_model.config.hidden_size

        # 自回归头: 小型Transformer
        self.ar_head = nn.TransformerDecoder(
            nn.TransformerDecoderLayer(
                d_model=hidden_size,
                nhead=hidden_size // 64,
                dim_feedforward=ar_hidden or hidden_size * 2,
                batch_first=True
            ),
            num_layers=ar_layers
        )

        # 特征投影
        self.feature_proj = nn.Linear(hidden_size, hidden_size)

    def forward(self, input_ids):
        # 获取基础模型特征
        outputs = self.base_model(input_ids, output_hidden_states=True)
        features = outputs.hidden_states[-1] # [batch, seq, hidden]

        return features

    def speculate_features(self, features, num_steps):
        """使用自回归头投机未来特征"""
        speculated = []
        current = features

        for _ in range(num_steps):
            # 自回归预测下一个特征
            next_feature = self.ar_head(current, current)
            next_feature = self.feature_proj(next_feature[:, -1:, :])
            speculated.append(next_feature)
            current = torch.cat([current, next_feature], dim=1)

        return torch.cat(speculated, dim=1)

    def features_to_tokens(self, features):
        """将特征转换为token分布"""
        logits = self.base_model.lm_head(features)
        return F.softmax(logits, dim=-1)

    def generate(self, prompt, max_tokens, gamma=5):
        tokens = tokenize(prompt)

        while len(tokens) < max_tokens:
```

```

# 获取当前特征
features = self.forward(tokens)

# 投机未来特征
spec_features = self.speculate_features(features[:, -4:, :], g

# 转换为token分布
draft_dists = self.features_to_tokens(spec_features)
draft_tokens = [sample(dist) for dist in draft_dists[0]]

# 大模型验证
verified = self.verify(tokens, draft_tokens, draft_dists[0])
tokens.extend(verified)

return detokenize(tokens)

def verify(self, prefix, draft_tokens, draft_dists):
    """验证候选token"""
    all_tokens = prefix + draft_tokens
    p_dists = self.base_model(all_tokens, return_all_probs=True)

    accepted = []
    for i, (x, q_dist) in enumerate(zip(draft_tokens, draft_dists)):
        p_dist = p_dists[len(prefix) + i]
        p_x = p_dist[x]
        q_x = q_dist[x]

        if random() < min(1.0, p_x / q_x):
            accepted.append(x)
        else:
            adjusted = normalize(max(0, p_dist - q_dist))
            accepted.append(sample(adjusted))
            break

    return accepted

```

训练方法

EAGLE的训练分为两个阶段：

1. 特征收集阶段：

2. 用基础模型处理训练数据
3. 保存每层的隐藏状态

4. 自回归头训练：

5. 输入：历史特征序列 $[h_{t-k}, \dots, h_t]$
6. 目标：未来特征序列 $[h_{t+1}, \dots, h_{t+m}]$

7. 损失：MSE损失 $\mathcal{L} = \sum_{i=1}^m ||\hat{h}_{t+i} - h_{t+i}||^2$

性能分析

EAGLE相比Medusa的优势： - 更高的接受率（特征级预测更准确） - 更好的长距离依赖建模 - 在复杂任务上表现更稳定

实测加速比： - LLaMA-2-7B: 2.5-3.0x - LLaMA-2-70B: 2.0-2.5x

7.2.4 Lookahead Decoding

问题背景

投机解码、Medusa和EAGLE都需要额外的模型或训练。Lookahead Decoding提出了一种**无需任何额外模型或训练**的并行解码方法。

核心思想

Lookahead Decoding基于Jacobi迭代的思想： 1. **打破顺序依赖**：将自回归生成视为求解方程组 2. **并行猜测**：基于n-gram匹配生成多个候选token 3. **并行验证**：一次前向传播验证所有猜测

Jacobi迭代视角

自回归生成可以表示为：

$$x_{t+1} = f(x_1, x_2, \dots, x_t)$$

Jacobi迭代同时更新所有位置：

$$x_i^{(k+1)} = f(x_1^{(k)}, x_2^{(k)}, \dots, x_{i-1}^{(k)})$$

对于语言模型，这意味着可以并行猜测多个未来token。

n-gram匹配策略

Lookahead Decoding维护一个n-gram窗口：

已生成序列: [The, cat, sat, on, the, ...]
|____| (2-gram: "cat sat")

在窗口中查找匹配的n-gram:

位置3: "cat sat" → 下一个token是 "on"

位置10: "cat sat" → 下一个token是 "under"

候选token: ["on", "under", ...]

Lookahead Decoding 伪代码

```
class LookaheadDecoding:
    def __init__(self, model, window_size=5, ngram_size=3, num_candidates=10):
        self.model = model
        self.window_size = window_size
        self.ngram_size = ngram_size
        self.num_candidates = num_candidates
        self.ngram_cache = {} # n-gram -> 后续token列表

    def update_cache(self, tokens):
        """更新n-gram缓存"""
        for i in range(len(tokens) - self.ngram_size):
            ngram = tuple(tokens[i:i + self.ngram_size])
            next_token = tokens[i + self.ngram_size]

            if ngram not in self.ngram_cache:
                self.ngram_cache[ngram] = []
            if next_token not in self.ngram_cache[ngram]:
                self.ngram_cache[ngram].append(next_token)

    def generate_candidates(self, tokens):
        """基于n-gram匹配生成候选token"""
        candidates = []

        # 使用最近的n-gram查找
        for n in range(self.ngram_size, 1, -1):
            if len(tokens) < n:
                continue

            ngram = tuple(tokens[-n:])
            if ngram in self.ngram_cache:
                candidates.extend(self.ngram_cache[ngram])

        # 去重并限制数量
        candidates = list(dict.fromkeys(candidates))[:self.num_candidates]
        return candidates

    def verify_candidates(self, tokens, candidates):
        """并行验证候选token"""
        if not candidates:
            return [self.model.generate_next(tokens)]

        # 构建验证批次
        sequences = [tokens + [cand] for cand in candidates]

        # 并行前向
        logits = self.model.batch_forward(sequences)

        # 检查哪些猜测是正确的
        accepted = []
        for i, cand in enumerate(candidates):
```



```
# 如果模型预测的下一个token与猜测一致
predicted = logits[i].argmax(dim=-1)
if predicted == cand:
    accepted.append(cand)
else:
    # 添加正确的token并停止
    accepted.append(predicted.item())
    break

return accepted

def generate(self, prompt, max_tokens):
    tokens = tokenize(prompt)

    while len(tokens) < max_tokens:
        # 生成候选token
        candidates = self.generate_candidates(tokens)

        # 验证候选
        verified = self.verify_candidates(tokens, candidates)

        # 更新序列和缓存
        tokens.extend(verified)
        self.update_cache(tokens)

    return detokenize(tokens)
```

复杂度分析

操作	复杂度	说明
n-gram查找	$O(1)$	哈希表实现
候选生成	$O(k)$	k为候选数
并行验证	$O(1)$	单次batch前向
缓存更新	$O(w)$	w为窗口大小

实际应用与限制

优势： - 无需额外模型或训练 - 实现简单 - 适用于任何自回归模型

限制： 1. **n-gram稀疏性：**长n-gram匹配率低 2. **内容依赖：**在模板化内容上效果好，在创造性内容上效果差 3. **缓存开销：**需要维护n-gram缓存

典型加速比： - 代码生成：1.5-2.0x - 对话：1.2-1.5x - 创意写作：<1.2x

7.3 调度算法

在大模型服务系统中，调度算法决定了请求的批处理方式、执行顺序和资源分配，直接影响系统的吞吐量和延迟。本节介绍从静态批处理到动态调度的演进。

7.3.1 Continuous Batching（连续批处理）

问题背景

传统的静态批处理（Static Batching）存在严重问题：

静态批处理示例：

请求A： [====PREFILL====] [=D1=] [=D2=] [=D3=]

请求B： [====PREFILL====] [=D1=] [=D2=]

请求C： [====PREFILL====] [=D1=] [=D2=] [=D3=] [=D4=]

问题：

1. 请求B完成后，GPU空闲等待A和C
2. 新请求D必须等待整个批次完成
3. GPU利用率低，延迟高

核心思想

Continuous Batching（也称为In-flight Batching或Dynamic Batching）允许：

1. **请求动态加入**：新请求可以在任何迭代加入批次
2. **请求动态退出**：完成的请求立即释放资源
3. **迭代级调度**：每个迭代重新组织批次

关键机制

Continuous Batching示例：

时间步1： [A_pre][B_pre][C_pre]

时间步2： [A_d1][B_d1][C_d1][D_pre] <- D加入

时间步3： [A_d2][B_d2][C_d2][D_d1] <- B完成退出

时间步4： [A_d3][C_d3][D_d2][E_pre] <- E加入

实现细节

迭代级调度

```
class ContinuousBatcher:
    def __init__(self, model, max_batch_size, max_tokens):
        self.model = model
        self.max_batch_size = max_batch_size
        self.max_tokens = max_tokens
        self.waiting_queue = deque()  # 等待队列
        self.running_batch = []       # 运行中的请求

    def schedule(self):
        """迭代级调度"""
        # 1. 移除完成的请求
        self.running_batch = [req for req in self.running_batch if not req.is_finished()]

        # 2. 尝试添加新请求
        while (len(self.running_batch) < self.max_batch_size and
               self.waiting_queue):
            new_req = self.waiting_queue.popleft()
            if self.can_fit(new_req):
                self.running_batch.append(new_req)
            else:
                self.waiting_queue.appendleft(new_req)
                break

        # 3. 执行当前批次
        if self.running_batch:
            self.execute_batch(self.running_batch)

    def can_fit(self, request):
        """检查是否能容纳新请求"""
        total_tokens = sum(req.num_tokens() for req in self.running_batch)
        total_tokens += request.num_tokens()
        return total_tokens <= self.max_tokens

    def execute_batch(self, batch):
        """执行批次"""
        # 构建输入张量
        input_ids = pad_and_stack([req.get_input() for req in batch])
        attention_mask = create_attention_mask(batch)
        position_ids = create_position_ids(batch)

        # 前向传播
        outputs = self.model(input_ids, attention_mask, position_ids)

        # 分发结果
        for i, req in enumerate(batch):
            req.update(outputs[i])
```

请求状态管理

```
class Request:
    def __init__(self, prompt, max_new_tokens):
        self.prompt_tokens = tokenize(prompt)
        self.generated_tokens = []
        self.max_new_tokens = max_new_tokens
        self.phase = 'PREFILL' # PREFILL或DECODE

    def num_tokens(self):
        return len(self.prompt_tokens) + len(self.generated_tokens)

    def is_done(self):
        return (len(self.generated_tokens) >= self.max_new_tokens or
                self.generated_tokens[-1] == EOS_TOKEN)

    def get_input(self):
        if self.phase == 'PREFILL':
            return self.prompt_tokens
        else:
            return [self.generated_tokens[-1]]

    def update(self, new_token):
        self.generated_tokens.append(new_token)
        self.phase = 'DECODE'
```

吞吐量优化

批大小权衡

Continuous Batching需要在以下因素间权衡：

批大小	吞吐量	延迟	GPU利用率
小	低	低	低
中	高	中	高
大	饱和	高	高

最优批大小取决于： - 模型大小和内存占用 - 请求到达模式 - 延迟SLA要求

自适应批大小

```
def adaptive_batch_size(self, current_batch, waiting_requests):
    """自适应调整批大小"""
    # 基于当前负载和等待队列长度
    queue_length = len(waiting_requests)
    avg_wait_time = self.get_avg_wait_time()

    if queue_length > 10 and avg_wait_time > 5.0:
        # 队列压力大，增加批大小
        return min(self.max_batch_size, len(current_batch) + 2)
    elif avg_wait_time < 1.0 and len(current_batch) > 1:
        # 负载轻，减小批大小以降低延迟
        return max(1, len(current_batch) - 1)

    return len(current_batch)
```

性能分析

Continuous Batching相比Static Batching的提升：

指标	Static Batching	Continuous Batching	提升
吞吐量	1.0x	2.0-3.0x	2-3x
平均延迟	1.0x	0.8-1.2x	相当
P99延迟	1.0x	0.5-0.8x	更好
GPU利用率	40-60%	70-90%	显著提升

7.3.2 Chunked Prefill

问题背景

在Continuous Batching中，Prefill阶段（处理输入prompt）和Decode阶段（生成token）存在冲突：

- 1. **Prefill计算密集**：需要处理长序列，占用大量计算资源
- 2. **Decode内存带宽密集**：需要频繁访问KV Cache
- 3. **混合执行困难**：同时执行Prefill和Decode会互相干扰

传统方法： - 方案A：所有请求先完成Prefill，再一起Decode → 新请求等待时间长 - 方案B：每个请求独立Prefill和Decode → 批处理效果差

核心思想

Chunked Prefill将长Prefill分解为多个小块，与Decode迭代交错执行：

传统方式：

`[A_pre][A_dec][A_dec] ... [B_pre][B_dec][B_dec] ...`

Chunked Prefill：

`[A_pre_1][A_pre_2][B_pre_1][A_dec][B_pre_2][A_dec][B_dec] ...`

^ 小块Prefill与Decode交错

分块策略

将长序列的Prefill分成固定大小的块（如512 tokens）：

$$\text{num_chunks} = \left\lceil \frac{\text{seq_len}}{\text{chunk_size}} \right\rceil$$

每个Prefill块与Decode迭代一起批处理执行。

实现细节

混合批次构建

```
class ChunkedPrefillScheduler:
    def __init__(self, model, chunk_size=512, max_batch_tokens=4096):
        self.model = model
        self.chunk_size = chunk_size
        self.max_batch_tokens = max_batch_tokens
        self.prefill_queue = deque() # 等待Prefill的请求
        self.decode_queue = deque() # 等待Decode的请求

    def schedule(self):
        """构建混合批次"""
        batch = []
        total_tokens = 0

        # 1. 优先添加Decode请求（低延迟敏感）
        while self.decode_queue and total_tokens < self.max_batch_tokens:
            req = self.decode_queue.popleft()
            batch.append(('DECODE', req))
            total_tokens += 1 # Decode每次1个token

        # 2. 添加Prefill块
        while self.prefill_queue and total_tokens < self.max_batch_tokens:
            req = self.prefill_queue[0]

            # 计算当前Prefill块的大小
            remaining = req.prefill_remaining()
            chunk = min(remaining, self.chunk_size)

            if total_tokens + chunk <= self.max_batch_tokens:
                batch.append(('PREFILL', req, chunk))
                total_tokens += chunk
                req.advance_prefill(chunk)

            if req.prefill_done():
                self.prefill_queue.popleft()
                self.decode_queue.append(req)
            else:
                break

        return batch

    def execute_batch(self, batch):
        """执行混合批次"""
        # 分离Prefill和Decode
        prefill_items = [item for item in batch if item[0] == 'PREFILL']
        decode_items = [item for item in batch if item[0] == 'DECODE']

        # 构建统一的输入
        input_ids = []
        position_ids = []
```

```
for _, req, chunk in prefill_items:
    input_ids.extend(req.get_prefill_chunk(chunk))
    position_ids.extend(range(req.prefill_start, req.prefill_start + chunk))

for _, req in decode_items:
    input_ids.append(req.get_last_token())
    position_ids.append(req.get_position())

# 单次前向
outputs = self.model(input_ids, position_ids)

# 分发结果
idx = 0
for _, req, chunk in prefill_items:
    req.update_kv_cache(outputs[idx:idx+chunk])
    idx += chunk

for _, req in decode_items:
    req.append_token(outputs[idx])
    idx += 1
```

注意力掩码处理

混合批次需要特殊的注意力掩码：

```
def create_mixed_attention_mask(batch):
    """
    创建混合批次的注意力掩码
    Prefill: 因果掩码
    Decode: 全可见（只需要最后一个token的KV）
    """
    total_len = sum(chunk if op == 'PREFILL' else 1 for op, *rest in batch)
    mask = torch.zeros(total_len, total_len)

    pos = 0
    for op, *rest in batch:
        if op == 'PREFILL':
            _, _, chunk = rest
            # Prefill使用因果掩码
            mask[pos:pos+chunk, pos:pos+chunk] = torch.tril(torch.ones(chunk, chunk))
            pos += chunk
        else:
            # Decode只关注自己
            mask[pos, pos] = 1
            pos += 1

    return mask
```

延迟优化分析

首Token延迟 (TTFT)

Chunked Prefill对首token延迟的影响：

策略	TTFT	说明
完整Prefill	高	新请求等待前面所有Prefill完成
Chunked Prefill	低	新请求可以快速加入批次

交错延迟

场景：请求A（长Prefill）和请求B（短Prefill）同时到达

完整Prefill:

时间：0 1 2 3 4 5

A: [=====PREFILL=====]

B: 等待... 等待... 等待... [PREFILL][D1][D2]

Chunked Prefill:

时间：0 1 2 3 4 5

A: [chunk1][chunk2][chunk3]...

B: [chunk1][D1][chunk2][D2]...

^ B可以更快开始生成

性能分析

Chunked Prefill的优势： - 更好的公平性：短请求不会被长请求阻塞 - 更高的GPU利用率：计算和内存带宽操作更好地重叠 - 更可预测的延迟：避免长Prefill造成的延迟尖峰

实测数据（vLLM）： - 在混合工作负载上，P99延迟降低30-50% - 吞吐量提升10-20%

7.3.3 SARATHI-Serve调度

问题背景

现有调度算法往往只关注吞吐量或延迟的单一优化目标。SARATHI-Serve提出了一种吞吐量-延迟联合优化的调度策略。

核心思想

- SARATHI-Serve的核心洞察：
- 1. 预填充和解耦的权衡：大batch提升吞吐量但增加延迟
 - 2. 请求特性感知：不同请求有不同的延迟敏感度
 - 3. 自适应调度：根据系统状态动态调整

策略

关键概念

请求分类： - **延迟敏感型**：交互式应用（如聊天），要求低TTFT - **吞吐量敏感型**：批处理应用（如文档处理），要求高吞吐

调度模式： - **eager模式**：低延迟优先，小batch快速响应 - **batch模式**：高吞吐优先，大batch最大化利用率

调度策略

自适应模式切换

```
class SarathiScheduler:
    def __init__(self, model):
        self.model = model
        self.mode = 'BATCH' # 或 'EAGER'
        self.latency_threshold = 2.0 # 延迟阈值 (秒)
        self.throughput_history = deque(maxlen=100)

    def select_mode(self):
        """选择调度模式"""
        current_latency = self.measure_avg_latency()
        queue_length = len(self.waiting_queue)

        if current_latency > self.latency_threshold:
            # 延迟过高, 切换到eager模式
            self.mode = 'EAGER'
        elif queue_length > 10:
            # 队列积压, 切换到batch模式
            self.mode = 'BATCH'

    def schedule_eager(self):
        """Eager模式: 低延迟优先"""
        batch = []

        # 优先处理延迟敏感请求
        latency_sensitive = [r for r in self.waiting_queue if r.is_latency_sensitive]
        throughput_sensitive = [r for r in self.waiting_queue if not r.is_latency_sensitive]

        # 小batch快速处理
        for req in latency_sensitive[:self.small_batch_size]:
            batch.append(req)

        return batch

    def schedule_batch(self):
        """Batch模式: 高吞吐优先"""
        batch = []
        total_tokens = 0

        # 尽可能填充批次
        for req in self.waiting_queue:
            if total_tokens + req.num_tokens() <= self.max_tokens:
                batch.append(req)
                total_tokens += req.num_tokens()

        return batch
```

请求优先级

```
class Request:
    def __init__(self, prompt, priority='normal', max_latency=None):
        self.prompt = prompt
        self.priority = priority # 'high', 'normal', 'low'
        self.max_latency = max_latency
        self.arrival_time = time.time()

    def get_priority_score(self):
        """计算优先级分数（越低越优先）"""
        wait_time = time.time() - self.arrival_time

        # 基于优先级的基准分数
        base_score = {'high': 0, 'normal': 10, 'low': 20}[self.priority]

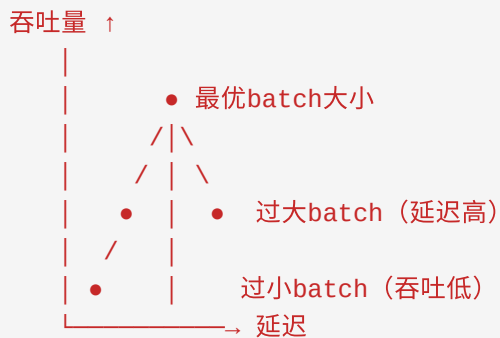
        # 等待时间加权
        urgency = wait_time / self.max_latency if self.max_latency else 0

        return base_score - urgency * 10
```

吞吐量-延迟权衡

Pareto最优分析

SARATHI-Serve探索了吞吐量和延迟的Pareto前沿：



动态batch大小

```
def dynamic_batch_size(self, requests, latency_sla):
    """
    动态确定最优batch大小

    目标：在满足延迟SLA的前提下最大化吞吐量
    """
    # 估计不同batch大小的延迟
    for batch_size in range(1, self.max_batch_size + 1):
        estimated_latency = self.estimate_latency(requests[:batch_size])

        if estimated_latency <= latency_sla:
            best_batch_size = batch_size
        else:
            break

    return best_batch_size

def estimate_latency(self, batch):
    """估计批次执行延迟"""
    # 基于历史数据和模型特性
    num_tokens = sum(r.num_tokens() for r in batch)

    # 线性模型（简化）
    latency = self.latency_intercept + self.latency_per_token * num_tokens

    return latency
```

性能分析

SARATHI-Serve在不同工作负载下的表现：

工作负载	吞吐量提升	P99延迟降低
聊天	1.2x	40%
代码生成	1.5x	25%
文档处理	1.8x	10%
混合	1.4x	30%

7.4 压缩算法

大模型推理面临严峻的内存挑战。以LLaMA-3-70B为例，FP16权重需要约140GB内存，加上KV Cache后很容易超出单卡容量。量化技术通过降低数值精度来压缩模型和激活，是解决这一问题的关键技术。

7.4.1 KV Cache量化

问题背景

KV Cache是推理阶段的主要内存消耗来源：

$$\text{Memory}_{\text{KV}} = 2 \times n_{\text{layers}} \times n_{\text{heads}} \times d_{\text{head}} \times n_{\text{tokens}} \times \text{bytes}_{\text{dtype}}$$

对于长序列，KV Cache可能超过模型权重本身： - LLaMA-3-70B，序列长度32K，FP16：约84GB KV Cache vs 140GB权重

7.4.1.1 INT8量化

对称量化

对称INT8量化将FP16值映射到[-128, 127]范围：

$$x_{\text{int8}} = \text{round}\left(\frac{x_{\text{fp16}}}{s}\right)$$

$$x_{\text{dequant}} = x_{\text{int8}} \times s$$

其中缩放因子 s 计算为：

$$s = \frac{\max(|x|)}{127}$$

非对称量化

非对称量化使用零点（zero-point）：

$$x_{\text{int8}} = \text{round}\left(\frac{x_{\text{fp16}} - z}{s}\right)$$

$$x_{\text{dequant}} = x_{\text{int8}} \times s + z$$

其中： $s = \frac{\max(x) - \min(x)}{255}$ $z = \text{round}\left(\frac{-\min(x)}{s}\right)$

逐通道量化

KV Cache通常按通道（channel-wise）量化以获得更好的精度：

```
def quantize_kv_cache_int8(K_cache, V_cache, axis=-1):
    """
    对KV Cache进行INT8量化
    K_cache, V_cache: [batch, heads, seq_len, head_dim]
    """
    # 计算每个通道的缩放因子
    K_max = K_cache.abs().amax(dim=axis, keepdim=True)
    V_max = V_cache.abs().amax(dim=axis, keepdim=True)

    K_scale = K_max / 127.0
    V_scale = V_max / 127.0

    # 量化
    K_int8 = torch.round(K_cache / K_scale).clamp(-128, 127).to(torch.int8)
    V_int8 = torch.round(V_cache / V_scale).clamp(-128, 127).to(torch.int8)

    return K_int8, V_int8, K_scale, V_scale

def dequantize_kv_cache_int8(K_int8, V_int8, K_scale, V_scale):
    """反量化KV Cache"""
    K_fp16 = K_int8.to(torch.float16) * K_scale
    V_fp16 = V_int8.to(torch.float16) * V_scale
    return K_fp16, V_fp16
```

精度分析

INT8 KV Cache量化的精度损失：

模型	任务	FP16 PPL	INT8 PPL	相对增长
LLaMA-2-7B	WikiText	5.12	5.18	1.2%
LLaMA-2-70B	WikiText	3.32	3.35	0.9%
LLaMA-3-8B	HumanEval	28.0%	27.5%	-1.8%

结论：INT8 KV Cache量化通常带来<2%的精度损失，可接受。

7.4.1.2 INT4量化

分组量化

INT4的范围仅为[-8, 7]，需要更精细的量化策略。分组量化（group-wise quantization）将通道分成小组：

```
def quantize_kv_cache_int4(K_cache, group_size=128):
    """
    INT4分组量化
    """
    batch, heads, seq_len, head_dim = K_cache.shape
    num_groups = head_dim // group_size

    # reshape为 [..., num_groups, group_size]
    K_resaped = K_cache.reshape(batch, heads, seq_len, num_groups, group_size)

    # 每组计算缩放因子
    K_max = K_resaped.abs().amax(dim=-1, keepdim=True)
    K_scale = K_max / 7.0

    # 量化到INT4（实际存储为INT8，但只使用4bit）
    K_int4 = torch.round(K_resaped / K_scale).clamp(-8, 7)

    # 打包存储（两个INT4打包到一个INT8）
    K_packed = pack_int4(K_int4)

    return K_packed, K_scale
```

精度保持技术

INT4量化需要额外的精度保持技术：

- 1. **异常值处理**：识别并特殊处理离群值
- 2. **动态缩放**：根据激活分布动态调整缩放因子
- 3. **混合精度**：关键层使用INT8，其他层使用INT4

精度分析

量化方案	压缩比	典型精度损失
INT8	2x	<2%
INT4	4x	3-8%
INT4 + 异常值处理	3.5x	2-4%

7.4.1.3 FP8量化

E4M3和E5M2格式

FP8有两种主要格式： - **E4M3**：1位符号，4位指数，3位尾数，范围±448 - **E5M2**：1位符号，5位指数，2位尾数，范围±57344

E4M3: S EEEE MMM (动态范围小, 精度高)
E5M2: S EEEEE MM (动态范围大, 精度低)

动态缩放

FP8量化通常使用动态缩放因子:

$$s = \frac{\text{FP8_max}}{\max(|x|) + \epsilon}$$

$$x_{\text{fp8}} = x \times s$$

实现 (H100)

```
def quantize_fp8_e4m3(tensor):  
    """使用E4M3格式量化到FP8"""  
    # H100支持原生FP8张量核心  
    fp8_max = 448.0  
  
    # 计算缩放因子  
    amax = tensor.abs().max()  
    scale = fp8_max / amax  
  
    # 量化 (使用H100的FP8指令)  
    tensor_fp8 = torch._scaled_quantize(tensor, scale, "E4M3")  
  
    return tensor_fp8, scale
```

FP8 vs INT8

特性	INT8	FP8
动态范围	固定	可调
硬件支持	广泛	Hopper+
精度	高	中高
速度	快	更快 (张量核心)

7.4.1.4 精度保持技术

离群值感知量化

KV Cache中存在少量离群值 (outliers), 严重影响量化精度:

```
def outlier_aware_quantize(tensor, outlier_threshold=6.0):
    """
    离群值感知量化
    """
    # 识别离群值
    mean = tensor.mean()
    std = tensor.std()
    outliers = (tensor - mean).abs() > outlier_threshold * std

    # 离群值使用更高精度
    normal_mask = ~outliers

    # 量化非离群值
    normal_quantized = quantize(tensor[normal_mask])

    # 离群值保持FP16
    outliers_fp16 = tensor[outliers]

    return normal_quantized, outliers_fp16, normal_mask
```

混合精度策略

不同层使用不同精度： - 浅层（输入层）：FP16（保留更多信息） - 中层：INT8 - 深层：INT4（对精度影响较小）

7.4.2 模型量化

7.4.2.1 GPTQ

问题背景

训练后量化（Post-Training Quantization, PTQ）需要在不重新训练的情况下将预训练模型量化到低位宽。GPTQ是一种基于近似二阶信息的逐层量化方法。

核心思想

GPTQ基于OBS（Optimal Brain Surgeon）框架，通过海森矩阵的逆来指导量化：

$$\Delta w = -\frac{w_q - w}{[H^{-1}]_{ij}} \cdot H^{-1}_{:,i}$$

其中： - w 是原始权重 - w_q 是量化后的权重 - H 是海森矩阵

逐层量化流程

```
def gptq_quantize_layer(layer_weight, bits=4, group_size=128):
    """
    GPTQ逐层量化
    """
    W = layer_weight.data.clone()
    rows, cols = W.shape

    # 计算海森矩阵的逆（使用Cholesky分解）
    H = compute_hessian(W)
    H_inv = cholesky_inverse(H)

    # 逐列量化
    for i in range(cols):
        # 量化当前列
        w_col = W[:, i]
        scale = w_col.abs().max() / (2**(bits-1) - 1)
        w_q = torch.round(w_col / scale).clamp(-2**(bits-1), 2**(bits-1)-1)

        # 计算误差
        err = (w_col - w_q).unsqueeze(1)

        # 更新剩余列（补偿量化误差）
        if i < cols - 1:
            W[:, i+1:] -= err @ H_inv[i, i+1:].unsqueeze(0)

        W[:, i] = w_q

    return W
```

分组量化

为减少海森矩阵计算开销，GPTQ使用分组量化：

```
def gptq_quantize_grouped(model, bits=4, group_size=128):
    """GPTQ分组量化"""
    for name, layer in model.named_modules():
        if isinstance(layer, nn.Linear):
            weight = layer.weight.data

            # 按group_size分组
            for i in range(0, weight.shape[1], group_size):
                end = min(i + group_size, weight.shape[1])
                weight_group = weight[:, i:end]

                # 每组独立量化
                weight_q = gptq_quantize_layer(weight_group, bits, group_size)
                weight[:, i:end] = weight_q

            layer.weight.data = weight
```

精度分析
GPTQ在LLaMA模型上的表现：

模型	FP16	GPTQ-4bit	精度损失
LLaMA-7B	5.68	5.84	2.8%
LLaMA-13B	5.09	5.25	3.1%
LLaMA-30B	4.10	4.25	3.7%
LLaMA-65B	3.53	3.68	4.2%

7.4.2.2 AWQ

核心思想
AWQ（Activation-aware Weight Quantization）发现：**并非所有权重的bit都同等重要**。保护对激活敏感的权重可以显著提升量化精度。

激活感知缩放
AWQ通过分析激活分布来识别重要权重：

$$s = \left(\frac{|W|}{|X|^{\alpha}}\right)^{\beta}$$

其中 s 是激活值， α, β 是超参数。

保护重要权重

```
def awq_quantize_layer(weight, activation, bits=4, group_size=128):
    """
    AWQ量化
    """
    # 计算每个通道的激活幅度
    act_scale = activation.abs().mean(dim=0)

    # 计算权重重要性
    weight_scale = weight.abs().max(dim=0)[0]

    # 计算保护缩放因子
    importance = act_scale ** 0.5 * weight_scale

    # 分组量化, 使用重要性指导
    for i in range(0, weight.shape[1], group_size):
        end = min(i + group_size, weight.shape[1])
        group_importance = importance[i:end]

        # 重要组使用更高精度或特殊处理
        if group_importance.mean() > threshold:
            # 保护重要权重
            weight[:, i:end] = quantize_with_protection(
                weight[:, i:end], bits, group_importance
            )
        else:
            weight[:, i:end] = normal_quantize(weight[:, i:end], bits)

    return weight
```

与GPTQ对比

特性	GPTQ	AWQ
核心思想	海森矩阵补偿	激活感知保护
校准数据	需要	需要
量化时间	较长	较短
典型精度	高	更高
实现复杂度	中等	较低

7.4.2.3 SmoothQuant

核心思想

SmoothQuant解决激活量化的难题：激活比权重更难量化，因为激活有显著的离群值。

核心洞察：将量化难度从激活迁移到权重。

数学推导

对于线性层 $Y = XW$ ，引入平滑因子 s ：

$$Y = (X \cdot \text{diag}(s)^{-1}) \cdot (\text{diag}(s) \cdot W) = \tilde{X} \cdot \tilde{W}$$

选择 s 使得： $s_j = \frac{\max(|X_j|)^\alpha}{\max(|W_j|)^{1-\alpha}}$

其中 α 是迁移强度（通常0.5）。

实现

```
def smoothquant_fuse_ln_linear(ln_layer, linear_layer, alpha=0.5):
    """
    融合LayerNorm和Linear层，应用SmoothQuant
    """
    # 获取激活统计
    act_scale = get_activation_scale(ln_layer)

    # 获取权重统计
    weight_scale = linear_layer.weight.abs().max(dim=0)[0]

    # 计算平滑因子
    smooth_scale = (act_scale ** alpha) / (weight_scale ** (1 - alpha))
    smooth_scale = smooth_scale.clamp(min=1e-5)

    # 应用平滑
    # 1. 缩放LayerNorm权重
    ln_layer.weight.data /= smooth_scale
    ln_layer.bias.data /= smooth_scale

    # 2. 缩放Linear权重
    linear_layer.weight.data *= smooth_scale.unsqueeze(0)
    if linear_layer.bias is not None:
        linear_layer.bias.data *= smooth_scale

    return ln_layer, linear_layer
```

精度分析

SmoothQuant在W8A8（权重INT8，激活INT8）配置下的表现：

模型	FP16	SmoothQuant W8A8	精度损失
OPT-175B	8.34	8.46	1.4%
LLaMA-65B	3.53	3.58	1.4%
LLaMA-2-70B	3.32	3.38	1.8%

7.4.2.4 量化感知训练

问题背景

训练后量化虽然方便，但无法完全恢复量化带来的精度损失。量化感知训练（Quantization-Aware Training, QAT）在训练过程中模拟量化效果。

直通估计器

QAT的核心挑战：量化函数不可导。使用直通估计器（Straight-Through Estimator, STE）：

前向： $y = \text{round}(x / s) * s$

反向： $dy/dx = 1$ （忽略round的梯度）

伪代码

```
class QuantizedLinear(nn.Module):
    def __init__(self, in_features, out_features, bits=8):
        super().__init__()
        self.weight = nn.Parameter(torch.randn(out_features, in_features))
        self.bits = bits
        self.scale = None

    def forward(self, x):
        # 训练时模拟量化
        if self.training:
            # 计算缩放因子
            w_scale = self.weight.abs().max() / (2**(self.bits-1) - 1)

            # 伪量化（前向量化，反向直通）
            weight_quantized = fake_quantize(self.weight, w_scale, self.bits)

            return F.linear(x, weight_quantized, self.bias)
        else:
            # 推理时使用真实量化
            return F.linear(x, self.quantized_weight, self.bias)

def fake_quantize(tensor, scale, bits):
    """伪量化：前向量化，反向直通"""
    quantized = torch.round(tensor / scale).clamp(-2**(bits-1), 2**(bits-1))
    dequantized = quantized * scale

    # STE：前向使用dequantized，反向使用原始梯度
    return tensor + (dequantized - tensor).detach()
```

QAT流程

1. **预训练**：使用FP16训练模型到收敛
2. **插入量化节点**：在需要量化的层插入伪量化节点
3. **微调**：使用小学习率继续训练（通常1-10%的原始训练步数）
4. **转换**：将伪量化转换为真实量化

精度对比

方法	配置	LLaMA-7B PPL	备注
FP16	-	5.68	baseline
PTQ	INT8	5.75	1.2%损失
PTQ	INT4	6.20	9.2%损失
QAT	INT4	5.85	3.0%损失

7.5 MoE相关算法

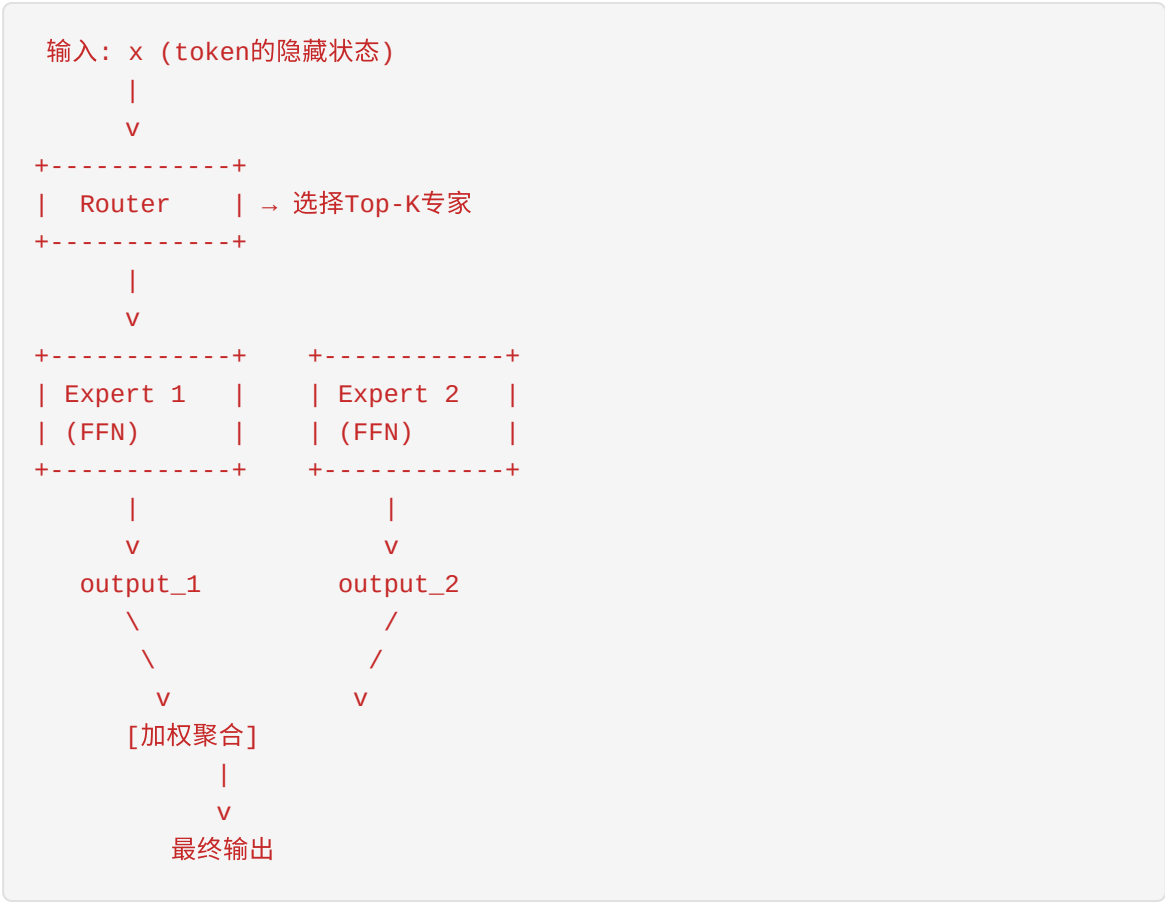
混合专家模型（Mixture of Experts, MoE）通过条件计算实现模型容量的扩展，而不显著增加推理成本。DeepSeek-V2/V3、Mixtral等模型都采用了MoE架构，使其成为当前大模型的重要技术路线。

7.5.1 Expert Routing

问题背景

在MoE模型中，每个输入token只激活部分专家（experts），需要一个路由机制来决定token应该由哪些专家处理。

MoE层结构



7.5.1.1 Top-K选择

基本路由

对于输入 x ，路由分数计算为：

$$g(x) = \text{Softmax}(W_g \cdot x)$$

其中 $W_g \in \mathbb{R}^{E \times d}$ 是路由权重矩阵， E 是专家数量。

Top-K选择：

$$\text{TopK}(g(x), k) = \{(i, g_i) \mid i \in \text{topk_indices}(g(x), k)\}$$

带噪声的Top-K

为防止路由崩溃（所有token都路由到少数专家），引入噪声：

$$g(x) = \text{Softmax}(W_g \cdot x + \epsilon \cdot \text{Normal}(0, 1))$$

其中 ϵ 是噪声强度，训练时通常非零，推理时为零。

路由实现

```

class TopKRouter(nn.Module):
    def __init__(self, hidden_dim, num_experts, top_k=2, noise_std=1.0):
        super().__init__()
        self.num_experts = num_experts
        self.top_k = top_k
        self.noise_std = noise_std

        # 路由权重
        self.gate = nn.Linear(hidden_dim, num_experts, bias=False)

    def forward(self, x, training=True):
        """
        x: [batch, seq_len, hidden_dim]
        返回: (expert_indices, expert_weights)
        """
        # 计算路由分数
        router_logits = self.gate(x) # [batch, seq_len, num_experts]

        # 添加噪声（仅训练时）
        if training and self.noise_std > 0:
            noise = torch.randn_like(router_logits) * self.noise_std
            router_logits = router_logits + noise

        # Softmax归一化
        router_probs = F.softmax(router_logits, dim=-1)

        # Top-K选择
        expert_weights, expert_indices = torch.topk(
            router_probs, self.top_k, dim=-1
        ) # [batch, seq_len, top_k]

        # 重新归一化权重
        expert_weights = expert_weights / expert_weights.sum(dim=-1, keepdim=True)

        return expert_indices, expert_weights, router_probs

```

7.5.1.2 Load Balancing

负载不均衡问题

如果路由机制不加约束，会出现： - 少数专家过载（处理大量token） - 多数专家闲置（处理很少token） - 训练不稳定，专家专业化不足

辅助损失函数

负载均衡损失 (Load Balancing Loss)

$$\mathcal{L}_{\text{load}} = \alpha \cdot E \cdot \sum_{i=1}^E f_i \cdot P_i$$

其中： - f_i 是专家 i 被选择的token比例 - P_i 是路由器分配给专家 i 的平均概率 - α 是损失权重（通常0.01）

```
def compute_load_balancing_loss(router_probs, expert_indices, num_experts)
    """
    计算负载均衡损失

    router_probs: [batch, seq_len, num_experts]
    expert_indices: [batch, seq_len, top_k]
    """
    # 每个专家被选择的token数
    expert_mask = F.one_hot(expert_indices, num_experts).sum(dim=-2) # [batch, seq_len, top_k]

    # 计算f_i (每个专家处理的token比例)
    tokens_per_expert = expert_mask.float().sum(dim=1) # [batch, num_experts]
    f = tokens_per_expert / tokens_per_expert.sum(dim=-1, keepdim=True)

    # 计算P_i (平均路由概率)
    P = router_probs.mean(dim=[0, 1])

    # 负载均衡损失
    load_loss = num_experts * (f * P).sum()

    return load_loss
```

重要性损失 (Importance Loss)

$$\mathcal{L}_{\text{importance}} = \beta \cdot \text{CV}(\text{importance})$$

其中importance是每个专家被分配的路由概率之和，CV是变异系数。

容量因子

限制每个专家能处理的token数量：

$$\text{capacity} = \frac{\text{num_tokens} \times k}{E} \times \text{capacity_factor}$$

超出容量的token被丢弃或路由到备用专家。

```

class CapacityLimitedRouter(TopKRouter):
    def __init__(self, hidden_dim, num_experts, top_k=2, capacity_factor=1):
        super().__init__(hidden_dim, num_experts, top_k)
        self.capacity_factor = capacity_factor

    def forward(self, x):
        batch, seq_len, _ = x.shape
        num_tokens = batch * seq_len

        # 计算容量
        capacity = int(num_tokens * self.top_k / self.num_experts * self.capacity_factor)

        expert_indices, expert_weights, router_probs = super().forward(x)

        # 应用容量限制
        for i in range(self.num_experts):
            expert_mask = (expert_indices == i).any(dim=-1)
            expert_count = expert_mask.sum()

            if expert_count > capacity:
                # 超出容量，需要丢弃部分token
                excess = expert_count - capacity
                # 按路由概率排序，丢弃低概率的
                expert_probs = router_probs[..., i]
                _, drop_indices = torch.topk(expert_probs[expert_mask], excess)
                # 标记这些token为需要重路由
                ...

        return expert_indices, expert_weights

```

7.5.1.3 辅助损失详解

总损失函数

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{LM}} + \lambda_1 \mathcal{L}_{\text{load}} + \lambda_2 \mathcal{L}_{\text{router_zloss}}$$

其中： - $\mathcal{L}_{\text{LM}}$ ：语言模型损失 - $\mathcal{L}_{\text{load}}$ ：负载均衡损失 - $\mathcal{L}_{\text{router_zloss}}$ ：路由器z-loss（防止logits过大）

Router Z-Loss

$$\mathcal{L}_{\text{router_zloss}} = \frac{1}{B} \sum_{i=1}^B \left(\log \sum_{j=1}^E e^{z_{ij}} \right)^2$$

其中 z_{ij} 是路由器logits，这个损失鼓励logits保持较小值，提高数值稳定性。

```
def compute_router_z_loss(router_logits):  
    """  
    计算router z-loss  
    router_logits: [batch, seq_len, num_experts]  
    """  
    # log(sum(exp(z)))  
    log_sum_exp = torch.logsumexp(router_logits, dim=-1)  
  
    # 平方损失  
    z_loss = (log_sum_exp ** 2).mean()  
  
    return z_loss
```

7.5.2 Expert Parallelism

问题背景

MoE模型的专家数量通常很大（如64、128个），单个GPU无法容纳所有专家。Expert Parallelism（EP）将专家分布到多个GPU上。

7.5.2.1 EP并行策略

专家分片

将 E 个专家分布到 N 个GPU上：

$$\text{experts_per_gpu} = \frac{E}{N}$$

每个GPU负责 $\frac{E}{N}$ 个专家的计算。

数据并行 + 专家并行

3个GPU，6个专家（每个GPU 2个专家）

GPU 0: [Expert 0, Expert 1] + DP副本

GPU 1: [Expert 2, Expert 3] + DP副本

GPU 2: [Expert 4, Expert 5] + DP副本

输入数据：

Batch 0 → GPU 0（需要Expert 0,1,4）

Batch 1 → GPU 1（需要Expert 2,3,5）

Batch 2 → GPU 2（需要Expert 0,2,5）

7.5.2.2 All-to-All通信

通信模式

Expert Parallelism的核心是All-to-All通信：

1. **Dispatch阶段**：将token发送到对应的专家所在GPU
2. **Compute阶段**：各GPU并行处理分配给它的token
3. **Combine阶段**：收集结果并返回原始位置

```

def expert_parallel_forward(x, expert_indices, expert_weights, experts, group):
    """
    专家并行前向传播

    x: [local_batch, seq_len, hidden_dim]
    expert_indices: [local_batch, seq_len, top_k]
    expert_weights: [local_batch, seq_len, top_k]
    experts: 当前GPU上的专家列表
    group: 专家并行通信组
    """

    batch, seq_len, _ = x.shape
    top_k = expert_indices.shape[-1]
    num_local_experts = len(experts)
    world_size = dist.get_world_size(group)

    # === Dispatch阶段 ===
    # 准备发送缓冲区
    send_buffer = [[] for _ in range(world_size)]

    for b in range(batch):
        for s in range(seq_len):
            for k in range(top_k):
                expert_id = expert_indices[b, s, k].item()
                target_rank = expert_id // num_local_experts
                send_buffer[target_rank].append({
                    'token': x[b, s],
                    'expert_id': expert_id,
                    'weight': expert_weights[b, s, k],
                    'position': (b, s, k)
                })

    # All-to-All通信
    recv_buffer = all_to_all(send_buffer, group)

    # === Compute阶段 ===
    outputs = {}
    for item in recv_buffer:
        expert_id = item['expert_id']
        local_expert_id = expert_id % num_local_experts
        expert = experts[local_expert_id]

        token = item['token']
        output = expert(token)
        outputs[item['position']] = output * item['weight']

    # === Combine阶段 ===
    # 准备发送回原始位置的缓冲区
    send_back_buffer = [[] for _ in range(world_size)]
    for pos, output in outputs.items():
        source_rank = pos[0] // (batch // world_size)

```



```
        send_back_buffer[source_rank].append({
            'output': output,
            'position': pos
        })

# All-to-All通信返回结果
recv_back_buffer = all_to_all(send_back_buffer, group)

# 聚合结果
final_output = torch.zeros_like(x)
for item in recv_back_buffer:
    b, s, k = item['position']
    final_output[b, s] += item['output']

return final_output
```

All-to-All优化

1. **通信与计算重叠**：在等待通信时执行其他计算
2. **分组All-to-All**：将大All-to-All拆分为多个小All-to-All
3. **FP16/BF16通信**：降低通信带宽需求

```
def optimized_all_to_all(send_buffers, group):
    """优化的All-to-All通信"""
    world_size = dist.get_world_size(group)

    # 将数据打包成连续张量
    send_tensors = []
    send_sizes = []
    for buf in send_buffers:
        if len(buf) > 0:
            tensor = torch.stack([item['token'] for item in buf])
        else:
            tensor = torch.empty(0)
        send_tensors.append(tensor)
        send_sizes.append(len(buf))

    # 交换大小信息
    recv_sizes = [torch.tensor(0) for _ in range(world_size)]
    dist.all_to_all(recv_sizes, [torch.tensor(s) for s in send_sizes], group=group)

    # 分配接收缓冲区
    recv_tensors = [
        torch.empty(recv_sizes[i], send_tensors[0].shape[1],
                    dtype=send_tensors[0].dtype, device=send_tensors[0].device)
        for i in range(world_size)
    ]

    # 执行All-to-All
    dist.all_to_all(recv_tensors, send_tensors, group=group)

    return recv_tensors
```

7.5.2.3 DeepSeek的EP实现

DeepSeek-V2/V3架构

DeepSeek采用了创新的MoE架构： - **共享专家**：所有token都经过的共享专家（2个） - **路由专家**：通过路由选择的专家（64个，激活6个） - **细粒度专家**：每个专家更小但数量更多

负载均衡创新

DeepSeek引入了设备级负载均衡：

$$\mathcal{L}_{\text{device}} = \alpha \cdot N_{\text{devices}} \cdot \sum_{d=1}^{N_{\text{devices}}} f_d \cdot P_d$$

其中 f_d 是设备 d 上的token比例， P_d 是路由到设备 d 的概率。

通信优化

DeepSeek的EP优化策略：

1. **双批次重叠：**
 2. 批次1的通信与批次2的计算重叠
 3. 批次2的通信与批次1的计算重叠
4. **细粒度调度：**
 5. 将序列分成多个micro-batch
 6. 独立调度每个micro-batch

```
class DeepSeekMoELayer(nn.Module):
    def __init__(self, config, ep_group):
        super().__init__()
        self.num_shared_experts = config.num_shared_experts
        self.num_routed_experts = config.num_routed_experts
        self.top_k = config.top_k
        self.ep_group = ep_group

        # 共享专家（所有GPU都有）
        self.shared_experts = nn.ModuleList([
            Expert(config.hidden_size, config.intermediate_size)
            for _ in range(self.num_shared_experts)
        ])

        # 路由专家（EP分片）
        self.routed_experts = self.create_routed_experts()

        # 路由器
        self.router = DeepSeekRouter(
            config.hidden_size,
            self.num_routed_experts,
            self.top_k
        )

    def forward(self, x):
        # 共享专家（本地计算）
        shared_output = sum(expert(x) for expert in self.shared_experts)

        # 路由专家（EP并行）
        expert_indices, expert_weights = self.router(x)
        routed_output = expert_parallel_forward(
            x, expert_indices, expert_weights,
            self.routed_experts, self.ep_group
        )

        return shared_output + routed_output
```

7.5.2.4 EP与其他并行的组合

3D并行 (EP + TP + DP)

全局并行策略：

- Data Parallelism (DP)：复制完整模型
- Tensor Parallelism (TP)：层内分片
- Expert Parallelism (EP)：专家分片

示例配置 (8 GPU)：

- DP = 2 (2个副本)
- TP = 2 (每层分2片)
- EP = 2 (专家分2组)

每个GPU组：

GPU 0-1：TP组1, EP组1, DP副本1

GPU 2-3：TP组2, EP组1, DP副本1

GPU 4-5：TP组1, EP组2, DP副本2

GPU 6-7：TP组2, EP组2, DP副本2

通信复杂度分析

并行类型	通信操作	通信量	频率
DP	All-Reduce	2x模型大小	每个梯度步
TP	All-Reduce	2x激活大小	每层
EP	All-to-All	2x token数	每层MoE

最优并行配置

选择并行策略的考虑因素： 1. **模型大小**：大模型需要更多TP 2. **序列长度**：长序列需要更多EP 3. **GPU数量**：决定并行度上限 4. **网络带宽**：影响通信效率

7.6 总结与展望

核心算法对比

算法类别	代表算法	核心优化	典型加速比
Attention优化	FlashAttention	IO-aware计算	2-4x
Decode优化	FlashDecoding	序列并行	3-8x
内存管理	PagedAttention	虚拟内存	2-3x吞吐
推理加速	Speculative Decoding	投机执行	1.5-2.5x
调度优化	Continuous Batching	动态批处理	2-3x吞吐
模型压缩	GPTQ/AWQ	训练后量化	4x压缩
MoE优化	Expert Parallelism	专家并行	线性扩展

技术趋势

- 1. **硬件-软件协同优化**：针对特定硬件（如H100、TPU）的专用优化
- 2. **自适应算法**：根据输入和工作负载动态选择最优策略
- 3. **多模态统一**：将LLM优化技术扩展到多模态模型
- 4. **边缘部署**：针对移动设备和边缘场景的轻量化算法

实践建议

对于Mooncake实习生： 1. **深入理解原理**：不仅要知道怎么用，更要理解为什么 2. **动手实验**：使用vLLM、TensorRT-LLM等框架进行实验 3. **性能分析**：学会使用Nsight、PyTorch Profiler等工具 4. **关注前沿**：跟踪最新的论文和开源实现

本章内容由Mooncake团队技术文档组编写，旨在帮助新加入的实习生快速了解AI Infra核心算法。如有疑问，请联系团队mentor。

AI Infra技术图表集

本文档包含AI Infra相关的技术图表，使用Mermaid语法绘制。

1. AI Infra整体架构图

架构说明

AI Infra整体架构包含四个核心层次：

- **基础设施层**：GPU/TPU计算节点、高速网络（InfiniBand/RoCE）、分布式存储
- **平台层**：Kubernetes编排、资源调度器、监控运维
- **框架层**：PyTorch、TensorFlow、vLLM、DeepSpeed等训练和推理框架
- **应用层**：大模型服务、AI Agent、多模态应用

flowchart TB

subgraph AppLayer["🚀 应用层"]

A1[大模型对话服务]

A2[AI Agent平台]

A3[代码生成助手]

A4[多模态应用]

end

subgraph FrameworkLayer["⚙️ 框架层"]

F1[PyTorch / TensorFlow]

F2[vLLM / TensorRT-LLM]

F3[DeepSpeed / Megatron]

F4[Ray / Triton]

end

subgraph PlatformLayer["🔧 平台层"]

P1[Kubernetes编排]

P2[资源调度系统]

P3[模型管理系统]

P4[监控与可观测性]

end

subgraph InfraLayer["🏠 基础设施层"]

subgraph Compute["计算资源"]

C1[GPU Cluster
A100/H100/H800]

C2[CPU Cluster]

end

subgraph Network["网络基础设施"]

N1[InfiniBand
400Gbps]

N2[RoCE v2]

N3[RDMA网络]

end

subgraph Storage["存储系统"]

S1[高性能并行存储
Lustre/GPFS]

S2[对象存储
S3/OSS]

S3[KV Cache存储
Redis/SSD]

end

end

AppLayer --> FrameworkLayer

FrameworkLayer --> PlatformLayer

PlatformLayer --> InfraLayer

style AppLayer fill:#e1f5fe

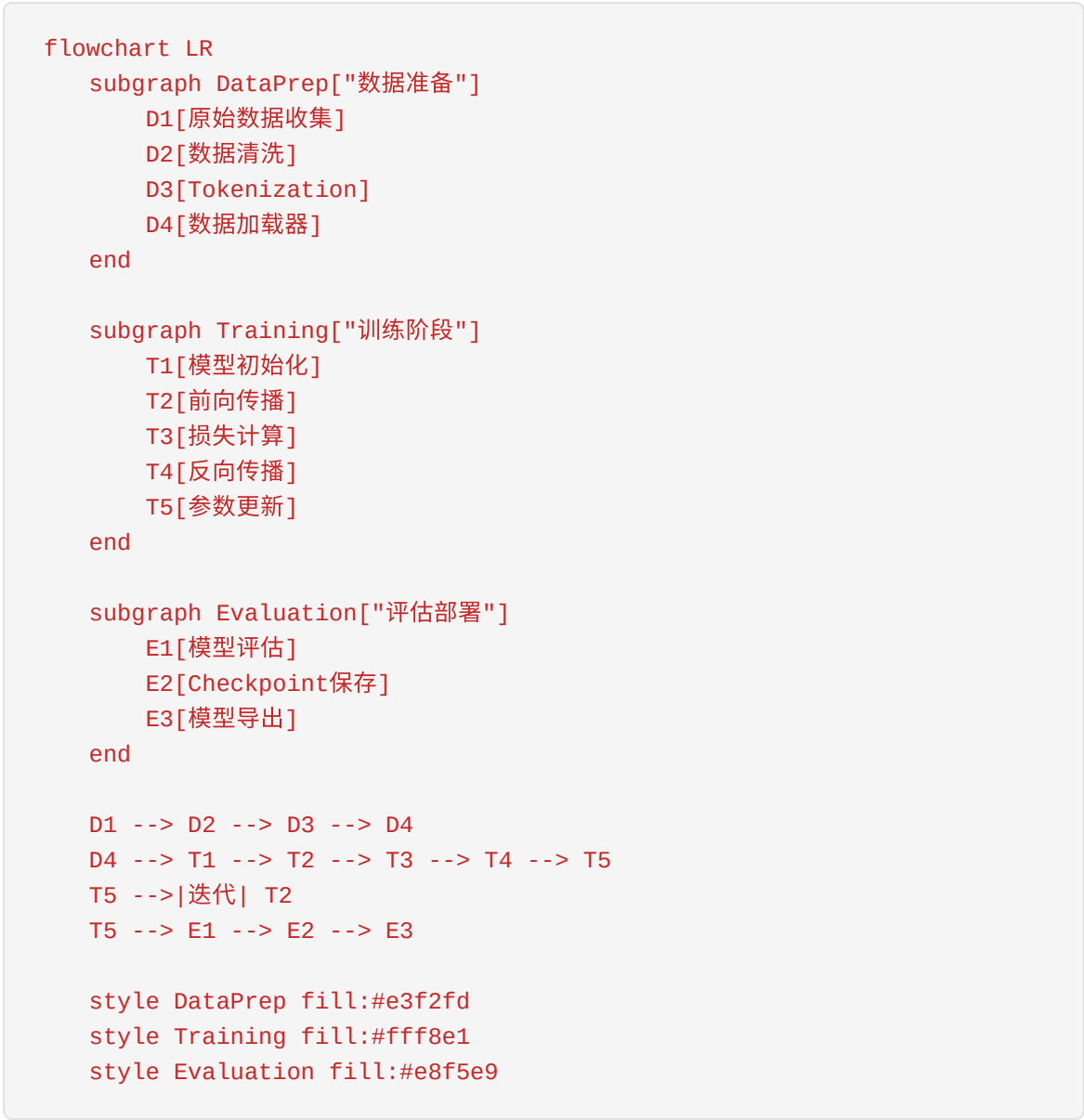
style FrameworkLayer fill:#fff3e0

style PlatformLayer fill:#f3e5f5

style InfraLayer fill:#e8f5e9

2. LLM训练和推理流程图

2.1 训练流程



2.2 推理流程


```
graph TD
    subgraph Request ["请求处理"]
        R1[用户请求]
        R2[请求队列]
        R3[Batch组装]
    end

    subgraph Prefill ["Prefill阶段"]
        P1[输入Token编码]
        P2[全量Attention计算]
        P3[生成KV Cache]
    end

    subgraph Decode ["Decode阶段"]
        D1[逐Token生成]
        D2[增量Attention]
        D3[采样策略]
        D4[终止条件检查]
    end

    subgraph Response ["响应输出"]
        O1[Token解码]
        O2[流式返回]
    end

    R1 --> R2 --> R3
    R3 --> P1 --> P2 --> P3
    P3 --> D1 --> D2 --> D3 --> D4
    D4 -->|继续生成| D1
    D4 -->|生成完成| O1 --> O2

    style Request fill:#fce4ec
    style Prefill fill:#e8eaf6
    style Decode fill:#fff3e0
    style Response fill:#e0f2f1
```

3. Transformer架构图

3.1 整体架构

```
flowchart TB
    subgraph Input["输入层"]
        I1[Input Embedding]
        I2[Positional Encoding]
    end

    subgraph Encoder["Encoder堆叠 ×N"]
        E1[Multi-Head Attention]
        E2[Add & Norm]
        E3[Feed Forward]
        E4[Add & Norm]
    end

    subgraph Decoder["Decoder堆叠 ×N"]
        D1[Masked Multi-Head Attention]
        D2[Add & Norm]
        D3[Cross Attention]
        D4[Add & Norm]
        D5[Feed Forward]
        D6[Add & Norm]
    end

    subgraph Output["输出层"]
        O1[Linear]
        O2[Softmax]
        O3[Output Probabilities]
    end

    I1 --> I2 --> E1
    E1 --> E2 --> E3 --> E4
    E4 -->|...N层| E1
    E4 --> D3

    I2 --> D1
    D1 --> D2 --> D3 --> D4 --> D5 --> D6
    D6 -->|...N层| D1
    D6 --> O1 --> O2 --> O3

    style Input fill:#e3f2fd
    style Encoder fill:#e8f5e9
    style Decoder fill:#fff8e1
    style Output fill:#fce4ec
```

3.2 Multi-Head Attention详细结构

```
graph LR
    subgraph LR
        Input["输入"]
        IN["Q/K/V输入"]
    end

    subgraph Linear["线性变换"]
        L1["W_Q"]
        L2["W_K"]
        L3["W_V"]
    end

    subgraph Split["分割多头"]
        S1["Head 1"]
        S2["Head 2"]
        S3["Head h"]
    end

    subgraph Attention["Scaled Dot-Product Attention"]
        A1["Q × K^T"]
        A2["÷ √d_k"]
        A3["Softmax"]
        A4["× V"]
    end

    subgraph Concat["拼接与输出"]
        C1["Concat"]
        C2["W_O"]
        C3["输出"]
    end

    IN --> L1 & L2 & L3
    L1 & L2 & L3 --> S1 & S2 & S3
    S1 & S2 & S3 --> A1 --> A2 --> A3 --> A4
    A4 --> C1 --> C2 --> C3

    style Input fill:#e3f2fd
    style Linear fill:#fff8e1
    style Split fill:#f3e5f5
    style Attention fill:#e8f5e9
    style Concat fill:#fce4ec
```

3.3 FFN前馈网络

```
flowchart LR
    Input[输入x] --> Linear1[Linear<br/>d_model → 4×d_model]
    Linear1 --> Activation[激活函数<br/>GELU/SiLU]
    Activation --> Linear2[Linear<br/>4×d_model → d_model]
    Linear2 --> Output[输出]

    style Input fill:#e3f2fd
    style Linear1 fill:#fff8e1
    style Activation fill:#e8f5e9
    style Linear2 fill:#fff8e1
    style Output fill:#fce4ec
```

4. Mooncake架构图

4.1 整体架构

```
flowchart TB
    subgraph Client["客户端"]
        C1[用户请求]
    end

    subgraph GlobalScheduler["全局调度器"]
        GS1[请求路由]
        GS2[负载均衡]
        GS3[集群选择<br/>Prefill/Decode]
    end

    subgraph PrefillCluster["Prefill集群"]
        P1[Prefill Instance 1]
        P2[Prefill Instance 2]
        P3[Prefill Instance N]
    end

    subgraph TransferEngine["传输引擎"]
        TE1[RDMA传输]
        TE2[KV Cache序列化]
        TE3[异步传输]
    end

    subgraph DecodeCluster["Decode集群"]
        D1[Decode Instance 1]
        D2[Decode Instance 2]
        D3[Decode Instance N]
    end

    subgraph KVCachePool["KV Cache池"]
        K1[DRAM缓存]
        K2[SSD缓存]
        K3[远程存储]
    end

    C1 --> GS1
    GS1 --> GS2 --> GS3
    GS3 -->|分配Prefill| P1 & P2 & P3
    P1 & P2 & P3 -->|生成KV Cache| TE1
    TE1 --> TE2 --> TE3
    TE3 -->|传输KV Cache| D1 & D2 & D3
    D1 & D2 & D3 <-->|读取/写入| K1 & K2 & K3
    D1 & D2 & D3 -->|返回结果| C1

    style Client fill:#e3f2fd
    style GlobalScheduler fill:#fff8e1
    style PrefillCluster fill:#e8f5e9
    style TransferEngine fill:#f3e5f5
```

```
style DecodeCluster fill:#fce4ec  
style KVCachePool fill:#e0f2f1
```

4.2 核心组件说明

```
flowchart TB
    subgraph Components["Mooncake核心组件"]
        direction TB

        subgraph Conductor["Conductor<br/>全局调度"]
            C01[请求分析]
            C02[集群状态收集]
            C03[调度决策]
        end

        subgraph PrefillNode["Prefill节点"]
            PR1[Attention计算]
            PR2[KV Cache生成]
            PR3[本地缓存]
        end

        subgraph TransferEngineDetail["Transfer Engine<br/>传输引擎"]
            TE[RDMA/NCCL]
            TS[序列化/反序列化]
            TP[流水线传输]
        end

        subgraph DecodeNode["Decode节点"]
            DE1[增量Attention]
            DE2[Token生成]
            DE3[KV Cache管理]
        end

        subgraph CachePool["KV Cache Pool"]
            CP1[分层存储]
            CP2[缓存策略]
            CP3[内存池管理]
        end
    end

    C01 --> C02 --> C03
    PR1 --> PR2 --> PR3
    TE --> TS --> TP
    DE1 --> DE2 --> DE3
    CP1 --> CP2 --> CP3

    style Conductor fill:#fff8e1
    style PrefillNode fill:#e8f5e9
    style TransferEngineDetail fill:#f3e5f5
    style DecodeNode fill:#fce4ec
    style CachePool fill:#e0f2f1
```

5. PD分离架构对比图

5.1 传统同构架构

```
flowchart TB
    subgraph Traditional["传统同构架构"]
        direction TB

        subgraph RequestQueue["请求队列"]
            R1[Req1]
            R2[Req2]
            R3[Req3]
        end

        subgraph HomogeneousNodes["同构GPU节点"]
            direction LR
            N1["GPU Node 1<br/>Prefill + Decode"]
            N2["GPU Node 2<br/>Prefill + Decode"]
            N3["GPU Node 3<br/>Prefill + Decode"]
        end

        R1 & R2 & R3 --> N1 & N2 & N3
    end

    subgraph Problems["问题"]
        P1[✗ 资源争抢]
        P2[✗ Prefill阻塞Decode]
        P3[✗ 无法独立扩展]
        P4[✗ 资源利用率低]
    end

    Traditional --> Problems

    style Traditional fill:#ffebee
    style Problems fill:#ffcdd2
```

5.2 PD分离架构


```
graph TD
    subgraph PDDissociation ["PD分离架构"]
        direction TB
        subgraph GlobalScheduler ["全局调度器"]
            GS["智能路由决策"]
        end
        subgraph PrefillTier ["Prefill层<br/>高吞吐计算"]
            direction LR
            P1["Prefill Node 1"]
            P2["Prefill Node 2"]
            P3["Prefill Node N"]
        end
        subgraph Transfer ["KV Cache传输"]
            T["高速RDMA传输"]
        end
        subgraph DecodeTier ["Decode层<br/>低延迟响应"]
            direction LR
            D1["Decode Node 1"]
            D2["Decode Node 2"]
            D3["Decode Node N"]
        end
    end

    GS --> P1 & P2 & P3
    P1 & P2 & P3 --> T
    T --> D1 & D2 & D3
    PDDissociation --> Benefits

    subgraph Benefits ["优势"]
        B1["✔ 资源隔离"]
        B2["✔ 独立扩缩容"]
        B3["✔ 专业化优化"]
        B4["✔ 高资源利用率"]
    end
```

style PDDissociation fill:#e8f5e9
style PrefillTier fill:#e3f2fd
style DecodeTier fill:#fff8e1
style Transfer fill:#f3e5f5
style Benefits fill:#c8e6c9

5.3 数据流对比

```
sequenceDiagram
    participant User as 用户
    participant Scheduler as 调度器
    participant Prefill as Prefill节点
    participant Transfer as 传输层
    participant Decode as Decode节点

    rect rgb(255, 235, 238)
        Note over User,Decode: 传统同构架构
        User->>Scheduler: 发送请求
        Scheduler->>Prefill: 分配节点
        Prefill->>Prefill: Prefill计算
        Prefill->>Prefill: Decode计算
        Prefill-->>User: 返回结果
    end

    rect rgb(232, 245, 233)
        Note over User,Decode: PD分离架构
        User->>Scheduler: 发送请求
        Scheduler->>Prefill: 路由到Prefill集群
        Prefill->>Prefill: 全量Attention计算
        Prefill->>Transfer: 输出KV Cache
        Transfer->>Decode: 传输KV Cache
        Decode->>Decode: 增量Attention计算
        loop Token生成
            Decode->>Decode: 生成下一个Token
        end
        Decode-->>User: 流式返回结果
    end
```

6. KV Cache管理流程图

6.1 生命周期管理

```
flowchart TB
    subgraph Lifecycle["KV Cache生命周期"]
        direction TB

        Start([开始]) --> Allocate[分配内存块]

        Allocate --> Use[使用阶段<br/>存储K/V张量]

        Use --> Decision{是否需要<br/>传输?}

        Decision -->|是| Transfer[传输阶段]
        Decision -->|否| Reuse[复用阶段]

        Transfer --> Remote[远程节点使用]

        Reuse --> Local[本地Decode使用]

        Remote --> ReleaseDecision{是否释放?}
        Local --> ReleaseDecision

        ReleaseDecision -->|是| Release[释放内存]
        ReleaseDecision -->|否| Persist[持久化存储]

        Release --> End([结束])
        Persist --> CachePool[缓存池]
        CachePool --> Reuse
    end

    end

    subgraph Operations["操作类型"]
        01[Allocate - 分配]
        02[Write - 写入]
        03[Read - 读取]
        04[Transfer - 传输]
        05[Evict - 驱逐]
        06[Free - 释放]
    end

    end

    style Start fill:#e8f5e9
    style End fill:#ffcdd2
    style Allocate fill:#e3f2fd
    style Use fill:#fff8e1
    style Transfer fill:#f3e5f5
    style Release fill:#ffebee
```

6.2 分层存储架构

```
flowchart TB
    subgraph StorageHierarchy["KV Cache分层存储"]
        direction TB

        subgraph Hot["热数据层<br/>GPU HBM"]
            H1[当前Batch KV Cache]
            H2[高频访问Cache]
        end

        subgraph Warm["温数据层<br/>DRAM"]
            W1[等待传输Cache]
            W2[近期使用Cache]
        end

        subgraph Cold["冷数据层<br/>SSD/本地存储"]
            C1[低频访问Cache]
            C2[预加载Cache]
        end

        subgraph Remote["远程层<br/>网络存储"]
            R1[历史对话Cache]
            R2[备份Cache]
        end
    end

    end

    subgraph Management["管理策略"]
        M1[LRU淘汰]
        M2[预加载策略]
        M3[压缩编码]
        M4[异步迁移]
    end

    end

    Hot <-->|命中| Warm
    Warm <-->|未命中| Cold
    Cold <-->|加载| Remote

    Management -. -> Hot & Warm & Cold & Remote

    style Hot fill:#ffcdd2
    style Warm fill:#fff9c4
    style Cold fill:#c8e6c9
    style Remote fill:#e1f5fe
```

7. PagedAttention原理图

7.1 虚拟内存映射

```
flowchart TB
    subgraph Logical["逻辑视图"]
        direction TB
        L1["Sequence 1<br/>Tokens: 1-100"]
        L2["Sequence 2<br/>Tokens: 1-50"]
        L3["Sequence 3<br/>Tokens: 1-200"]
    end

    subgraph BlockTable["Block Table<br/>映射表"]
        BT1["Seq1: [0, 5, 12, 23]"]
        BT2["Seq2: [1, 8, 15]"]
        BT3["Seq3: [2, 7, 11, 19, 25, 31]"]
    end

    subgraph Physical["物理存储<br/>固定大小Blocks"]
        direction LR
        B0["Block 0"]
        B1["Block 1"]
        B2["Block 2"]
        B5["Block 5"]
        B7["Block 7"]
        B8["Block 8"]
        B11["Block 11"]
        B12["Block 12"]
        B15["Block 15"]
        B19["Block 19"]
        B23["Block 23"]
        B25["Block 25"]
        B31["Block 31"]

        B0 -. -> |空闲| F1[...]
    end

    L1 --> BT1
    L2 --> BT2
    L3 --> BT3

    BT1 -. -> B0 & B5 & B12 & B23
    BT2 -. -> B1 & B8 & B15
    BT3 -. -> B2 & B7 & B11 & B19 & B25 & B31

    style Logical fill:#e3f2fd
    style BlockTable fill:#fff8e1
    style Physical fill:#e8f5e9
```

7.2 Block分配与共享

```
flowchart LR
    subgraph Allocation["Block分配策略"]
        A1[请求到达]
        A2[计算所需Blocks]
        A3[分配物理Blocks]
        A4[更新Block Table]
    end

    subgraph Sharing["Copy-on-Write共享"]
        S1[父序列Block]
        S2[引用计数+1]
        S3[子序列指向相同Block]
        S4[修改时复制]
    end

    subgraph InternalFragmentation["内部碎片处理"]
        I1[非完整Block]
        I2[预留空间]
        I3[追加写入]
    end

    A1 --> A2 --> A3 --> A4
    S1 --> S2 --> S3 --> S4
    I1 --> I2 --> I3

    style Allocation fill:#e3f2fd
    style Sharing fill:#e8f5e9
    style InternalFragmentation fill:#fff8e1
```

7.3 内存布局对比

```
flowchart TB
    subgraph TraditionalAlloc["传统连续分配"]
        direction LR
        T1["Seq1<br/>"]
        T2["Seq2<br/>"]
        T3["Seq3<br/>"]

        Note over T1,T3
        外部碎片: 
        内存利用率低
        End Note
    end

    subgraph PagedAlloc["PagedAttention分配"]
        direction LR
        P1["Block Pool"]
        P2["[B0][B1][B2][B3][B4][B5]..."]

        P3["Seq1: B0→B2→B5"]
        P4["Seq2: B1→B3"]
        P5["Seq3: B4→B6→B7→B8"]

        Note over P1,P5
        无外部碎片
        支持动态扩展
        支持Copy-on-Write
        End Note
    end

    style TraditionalAlloc fill:#ffebee
    style PagedAlloc fill:#e8f5e9
```

8. 分布式训练并行策略对比图

8.1 并行策略总览

```
flowchart TB
    subgraph ParallelStrategies["分布式训练并行策略"]
        direction TB

        subgraph DP["数据并行 DP"]
            DP1["每个GPU保存完整模型"]
            DP2["不同数据分片"]
            DP3["梯度AllReduce同步"]
        end

        subgraph TP["张量并行 TP"]
            TP1["模型层内切分"]
            TP2["Attention/FFN分割"]
            TP3["激活值AllReduce"]
        end

        subgraph PP["流水线并行 PP"]
            PP1["模型层间切分"]
            PP2["不同Stage"]
            PP3["激活值P2P传输"]
        end

        subgraph SP["序列并行 SP"]
            SP1["序列维度切分"]
            SP2["长序列处理"]
            SP3["Ring Attention"]
        end

        subgraph EP["专家并行 EP<br/>MoE专用"]
            EP1["不同Expert分片"]
            EP2["All-to-All通信"]
            EP3["门控路由"]
        end
    end

    DP --> TP --> PP --> SP --> EP

    style DP fill:#e3f2fd
    style TP fill:#fff8e1
    style PP fill:#e8f5e9
    style SP fill:#f3e5f5
    style EP fill:#fce4ec
```

8.2 数据并行(DP)详解


```
flowchart TB
    subgraph DataParallel["数据并行 Data Parallel"]
        direction TB

        subgraph InputData["输入数据"]
            I1[Batch Size = N]
        end

        subgraph Split["数据切分"]
            S1[GPU0: Batch 0-N/4]
            S2[GPU1: Batch N/4-N/2]
            S3[GPU2: Batch N/2-3N/4]
            S4[GPU3: Batch 3N/4-N]
        end

        subgraph Replicas["模型副本"]
            R1[GPU0: 完整模型]
            R2[GPU1: 完整模型]
            R3[GPU2: 完整模型]
            R4[GPU3: 完整模型]
        end

        subgraph Forward["前向传播"]
            F1[独立计算Loss]
        end

        subgraph Backward["反向传播"]
            B1[计算本地梯度]
        end

        subgraph Sync["梯度同步"]
            G1[AllReduce操作]
            G2[梯度平均]
        end

        subgraph Update["参数更新"]
            U1[同步更新参数]
        end
    end

    I1 --> S1 & S2 & S3 & S4
    S1 --> R1 --> F1 --> B1
    S2 --> R2 --> F1
    S3 --> R3 --> F1
    S4 --> R4 --> F1
    B1 --> G1 --> G2 --> U1

    style InputData fill:#e3f2fd
    style Split fill:#fff8e1
```

```
style Replicas fill:#e8f5e9
style Sync fill:#f3e5f5
```

8.3 张量并行(TP)详解

```
flowchart TB
    subgraph TensorParallel["张量并行 Tensor Parallel"]
        direction TB

        subgraph Input["输入"]
            IN["Hidden States<br/>[B, S, H]"]
        end

        subgraph MHA["Multi-Head Attention切分"]
            M1["Q/K/V线性层切分"]
            M2["Attention计算"]
            M3["输出AllGather"]
        end

        subgraph FFN["FFN切分"]
            F1["第一个Linear切分"]
            F2["GELU激活"]
            F3["第二个Linear切分"]
            F4["输出AllReduce"]
        end

        subgraph Output["输出"]
            OUT["完整输出<br/>[B, S, H]"]
        end

        IN --> M1 --> M2 --> M3
        M3 --> F1 --> F2 --> F3 --> F4
        F4 --> OUT

        style Input fill:#e3f2fd
        style MHA fill:#fff8e1
        style FFN fill:#e8f5e9
        style Output fill:#fce4ec
    end

    subgraph Example["切分示例 (TP=2)"]
        E1["GPU0: W_Q[:, :H/2]"]
        E2["GPU1: W_Q[:, H/2:]"]
        E3["计算后AllGather合并"]
    end
```

8.4 流水线并行(PP)详解

```
flowchart LR
    subgraph PipelineParallel["流水线并行 Pipeline Parallel"]
        direction LR

        subgraph Stage0["Stage 0 (GPU0)"]
            S01[Embedding]
            S02[Layer 0-3]
        end

        subgraph Stage1["Stage 1 (GPU1)"]
            S11[Layer 4-7]
        end

        subgraph Stage2["Stage 2 (GPU2)"]
            S21[Layer 8-11]
        end

        subgraph Stage3["Stage 3 (GPU3)"]
            S31[Layer 12-15]
            S32[Output]
        end
    end

    subgraph Scheduling["调度策略"]
        G0[GPipe<br/>微批次填充]
        G1[PipeDream<br/>1F1B调度]
        G2[Interleaved<br/>交错调度]
    end

    S01 --> S02 --> S11 --> S21 --> S31 --> S32

    style Stage0 fill:#e3f2fd
    style Stage1 fill:#fff8e1
    style Stage2 fill:#e8f5e9
    style Stage3 fill:#fce4ec
```

8.5 并行策略组合

```
flowchart TB
    subgraph 3DParallelism["3D并行组合"]
        direction TB

        subgraph DataParallelDim["数据并行 (DP=2)"]
            DP1[Replica 0]
            DP2[Replica 1]
        end

        subgraph TensorParallelDim["张量并行 (TP=4)"]
            TP1[TP Rank 0]
            TP2[TP Rank 1]
            TP3[TP Rank 2]
            TP4[TP Rank 3]
        end

        subgraph PipelineParallelDim["流水线并行 (PP=4)"]
            PP1[Stage 0]
            PP2[Stage 1]
            PP3[Stage 2]
            PP4[Stage 3]
        end
    end

    subgraph GPUAllocation["GPU分配 (2×4×4=32 GPUs)"]
        A1["DP0-TP0-PP0: GPU0"]
        A2["DP0-TP0-PP1: GPU1"]
        A3["..."]
        A4["DP1-TP3-PP3: GPU31"]
    end

    DP1 & DP2 --> TP1 & TP2 & TP3 & TP4
    TP1 & TP2 & TP3 & TP4 --> PP1 & PP2 & PP3 & PP4

    style DataParallelDim fill:#e3f2fd
    style TensorParallelDim fill:#fff8e1
    style PipelineParallelDim fill:#e8f5e9
```

9. 推理调度时序图

9.1 Continuous Batching

```
sequenceDiagram
    participant User1 as 用户1
    participant User2 as 用户2
    participant User3 as 用户3
    participant Scheduler as 调度器
    participant GPU as GPU

    Note over User1,GPU: Continuous Batching 持续批处理

    User1->>Scheduler: 请求1 (Prompt: 100 tokens)
    Scheduler->>GPU: Batch {Req1} Prefill
    GPU-->>Scheduler: KV Cache + 1st Token

    User2->>Scheduler: 请求2 (Prompt: 50 tokens)
    Scheduler->>GPU: Batch {Req1 Decode, Req2 Prefill}
    Note right of GPU: 动态添加新请求
    GPU-->>Scheduler: Tokens

    User3->>Scheduler: 请求3 (Prompt: 80 tokens)
    Scheduler->>GPU: Batch {Req1 Decode, Req2 Decode, Req3 Prefill}
    GPU-->>Scheduler: Tokens

    loop 持续生成
        Scheduler->>GPU: Batch {活跃请求}
        GPU-->>Scheduler: 各请求Token

        alt 请求完成
            Scheduler-->>User1: 返回完整响应
        else 新请求到达
            User3->>Scheduler: 新请求
        end
    end

    Scheduler-->>User2: 返回完整响应
    Scheduler-->>User3: 返回完整响应
```

9.2 Chunked Prefill

```
sequenceDiagram
    participant User as 用户
    participant Scheduler as 调度器
    participant PrefillQueue as Prefill队列
    participant DecodeQueue as Decode队列
    participant GPU as GPU引擎

    Note over User, GPU: Chunked Prefill 分块Prefill

    User->>Scheduler: 长文本请求 (4K tokens)

    Scheduler->>PrefillQueue: 加入队列

    loop 分块处理
        Scheduler->>GPU: Prefill Chunk 1 (1K tokens)
        GPU-->>Scheduler: Partial KV Cache

        Scheduler->>GPU: Prefill Chunk 2 (1K tokens)
        Note right of GPU: 与Decode交替执行
        GPU-->>Scheduler: Partial KV Cache

        Scheduler->>DecodeQueue: 检查Decode请求
        GPU->>GPU: 执行Decode Batch
    end

    Scheduler->>GPU: Final Prefill Chunk
    GPU-->>Scheduler: Complete KV Cache + 1st Token

    Scheduler->>DecodeQueue: 转入Decode阶段

    loop Token生成
        Scheduler->>GPU: Decode Batch
        GPU-->>Scheduler: Next Token
    end

    Scheduler-->>User: 返回完整响应
```

9.3 调度策略对比

```
flowchart TB
    subgraph StaticBatching["静态批处理"]
        S1[等待所有请求到达]
        S2[固定Batch执行]
        S3[等待最慢请求]
        S4[❌ GPU空闲气泡]
    end

    subgraph ContinuousBatching["Continuous Batching"]
        C1[动态添加请求]
        C2[迭代级调度]
        C3[请求完成后立即补充]
        C4[✅ 高GPU利用率]
    end

    subgraph ChunkedPrefill["Chunked Prefill"]
        P1[长文本分块]
        P2[Prefill/Decode交替]
        P3[减少TTFT延迟]
        P4[✅ 更好的交互性]
    end

    S1 --> S2 --> S3 --> S4
    C1 --> C2 --> C3 --> C4
    P1 --> P2 --> P3 --> P4

    style StaticBatching fill:#ffebee
    style ContinuousBatching fill:#e8f5e9
    style ChunkedPrefill fill:#e3f2fd
```

10. 投机解码流程图

10.1 整体流程

```
flowchart TB
    subgraph SpeculativeDecoding["投机解码 Speculative Decoding"]
        direction TB

        subgraph Draft["起草阶段"]
            D1[输入上下文]
            D2[小模型起草<br/>y个Token]
            D3[生成草稿序列]
        end

        subgraph Verify["验证阶段"]
            V1[大模型并行验证]
            V2[一次前向传播]
            V3[验证y个Token]
        end

        subgraph Accept["接受决策"]
            A1{Token是否<br/>接受?}
            A2[接受Token]
            A3[拒绝Token]
            A4[从拒绝处<br/>重新采样]
        end

        subgraph Output["输出"]
            O1[返回接受的Tokens]
            O2[更新上下文]
        end
    end

    D1 --> D2 --> D3
    D3 --> V1 --> V2 --> V3
    V3 --> A1
    A1 -->|是| A2
    A1 -->|否| A3 --> A4
    A2 --> O1 --> O2
    A4 --> O1
    O2 -->|继续| D1

    style Draft fill:#e3f2fd
    style Verify fill:#fff8e1
    style Accept fill:#e8f5e9
    style Output fill:#fce4ec
```

10.2 详细时序


```
sequenceDiagram
    participant User as 用户
    participant DraftModel as 起草模型<br/>小模型
    participant TargetModel as 目标模型<br/>大模型

    Note over User,TargetModel: 投机解码时序

    User->>DraftModel: 输入上下文

    loop y次迭代
        DraftModel->>DraftModel: 生成草稿Token
    end

    DraftModel-->>TargetModel: 草稿序列 [t+1, t+y]

    Note right of TargetModel: 单次前向传播<br/>验证所有草稿Token

    TargetModel->>TargetModel: 并行计算y个位置

    alt 所有Token接受
        TargetModel-->>User: 返回y个Token
    else 第k个Token拒绝
        TargetModel-->>User: 返回前k-1个Token
        TargetModel->>TargetModel: 从位置k重新采样
        TargetModel-->>User: 第k个新Token
    end

    Note over User,TargetModel: 平均接受率 ~70-80%<br/>速度提升 1.5x-2.5x
```

10.3 接受概率计算

```
flowchart TB
    subgraph Acceptance["Token接受机制"]
        direction TB

        subgraph Prob["概率分布"]
            P1["q(x): 起草模型概率"]
            P2["p(x): 目标模型概率"]
        end

        subgraph Method["接受方法"]
            M1["接受概率: min(1, p(x)/q(x))"]
            M2["高概率Token更容易接受"]
            M3["低概率Token可能拒绝"]
        end

        subgraph Rejection["拒绝处理"]
            R1["找到拒绝位置"]
            R2["从调整后的分布采样"]
            R3["p'(x) = norm(max(0, p(x) - q(x)))"]
        end
    end

    P1 & P2 --> M1 --> M2 --> M3
    M3 -->|拒绝| R1 --> R2 --> R3

    style Prob fill:#e3f2fd
    style Method fill:#fff8e1
    style Rejection fill:#ffebee
```

10.4 变体对比

```
graph TD
    subgraph Variants [投机解码变体]
        direction TB
        subgraph StandardSD [标准投机解码]
            SD1[独立小模型]
            SD2[需要额外内存]
            SD3[通用性强]
        end
        subgraph LookaheadSD [Lookahead解码]
            LS1[无额外模型]
            LS2[Jacobi迭代]
            LS3[基于n-gram]
        end
        subgraph MedusaSD [Medusa]
            M1[训练 draft heads]
            M2[与基础模型共享]
            M3[树形注意力]
        end
        subgraph EAGLESD [EAGLE]
            E1[特征层投机]
            E2[Auto-regressive head]
            E3[更高接受率]
        end
    end

    SD1 --> SD2 --> SD3
    LS1 --> LS2 --> LS3
    M1 --> M2 --> M3
    E1 --> E2 --> E3

    style StandardSD fill:#e3f2fd
    style LookaheadSD fill:#fff8e1
    style MedusaSD fill:#e8f5e9
    style EAGLESD fill:#fce4ec
```

附录：图表速查表

图表编号	名称	用途
1	AI Infra整体架构图	展示AI基础设施的层次结构
2	LLM训练和推理流程图	说明训练和推理的完整流程
3	Transformer架构图	展示Transformer的内部结构
4	Mooncake架构图	介绍PD分离推理系统架构
5	PD分离架构对比图	对比传统架构和PD分离架构
6	KV Cache管理流程图	说明KV Cache的生命周期管理
7	PagedAttention原理图	展示PagedAttention内存管理机制
8	分布式训练并行策略对比图	对比各种并行训练策略
9	推理调度时序图	展示Continuous Batching等调度机制
10	投机解码流程图	说明Speculative Decoding原理

文档生成时间：2024年 图表使用Mermaid语法绘制